Solutions Manual

Theory of Finite Automata with an Introduction to Formal Languages

John Carroll and Darrell Long

March 26, 2026

Contents

Chapter 0

Preliminaries — Solutions

0.1. **Construct truth tables for:**

(a) $\neg r \vee (\neg p \downarrow \neg q)$

Recall that $p \downarrow q$ (NOR) is $\neg(p \vee q)$.

p	q	r	$\neg p$	$\neg q$	$\neg p \downarrow \neg q$	$\neg r$	$\neg r \vee (\neg p \downarrow \neg q)$
0	0	0	1	1	0	1	1
0	0	1	1	1	0	0	0
0	1	0	1	0	0	1	1
0	1	1	1	0	0	0	0
1	0	0	0	1	0	1	1
1	0	1	0	1	0	0	0
1	1	0	0	0	1	1	1
1	1	1	0	0	1	0	1

(b) $(p \wedge \neg q) \vee \neg(p \uparrow q)$

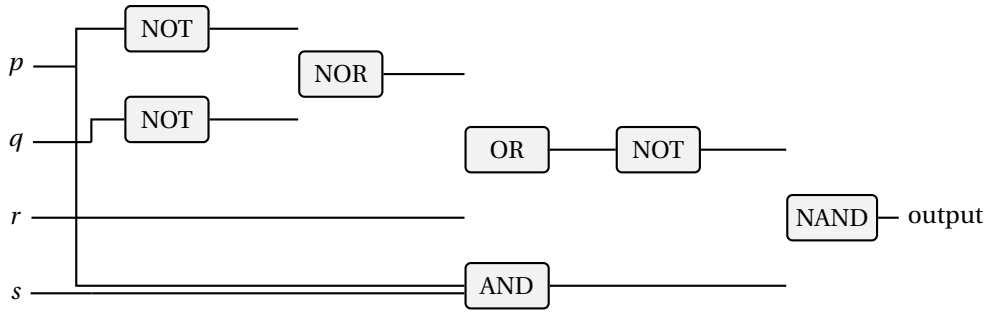
Recall that $p \uparrow q$ (NAND) is $\neg(p \wedge q)$.

p	q	$p \wedge \neg q$	$p \uparrow q$	$\neg(p \uparrow q)$	$(p \wedge \neg q) \vee \neg(p \uparrow q)$
0	0	0	1	0	0
0	1	0	1	0	0
1	0	1	1	0	1
1	1	0	0	1	1

0.2. **Draw circuit diagrams for:**

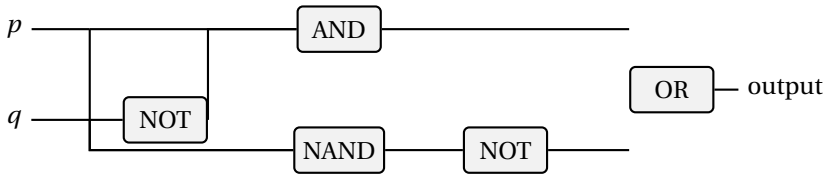
(a) $\neg(r \vee (\neg p \downarrow \neg q)) \uparrow (s \wedge p)$

One corresponding circuit is:



(b) $(p \wedge \neg q) \vee \neg(p \uparrow q)$

One corresponding circuit is:



0.3. **Show that** $\{1,2\} \times \{a,b\}$ **and** $\{a,b\} \times \{1,2\}$ **are not equal.**

$$\{1,2\} \times \{a,b\} = \{\langle 1,a\rangle, \langle 1,b\rangle, \langle 2,a\rangle, \langle 2,b\rangle\}$$

$$\{a,b\} \times \{1,2\} = \{\langle a,1\rangle, \langle a,2\rangle, \langle b,1\rangle, \langle b,2\rangle\}$$

Since $\langle 1,a\rangle \in \{1,2\} \times \{a,b\}$ but $\langle 1,a\rangle \notin \{a,b\} \times \{1,2\}$ (the first component of any element of $\{a,b\} \times \{1,2\}$ is a letter, not a number), the two sets are not equal.

0.4. **Let** $X = \{1,2,3,4\}$**.**

(a) $< = \{\langle 1,2\rangle, \langle 1,3\rangle, \langle 1,4\rangle, \langle 2,3\rangle, \langle 2,4\rangle, \langle 3,4\rangle\}$

(b) $= = \{\langle 1,1\rangle, \langle 2,2\rangle, \langle 3,3\rangle, \langle 4,4\rangle\}$

(c) $= \cup < = \{\langle 1,1\rangle, \langle 2,2\rangle, \langle 3,3\rangle, \langle 4,4\rangle, \langle 1,2\rangle, \langle 1,3\rangle, \langle 1,4\rangle, \langle 2,3\rangle, \langle 2,4\rangle, \langle 3,4\rangle\}$

(d) $\leq == \cup <$, which is the set computed in part (c).

0.5. **Show that** $\equiv_n$ **is an equivalence relation.**

We must show reflexivity, symmetry, and transitivity.

Reflexivity: $a - a = 0 = 0 \cdot n$, so $a \equiv_n a$.

Symmetry: If $a \equiv_n b$, then $n \mid (a-b)$, so $a - b = kn$ for some $k \in \mathbb{Z}$. Then $b - a = (-k)n$, so $n \mid (b-a)$, giving $b \equiv_n a$.

Transitivity: If $a \equiv_n b$ and $b \equiv_n c$, then $a - b = kn$ and $b - c = mn$ for integers k, m. Then $a - c = (a-b) + (b-c) = (k+m)n$, so $a \equiv_n c$.

0.6. **Equivalence classes of** $\equiv_0$ **on** $\mathbb{N}$**.**

$a \equiv_0 b$ means $0 \mid (a-b)$, i.e., $a - b = 0$, i.e., $a = b$. So the equivalence classes are all singletons: $\{0\}, \{1\}, \{2\}, \ldots$

0.7. **Equivalence classes of $\equiv_1$ on $\mathbb{N}$.**

$a \equiv_1 b$ means $1 \mid (a-b)$, which is always true. So there is exactly one equivalence class: $\mathbb{N}$ itself.

0.8. **Equivalence classes of $\equiv_1$ on $\mathbb{R}$.**

Here $x \equiv_1 y$ means $1 \mid (x-y)$, i.e., $x - y \in \mathbb{Z}$ (the difference is an integer). Two reals are equivalent iff they have the same fractional part $\{x\} = x - \lfloor x \rfloor$. The equivalence classes are the cosets of $\mathbb{Z}$ in $(\mathbb{R}, +)$:

$$[r]_{\equiv_1} = \{r + n \mid n \in \mathbb{Z}\}, \qquad r \in [0,1).$$

There are uncountably many such classes, one for each $r \in [0,1)$.

0.9. **Prove the distinct equivalence classes of R form a partition of X.**

Let $[x]_R = \{y \in X \mid y\,R\,x\}$.

Coverage: For any $x \in X$, reflexivity gives $x \in [x]_R$, so every element of X belongs to at least one class.

Disjointness: Suppose $[x]_R \cap [y]_R \neq \emptyset$; pick z with $z\,R\,x$ and $z\,R\,y$. By symmetry $x\,R\,z$, and by transitivity $x\,R\,y$. For any $w \in [x]_R$ we have $w\,R\,x\,R\,y$, so $w \in [y]_R$; thus $[x]_R \subseteq [y]_R$. By symmetry $[y]_R \subseteq [x]_R$, so $[x]_R = [y]_R$.

Hence the distinct classes are disjoint and cover X, forming a partition.

0.10. **Prove X equals the union of the sets in P.**

Let $P = \{A_1, \ldots, A_n\}$ be a partition of X.

By definition of partition, each $A_i \subseteq X$, so $\bigcup_i A_i \subseteq X$.

Also by definition, every $x \in X$ belongs to some A_i, so $X \subseteq \bigcup_i A_i$.

Therefore $X = \bigcup_{i=1}^{n} A_i$.

0.11. **Prove $R(P)$ is an equivalence relation.**

Define $x\,R(P)\,y$ iff x and y belong to the same block of P.

Reflexivity: x is in the same block as itself.

Symmetry: If x and y are in the same block, so are y and x.

Transitivity: If x, y are in block A_i and y, z are in block A_j, then since $y \in A_i \cap A_j$ and blocks are disjoint, $A_i = A_j$. So x and z are in the same block.

0.12. **Let $X = \{1,2,3,4\}$.**

(a) Example where $R(P)$ is a function: $P = \{\{1\},\{2\},\{3\},\{4\}\}$. Then $R(P) = \{\langle x, x\rangle \mid x \in X\}$, which is the identity function.

(b) Example where $R(P)$ is not a function: $P = \{\{1,2\},\{3,4\}\}$. Then $\langle 1,1\rangle, \langle 1,2\rangle \in R(P)$, so 1 maps to two elements; it is not a function.

0.13. **Find the flaw in the "proof" that symmetric and transitive implies reflexive.**

The flaw is in the first step. Symmetry says: if $x\,R\,y$ then $y\,R\,x$. This requires the hypothesis that $x\,R\,y$. If x is not related to any y at all, we cannot conclude $x\,R\,x$.

Counterexample: Let $X = \{1,2,3\}$ and $R = \{\langle 1,1\rangle, \langle 2,2\rangle, \langle 1,2\rangle, \langle 2,1\rangle\}$. Then R is symmetric and transitive but $3 \not R\, 3$, so R is not reflexive.

0.14. **Prove equality refines R.**

We need: $x = y \Rightarrow x\,R\,y$. If $x = y$, then by reflexivity of R, $x\,R\,x = x\,R\,y$. $\square$

0.15. **The "function" $t : \mathbb{R} \to \mathbb{R}$, $x \mapsto \arccos(x)$.**

(a) t is not well defined. A function must assign *exactly one* output to each input in its domain. But if $x \in [-1,1]$, there are generally many real numbers whose cosine is x. For example,

$$\cos\left(\frac{\pi}{3}\right) = \frac{1}{2} = \cos\left(\frac{5\pi}{3}\right),$$

so the input $x = \frac{1}{2}$ would have more than one output. Also, if $x \notin [-1,1]$, there is no real number whose cosine is x, so the proposed rule is not even defined everywhere on $\mathbb{R}$.

(b) Restrict domain to $[-1,1]$ and range to $[0,\pi]$: $t : [-1,1] \to [0,\pi]$, $t(x) = \arccos(x)$, is a valid function.

0.16. **Show the converse of $s' : \mathbb{R} \to \mathbb{R}$, $s'(x) = x^2$, is not a function.**

s'^{-1} (converse) is $\{\langle y, x\rangle \mid x^2 = y\}$. For $y = 1$, both $x = 1$ and $x = -1$ satisfy $x^2 = 1$, so $\langle 1,1\rangle$ and $\langle 1,-1\rangle$ are both in the converse, which is therefore not a function (1 maps to two values).

0.17. **Show s^{-1} exists for $s : \mathbb{P} \to \mathbb{P}$, $s(x) = x^2$, where $\mathbb{P}$ is nonneg. reals.**

s is injective: if $x^2 = y^2$ and $x, y \geq 0$ then $x = y$. s is surjective onto $\mathbb{P}$: for any $y \geq 0$, $x = \sqrt{y} \geq 0$ satisfies $s(x) = y$. Since s is a bijection, $s^{-1}(y) = \sqrt{y}$ exists.

0.18. **Prove: converse of $f : X \to Y$ is a function iff f is a bijection.**

($\Rightarrow$) Suppose $\tilde{f} = \{\langle y, x\rangle \mid \langle x, y\rangle \in f\}$ is a function (from Y to X). Then every $y \in Y$ has a unique pre-image under f. *Injectivity*: if $f(x_1) = f(x_2) = y$, then $\tilde{f}(y)$ must equal both x_1 and x_2; since $\tilde{f}$ is a function, $x_1 = x_2$. *Surjectivity*: since $\tilde{f}$ is a function *with domain* Y, every $y \in Y$ is in the domain of $\tilde{f}$, meaning there exists x with $f(x) = y$. Hence f is surjective. (Note: totality of f does not by itself imply surjectivity; surjectivity follows here from $\tilde{f}$ being defined on *all* of Y.)

($\Leftarrow$) If f is injective, each y has at most one pre-image, so $\tilde{f}$ assigns at most one value. If f is surjective, each $y \in Y$ has at least one pre-image, so $\tilde{f}$ is defined everywhere on Y. Together, $\tilde{f}$ is a function.

0.19. (a) $\sim(\sim A) = A$: By definition, $\sim A = \{x \mid x \notin A\}$. Then $\sim(\sim A) = \{x \mid x \notin \sim A\} = \{x \mid x \in A\} = A$.

(b) $\sim(\sim R) = R$ for a relation R (where $\sim R$ is the converse): $\sim R = \{\langle y, x\rangle \mid \langle x, y\rangle \in R\}$. Then $\sim(\sim R) = \{\langle x, y\rangle \mid \langle y, x\rangle \in \sim R\} = \{\langle x, y\rangle \mid \langle x, y\rangle \in R\} = R$.

0.20. **If $f : X \to Y$ and $g : Y \to Z$ are one-to-one, prove $g \circ f$ is one-to-one.**

Suppose $(g \circ f)(x_1) = (g \circ f)(x_2)$. Then $g(f(x_1)) = g(f(x_2))$. Since g is injective, $f(x_1) = f(x_2)$. Since f is injective, $x_1 = x_2$.

0.21. **If $f : X \to Y$ and $g : Y \to Z$ are onto, prove $g \circ f$ is onto.**

Let $z \in Z$. Since g is surjective, $\exists y \in Y$ with $g(y) = z$. Since f is surjective, $\exists x \in X$ with $f(x) = y$. Then $(g \circ f)(x) = g(f(x)) = g(y) = z$.

0.22. **Define two functions for which $f \circ g = g \circ f$.**

Let $f, g : \mathbb{R} \to \mathbb{R}$, $f(x) = x+1$, $g(x) = x+2$. Then $(f \circ g)(x) = f(x+2) = x+3 = g(x+1) = (g \circ f)(x)$.

0.23. **Define bijections.**

(a) $\mathbb{N}$ and $\mathbb{Z}$: $f : \mathbb{N} \to \mathbb{Z}$ defined by

$$f(n) = \begin{cases} n/2 & \text{if } n \text{ is even} \\ -(n+1)/2 & \text{if } n \text{ is odd} \end{cases}$$

maps $0 \mapsto 0, 1 \mapsto -1, 2 \mapsto 1, 3 \mapsto -2, 4 \mapsto 2, \ldots$

(b) $\mathbb{N}$ and $\mathbb{N} \times \mathbb{N}$: Use the Cantor pairing function $f(m,n) = \frac{(m+n)(m+n+1)}{2} + n$. Its inverse gives a bijection $\mathbb{N} \to \mathbb{N} \times \mathbb{N}$.

(c) $\mathbb{N}$ and $\mathbb{Q}$: The positive rationals can be arranged in a grid and enumerated by diagonals (Cantor's enumeration). Combined with the bijection $\mathbb{N} \to \mathbb{Z}$ from part (a), one obtains a bijection $\mathbb{N} \to \mathbb{Q}$.

(d) $\mathbb{N}$ and $\{a, b, c\}$: No bijection exists, since $\mathbb{N}$ is infinite and $\{a, b, c\}$ has only 3 elements.

0.24. **Use induction to prove $\sum_{k=1}^{n} k^3 = \frac{n^2(n+1)^2}{4}$.**

Base case ($n = 1$): $1^3 = 1 = \frac{1 \cdot 4}{4}$. ✓

Inductive step: Assume $\sum_{k=1}^{n} k^3 = \frac{n^2(n+1)^2}{4}$. Then

$$\begin{aligned} \sum_{k=1}^{n+1} k^3 &= \frac{n^2(n+1)^2}{4} + (n+1)^3 \\ &= \frac{n^2(n+1)^2 + 4(n+1)^3}{4} \\ &= \frac{(n+1)^2(n^2 + 4(n+1))}{4} \\ &= \frac{(n+1)^2(n^2 + 4n + 4)}{4} \\ &= \frac{(n+1)^2(n+2)^2}{4}. \end{aligned}$$

This is the formula with n replaced by $n+1$. □

0.25. **Prove $\sum_{k=1}^{n} k < n^2$ for $n > 1$.**

Base case ($n = 2$): $1 + 2 = 3 < 4 = 2^2$. ✓

Inductive step: Assume $\sum_{k=1}^{n} k < n^2$. Then

$$\sum_{k=1}^{n+1} k = \sum_{k=1}^{n} k + (n+1) < n^2 + n + 1.$$

We need $n^2 + n + 1 \leq (n+1)^2 = n^2 + 2n + 1$, which holds since $n \geq 1$. □

0.26. **Prove $n! > n^2$ for $n > 3$.**

Base case ($n = 4$): $4! = 24 > 16 = 4^2$. ✓

Inductive step: Assume $n! > n^2$ for some $n \geq 4$. Then

$$(n+1)! = (n+1) \cdot n! > (n+1) \cdot n^2.$$

We need $(n+1)n^2 \geq (n+1)^2$, i.e., $n^2 \geq n+1$, i.e., $n^2 - n - 1 \geq 0$. For $n \geq 4$, $n^2 \geq 16 > 5 > n+1$. □

0.27. **Prove $n! > 2^n$ for $n > 3$.**

Base case ($n = 4$): $24 > 16$. ✓

Inductive step: Assume $n! > 2^n$ for $n \geq 4$. Then

$$(n+1)! = (n+1) \cdot n! > (n+1) \cdot 2^n \geq 5 \cdot 2^n > 2 \cdot 2^n = 2^{n+1}. \quad \square$$

0.28. **Prove $1^2 + 2^2 + \cdots + n^2 = \frac{n(n+1)(2n+1)}{6}$.**

Base case ($n = 1$): $1 = \frac{1 \cdot 2 \cdot 3}{6} = 1$. ✓

Inductive step: Assume the formula holds for n. Then

$$\begin{aligned}
\sum_{k=1}^{n+1} k^2 &= \frac{n(n+1)(2n+1)}{6} + (n+1)^2 \\
&= \frac{n(n+1)(2n+1) + 6(n+1)^2}{6} \\
&= \frac{(n+1)[n(2n+1) + 6(n+1)]}{6} \\
&= \frac{(n+1)(2n^2 + 7n + 6)}{6} \\
&= \frac{(n+1)(n+2)(2n+3)}{6}. \quad \square
\end{aligned}$$

0.29. **Prove $X \cap (X_1 \cup \cdots \cup X_n) = (X \cap X_1) \cup \cdots \cup (X \cap X_n)$ by induction.**

Base case ($n = 1$): $X \cap X_1 = X \cap X_1$. ✓

Inductive step: Assume the result for n. Then

$$\begin{aligned}
X \cap \Big(\bigcup_{i=1}^{n+1} X_i\Big) &= X \cap \Big(\Big(\bigcup_{i=1}^{n} X_i\Big) \cup X_{n+1}\Big) \\
&= \Big(X \cap \bigcup_{i=1}^{n} X_i\Big) \cup (X \cap X_{n+1}) \quad \text{(distributivity for } n = 2\text{)} \\
&= \Big(\bigcup_{i=1}^{n} (X \cap X_i)\Big) \cup (X \cap X_{n+1}) \quad \text{(inductive hypothesis)} \\
&= \bigcup_{i=1}^{n+1} (X \cap X_i). \quad \square
\end{aligned}$$

0.30. **Prove $\sim(X_1 \cup \cdots \cup X_n) = (\sim X_1) \cap \cdots \cap (\sim X_n)$ by induction.**

Base case ($n = 2$): De Morgan's law: $\sim(X_1 \cup X_2) = (\sim X_1) \cap (\sim X_2)$. ✓

Inductive step: Assume for n. Then

$$\sim\left(\bigcup_{i=1}^{n+1} X_i\right) = \sim\left(\bigcup_{i=1}^{n} X_i \cup X_{n+1}\right) = \sim\left(\bigcup_{i=1}^{n} X_i\right) \cap (\sim X_{n+1}) = \left(\bigcap_{i=1}^{n} \sim X_i\right) \cap (\sim X_{n+1}) = \bigcap_{i=1}^{n+1} \sim X_i. \quad \square$$

0.31. **Prove $|\wp(A)| = 2^{|A|}$ by induction.**

Base case ($|A| = 0$): $\wp(\emptyset) = \{\emptyset\}$, so $|\wp(\emptyset)| = 1 = 2^0$. ✓

Inductive step: Assume $|\wp(A)| = 2^n$ for all sets with $|A| = n$. Let $|B| = n+1$ and pick any $b \in B$. Set $A = B \setminus \{b\}$. Every subset of B either contains b or not. Those not containing b are exactly $\wp(A)$ (2^n of them). Those containing b are in bijection with $\wp(A)$ via $S \mapsto S \setminus \{b\}$ (2^n more). Total: $2^n + 2^n = 2^{n+1}$. $\square$

0.32. **Prove strong induction is equivalent to ordinary induction.**

Let $P(n)$ be a statement and $Q(n) = (\forall i \le n) P(i)$.

($\Rightarrow$) Assume ordinary induction. The strong induction hypotheses are $P(0)$ and $(\forall m)((\forall i \le m) P(i) \Rightarrow P(m+1))$. Define $Q(n) = (\forall i \le n) P(i)$. Clearly $Q(0) \Leftrightarrow P(0)$. The strong hypothesis says $Q(m) \Rightarrow P(m+1)$, which means $Q(m) \Rightarrow Q(m+1)$ (since $Q(m+1) = Q(m) \wedge P(m+1)$). By ordinary induction on Q, $(\forall n) Q(n)$, hence $(\forall n) P(n)$.

($\Leftarrow$) Ordinary induction is the special case of strong induction where the hypothesis uses only $P(m)$ instead of $(\forall i \le m) P(i)$.

0.33. **Determine what strings the BNF defines.**

The grammar defines *real number constants* (possibly with a sign and decimal/exponent part) in the style of FORTRAN: integers, signed integers, decimals, and floating-point constants with an **E** exponent. Examples: `42`, `-3.14`, `+2.5E-10`, `0.001E3`.

0.34. **Cofinite sets with universal set $\mathbb{Z}$.**

(a) A finite set: $\{-5, 0, 3, 7\}$.

(b) A cofinite set: $\mathbb{Z} \setminus \{0\} = \{n \in \mathbb{Z} \mid n \neq 0\}$ (complement is $\{0\}$, which is finite).

(c) Neither: $2\mathbb{Z} = \{\ldots, -4, -2, 0, 2, 4, \ldots\}$, the even integers. Its complement is the set of odd integers, which is infinite; so it is not cofinite. It is infinite, so not finite.

0.35. **Prove the equipotent relation is an equivalence relation.**

Recall: $A \sim B$ iff there exists a bijection $f : A \to B$.

(a) *Reflexive*: $\mathrm{id}_A : A \to A$ is a bijection, so $A \sim A$.

(b) *Symmetric*: If $f : A \to B$ is a bijection, then $f^{-1} : B \to A$ is also a bijection.

(c) *Transitive*: If $f : A \to B$ and $g : B \to C$ are bijections, then $g \circ f : A \to C$ is a bijection.

0.36. **Define a function showing $|\mathbb{N}| = |\mathbb{N} \times \mathbb{N}|$.**

Use the Cantor pairing function: $f : \mathbb{N} \times \mathbb{N} \to \mathbb{N}$, $f(m,n) = \frac{(m+n)(m+n+1)}{2} + n$. This is a bijection. Its inverse $f^{-1} : \mathbb{N} \to \mathbb{N} \times \mathbb{N}$ is the desired function.

0.37. **Show $\mathbb{N}$ is equipotent to $\mathbb{Z}$.**

One bijection $f : \mathbb{N} \to \mathbb{Z}$ is

$$f(n) = \begin{cases} n/2, & \text{if } n \text{ is even,} \\ -(n+1)/2, & \text{if } n \text{ is odd.} \end{cases}$$

Thus

$$0 \mapsto 0, \quad 1 \mapsto -1, \quad 2 \mapsto 1, \quad 3 \mapsto -2, \quad 4 \mapsto 2, \quad 5 \mapsto -3, \ldots$$

so every integer appears exactly once. Hence $\mathbb{N}$ and $\mathbb{Z}$ are equipotent.

0.38. **Show $\mathbb{N}$ is equipotent to $\mathbb{Q}$.**

The positive rationals $\mathbb{Q}^+$ can be enumerated by listing p/q in a grid (rows indexed by p, columns by q) and enumerating by anti-diagonals (skipping non-reduced fractions). Combined with the bijection from $\mathbb{N}$ to $\mathbb{Z}$ (hence handling negative rationals and zero), one obtains a bijection $\mathbb{N} \to \mathbb{Q}$.

0.39. **Show $\wp(\mathbb{N})$ is equipotent to $\{f : \mathbb{N} \to \{\textbf{Yes,No}\}\}$.**

Define $\phi : \wp(\mathbb{N}) \to \{f : \mathbb{N} \to \{\text{Yes,No}\}\}$ by $\phi(A) = \chi_A$, where $\chi_A(n) = \text{Yes}$ if $n \in A$ and No otherwise. This is clearly a bijection (its inverse maps f to $\{n \mid f(n) = \text{Yes}\}$).

0.40. **Show $\wp(\mathbb{N})$ is equipotent to $\mathbb{R}$.**

By Exercise 0.39, $\wp(\mathbb{N})$ is equipotent to the set

$$B = \{(a_0, a_1, a_2, \ldots) \mid a_i \in \{0,1\}\}$$

of infinite binary sequences. It therefore suffices to show that B is equipotent to $\mathbb{R}$.

Let $E \subseteq B$ be the set of sequences that are eventually all 1s. This set is countable, because such a sequence is determined by a finite initial segment and the position after which only 1s occur. Let

$$D = \left\{ \frac{k}{2^n} \;\middle|\; n \in \mathbb{N},\ 0 \le k \le 2^n \right\}$$

be the set of dyadic rationals in $[0,1]$; D is also countable. Choose a bijection $\eta : E \to D$.

Now define $\Phi : B \to [0,1]$ by

$$\Phi(a_0, a_1, a_2, \ldots) = \begin{cases} \displaystyle\sum_{i=0}^{\infty} \frac{a_i}{2^{i+1}}, & \text{if } (a_0, a_1, a_2, \ldots) \notin E, \\ \eta(a_0, a_1, a_2, \ldots), & \text{if } (a_0, a_1, a_2, \ldots) \in E. \end{cases}$$

If a sequence is not eventually all 1s, its binary expansion is the unique binary expansion of a nondyadic real in $[0,1]$. Thus the first clause is a bijection from $B \setminus E$ onto $[0,1] \setminus D$. The second clause is a bijection from E onto D. Therefore Φ is a bijection from B onto $[0,1]$.

It remains to show that $[0,1]$ is equipotent to $\mathbb{R}$. First define a bijection $j : [0,1] \to (0,1)$ by

$$j(x) = \begin{cases} \frac{1}{2}, & x = 0, \\ \frac{1}{4}, & x = 1, \\ \frac{1}{2^{n+2}}, & x = \frac{1}{2^n} \text{ for } n \geq 1, \\ x, & \text{otherwise.} \end{cases}$$

This simply shifts the countable set $\{1, 1/2, 1/4, 1/8, \ldots\}$ inward and fixes all other points. Next define

$$g : (0,1) \to \mathbb{R}, \qquad g(x) = \tan\left(\pi x - \frac{\pi}{2}\right).$$

This is a bijection from $(0,1)$ onto $\mathbb{R}$. Hence $g \circ j$ is a bijection from $[0,1]$ onto $\mathbb{R}$.

Since $\wp(\mathbb{N}) \sim B$, $B \sim [0,1]$, and $[0,1] \sim \mathbb{R}$, the equipotent relation is transitive, so $\wp(\mathbb{N}) \sim \mathbb{R}$.

0.41. **Draw a circuit for q (Figure 0.8).**

The truth table shows $q = 1$ when $(p_1, p_2, p_3) \in \{(0,1,0), (1,0,0), (1,1,1)\}$.

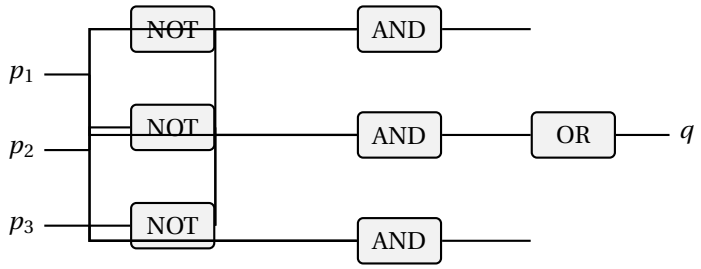
The minimal sum of products (minterms 2, 4, 7):

$$q = (\bar{p}_1 p_2 \bar{p}_3) \vee (p_1 \bar{p}_2 \bar{p}_3) \vee (p_1 p_2 p_3).$$

This can be factored as:

$$q = \neg p_3 (p_1 \oplus p_2) \vee p_1 p_2 p_3.$$

One corresponding AND-OR circuit is:



0.42. **Draw circuits for q_1, q_2, q_3 (Figure 0.9).**

Reading the truth table row by row (rows indexed 0–15, with $p_1 p_2 p_3 p_4$ in binary):

$q_1 = 1$ at rows 0,2,3,5,6,8,9,12 (i.e., minterms $m_0, m_2, m_3, m_5, m_6, m_8, m_9, m_{12}$): (0000), (0010), (0011), (0101), (0110), (1

$q_2 = 1$ at rows 0,1,3,4,6,10,12,13,14 (minterms $m_0, m_1, m_3, m_4, m_6, m_{10}, m_{12}, m_{13}, m_{14}$): (0000), (0001), (0011), (0100), (

$q_3 = 1$ at rows 0,4,8,12,14 (minterms $m_0, m_4, m_8, m_{12}, m_{14}$): (0000), (0100), (1000), (1100), (1110).

Applying four-variable Karnaugh maps over p_1, p_2, p_3, p_4 yields the following minimal sum-of-products expressions (bars denote complementation):

$$q_1 \;=\; p_1 \bar{p}_2 \bar{p}_3 + \; p_1 \bar{p}_3 \bar{p}_4 + \; \bar{p}_1 \bar{p}_2 p_3 + \; \bar{p}_1 p_3 \bar{p}_4 + \; \bar{p}_1 p_2 \bar{p}_3 p_4 + \; \bar{p}_2 \bar{p}_3 \bar{p}_4$$

(6 prime implicants; $m_5 = \bar{p}_1 p_2 \bar{p}_3 p_4$ is an isolated essential implicant.)

$$q_2 = p_2\bar{p}_4 + p_1p_3\bar{p}_4 + p_1p_2\bar{p}_3 + \bar{p}_1\bar{p}_2p_4 + \bar{p}_1\bar{p}_2\bar{p}_3$$

(The quad $p_2\bar{p}_4$ covers m_4, m_6, m_{12}, m_{14}; the last two terms together cover m_0, m_1, m_3.)

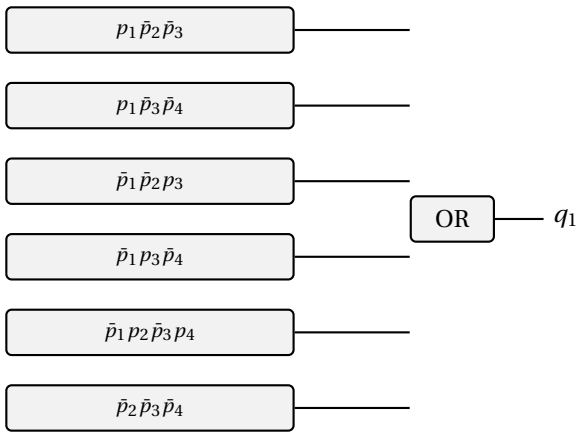
$$q_3 = \bar{p}_3\bar{p}_4 + p_1p_2\bar{p}_4$$

(The quad $\bar{p}_3\bar{p}_4$ covers m_0, m_4, m_8, m_{12}; the pair $p_1p_2\bar{p}_4$ covers m_{12}, m_{14}.)

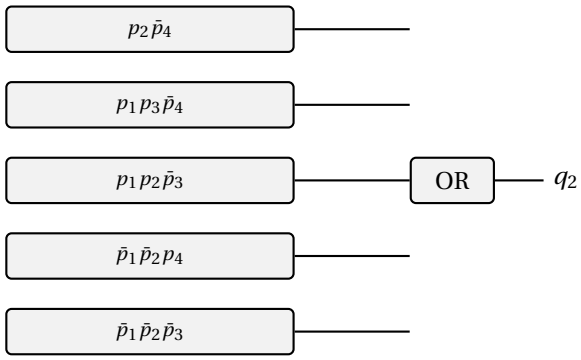
Each minimized expression is implemented as a two-level AND-OR circuit (one AND gate per product term, one final OR gate per output), with NOT gates on the complemented inputs.

The following block diagrams show those two-level realizations. In each case, the rectangles on the left are AND subcircuits computing the indicated product terms from p_1, p_2, p_3, p_4 and their complements.

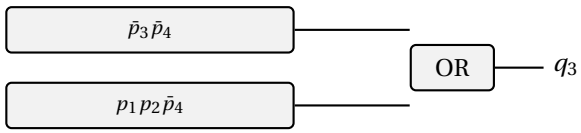
(a) For q_1:



(b) For q_2:



(c) For q_3:



Chapter 1

Introduction and Basic Definitions — Solutions

1.1. **Show $\overline{\delta}_t = \overline{\delta}_h$.**

Let $P(n)$: $(\forall s \in S)(\forall x \in \Sigma^n)(\overline{\delta}_t(s,x) = \overline{\delta}_h(s,x))$.

Base: $n = 0$: $\overline{\delta}_t(s,\lambda) = s = \overline{\delta}_h(s,\lambda)$. For $n = 1$: $\overline{\delta}_t(s,\mathbf{a}) = \delta(s,\mathbf{a}) = \overline{\delta}_h(s,\mathbf{a})$.

Inductive step: Let $x = ya$ where $|y| = n$ and $a \in \Sigma$. Assume $\overline{\delta}_t(s,y) = \overline{\delta}_h(s,y)$ for all s, y with $|y| \le n$.

We prove $\overline{\delta}_t(s,wa) = \delta(\overline{\delta}_t(s,w),a)$ for all $w \in \Sigma^*$, $a \in \Sigma$, $s \in S$, by induction on $|w|$.

If $|w| = 0$: $\overline{\delta}_t(s,a) = \delta(s,a)$. ✓

If $|w| = n+1$, write $w = bv$ with $b \in \Sigma$: $\overline{\delta}_t(s,bva) = \overline{\delta}_t(\delta(s,b),va) = \delta(\overline{\delta}_t(\delta(s,b),v),a)$ [IH] $= \delta(\overline{\delta}_t(s,bv),a)$. This establishes the claim. Now:

$$\overline{\delta}_h(s,wa) = \delta(\overline{\delta}_h(s,w),a) = \delta(\overline{\delta}_t(s,w),a) \text{ [IH]} = \overline{\delta}_t(s,wa). \quad \square$$

1.2. **Modify vending machine to include coin-return r.**

Add a new input symbol **r** (coin return). When **r** is pressed, transition back to the initial state s_0 (0 cents deposited) from any state. All other transitions remain unchanged. Formally, for every state s_i (representing i cents), $\delta(s_i,\mathbf{r}) = s_0$.

1.3. **The DFA of Figure 1.26.**

(a) The quintuple is

$$A = \langle\{\mathbf{0},\mathbf{1}\},\{q_0,q_1\},q_0,\delta,\{q_1\}\rangle,$$

where

$$\delta(q_0,\mathbf{0}) = q_1, \quad \delta(q_0,\mathbf{1}) = q_1, \quad \delta(q_1,\mathbf{0}) = q_1, \quad \delta(q_1,\mathbf{1}) = q_0.$$

(b) The language is

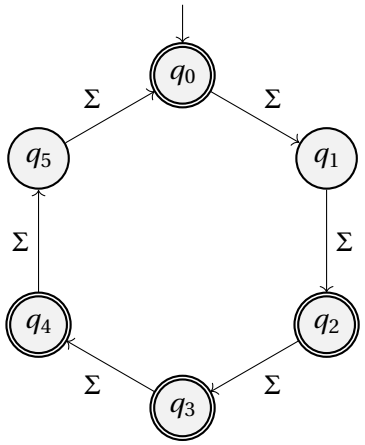
$$L(A) = \Sigma^*\mathbf{0}(\mathbf{11})^* \cup \mathbf{1}(\mathbf{11})^*, \qquad \Sigma = \{\mathbf{0},\mathbf{1}\}.$$

Equivalently, a word is accepted iff it either ends in **0** followed by an even number of trailing **1**s, or consists entirely of an odd number of **1**s.

1.4. **Build a DFA accepting $L = \{x \mid |x| \text{ is a multiple of 2 or 3}\}$ over $\{\mathbf{a}, \mathbf{b}, \mathbf{c}\}$.**

We need $|x| \equiv 0 \pmod 2$ or $|x| \equiv 0 \pmod 3$, i.e., $|x| \in \{0,2,3,4,6,8,9,\ldots\}$. The complement is $\{|x| \equiv 1 \pmod 6$ or $|x| \equiv 5 \pmod 6\}$. Use a 6-state DFA tracking $|x| \bmod 6$, with final states $\{0,2,3,4\}$ (i.e., everything except 1 and 5 mod 6).

$S = \{q_0, q_1, q_2, q_3, q_4, q_5\}$, $s_0 = q_0$, $F = \{q_0, q_2, q_3, q_4\}$, $\delta(q_i, a) = q_{(i+1) \bmod 6}$ for all $a \in \Sigma$.



1.5. **DFAs for L_1, L_2, L_3 over $\{0,1\}$.**

(a) $L_1 = \{x \mid |x| \bmod 7 = 4\}$: 7-state DFA with states $q_0, \ldots, q_6$, $F = \{q_4\}$, $\delta(q_i, a) = q_{(i+1) \bmod 7}$.

(b) $L_2 = \Sigma^* \setminus \{w \mid w \text{ ends in } 1\}$: The complement is all words ending in **1**. So L_2 accepts words not ending in **1**: either λ, or words ending in **0**. Two-state DFA: s_0 (start, final – not ending in 1, or empty), s_1 (ending in 1, nonfinal). $\delta(s_0, \mathbf{0}) = s_0$, $\delta(s_0, \mathbf{1}) = s_1$, $\delta(s_1, \mathbf{0}) = s_0$, $\delta(s_1, \mathbf{1}) = s_1$.

(c) $L_3 = \{y \mid |y|_0 = |y|_1\}$: not FAD. Suppose some DFA with n states accepted it. Consider the $n+1$ strings

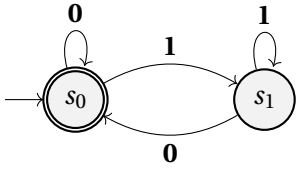
$$\lambda, \mathbf{0}, \mathbf{00}, \ldots, \mathbf{0}^n.$$

Two of them, say $\mathbf{0}^i$ and $\mathbf{0}^j$ with $i < j$, must lead from the start state to the same state. Then appending $\mathbf{1}^i$ would give the same final outcome for both strings. But

$$\mathbf{0}^i\mathbf{1}^i \in L_3 \quad \text{while} \quad \mathbf{0}^j\mathbf{1}^i \notin L_3,$$

since the second string has more **0**s than **1**s. This contradiction shows that no DFA can accept L_3.

The 2-state DFA for L_2:

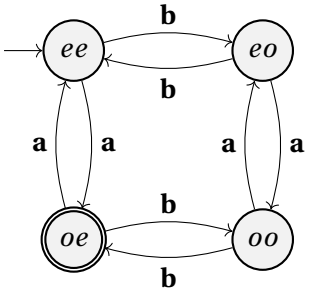


1.6. **DFAs A_1, A_2, A_3, A_4 over $\{\mathbf{a}, \mathbf{b}\}$.**

(a) Use a 4-state DFA with states $\{ee, eo, oe, oo\}$ (parity of #a, parity of #b). A_1: $F = \{oe\}$ (#a odd, #b even). A_2: $F = \{eo, oe, oo\}$ (not ee). A_3: $F = \{oe, eo\}$ (exactly one count even). A_4: $F = \{ee, oe\}$ (#a even).

(b) Each machine is a quotient or sub-machine of the 4-state product machine from Example 1.10, which tracks parities of #a and #b.

The common 4-state machine (shown here for A_1 with $F = \{oe\}$; change F for the other sub-machines):



1.7. **Modify M to accept $L(M) - \{\lambda\}$.**

If $s_0 \notin F$ then $\lambda \notin L(M)$ and M already accepts $L(M) \setminus \{\lambda\} = L(M)$; nothing to do.

If $s_0 \in F$, add a fresh nonfinal start state s_0' and set

$$\delta'(s_0', a) = \delta(s_0, a) \quad \text{for all } a \in \Sigma,$$

leaving all other transitions and the final-state set F unchanged. The new machine $M' = (\Sigma, S \cup \{s_0'\}, s_0', \delta', F)$ accepts exactly $L(M) \setminus \{\lambda\}$.

Correctness: λ is rejected because $s_0' \notin F$. For any nonempty $x = a_1 \cdots a_k$, the computation in M' follows $s_0' \xrightarrow{a_1} \delta(s_0, a_1) \xrightarrow{a_2} \cdots$, which mirrors the computation of M from s_0 on x, so $x \in L(M')$ iff $x \in L(M)$.

Why not remove s_0 from F? That would reject nonempty strings x with $\bar{\delta}(s_0, x) = s_0$, which belong to $L(M)$ and should remain accepted.

1.8. **General procedure to modify M to accept $L(M) \cup \{\lambda\}$.**

If $s_0 \in F$ already, M already accepts λ. Otherwise, add s_0 to F: $F' = F \cup \{s_0\}$. All transitions stay the same. This accepts $L(M) \cup \{\lambda\}$.

1.9. **DFA for $\Psi = \{x \mid (x$ begins with $\mathbf{d}) \vee (x$ contains $\mathbf{bb})\}$.**

The machine must distinguish "first character is **d**" (which makes the string accepted regardless of what follows) from "**d** appears later". A 3-state machine conflates these; the minimal DFA has **4 states**:

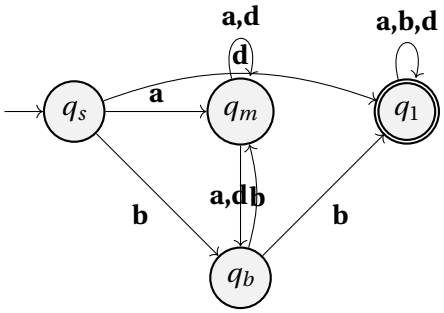
- q_s: start (no characters read yet).

- q_m: at least one character read; string does not start with **d** and no **bb** seen; last char was not **b**.
- q_b: same as q_m but last char was **b** (pending **bb**).
- q_1: accepting sink (string starts with **d**, or **bb** has been seen).

$F = \{q_1\}$.

	a	**b**	**d**
q_s	q_m	q_b	q_1
q_m	q_m	q_b	q_m
q_b	q_m	q_1	q_m
q_1	q_1	q_1	q_1

Spot-check: **bd** — $q_s \xrightarrow{b} q_b \xrightarrow{d} q_m$, rejected. ✓ **bb** — $q_s \xrightarrow{b} q_b \xrightarrow{b} q_1$, accepted. ✓ **d** — $q_s \xrightarrow{d} q_1$, accepted. ✓



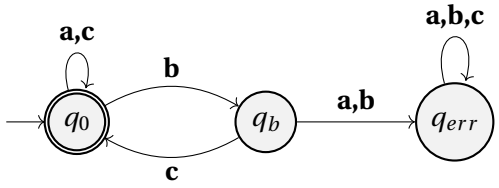
1.10. **DFA for $\Phi = \{x \mid$ every b in x is immediately followed by c$\}$.**

States: q_0 (ok, last char not **b**), q_b (last char was **b**), q_{err} (dead state). $F = \{q_0\}$.

$$\delta(q_0, \mathbf{a}) = q_0, \quad \delta(q_0, \mathbf{b}) = q_b, \quad \delta(q_0, \mathbf{c}) = q_0,$$

$$\delta(q_b, \mathbf{c}) = q_0, \quad \delta(q_b, \mathbf{a}) = q_{err}, \quad \delta(q_b, \mathbf{b}) = q_{err},$$

$$\delta(q_{err}, \mathbf{a}) = \delta(q_{err}, \mathbf{b}) = \delta(q_{err}, \mathbf{c}) = q_{err}.$$



1.11. **DFA for Ω (numbers divisible by 7).**

The key insight: if the current state represents n mod 7 (the value of the number read so far), then reading digit d gives the new value $(10n + d)$ mod 7.

States $q_0, \ldots, q_6$ (representing $0, \ldots, 6$ mod 7). $s_0 = q_0$, $F = \{q_0\}$.

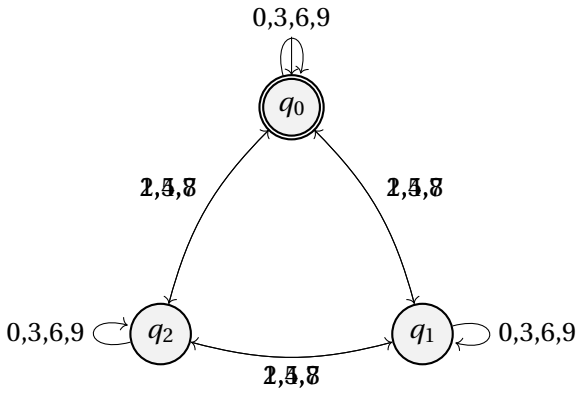
$$\delta(q_i, d) = q_{(10i+d) \bmod 7}.$$

1.12. **DFA for Γ (divisible by 3).**

Three states q_0, q_1, q_2 (representing $0, 1, 2 \bmod 3$). $F = \{q_0\}$.

$$\delta(q_i, d) = q_{(i+d) \bmod 3}$$

since $(10i + d) \bmod 3 = (i + d) \bmod 3$ (because $10 \equiv 1 \pmod 3$).



1.13. **DFA for K (divisible by 5).**

Since $10 \equiv 0 \pmod 5$, we have $(10i + d) \bmod 5 = d \bmod 5$. So only the last digit matters.

Five-state DFA: $\delta(q_i, d) = q_{d \bmod 5}$; $F = \{q_0\}$.

Two-state DFA: q_0 (final, last digit in $\{0,5\}$), q_1 (nonfinal, last digit not in $\{0,5\}$). $\delta(q_i, d) = q_0$ if $d \in \{0,5\}$, else q_1.

1.14. **DFA for the first eight primes:** $\{2,3,5,7,11,13,17,19\}$**.**

This language is finite, so it is FAD. A concrete DFA is the prefix-tree machine for the set

$$P = \{\lambda, \mathbf{1}, \mathbf{11}, \mathbf{13}, \mathbf{17}, \mathbf{19}, \mathbf{2}, \mathbf{3}, \mathbf{5}, \mathbf{7}\}.$$

Let the states be $\{q_x \mid x \in P\} \cup \{q_{dead}\}$, with start state q_λ and final states

$$F = \{q_{\mathbf{2}}, q_{\mathbf{3}}, q_{\mathbf{5}}, q_{\mathbf{7}}, q_{\mathbf{11}}, q_{\mathbf{13}}, q_{\mathbf{17}}, q_{\mathbf{19}}\}.$$

For any state q_x and any digit $\boldsymbol{d}$, define

$$\delta(q_x, \boldsymbol{d}) = \begin{cases} q_{x\boldsymbol{d}} & \text{if } x\boldsymbol{d} \in P, \\ q_{dead} & \text{otherwise.} \end{cases}$$

Also $\delta(q_{dead}, \boldsymbol{d}) = q_{dead}$ for every digit $\boldsymbol{d}$. Thus q_λ branches to $q_{\mathbf{2}}, q_{\mathbf{3}}, q_{\mathbf{5}}, q_{\mathbf{7}}$, and to $q_{\mathbf{1}}$ on input $\mathbf{1}$; then $q_{\mathbf{1}}$ branches to $q_{\mathbf{11}}, q_{\mathbf{13}}, q_{\mathbf{17}}, q_{\mathbf{19}}$ on inputs $\mathbf{1}, \mathbf{3}, \mathbf{7}, \mathbf{9}$, respectively. Every other transition goes to q_{dead}, so the machine accepts exactly the first eight primes.

1.15. **Combinations of u, v, w with $uvw =$ cab.**

(a) The string **cab** has length 3. We must split it as uvw with $u, v, w \in \Sigma^*$. The split points are $i = |u|$, $j = |u| + |v|$ with $0 \le i \le j \le 3$. There are $\binom{n+2}{2} = \binom{5}{2} = 10$ ways for length 3:
(u, v, w): $(\lambda, \lambda, \mathbf{cab})$, $(\lambda, \mathbf{c}, \mathbf{ab})$, $(\lambda, \mathbf{ca}, \mathbf{b})$, $(\lambda, \mathbf{cab}, \lambda)$, $(\mathbf{c}, \lambda, \mathbf{ab})$, $(\mathbf{c}, \mathbf{a}, \mathbf{b})$, $(\mathbf{c}, \mathbf{ab}, \lambda)$, $(\mathbf{ca}, \lambda, \mathbf{b})$, $(\mathbf{ca}, \mathbf{b}, \lambda)$, $(\mathbf{cab}, \lambda, \lambda)$.

(b) For a string of length n: the number of ways to choose split points $0 \le i \le j \le n$ is $\binom{n+2}{2} = \frac{(n+1)(n+2)}{2}$.

1.16. **DFA for $\Xi = \{x \mid x$ contains ≥ 2 consecutive bs and does not contain aa$\}$.**

The DFA tracks two conditions simultaneously: whether **aa** has ever appeared, and whether **bb** has ever appeared. Once **aa** appears the string is rejected regardless; once **bb** appears we continue watching for **aa**.

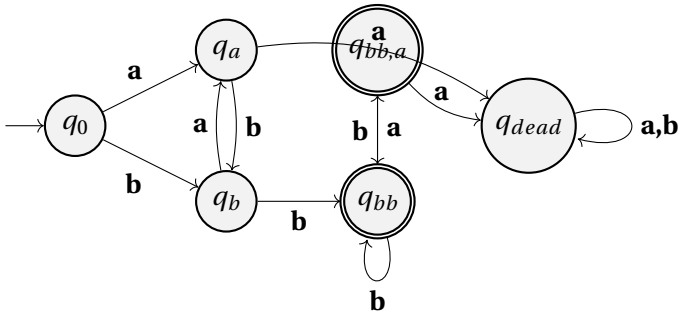
States (6 total):

- q_0: start; no **bb**, no **aa**, last char none.
- q_a: no **bb**, no **aa**, last char = **a**.
- q_b: no **bb**, no **aa**, last char = **b**.
- q_{bb}: seen **bb**, no **aa**, last char = **b**.
- $q_{bb,a}$: seen **bb**, no **aa**, last char = **a**.
- q_{dead}: seen **aa** (trap).

$F = \{q_{bb}, q_{bb,a}\}$.

Transitions:

	a	**b**
q_0	q_a	q_b
q_a	q_{dead}	q_b
q_b	q_a	q_{bb}
q_{bb}	$q_{bb,a}$	q_{bb}
$q_{bb,a}$	q_{dead}	q_{bb}
q_{dead}	q_{dead}	q_{dead}



1.17. **Modify Figure 1.11 to reject DO and DATA.**

Keep the structure of Example 1.9, but split the ordinary identifier states so that the machine can remember the exceptional prefixes ***D***, ***DO***, ***DA***, ***DAT***, and ***DATA*** until it is clear whether the identifier is exactly one of the reserved words.

One convenient design uses the states

$$s_0,\ d_1,\ do_2,\ da_2,\ dat_3,\ data_4,\ g_1,\ g_2,\ g_3,\ g_4,\ g_5,\ g_6,\ s_7,$$

where g_i means "a legal identifier of length i whose prefix is no longer one of the exceptional words," and s_7 is the dead state. Embedded blanks behave exactly as in Example 1.9: they loop in every nondead state.

The key transitions are:

- From s_0, the letter ***D*** goes to d_1, every other letter goes to g_1, digits and characters from Γ go to s_7, and $\diamond$ loops to s_0.
- From d_1, the letter ***O*** goes to do_2, ***A*** goes to da_2, and every other letter or digit goes to g_2.
- From do_2, any further letter or digit goes to g_3; if the identifier ends while the machine is still in do_2, the word is rejected.
- From da_2, the letter ***T*** goes to dat_3; every other letter or digit goes to g_3.
- From dat_3, the letter ***A*** goes to $data_4$; every other letter or digit goes to g_4.
- From $data_4$, any further letter or digit goes to g_5; if the identifier ends while the machine is still in $data_4$, the word is rejected.
- The states $g_1, \ldots, g_6$ behave exactly like the length-counting states $s_1, \ldots, s_6$ of Example 1.9.

The final states are

$$\{d_1, da_2, dat_3, g_1, g_2, g_3, g_4, g_5, g_6\}.$$

So identifiers such as ***D***, ***DA***, ***DAT***, ***DOG***, and ***DATA*1** are accepted, while exactly ***DO*** and exactly ***DATA*** are rejected.

1.18. **Modify machine for real-number constants (extended BNF).**

The new BNF allows leading decimal points like `.5` and `+.5`. Modify the DFA from Figure 1.13 to add transitions that accept the input immediately after a sign symbol or at the start by going to a decimal-point state, from which digits are read as the fractional part.

(a) A concrete modified DFA is obtained by using the states

$$s_0 = \text{start}, \quad s_1 = \text{sign}, \quad s_2 = \text{integer part}, \quad s_3 = \text{dot after integer},$$

$$s_4 = \text{E just read}, \quad s_5 = \text{sign after E}, \quad s_6 = \text{exponent digits},$$

$$s_8 = \text{fractional digits}, \quad s_9 = \text{leading dot before any digit}, \quad s_7 = \text{dead}.$$

The accepting states are $\{s_2, s_3, s_8, s_6\}$. The important transitions are:

$$\begin{array}{llll}
\delta(s_0, \text{digit}) = s_2, & \delta(s_0, \mathbf{+}) = s_1, & \delta(s_0, \mathbf{-}) = s_1, & \delta(s_0, \mathbf{.}) = s_9, \\
\delta(s_1, \text{digit}) = s_2, & \delta(s_1, \mathbf{.}) = s_9, & & \\
\delta(s_2, \text{digit}) = s_2, & \delta(s_2, \mathbf{.}) = s_3, & & \\
\delta(s_3, \text{digit}) = s_8, & & & \\
\delta(s_8, \text{digit}) = s_8, & \delta(s_8, \boldsymbol{E}) = s_4, & & \\
\delta(s_9, \text{digit}) = s_8, & & & \\
\delta(s_4, \text{digit}) = s_6, & \delta(s_4, \mathbf{+}) = s_5, & \delta(s_4, \mathbf{-}) = s_5, & \\
\delta(s_5, \text{digit}) = s_6, & & & \\
\delta(s_6, \text{digit}) = s_6. & & &
\end{array}$$

All unspecified transitions go to s_7, and s_7 loops to itself. This accepts strings such as `.5`, `+.5`, and `-.5E7`, while still rejecting strings such as `8.E-10`, because there must be at least one digit after the decimal point if an exponent is present.

(b) Here is a Pascal implementation of that DFA.

```
program RealConst(input, output);
type
   State = (start, sign, intpart, dotafterint, expmark,
            expsign, expdigits, dead, fracpart, leadingdot);
var
   s : State;
   ch : char;
function Accepting(t : State) : boolean;
begin
   Accepting := t in [intpart, dotafterint, fracpart, expdigits];
end;
begin
   s := start;
   while not eoln(input) do
      begin
         ch := input^;
         case s of
            start:
               if ch in ['0'..'9'] then s := intpart
               else if ch in ['+','-'] then s := sign
               else if ch = '.' then s := leadingdot
               else s := dead;
            sign:
               if ch in ['0'..'9'] then s := intpart
               else if ch = '.' then s := leadingdot
               else s := dead;
            intpart:
               if ch in ['0'..'9'] then s := intpart
```

```
                  else if ch = '.' then s := dotafterint
                  else s := dead;
               dotafterint:
                  if ch in ['0'..'9'] then s := fracpart
                  else s := dead;
               leadingdot:
                  if ch in ['0'..'9'] then s := fracpart
                  else s := dead;
               fracpart:
                  if ch in ['0'..'9'] then s := fracpart
                  else if ch = 'E' then s := expmark
                  else s := dead;
               expmark:
                  if ch in ['0'..'9'] then s := expdigits
                  else if ch in ['+','-'] then s := expsign
                  else s := dead;
               expsign:
                  if ch in ['0'..'9'] then s := expdigits
                  else s := dead;
               expdigits:
                  if ch in ['0'..'9'] then s := expdigits
                  else s := dead;
               dead:
                  s := dead
            end;
            get(input)
         end;
      if Accepting(s) then
         writeln(output, 'Accepted')
      else
         writeln(output, 'Rejected')
   end.
```

1.19. **Show Definition 1.11(i) is implied by (ii) and (iii).**

In Definition 1.11, part (i) is the statement

$$(\forall s \in S)(\forall \boldsymbol{a} \in \Sigma)\ \overline{\delta}(s, \boldsymbol{a}) = \delta(s, \boldsymbol{a}).$$

Assume only parts (ii) and (iii). Then for any state s and any letter $\boldsymbol{a}$,

$$\overline{\delta}(s, \boldsymbol{a}) = \overline{\delta}(s, \boldsymbol{a}\lambda)$$

because $\boldsymbol{a}\lambda = \boldsymbol{a}$. Now apply part (iii) with $x = \lambda$:

$$\overline{\delta}(s, \boldsymbol{a}\lambda) = \overline{\delta}(\delta(s, \boldsymbol{a}), \lambda).$$

Finally, by part (ii),

$$\overline{\delta}(\delta(s, \boldsymbol{a}), \lambda) = \delta(s, \boldsymbol{a}).$$

Therefore $\overline{\delta}(s, \boldsymbol{a}) = \delta(s, \boldsymbol{a})$ for every $s \in S$ and every $\boldsymbol{a} \in \Sigma$, which is exactly part (i).

1.20. **Develop a succinct description of the transition function from Example 1.9.**

Let Γ be the set defined in Example 1.9, so that the alphabet is partitioned into letters, digits, blanks $\diamond$, and the remaining symbols Γ. Then the transition function can be described uniformly as follows:

$$\begin{aligned}
\delta(s_0, \text{letter}) &= s_1, & \delta(s_0, \text{digit}) &= s_7, \quad \delta(s_0, \diamond) = s_0, \quad \delta(s_0, \gamma) = s_7 \quad (\gamma \in \Gamma),\\
\delta(s_i, \text{letter or digit}) &= s_{i+1} \qquad (1 \le i \le 5),\\
\delta(s_i, \diamond) &= s_i \qquad (1 \le i \le 6),\\
\delta(s_i, \gamma) &= s_7 \qquad (1 \le i \le 6,\ \gamma \in \Gamma),\\
\delta(s_6, \text{letter or digit}) &= s_7,\\
\delta(s_7, \sigma) &= s_7 \qquad (\sigma \in \Sigma).
\end{aligned}$$

So s_i means "a legal identifier of length i has been seen," blanks do not change the current length, and any illegal character or seventh nonblank character forces the machine into the dead state s_7.

1.21. **Finite, cofinite, neither over $\{\mathbf{a}, \mathbf{b}\}^*$.**

(a) Finite: $\{\mathbf{a}, \mathbf{ab}\}$.

(b) Cofinite: $\{\mathbf{a}, \mathbf{b}\}^* \setminus \{\mathbf{a}\}$ (complement is $\{\mathbf{a}\}$, finite).

(c) Neither: $\{\mathbf{a}^n \mid n \text{ even}\}$. Infinite, and its complement $\{\mathbf{a}^n \mid n \text{ odd}\} \cup \{x \mid x \text{ contains } \mathbf{b}\}$ is also infinite.

1.22. **DFA of Figure 1.27.**

(a) The quintuple is

$$\langle \{\boldsymbol{a}, \boldsymbol{b}\}, \{q_0, q_1, q_2, q_3, q_4\}, q_0, \delta, \{q_2, q_3\} \rangle,$$

where

$$\begin{aligned}
\delta(q_0, \boldsymbol{a}) &= q_1, & \delta(q_0, \boldsymbol{b}) &= q_0,\\
\delta(q_1, \boldsymbol{a}) &= q_2, & \delta(q_1, \boldsymbol{b}) &= q_1,\\
\delta(q_2, \boldsymbol{a}) &= q_2, & \delta(q_2, \boldsymbol{b}) &= q_4,\\
\delta(q_3, \boldsymbol{a}) &= q_3, & \delta(q_3, \boldsymbol{b}) &= q_2,\\
\delta(q_4, \boldsymbol{a}) &= q_4, & \delta(q_4, \boldsymbol{b}) &= q_3.
\end{aligned}$$

(b) The machine first waits for two occurrences of $\boldsymbol{a}$. The state q_2 is reached as soon as the second $\boldsymbol{a}$ is seen. After that, only the number of $\boldsymbol{b}$s modulo 3 matters:

$$q_2 \equiv 0, \qquad q_4 \equiv 1, \qquad q_3 \equiv 2 \pmod 3.$$

Since q_2 and q_3 are final and q_4 is not, the language is

$$\{\boldsymbol{b}^i \boldsymbol{a} \boldsymbol{b}^j \boldsymbol{a} x \mid i, j \ge 0,\ x \in \{\boldsymbol{a}, \boldsymbol{b}\}^*,\ |x|_{\boldsymbol{b}} \equiv 0 \text{ or } 2 \pmod 3\}.$$

Equivalently: the string must contain at least two $\boldsymbol{a}$s, and after the second $\boldsymbol{a}$ the number of $\boldsymbol{b}$s must not be congruent to 1 (mod 3).

1.23. **Names of everyone in China – FAD?**

The set is finite (China has a finite population), hence regular (every finite language is FAD). Yes.

1.24. **Legal infix arithmetic expressions – FAD?**

Yes. Without parentheses, normal precedence rules affect evaluation, not syntactic well-formedness. A DFA only has to remember whether the next symbol should be an operand or an operator. The symbol **−** is special because it can serve as unary negation while the machine is expecting an operand, and as subtraction while the machine is expecting an operator.

A suitable DFA has three states:

- q_E: expecting an operand,
- q_O: expecting an operator or the end of the string,
- q_D: dead state.

The start state is q_E, the only final state is q_O, and the transitions are

	A	***B***	**+**	**−**	**×**	**/**
q_E	q_O	q_O	q_D	q_E	q_D	q_D
q_O	q_D	q_D	q_E	q_E	q_E	q_E
q_D	q_D	q_D	q_D	q_D	q_D	q_D

Thus ***A***, **−*A***, ***A*−*B***, and ***A*×−*B*** are accepted, while strings such as **+*A***, ***AB***, and ***A/*** are rejected.

1.25. **Properties of a DFA M.**

(a) $\lambda \in L(M)$ iff $s_0 \in F$.

(b) $L(M) = \Sigma^*$ iff every *reachable* state is a final state. (Unreachable non-final states do not affect the language: a DFA with one reachable final start state and one unreachable non-final state still accepts Σ^*.)

(c) $L(M) = \emptyset$ iff no final state is reachable from s_0.

1.26. **DFAs for substring recognition.**

(a) For "***abc*** is a substring," use the states q_0, q_1, q_2, q_3, where q_i means that the longest suffix seen so far that is also a prefix of ***abc*** has length i. Let q_3 be an accepting sink. The transition table is

	a	***b***	***c***
q_0	q_1	q_0	q_0
q_1	q_1	q_2	q_0
q_2	q_1	q_0	q_3
q_3	q_3	q_3	q_3

(b) For "***acaba*** is a substring," use the states $p_0, p_1, p_2, p_3, p_4, p_5$, where p_5 is an accepting sink. The transition table is

	a	***b***	***c***
p_0	p_1	p_0	p_0
p_1	p_1	p_0	p_2
p_2	p_3	p_0	p_0
p_3	p_1	p_4	p_2
p_4	p_5	p_0	p_0
p_5	p_5	p_5	p_5

Again, p_i records the longest suffix that is also a prefix of ***acaba***.

1.27. **Add pennies to vending machine.**

The original machine tracks deposited amounts in multiples of 5¢, needing 7 states (0, 5, 10, 15, 20, 25, 30¢). Adding pennies requires tracking every cent from 0 to 30, so the minimum number of states is **31**: states $q_0, q_1, \ldots, q_{30}$ representing $0, 1, \ldots, 30$ cents deposited. State q_{30} is the accepting (dispensing) state; from q_{30} all additional input is ignored (or loops back). Each extra penny input on state q_i (for $i < 30$) transitions to q_{i+1}.

1.28. **The DFA (t_0, t_1) of Exercise 1.29.**

(a) The language: $L = \{x \in \{\mathbf{a}, \mathbf{b}\}^* \mid |x|_{\mathbf{b}} \text{ is odd}\}$, i.e., strings with an odd number of **b**s. (t_0 = even #b, t_1 = odd #b, $F = \{t_1\}$.)

(b) *Proof*: By induction on $|x| = n$. Let $P(n)$: $(\forall x \in \Sigma^n)(\overline{\delta}(t_0, x) = t_0 \Leftrightarrow |x|_{\mathbf{b}} \text{ even}) \wedge (\overline{\delta}(t_0, x) = t_1 \Leftrightarrow |x|_{\mathbf{b}} \text{ odd})$.

Base ($n = 0$): $\overline{\delta}(t_0, \lambda) = t_0$, $|\lambda|_{\mathbf{b}} = 0$ (even). ✓

Step: Let $x = ya$, $|y| = n$. Case $a = \mathbf{a}$: $\overline{\delta}(t_0, ya) = \delta(\overline{\delta}(t_0, y), \mathbf{a})$. Since $\delta(t_0, \mathbf{a}) = t_0$ and $\delta(t_1, \mathbf{a}) = t_1$, the state after ya equals the state after y, and $|ya|_{\mathbf{b}} = |y|_{\mathbf{b}}$. Case $a = \mathbf{b}$: δ flips the state; $|ya|_{\mathbf{b}} = |y|_{\mathbf{b}} + 1$ changes parity. Both cases follow from the IH. □

1.29. **Vending machine: pennies, nickels, dimes, quarters, 10¢ bars.**

(a) States $q_0, q_1, \ldots, q_{10}$ where $q_i = i$ cents deposited so far, q_{10} = accepting (dispensing) sink. $\delta(q_i, p) = q_{\min(i+1,10)}$, $\delta(q_i, n) = q_{\min(i+5,10)}$, $\delta(q_i, d) = q_{\min(i+10,10)}$, $\delta(q_i, q) = q_{\min(i+25,10)} \to q_{10}$. $F = \{q_{10}\}$.

(b) 11 states.

(c) The minimum is 11 states (must track exact amount from 0 to 10 cents).

1.30. **Vending machine: nickels, dimes, quarters, 10¢ bars.**

(a) States q_0, q_5, q_{10} (representing 0, 5, 10+ cents). $\delta(q_0, n) = q_5$, $\delta(q_0, d) = \delta(q_5, n) = q_{10}$, $\delta(q_0, q) = \delta(q_5, d) = \delta(q_5, q) = q_{10}$, $\delta(q_{10}, -) = q_{10}$. $F = \{q_{10}\}$.

(b) Minimum: 3 states (0, 5, ≥10 cents).

(c) Encode the states by

$$q_0 = \mathbf{00}, \qquad q_5 = \mathbf{01}, \qquad q_{10} = \mathbf{10},$$

with **11** unused. Encode the input alphabet {<*EOS*>, $\boldsymbol{n}, \boldsymbol{d}, \boldsymbol{q}$} by

$$<EOS> = \mathbf{00}, \qquad \boldsymbol{n} = \mathbf{01}, \qquad \boldsymbol{d} = \mathbf{10}, \qquad \boldsymbol{q} = \mathbf{11}.$$

Let the state bits be t_1, t_2, and the input bits be $\boldsymbol{a}_1, \boldsymbol{a}_2$. One corresponding circuit uses two D flip-flops with inputs

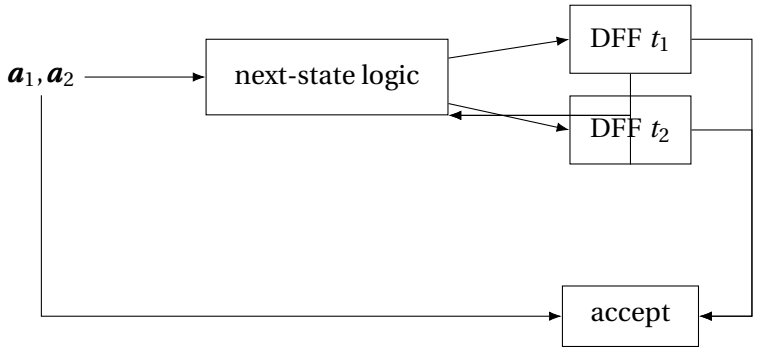
$$t_1' = (\neg \boldsymbol{a}_1 \wedge \neg \boldsymbol{a}_2 \wedge t_1) \vee (\neg \boldsymbol{a}_1 \wedge \boldsymbol{a}_2 \wedge (t_1 \vee t_2)) \vee \boldsymbol{a}_1,$$

$$t_2' = \neg \boldsymbol{a}_1 \wedge \big((\neg \boldsymbol{a}_2 \wedge t_2) \vee (\boldsymbol{a}_2 \wedge \neg t_1 \wedge \neg t_2)\big).$$

The accept light should turn on exactly when <EOS> is scanned in state q_{10}, so

$$\textbf{accept} = t_1 \wedge \neg t_2 \wedge \neg \boldsymbol{a}_1 \wedge \neg \boldsymbol{a}_2.$$

One corresponding circuit diagram is:



1.31. **Circuit for Exercise 1.29 DFA.**

(a) Use one state bit t_1, with $t_1 = \mathbf{0}$ for state t_0 and $t_1 = \mathbf{1}$ for state t_1. Encode

$$<EOS> = \mathbf{00}, \qquad \boldsymbol{a} = \mathbf{01}, \qquad \boldsymbol{b} = \mathbf{10}, \qquad <SOS> = \mathbf{11}.$$

Then <EOS> and $\boldsymbol{a}$ leave the parity unchanged, $\boldsymbol{b}$ toggles it, and <SOS> resets to the start state. Thus one D flip-flop with input

$$t_1' = (t_1 \wedge \neg \boldsymbol{a}_1) \vee (\neg t_1 \wedge \boldsymbol{a}_1 \wedge \neg \boldsymbol{a}_2)$$

implements the next-state logic. The accept circuitry is

$$\textbf{accept} = t_1 \wedge \neg \boldsymbol{a}_1 \wedge \neg \boldsymbol{a}_2,$$

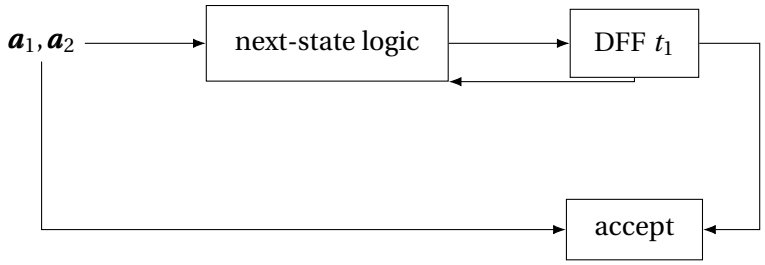
since the string is accepted exactly when <EOS> is scanned in the odd-$\boldsymbol{b}$ state.

(b) Without <SOS> and <EOS>, only one input bit is needed; let $x = \mathbf{0}$ represent $\boldsymbol{a}$ and $x = \mathbf{1}$ represent $\boldsymbol{b}$. The next state is just

$$t_1' = t_1 \oplus x.$$

So the circuit is a single D flip-flop whose input is the exclusive-or of the current state bit and the input bit.

A corresponding circuit diagram for the version with <SOS> and <EOS> is:



1.32. **Circuit for Exercise 1.7 DFA.**

It is convenient to implement the language of Exercise 1.7 by keeping the parity bits from Example 1.10 together with one extra bit u that records whether at least one input symbol has been read. Let

$$p = \text{parity of } |x|_{\boldsymbol{a}}, \qquad q = \text{parity of } |x|_{\boldsymbol{b}}, \qquad u = \text{“have seen input”}.$$

Then the machine accepts exactly when $p = q = \mathbf{0}$ and $u = \mathbf{1}$.

(a) With <SOS> and <EOS>, use the encoding

$$<\!EOS\!> = \mathbf{00}, \qquad \boldsymbol{a} = \mathbf{01}, \qquad \boldsymbol{b} = \mathbf{10}, \qquad <\!SOS\!> = \mathbf{11}.$$

One corresponding circuit uses three D flip-flops with inputs

$$p' = (\neg \boldsymbol{a}_1 \wedge \neg \boldsymbol{a}_2 \wedge p) \vee (\neg \boldsymbol{a}_1 \wedge \boldsymbol{a}_2 \wedge \neg p) \vee (\boldsymbol{a}_1 \wedge \neg \boldsymbol{a}_2 \wedge p),$$

$$q' = (\neg \boldsymbol{a}_1 \wedge \neg \boldsymbol{a}_2 \wedge q) \vee (\neg \boldsymbol{a}_1 \wedge \boldsymbol{a}_2 \wedge q) \vee (\boldsymbol{a}_1 \wedge \neg \boldsymbol{a}_2 \wedge \neg q),$$

$$u' = (\neg \boldsymbol{a}_1 \wedge \neg \boldsymbol{a}_2 \wedge u) \vee (\neg \boldsymbol{a}_1 \wedge \boldsymbol{a}_2) \vee (\boldsymbol{a}_1 \wedge \neg \boldsymbol{a}_2).$$

The accept light is

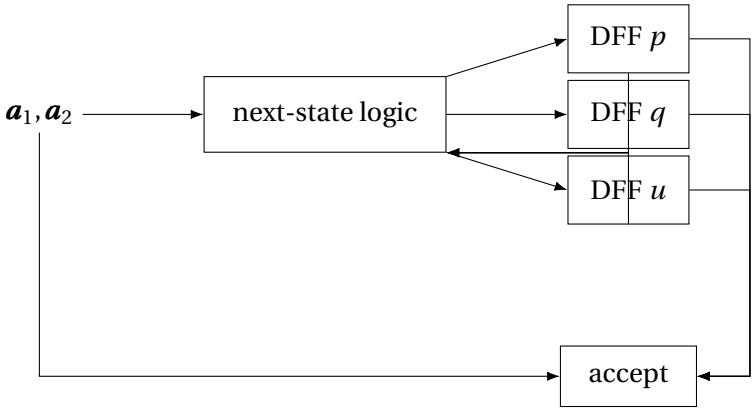
$$\textbf{accept} = \neg p \wedge \neg q \wedge u \wedge \neg \boldsymbol{a}_1 \wedge \neg \boldsymbol{a}_2.$$

(b) Without <SOS> and <EOS>, use one input bit x with $x = \mathbf{0}$ for $\boldsymbol{a}$ and $x = \mathbf{1}$ for $\boldsymbol{b}$. Then

$$p' = p \oplus \neg x, \qquad q' = q \oplus x, \qquad u' = \mathbf{1}.$$

After the last clock pulse, the machine accepts iff $\neg p \wedge \neg q \wedge u$.

One corresponding circuit diagram for the <SOS>/<EOS> version is:



1.33. **Modify Example 1.12 to correctly handle <SOS>.**

Use the same one-bit state encoding as Example 1.12: $s_0 = \mathbf{0}$ and $s_1 = \mathbf{1}$. Keep the book's input convention and encode

$$<EOS> = \mathbf{00}, \qquad \boldsymbol{b} = \mathbf{01}, \qquad \boldsymbol{c} = \mathbf{10}, \qquad <SOS> = \mathbf{11}.$$

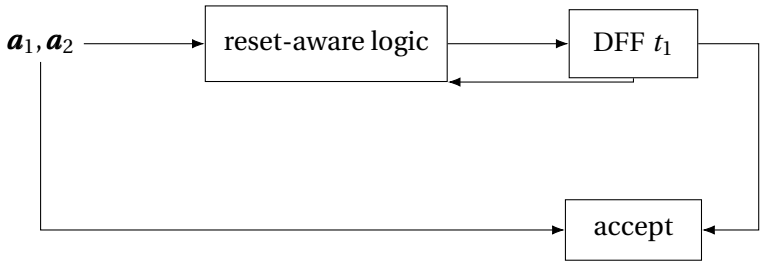
The only extra requirement is that scanning <SOS> from either state must send the machine to s_0. The resulting next-state function is

$$t_1' = (\neg \boldsymbol{a}_1 \wedge \neg \boldsymbol{a}_2 \wedge t_1) \vee (\boldsymbol{a}_1 \wedge \neg \boldsymbol{a}_2 \wedge \neg t_1).$$

This agrees with Example 1.12 on $\boldsymbol{b}$, $\boldsymbol{c}$, and <EOS>, and forces a reset to the start state on <SOS>. The accept circuitry is unchanged:

$$\mathbf{accept} = t_1 \wedge \neg \boldsymbol{a}_1 \wedge \neg \boldsymbol{a}_2.$$

So the corrected circuit is exactly the Example 1.12 circuit with the input combination **11** wired to force the next state to **0**.



1.34. **Circuit for Example 1.6 machine.**

(a) Encode the states by

$$s_0 = \mathbf{00}, \qquad s_1 = \mathbf{01}, \qquad s_2 = \mathbf{10},$$

with **11** unused. Use the standard input encoding

$$<EOS> = \mathbf{00}, \qquad \boldsymbol{a} = \mathbf{01}, \qquad \boldsymbol{b} = \mathbf{10}, \qquad <SOS> = \mathbf{11}.$$

If t_1, t_2 are the state bits, then a corresponding pair of D inputs is

$$t_1' = (\neg \boldsymbol{a}_1 \wedge \neg \boldsymbol{a}_2 \wedge t_1) \vee (\boldsymbol{a}_1 \wedge \neg \boldsymbol{a}_2 \wedge \neg t_1),$$

$$t_2' = (\neg \boldsymbol{a}_1 \wedge \neg \boldsymbol{a}_2 \wedge t_2) \vee (\neg \boldsymbol{a}_1 \wedge \boldsymbol{a}_2).$$

The machine accepts in state s_1, so the accept circuitry is

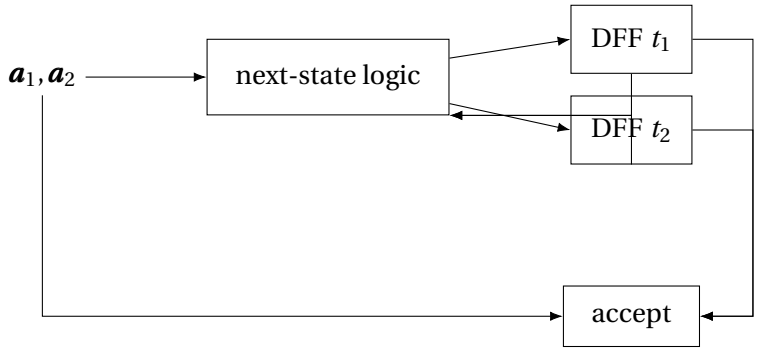
$$\mathbf{accept} = \neg t_1 \wedge t_2 \wedge \neg \boldsymbol{a}_1 \wedge \neg \boldsymbol{a}_2.$$

(b) Without <SOS> and <EOS>, only one input bit x is needed, with $x = \mathbf{0}$ for $\boldsymbol{a}$ and $x = \mathbf{1}$ for $\boldsymbol{b}$. Then

$$t_1' = x \wedge \neg t_1, \qquad t_2' = \neg x.$$

Thus two D flip-flops together with the above combinational logic implement the machine.

One corresponding circuit diagram for the <SOS>/<EOS> version is:



1.35. **Circuit for Example 1.10 machine.**

Let p and q be the parity bits for $|x|_{\boldsymbol{a}}$ and $|x|_{\boldsymbol{b}}$, respectively. Then the four states of Example 1.10 are exactly the four combinations of (p, q).

(a) With <SOS> and <EOS>, use

$$<\!EOS\!> = \mathbf{00}, \qquad \boldsymbol{a} = \mathbf{01}, \qquad \boldsymbol{b} = \mathbf{10}, \qquad <\!SOS\!> = \mathbf{11}.$$

Then <EOS> leaves the state unchanged, $\boldsymbol{a}$ flips p, $\boldsymbol{b}$ flips q, and <SOS> resets both bits to $\mathbf{0}$. One corresponding circuit uses two D flip-flops with

$$p' = (\neg \boldsymbol{a}_1 \wedge \neg \boldsymbol{a}_2 \wedge p) \vee (\neg \boldsymbol{a}_1 \wedge \boldsymbol{a}_2 \wedge \neg p) \vee (\boldsymbol{a}_1 \wedge \neg \boldsymbol{a}_2 \wedge p),$$

$$q' = (\neg \boldsymbol{a}_1 \wedge \neg \boldsymbol{a}_2 \wedge q) \vee (\neg \boldsymbol{a}_1 \wedge \boldsymbol{a}_2 \wedge q) \vee (\boldsymbol{a}_1 \wedge \neg \boldsymbol{a}_2 \wedge \neg q).$$

The accept light turns on exactly at <EOS> when $p = q = \mathbf{0}$:

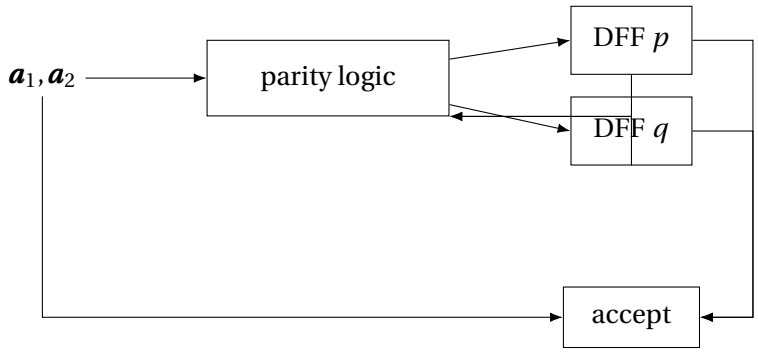
$$\textbf{accept} = \neg p \wedge \neg q \wedge \neg \boldsymbol{a}_1 \wedge \neg \boldsymbol{a}_2.$$

(b) Without <SOS> and <EOS>, let $x = \mathbf{0}$ encode $\boldsymbol{a}$ and $x = \mathbf{1}$ encode $\boldsymbol{b}$. Then

$$p' = p \oplus \neg x, \qquad q' = q \oplus x.$$

So the circuit is just two D flip-flops fed by these two exclusive-or subcircuits.

One corresponding circuit diagram for the <SOS>/<EOS> version is:



1.36. **Circuit for Example 1.14 (vending machine), with EOS.**

Encode the seven states of Example 1.14 by the binary codes for the numbers 0 through 6:

$$s_0 = \mathbf{000},\ s_5 = \mathbf{001},\ s_{10} = \mathbf{010},\ s_{15} = \mathbf{011},\ s_{20} = \mathbf{100},\ s_{25} = \mathbf{101},\ s_{30} = \mathbf{110},$$

leaving **111** unused. Encode the input alphabet by

$$<EOS> = \mathbf{00}, \qquad \boldsymbol{n} = \mathbf{01}, \qquad \boldsymbol{d} = \mathbf{10}, \qquad \boldsymbol{q} = \mathbf{11}.$$

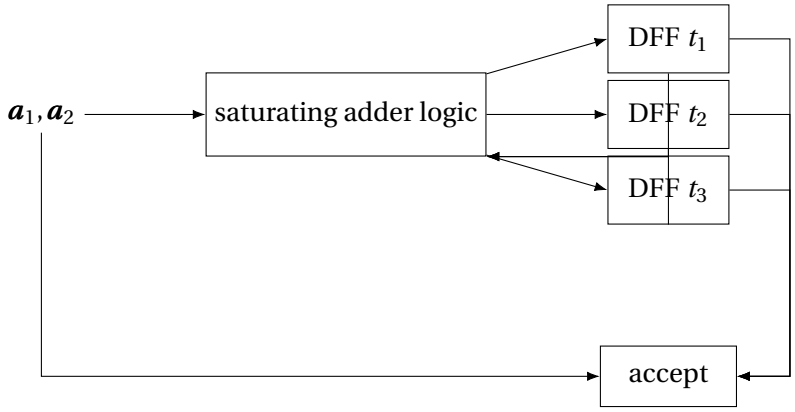
One clean circuit design uses three D flip-flops together with a combinational subcircuit that does the following:

- decode the current state bits as the number $i \in \{0, 1, \ldots, 6\}$,
- decode the input bits as the increment c, where $c = 0$ for <EOS>, $c = 1$ for $\boldsymbol{n}$, $c = 2$ for $\boldsymbol{d}$, and $c = 5$ for $\boldsymbol{q}$,
- compute $j = \min\{6, i + c\}$,
- write the three-bit encoding of j into the D inputs.

The accept light is on exactly when <EOS> is scanned in state s_{30}:

$$\textbf{accept} = t_1 \wedge t_2 \wedge \neg t_3 \wedge \neg \boldsymbol{a}_1 \wedge \neg \boldsymbol{a}_2,$$

if $t_1 t_2 t_3$ is the chosen encoding of the current state and $s_{30} = \mathbf{110}$.



1.37. **Circuit for Example 1.16, with SOS and EOS.**

The protocol machine has four states, so two state bits suffice. For example, use

$$A = \mathbf{00}, \qquad R = \mathbf{01}, \qquad RF = \mathbf{10}, \qquad RD = \mathbf{11}.$$

The input alphabet is $\{B, D, Z, H, S, <EOS>, <SOS>\}$, so use three input bits and encode

$$<EOS> = \mathbf{000}, \quad B = \mathbf{001}, \quad D = \mathbf{010}, \quad H = \mathbf{011},$$

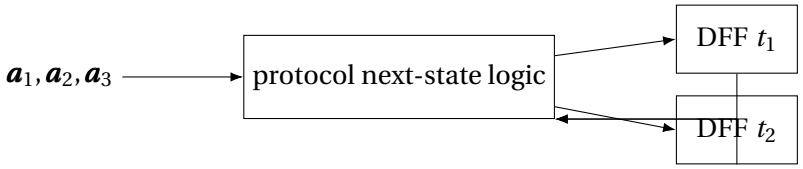
$$S = \mathbf{100}, \quad Z = \mathbf{101}, \quad <SOS> = \mathbf{111},$$

leaving **110** unused.

The next-state combinational block should implement exactly the protocol described in Example 1.16:

- scanning <SOS> from any state sends the machine to R,
- scanning <EOS> leaves the current state unchanged,
- from R, only S is legal and sends the machine to RF,
- from RF, only H is legal and sends the machine to RD,
- from RD, D stays in RD, Z goes to RF, and B goes to R,
- every unexpected packet sends the machine to A,
- from A, every packet except <SOS> leaves the machine in A.

Thus the circuit consists of two D flip-flops whose inputs are the Boolean functions defined by this complete transition table. Since Example 1.16 is being used as a control automaton rather than as a language acceptor, there is no separate accept light; the outputs of interest are simply the state bits themselves.



1.38. **DFA for $L = \{x \in \{\mathbf{a},\mathbf{b},\mathbf{c}\}^* \mid |x|_{\mathbf{b}} = 2\}$.**

(a) Three states q_0, q_1, q_2 (counting #b seen so far, capped at 2), plus a dead state q_3. $F = \{q_2\}$. $\delta(q_i, \mathbf{a}) = \delta(q_i, \mathbf{c}) = q_i$; $\delta(q_0, \mathbf{b}) = q_1$; $\delta(q_1, \mathbf{b}) = q_2$; $\delta(q_2, \mathbf{b}) = q_3$; $\delta(q_3, -) = q_3$.

(b) Quintuple: $\Sigma = \{\mathbf{a},\mathbf{b},\mathbf{c}\}$, $S = \{q_0, q_1, q_2, q_3\}$, $s_0 = q_0$, $F = \{q_2\}$, δ as above.

1.39. **DFA for every b eventually followed by c.**

Two states track whether there is an outstanding **b** with no subsequent **c**: q_0 (no unresolved **b**), q_b (at least one unresolved **b**). $F = \{q_0\}$.

$$\delta(q_0, \mathbf{a}) = q_0, \quad \delta(q_0, \mathbf{b}) = q_b, \quad \delta(q_0, \mathbf{c}) = q_0,$$

$$\delta(q_b, \mathbf{a}) = q_b, \quad \delta(q_b, \mathbf{b}) = q_b, \quad \delta(q_b, \mathbf{c}) = q_0.$$

Reading **c** in q_b resolves all pending **b**s; **a** or **b** keeps them pending. Accept iff input ends in q_0.

1.40. **DFA for no consecutive as and no consecutive bs.**

(a) States: q_0 (start/even alternation), q_a (last char **a**), q_b (last char **b**), q_{dead}. $F = \{q_0, q_a, q_b\}$. $\delta(q_0, \mathbf{a}) = q_a$, $\delta(q_0, \mathbf{b}) = q_b$, $\delta(q_a, \mathbf{a}) = q_{dead}$, $\delta(q_a, \mathbf{b}) = q_b$, $\delta(q_b, \mathbf{b}) = q_{dead}$, $\delta(q_b, \mathbf{a}) = q_a$, $\delta(q_{dead}, -) = q_{dead}$.

(b) Quintuple: $\Sigma = \{\mathbf{a},\mathbf{b}\}$, $S = \{q_0, q_a, q_b, q_{dead}\}$, $s_0 = q_0$, $F = \{q_0, q_a, q_b\}$, δ as above.

1.41. **DFA for $L = \{x \in \{\boldsymbol{a},\boldsymbol{b},\boldsymbol{c}\}^* \mid |x|_{\boldsymbol{a}} \equiv 0 \bmod 3\}$.**

(a) Use three states r_0, r_1, r_2, where r_i means that the number of $\boldsymbol{a}$s seen so far is congruent to i (mod 3). The start state is r_0, the only final state is r_0, and

$$\delta(r_i, \boldsymbol{a}) = r_{i+1 \bmod 3}, \qquad \delta(r_i, \boldsymbol{b}) = r_i, \qquad \delta(r_i, \boldsymbol{c}) = r_i.$$

(b) The quintuple is

$$\langle \{\boldsymbol{a}, \boldsymbol{b}, \boldsymbol{c}\}, \{r_0, r_1, r_2\}, r_0, \delta, \{r_0\} \rangle,$$

with δ defined exactly as above.

1.42. **DFAs for Pascal comments.**

(a) For "exactly one valid Pascal comment and no illegal comment fragments," a DFA can use the following states:

$$P_0, P_(, P_*, I_0, I_*, Q_0, Q_(, Q_*, D.$$

Here P means "before the unique comment," I means "inside the comment," Q means "after the unique comment," and D is dead. The one-character memory states $P_(, P_*, Q_(, Q_*$ remember a possible start or end delimiter.

The key transitions are:

- Before the comment:

$$\delta(P_0, \texttt{(}) = P_(, \quad \delta(P_0, \texttt{*}) = P_*,$$

and letters $\boldsymbol{a}, \boldsymbol{b}$ and the symbol **)** stay in P_0.

- A valid opening delimiter is recognized by

$$\delta(P_(, \texttt{*}) = I_0.$$

If the next symbol after **(** is anything else, the machine returns to the appropriate outside state.

- A stray closing delimiter outside a comment is illegal:

$$\delta(P_*, \texttt{)}) = D, \qquad \delta(Q_*, \texttt{)}) = D.$$

- Inside a comment, everything stays in I_0 except that * moves to I_*:

$$\delta(I_0, \texttt{*}) = I_*.$$

Then

$$\delta(I_*, \texttt{)}) = Q_0$$

closes the unique comment; on another * the machine stays in I_*, and on any other symbol it returns to I_0.

- After the unique comment, the outside-text behavior is the same as before, except that a second opening delimiter is forbidden:

$$\delta(Q_(, \texttt{*}) = D.$$

The accepting states are $Q_0, Q_(, Q_*$, because a complete input must contain exactly one closed comment and must not end inside a comment.

(b) For "zero or more valid unnested Pascal comments," the same idea works with only five essential states:

$$O_0, O_(, O_*, I_0, I_*, D.$$

Here O means "outside any comment." The transitions are exactly as above, except that after closing a comment

$$\delta(I_*,)) = O_0,$$

so another comment may begin later, and

$$\delta(O_(, *) = I_0$$

is always allowed. The accepting states are $O_0, O_(, O_*$, since the input is valid iff it ends outside a comment and never produces an illegal delimiter fragment.

1.43. **Postfix expressions.**

(a) Yes. A convenient DFA for the case "at most 2 operators" uses states

$$q_{d,m} \qquad (0 \le d \le 3,\ 0 \le m \le 2),$$

together with a dead state q_{dead}. Here d is the current stack surplus

$$d = (\#\text{operands read}) - (\#\text{operators read}),$$

and m is the number of operators used so far. The start state is $q_{0,0}$. On an operand $\boldsymbol{A}$ or $\boldsymbol{B}$,

$$\delta(q_{d,m}, \boldsymbol{A}) = \delta(q_{d,m}, \boldsymbol{B}) = \begin{cases} q_{d+1,m} & d < 3, \\ q_{dead} & d = 3, \end{cases}$$

and on any operator $\odot \in \{+, -, \times, /\}$,

$$\delta(q_{d,m}, \odot) = \begin{cases} q_{d-1,m+1} & d \ge 2 \text{ and } m < 2, \\ q_{dead} & \text{otherwise.} \end{cases}$$

The final states are the states $q_{1,0}, q_{1,1}, q_{1,2}$, because a complete postfix expression leaves exactly one value on the stack. This is the required machine.

(b) For four or fewer operators, use exactly the same construction with

$$q_{d,m} \qquad (0 \le d \le 5,\ 0 \le m \le 4),$$

again plus a dead state. The same transition rules apply, with the bounds 3 and 2 replaced by 5 and 4, respectively. The final states are all $q_{1,m}$ with $0 \le m \le 4$.

(c) For eight or fewer operators, use the same DFA schema once more, now with

$$q_{d,m} \qquad (0 \le d \le 9,\ 0 \le m \le 8),$$

plus a dead state. On operands the machine increments d; on operators it checks that $d \ge 2$ and then decrements d while incrementing m. The final states are all $q_{1,m}$ with $0 \le m \le 8$.

(d) The set of *all* postfix expressions is not FAD. Intuitively, a recognizer must keep track of how many partial results are currently on the stack, and that number is unbounded. More formally, the strings

$$\boldsymbol{A},\ \boldsymbol{AA},\ \boldsymbol{AAA},\ \ldots,\ \boldsymbol{A}^n,\ \ldots$$

have pairwise different futures: from $\boldsymbol{A}^n$, the continuation $+^{n-1}$ completes a legal postfix expression, but the same continuation does not complete a legal expression from $\boldsymbol{A}^m$ when $m \ne n$. Therefore infinitely many distinct states would be required.

1.44. **DFA for words that begin and end with different letters.**

(a) Use one start state s, and for each ordered pair $(u, v) \in \{\boldsymbol{a}, \boldsymbol{b}, \boldsymbol{c}\}^2$ a state q_{uv} meaning "the first letter seen was u and the current last letter is v." From the start state,

$$\delta(s, \boldsymbol{a}) = q_{aa}, \qquad \delta(s, \boldsymbol{b}) = q_{bb}, \qquad \delta(s, \boldsymbol{c}) = q_{cc}.$$

Thereafter,

$$\delta(q_{uv}, w) = q_{uw} \qquad \text{for all } u, v, w \in \{\boldsymbol{a}, \boldsymbol{b}, \boldsymbol{c}\}.$$

The accepting states are precisely the six states q_{uv} with $u \neq v$.

(b) The quintuple is

$$\langle \{\boldsymbol{a}, \boldsymbol{b}, \boldsymbol{c}\}, S, s, \delta, F \rangle,$$

where

$$S = \{s\} \cup \{q_{uv} \mid u, v \in \{\boldsymbol{a}, \boldsymbol{b}, \boldsymbol{c}\}\},$$

and

$$F = \{q_{uv} \mid u \neq v\}.$$

The transition function is exactly the one described in part (a).

1.45. **DFA that rejects words whose last two letters match.**

(a) A convenient DFA uses the seven states

$$s,\ a,\ b,\ c,\ aa,\ bb,\ cc.$$

The start state is s. The states a, b, c mean that the current last letter is $\boldsymbol{a}, \boldsymbol{b}, \boldsymbol{c}$ and the last two letters do *not* match; the states aa, bb, cc mean that the last two letters are equal. The transitions are:

$$\delta(s, \boldsymbol{a}) = a, \quad \delta(s, \boldsymbol{b}) = b, \quad \delta(s, \boldsymbol{c}) = c,$$

$$\delta(a, \boldsymbol{a}) = aa,\ \delta(a, \boldsymbol{b}) = b,\ \delta(a, \boldsymbol{c}) = c,$$

$$\delta(b, \boldsymbol{a}) = a,\ \delta(b, \boldsymbol{b}) = bb,\ \delta(b, \boldsymbol{c}) = c,$$

$$\delta(c, \boldsymbol{a}) = a,\ \delta(c, \boldsymbol{b}) = b,\ \delta(c, \boldsymbol{c}) = cc,$$

and from aa, bb, cc the same rule applies: a repeated last letter stays in the corresponding double-letter state, while a different last letter moves to $a, b,$ or c. The accepting states are s, a, b, c.

(b) The quintuple is

$$\langle \{\boldsymbol{a}, \boldsymbol{b}, \boldsymbol{c}\}, \{s, a, b, c, aa, bb, cc\}, s, \delta, \{s, a, b, c\} \rangle,$$

with δ defined as in part (a).

1.46. **DFA that rejects words whose first two letters match.**

(a) Use the six states

$$s,\ a,\ b,\ c,\ q_{acc},\ q_{rej}.$$

After the first input symbol, the machine remembers that symbol in one of the states a, b, c. On the second symbol it decides permanently:

$$\delta(a, \boldsymbol{a}) = q_{rej}, \quad \delta(a, \boldsymbol{b}) = \delta(a, \boldsymbol{c}) = q_{acc},$$

and similarly for b and c. Both q_{acc} and q_{rej} are sink states. The accepting states are s, a, b, c, q_{acc}, since words of length 0 or 1 do not have matching first two letters.

(b) The quintuple is

$$\langle \{\boldsymbol{a}, \boldsymbol{b}, \boldsymbol{c}\}, \{s, a, b, c, q_{acc}, q_{rej}\}, s, \delta, \{s, a, b, c, q_{acc}\} \rangle,$$

where

$$\delta(s, \boldsymbol{a}) = a, \qquad \delta(s, \boldsymbol{b}) = b, \qquad \delta(s, \boldsymbol{c}) = c,$$

the second-symbol transitions are as described above, and

$$\delta(q_{acc}, x) = q_{acc}, \qquad \delta(q_{rej}, x) = q_{rej}$$

for every $x \in \{\boldsymbol{a}, \boldsymbol{b}, \boldsymbol{c}\}$.

1.47. **Prove that the empty word is unique.**

Suppose x and y are both empty words. By definition, this means $|x| = 0$ and $|y| = 0$. The definition of equality of strings says that two strings are equal iff they have the same length and agree in every position. Here the lengths are equal, and there are no positions at all to check because the common length is 0. Thus the positional condition is vacuously satisfied, so $x = y$.

1.48. **Show that $|xy| = |x| + |y|$.**

We prove this by induction on $|y|$.

Base case: If $|y| = 0$, then $y = \lambda$, so

$$|xy| = |x\lambda| = |x| = |x| + 0 = |x| + |y|.$$

Inductive step: Assume that $|xz| = |x| + |z|$ whenever $|z| = n$. Let $|y| = n + 1$, so $y = \boldsymbol{a}z$ for some letter $\boldsymbol{a}$ and some string z of length n. Then

$$|xy| = |x\boldsymbol{a}z| = |(x\boldsymbol{a})z|.$$

By the induction hypothesis,

$$|(x\boldsymbol{a})z| = |x\boldsymbol{a}| + |z| = (|x| + 1) + |z| = |x| + (1 + |z|) = |x| + |y|.$$

Hence $|xy| = |x| + |y|$ for all strings x and y.

1.49. **The DFA C.**

(a) The state transition diagram has two states, t_0 and t_1, with t_0 as start state and t_1 final. The transitions are

$$\delta(t_0, \boldsymbol{a}) = t_0, \quad \delta(t_0, \boldsymbol{b}) = t_1, \quad \delta(t_0, \boldsymbol{c}) = t_1,$$
$$\delta(t_1, \boldsymbol{a}) = t_0, \quad \delta(t_1, \boldsymbol{b}) = t_1, \quad \delta(t_1, \boldsymbol{c}) = t_0.$$

(b) The letter $\boldsymbol{a}$ always resets the machine to t_0. After the last $\boldsymbol{a}$, the state depends only on the remaining suffix over $\{\boldsymbol{b},\boldsymbol{c}\}$. Starting from t_0, a $\boldsymbol{b}$ moves to t_1 and then further $\boldsymbol{b}$s leave the machine there, while each $\boldsymbol{c}$ toggles the state. Therefore $L(C)$ consists exactly of those strings for which, after the last $\boldsymbol{a}$, either

- there is no $\boldsymbol{b}$ and the number of $\boldsymbol{c}$s is odd, or
- there is a last $\boldsymbol{b}$ and it is followed by an even number of $\boldsymbol{c}$s.

(c) Use one state bit t, with $t = \mathbf{0}$ for t_0 and $t = \mathbf{1}$ for t_1. Encode

$$<EOS> = \mathbf{00}, \qquad \boldsymbol{a} = \mathbf{01}, \qquad \boldsymbol{b} = \mathbf{10}, \qquad \boldsymbol{c} = \mathbf{11}.$$

Then <EOS> leaves the state unchanged, $\boldsymbol{a}$ sends the machine to t_0, $\boldsymbol{b}$ sends it to t_1, and $\boldsymbol{c}$ toggles the state. So one D flip-flop with input

$$t' = (\neg t \wedge \boldsymbol{a}_1) \vee (t \wedge \neg \boldsymbol{a}_2)$$

implements the next-state logic. The accept circuitry is

$$\textbf{accept} = t \wedge \neg \boldsymbol{a}_1 \wedge \neg \boldsymbol{a}_2.$$

1.50. **Roman numerals.**

(a) For strict-order Roman numerals, a DFA is

$$\langle \{\boldsymbol{I},\boldsymbol{V},\boldsymbol{X},\boldsymbol{L},\boldsymbol{C},\boldsymbol{D},\boldsymbol{M}\}, \{q_s, q_M, q_D, q_C, q_L, q_X, q_V, q_I, q_{dead}\}, q_s, \delta, F \rangle,$$

where

$$F = \{q_M, q_D, q_C, q_L, q_X, q_V, q_I\}.$$

The states represent the highest-valued symbol class that may still appear. The transitions are:

$$\begin{array}{llllll}
\delta(q_s,\boldsymbol{M}) = q_M, & \delta(q_s,\boldsymbol{D}) = q_D, & \delta(q_s,\boldsymbol{C}) = q_C, & & & \\
\delta(q_s,\boldsymbol{L}) = q_L, & \delta(q_s,\boldsymbol{X}) = q_X, & \delta(q_s,\boldsymbol{V}) = q_V, & \delta(q_s,\boldsymbol{I}) = q_I, & & \\
\delta(q_M,\boldsymbol{M}) = q_M, & \delta(q_M,\boldsymbol{D}) = q_D, & \delta(q_M,\boldsymbol{C}) = q_C, & & & \\
\delta(q_M,\boldsymbol{L}) = q_L, & \delta(q_M,\boldsymbol{X}) = q_X, & \delta(q_M,\boldsymbol{V}) = q_V, & \delta(q_M,\boldsymbol{I}) = q_I, & & \\
\delta(q_D,\boldsymbol{C}) = q_C, & \delta(q_D,\boldsymbol{L}) = q_L, & \delta(q_D,\boldsymbol{X}) = q_X, & \delta(q_D,\boldsymbol{V}) = q_V, & \delta(q_D,\boldsymbol{I}) = q_I, & \\
\delta(q_C,\boldsymbol{C}) = q_C, & \delta(q_C,\boldsymbol{L}) = q_L, & \delta(q_C,\boldsymbol{X}) = q_X, & \delta(q_C,\boldsymbol{V}) = q_V, & \delta(q_C,\boldsymbol{I}) = q_I, & \\
\delta(q_L,\boldsymbol{X}) = q_X, & \delta(q_L,\boldsymbol{V}) = q_V, & \delta(q_L,\boldsymbol{I}) = q_I, & & & \\
\delta(q_X,\boldsymbol{X}) = q_X, & \delta(q_X,\boldsymbol{V}) = q_V, & \delta(q_X,\boldsymbol{I}) = q_I, & & & \\
\delta(q_V,\boldsymbol{I}) = q_I, & \delta(q_I,\boldsymbol{I}) = q_I. & & & &
\end{array}$$

Every unspecified transition goes to q_{dead}, and q_{dead} loops to itself. This accepts exactly the strings of the form

$$\boldsymbol{M}^*\boldsymbol{D}^{0,1}\boldsymbol{C}^*\boldsymbol{L}^{0,1}\boldsymbol{X}^*\boldsymbol{V}^{0,1}\boldsymbol{I}^*,$$

excluding the empty word.

(b) For ordinary Roman numerals, a DFA can be built from four deterministic phases: thousands, hundreds, tens, and ones. One explicit state set is

$$\{q_s, q_M, q_T, q_{HC}, q_{HCC}, q_{HCCC}, q_{HD}, q_{HDC}, q_{HDCC}, q_{HDCCC},$$

$$q_{TX}, q_{TXX}, q_{TXXX}, q_{TL}, q_{TLX}, q_{TLXX}, q_{TLXXX}, q_{OI}, q_{OII}, q_{OIII}, q_{OV}, q_{OVI}, q_{OVII}, q_{OVIII}, q_O, q_{done}, q_{dead}\}.$$

Here q_T means "the thousands and hundreds phases are complete and the tens phase has not yet begun," and q_O means "the tens phase is complete and the ones phase has not yet begun." The start state is q_s, the dead state is q_{dead}, and every state except q_s and q_{dead} that represents a completed legal prefix is final.

The transition rules are as follows.

- Thousands:

$$\delta(q_s, \boldsymbol{M}) = q_M, \qquad \delta(q_M, \boldsymbol{M}) = q_M.$$

From either q_s or q_M, a first hundreds symbol sends the machine into the hundreds module, a first tens symbol into the tens module, and a first ones symbol into the ones module:

$$\delta(q_s, \boldsymbol{C}) = \delta(q_M, \boldsymbol{C}) = q_{HC}, \qquad \delta(q_s, \boldsymbol{D}) = \delta(q_M, \boldsymbol{D}) = q_{HD},$$

$$\delta(q_s, \boldsymbol{X}) = \delta(q_M, \boldsymbol{X}) = q_{TX}, \qquad \delta(q_s, \boldsymbol{L}) = \delta(q_M, \boldsymbol{L}) = q_{TL},$$

$$\delta(q_s, \boldsymbol{I}) = \delta(q_M, \boldsymbol{I}) = q_{OI}, \qquad \delta(q_s, \boldsymbol{V}) = \delta(q_M, \boldsymbol{V}) = q_{OV}.$$

- Hundreds:

$$\delta(q_{HC}, \boldsymbol{M}) = q_T, \qquad \delta(q_{HC}, \boldsymbol{D}) = q_T, \qquad \delta(q_{HC}, \boldsymbol{C}) = q_{HCC},$$

and on $\boldsymbol{X}, \boldsymbol{L}, \boldsymbol{I}, \boldsymbol{V}$ the machine leaves the hundreds phase and enters the appropriate lower module:

$$\delta(q_{HC}, \boldsymbol{X}) = q_{TX},\ \delta(q_{HC}, \boldsymbol{L}) = q_{TL},\ \delta(q_{HC}, \boldsymbol{I}) = q_{OI},\ \delta(q_{HC}, \boldsymbol{V}) = q_{OV}.$$

Similarly,

$$\delta(q_{HCC}, \boldsymbol{C}) = q_{HCCC}, \qquad \delta(q_{HD}, \boldsymbol{C}) = q_{HDC}, \qquad \delta(q_{HDC}, \boldsymbol{C}) = q_{HDCC}, \qquad \delta(q_{HDCC}, \boldsymbol{C}) = q_{HDCCC},$$

and from each of $q_{HCC}, q_{HCCC}, q_{HD}, q_{HDC}, q_{HDCC}, q_{HDCCC}$ the symbols $\boldsymbol{X}, \boldsymbol{L}, \boldsymbol{I}, \boldsymbol{V}$ move to $q_{TX}, q_{TL}, q_{OI}, q_{OV}$, respectively. All other hundreds-phase violations go to q_{dead}.

- Tens:

$$\delta(q_T, \boldsymbol{X}) = q_{TX}, \qquad \delta(q_T, \boldsymbol{L}) = q_{TL}, \qquad \delta(q_T, \boldsymbol{I}) = q_{OI}, \qquad \delta(q_T, \boldsymbol{V}) = q_{OV}.$$

Then

$$\delta(q_{TX}, \boldsymbol{C}) = q_O, \qquad \delta(q_{TX}, \boldsymbol{L}) = q_O, \qquad \delta(q_{TX}, \boldsymbol{X}) = q_{TXX},$$

$$\delta(q_{TXX}, \boldsymbol{X}) = q_{TXXX},$$

and the symbols $\boldsymbol{I}, \boldsymbol{V}$ leave the tens phase from any of $q_{TX}, q_{TXX}, q_{TXXX}$:

$$\delta(q_{TX}, \boldsymbol{I}) = \delta(q_{TXX}, \boldsymbol{I}) = \delta(q_{TXXX}, \boldsymbol{I}) = q_{OI},$$

$$\delta(q_{TX}, \boldsymbol{V}) = \delta(q_{TXX}, \boldsymbol{V}) = \delta(q_{TXXX}, \boldsymbol{V}) = q_{OV}.$$

Likewise,

$$\delta(q_{TL}, \boldsymbol{X}) = q_{TLX}, \quad \delta(q_{TLX}, \boldsymbol{X}) = q_{TLXX}, \quad \delta(q_{TLXX}, \boldsymbol{X}) = q_{TLXXX},$$

and from any of $q_{TL}, q_{TLX}, q_{TLXX}, q_{TLXXX}$ the symbols $\boldsymbol{I}, \boldsymbol{V}$ move to q_{OI} or q_{OV}, respectively.

- Ones:

$$\delta(q_O, \boldsymbol{I}) = q_{OI}, \qquad \delta(q_O, \boldsymbol{V}) = q_{OV},$$

$$\delta(q_{OI}, \boldsymbol{X}) = q_{done}, \qquad \delta(q_{OI}, \boldsymbol{V}) = q_{done}, \qquad \delta(q_{OI}, \boldsymbol{I}) = q_{OII},$$

$$\delta(q_{OII}, \boldsymbol{I}) = q_{OIII},$$

$$\delta(q_{OV}, \boldsymbol{I}) = q_{OVI}, \qquad \delta(q_{OVI}, \boldsymbol{I}) = q_{OVII}, \qquad \delta(q_{OVII}, \boldsymbol{I}) = q_{OVIII}.$$

All outgoing transitions from q_{done} and q_{OIII} and q_{OVIII} go to q_{dead}, and every unspecified transition anywhere also goes to q_{dead}. This DFA accepts exactly the standard Roman numerals.

(c) The following Pascal program recognizes exactly the language of part (b).

```
program Roman(input, output);
var
   s : string;
   i, n : integer;
   ok : boolean;

function Take(c : char) : boolean;
begin
   if (i <= n) and (s[i] = c) then
      begin
         i := i + 1;
         Take := true
      end
   else
      Take := false
end;

procedure Thousands;
begin
   while Take('M') do
      ;
end;

function Block(one, five, ten : char) : boolean;
var
   start : integer;
begin
   start := i;
```

```
   if Take(one) then
      begin
         if Take(ten) then
            Block := true
         else if Take(five) then
            Block := true
         else
            begin
               Take(one);
               Take(one);
               if Take(one) then
                  begin
                     i := start;
                     Block := false
                  end
               else
                  Block := true
            end
      end
   else if Take(five) then
      begin
         Take(one);
         Take(one);
         Take(one);
         if Take(one) then
            begin
               i := start;
               Block := false
            end
         else
            Block := true
      end
   else
      Block := true
end;

begin
   readln(s);
   n := length(s);
   i := 1;
   if n = 0 then
      writeln(output, 'Rejected')
   else
      begin
         Thousands;
```

```
            ok := Block('C','D','M');
            if ok then
               ok := Block('X','L','C');
            if ok then
               ok := Block('I','V','X');
            if ok and (i > n) then
               writeln(output, 'Accepted')
            else
               writeln(output, 'Rejected')
         end
   end.
```

1.51. **Describe the set of words accepted by the DFA in Figure 1.9.**

Let y be the suffix of the input after the last occurrence of $\boldsymbol{b}$; if no $\boldsymbol{b}$ occurs, let y be the entire string. The DFA accepts exactly those words for which

$$|y|_{\boldsymbol{a}} + 2|y|_{\boldsymbol{c}} \equiv 2 \pmod 3.$$

Equivalently, $\boldsymbol{b}$ resets the machine to state s_0, while $\boldsymbol{a}$ adds 1 modulo 3 and $\boldsymbol{c}$ adds 2 modulo 3, so acceptance depends only on the contribution of the suffix after the last $\boldsymbol{b}$.

1.52. **The languages L_7, L_3, and L_n for digit sums.**

(a) For L_7, use seven states $q_0, \dots, q_6$, where q_i means that the sum of the digits seen so far is congruent to $i \pmod 7$. The start state is q_0, the only final state is q_0, and

$$\delta(q_i, \boldsymbol{d}) = q_{i+d \bmod 7}.$$

(b) The corresponding quintuple is

$$\langle \{\mathbf{0}, \mathbf{1}, \dots, \mathbf{9}\}, \{q_0, \dots, q_6\}, q_0, \delta, \{q_0\} \rangle,$$

with $\delta(q_i, \boldsymbol{d}) = q_{i+d \bmod 7}$.

(c) For L_3, use three states r_0, r_1, r_2, where r_i records the digit sum modulo 3. Again r_0 is both the start state and the only final state, and

$$\delta(r_i, \boldsymbol{d}) = r_{i+d \bmod 3}.$$

(d) The corresponding quintuple is

$$\langle \{\mathbf{0}, \mathbf{1}, \dots, \mathbf{9}\}, \{r_0, r_1, r_2\}, r_0, \delta, \{r_0\} \rangle,$$

with $\delta(r_i, \boldsymbol{d}) = r_{i+d \bmod 3}$.

(e) In general,

$$\langle \{\mathbf{0}, \mathbf{1}, \dots, \mathbf{9}\}, \{s_0, \dots, s_{n-1}\}, s_0, \delta, \{s_0\} \rangle$$

recognizes L_n if

$$\delta(s_i, \boldsymbol{d}) = s_{i+d \bmod n}$$

for every residue class i and every digit $\boldsymbol{d}$. The state always records the current digit sum modulo n.

1.53. **Explain the last row of Table 1.3.**

The last row corresponds to the case where the circuitry has somehow been initialized to the unused state **11** and then scans <SOS>, which is also encoded by **11**. Unlike the other rows containing unused state/input combinations, this one is *not* a don't-care, because the text explicitly requires <SOS> to reset the machine to the real start state. Therefore the outputs in that row must be fixed so that the circuit moves from the bogus state back to s_0 rather than behaving arbitrarily.

Chapter 2

Nerode's Theorem — Solutions

2.1. **Show Ψ is not a right congruence.**

$x \Psi y \Leftrightarrow |x| - |y|$ is odd. Take $x = \mathbf{a}$, $y = \mathbf{b}$ (so $|x| - |y| = 0$, not odd, thus $x \not\Psi y$). Also check: is Ψ even an equivalence relation? $|x| - |x| = 0$ (even), so $x \not\Psi x$ (not reflexive). Hence Ψ is not an equivalence relation and a fortiori not a right congruence.

2.2. **Q defined by $|x|_{\mathbf{a}} - |y|_{\mathbf{a}} \equiv 0 \pmod 3$.**

(a) *Equivalence*: Reflexive: $|x|_{\mathbf{a}} - |x|_{\mathbf{a}} = 0 \equiv 0$. Symmetric: if $3 \mid (|x|_{\mathbf{a}} - |y|_{\mathbf{a}})$ then $3 \mid (|y|_{\mathbf{a}} - |x|_{\mathbf{a}})$. Transitive: if $3 \mid (a-b)$ and $3 \mid (b-c)$ then $3 \mid (a-c)$.

(b) *Right congruence*: Suppose $x\,Q\,y$, i.e., $|x|_{\mathbf{a}} \equiv |y|_{\mathbf{a}} \pmod 3$. For any $z \in \Sigma^*$: $|xz|_{\mathbf{a}} = |x|_{\mathbf{a}} + |z|_{\mathbf{a}}$ and $|yz|_{\mathbf{a}} = |y|_{\mathbf{a}} + |z|_{\mathbf{a}}$. So $|xz|_{\mathbf{a}} - |yz|_{\mathbf{a}} = |x|_{\mathbf{a}} - |y|_{\mathbf{a}} \equiv 0 \pmod 3$, i.e., $xz\,Q\,yz$.

2.3. **All languages L over $\{\mathbf{a},\mathbf{b},\mathbf{c}\}$ with $\mathrm{rk}(R_L) = 1$.**

$\mathrm{rk}(R_L) = 1$ means R_L has exactly one equivalence class, namely all of Σ^*. This occurs iff $R_L = \Sigma^* \times \Sigma^*$, i.e., $x\,R_L\,y$ for all x, y. This happens iff $(\forall z)(xz \in L \Leftrightarrow yz \in L)$ for all x, y, z, which holds iff either $L = \emptyset$ or $L = \Sigma^*$.

Justification: If $L \neq \emptyset$ and $L \neq \Sigma^*$, pick $w \in L$ and $u \notin L$. Then $w\,R_L\,u$ fails (take $z = \lambda$), so R_L has at least 2 classes.

2.4. **Show P (with 3 equivalence classes as given) is a right congruence.**

The classes are $[\lambda]_P = \{\mathbf{0},\mathbf{1}\}^0$, $[\mathbf{1}]_P = \{\mathbf{0},\mathbf{1}\}^1$, $[\mathbf{00}]_P = \bigcup_{n\geq 2}\{\mathbf{0},\mathbf{1}\}^n$.

We must check: if $x\,P\,y$ then $xa\,P\,ya$ for all $a \in \{0,1\}$.

Case $x, y \in [\lambda]_P$: $|x| = |y| = 0$, so $|xa| = |ya| = 1$, thus $xa, ya \in [\mathbf{1}]_P$.

Case $x, y \in [\mathbf{1}]_P$: $|x| = |y| = 1$, so $|xa| = |ya| = 2$, thus $xa, ya \in [\mathbf{00}]_P$.

Case $x, y \in [\mathbf{00}]_P$: $|x|, |y| \geq 2$, so $|xa|, |ya| \geq 3 \geq 2$, thus $xa, ya \in [\mathbf{00}]_P$.

In every case $xa\,P\,ya$, so P is a right congruence. □

2.5. **All languages L with $R_L = P$ (from Exercise 2.4).**

Recall $A_0 = [\lambda]_P = \{\lambda\}$, $A_1 = [\mathbf{1}]_P = \Sigma^1$, $A_2 = [\mathbf{00}]_P = \Sigma^{\geq 2}$. For $R_L = P$ we need L to be a union of P-classes *and* no two distinct P-classes to be merged by R_L. Two classes A_i, A_j are merged (identified)

by R_L iff for every $z \in \Sigma^*$: $A_i \cdot z \subseteq L \Leftrightarrow A_j \cdot z \subseteq L$ (more precisely, for representative elements $u \in A_i$, $v \in A_j$: $uz \in L \Leftrightarrow vz \in L$ for all z).

Check each of the 6 nonempty proper unions:

- $L = A_0 = \{\lambda\}$: For any $z \neq \lambda$, both $\mathbf{0}z \notin L$ (length≥ 2) and $\mathbf{00}z \notin L$; for $z = \lambda$, $\mathbf{0} \notin L$ and $\mathbf{00} \notin L$. So A_1 and A_2 are *not* distinguished: $R_L \neq P$.
- $L = A_1 = \Sigma^1$: $z = \lambda$: $\lambda \notin L$ but $\mathbf{0} \in L$ (distinguishes A_0 from A_1). $\mathbf{0} \in L$ but $\mathbf{00} \notin L$ (distinguishes A_1 from A_2). All three classes remain distinct: $R_L = P$. ✓
- $L = A_2 = \Sigma^{\geq 2}$: $z = \mathbf{0}$: $\lambda\mathbf{0} = \mathbf{0} \notin L$ but $\mathbf{0} \cdot \mathbf{0} = \mathbf{00} \in L$ (distinguishes A_0 from A_1). $z = \lambda$: $\mathbf{0} \notin L$ but $\mathbf{00} \in L$ (distinguishes A_1 from A_2). So $R_L = P$. ✓
- $L = A_0 \cup A_1 = \Sigma^{\leq 1}$: $z = \mathbf{0}$: $\lambda \cdot \mathbf{0} = \mathbf{0} \in L$ but $\mathbf{0} \cdot \mathbf{0} = \mathbf{00} \notin L$ (distinguishes A_0 from A_1). $z = \lambda$: $\mathbf{0} \in L$ but $\mathbf{00} \notin L$ (distinguishes A_1 from A_2). So $R_L = P$. ✓
- $L = A_0 \cup A_2$: $z = \lambda$: $\lambda \in L$ but $\mathbf{0} \notin L$ (distinguishes A_0, A_1). $z = \lambda$: $\mathbf{0} \notin L$ but $\mathbf{00} \in L$ (distinguishes A_1, A_2). So $R_L = P$. ✓
- $L = A_1 \cup A_2 = \Sigma^+$: For any z, $\mathbf{0}z \in \Sigma^+$ and $\mathbf{00}z \in \Sigma^+$, both in L. So A_1 and A_2 are not distinguished: $R_L \neq P$.

Therefore exactly **4** languages satisfy $R_L = P$: A_1, A_2, $A_0 \cup A_1$, and $A_0 \cup A_2$.

2.6. **All languages L with $R_L = Q$ (from Exercise 2.2).**

Q has three equivalence classes: $[x]_Q$ depends on $|x|_{\mathbf{a}}$ mod 3. So Q-classes are $C_0 = \{x : |x|_{\mathbf{a}} \equiv 0\}$, $C_1 = \{x : |x|_{\mathbf{a}} \equiv 1\}$, $C_2 = \{x : |x|_{\mathbf{a}} \equiv 2\}$. Any nonempty proper union of these classes gives $R_L = Q$.

2.7. **Analysis of Q on $\{\mathbf{a}, \mathbf{b}\}^*$.**

(a) *Right congruence, classes*: $\lambda Q \lambda$ by definition; for non-empty strings, $x\,Q\,y$ iff ($|x|, |y|$ have the same parity. Classes: $[\lambda]_Q$ (just λ) and $C_e = \{x \neq \lambda : |x|$ even$\}$ and $C_o = \{x : |x|$ odd$\}$. This is a right congruence because appending any a flips parity of length for non-empty strings, consistently within each class.

(b) $L = [\lambda]_Q \cup [\mathbf{aa}]_Q = \{\lambda\} \cup C_e$. Simple description: $L = \{x : |x|$ is even$\}$. R_L has two classes: even-length strings and odd-length strings.

(c) A_Q: 3 states ($[\lambda]_Q$, C_e, C_o). Final states for L: $\{[\lambda]_Q, C_e\}$.

(d) A_{R_L}: 2 states (even, odd), final: even.

(e) A_Q with relabeled final states: Yes. $C_e \cup [\lambda]_Q$ vs. C_o – this collapses to A_{R_L}.

(f) The eight languages are the $2^3 = 8$ unions of subsets of $\{[\lambda]_Q, C_e, C_o\}$. A language K satisfies the criterion of part (e) iff $R_K = Q$, i.e., A_Q (relabeled with new final states for K) is already minimal. Checking all eight: $\emptyset$ and Σ^* give rk$(R_K) = 1$; $\{\lambda\}$, all-even ($\{\lambda\} \cup C_e$), C_o, and Σ^+ ($= C_e \cup C_o$) give rk$(R_K) = 2$ (two of the three Q-classes merge in each case). Only two choices keep all three Q-classes distinct:

- $K = C_e$: $z = \lambda$ distinguishes $[\lambda]$ from C_e (since $\lambda \notin K$ but $aa \in K$); $z = \lambda$ distinguishes C_e from C_o (since $aa \in K$ but $a \notin K$). So $R_K = Q$.
- $K = \{\lambda\} \cup C_o$: $z = aa$ distinguishes $[\lambda]$ from C_o ($\lambda \cdot aa = aa \notin K$ but $a \cdot aa = aaa \in K$); $z = \lambda$ distinguishes C_o from C_e ($a \in K$ but $aa \notin K$). So $R_K = Q$.

These are the only two of the eight languages satisfying part (e).

2.8. I **= identity on** $\{\mathbf{a}\}^*$**.**

(a) $x\,I\,y \Leftrightarrow x = y$. Right congruence: if $x = y$, then $xa = ya$ for any a. Yes.

(b) Each equivalence class is a singleton: $[\mathbf{a}^n]_I = \{\mathbf{a}^n\}$.

2.9. $L = \{\lambda\} \cup \{\mathbf{a}\} \cup \{\mathbf{aa}\}$**,** I **has infinite rank. Is** L **FAD?**

L is a finite set, hence FAD. Nerode's theorem says L is FAD iff R_L has finite rank, not iff I (or any other right congruence) has finite rank. Here R_L has rank equal to the number of R_L-classes, which is finite (since L is finite and FAD). The fact that I has infinite rank is irrelevant – we need I to *refine* R_L (which it does, trivially, since I is the finest relation), but Nerode's theorem requires R_L itself to have finite rank. □

2.10. **Machine** M **with** $\mathrm{rk}(R_{L(M)}) \neq |S_M|$**.**

Let M be the two-state DFA: $\Sigma = \{\mathbf{a}\}$, $S = \{q_0, q_1\}$, $s_0 = q_0$, $F = \{q_0, q_1\}$ (all states final), $\delta(q_0, \mathbf{a}) = q_1$, $\delta(q_1, \mathbf{a}) = q_0$. Then $L(M) = \Sigma^*$, so $R_{L(M)}$ has rank 1, but $|S_M| = 2$.

2.11. **Show** F_{R_L} **is well defined.**

$F_{R_L} = \{[x]_{R_L} \mid x \in L\}$. We must show: if $[x]_{R_L} = [y]_{R_L}$, then $x \in L \Leftrightarrow y \in L$ (so each R_L-class is either entirely inside L or entirely outside L, making the assignment of classes to F_{R_L} unambiguous). Since $[x]_{R_L} = [y]_{R_L}$, we have $x\,R_L\,y$, meaning $(\forall z \in \Sigma^*)(xz \in L \Leftrightarrow yz \in L)$. Taking $z = \lambda$: $x \in L \Leftrightarrow y \in L$. □

2.12. **Show** δ_{R_L} **is well defined.**

$\delta_{R_L}([x]_{R_L}, a) = [xa]_{R_L}$. We must show: if $[x]_{R_L} = [y]_{R_L}$, then $[xa]_{R_L} = [ya]_{R_L}$. Since $x\,R_L\,y$, for any z: $xaz \in L \Leftrightarrow x(az) \in L \Leftrightarrow y(az) \in L \Leftrightarrow yaz \in L$. Thus $xa\,R_L\,ya$. □

2.13. **Prove** $\overline{\delta}_{R_L}([x]_{R_L}, y) = [xy]_{R_L}$ **by induction on** $|y|$**.**

Base $|y| = 0$: $\overline{\delta}_{R_L}([x], \lambda) = [x] = [x\lambda]$. ✓

Step: $y = wa$, $|w| = n$. $\overline{\delta}_{R_L}([x], wa) = \delta_{R_L}(\overline{\delta}_{R_L}([x], w), a) = \delta_{R_L}([xw], a) = [xwa] = [xy]$. □

2.14. **Find** R^P **for automaton** P **in Example 2.6, check** $R^P = R_L$**.**

R^P is defined by $x\,R^P\,y \Leftrightarrow \overline{\delta}(s_0, x) = \overline{\delta}(s_0, y)$. Since the machine P in Example 2.6 is the minimal machine for L, each state is reachable and distinguishable. The relation R^P has the same classes as R_L (each state corresponds to one R_L-class). Hence $R^P = R_L$.

2.15. **Find** R^A **for machines from Chapter 1 exercises.**

For each DFA A built in Chapter 1: R^A is defined by $x\,R^A\,y \Leftrightarrow \overline{\delta}(s_0, x) = \overline{\delta}(s_0, y)$ (same state reached). Compare to $R_{L(A)}$: by definition of R^A, it always refines $R_{L(A)}$ (since if same state, same future acceptance). For minimal machines they coincide; for non-minimal ones R^A may be strictly finer.

2.16. **Prove the pumping lemma statement by induction.**

Let M be a DFA with states S, $|S| = n$. For a word $w = uvw'$ with $|uv| \leq n$, $|v| \geq 1$, and $uvw' \in L$. Let $s = \overline{\delta}(s_0, u)$, so $\overline{\delta}(s, v) = s$ (since the path through v loops back to s).

Claim: $(\forall i \in \mathbb{N})(\overline{\delta}(s_0, uv^i w') = \overline{\delta}(s_0, uvw') = f \in F)$.

Proof by induction on i:

$i=0$: $\overline{\delta}(s_0,uw')=\overline{\delta}(\overline{\delta}(s_0,u),w')=\overline{\delta}(s,w')$. And $\overline{\delta}(s_0,uvw')=\overline{\delta}(\overline{\delta}(s_0,uv),w')=\overline{\delta}(s,w')$ (since $\overline{\delta}(s,v)=s$). Equal. ✓

$i\to i{+}1$: By the inner induction, $\overline{\delta}(s_0,uv^i)=s$ (base: $\overline{\delta}(s_0,u)=s$; step: $\overline{\delta}(s_0,uv^{i+1})=\overline{\delta}(\overline{\delta}(s_0,uv^i),v)=\overline{\delta}(s,v)=s$). Therefore

$$\overline{\delta}(s_0,uv^{i+1}w')=\overline{\delta}(\overline{\delta}(s_0,uv^i),vw')=\overline{\delta}(s,vw')=\overline{\delta}(\overline{\delta}(s,v),w')=\overline{\delta}(s,w').$$

So $\overline{\delta}(s_0,uv^iw')=\overline{\delta}(s,w')=f$ for all i. □

2.17. **Prove Theorem 2.1.**

Theorem 2.1 states: L is FAD iff R_L has finite rank.

($\Rightarrow$) If $L=L(M)$ for a DFA M with n states, define $\mu:\Sigma^*\to S$ by $\mu(x)=\overline{\delta}(s_0,x)$. Then $x\,R^M\,y\Leftrightarrow\mu(x)=\mu(y)$, so R^M has at most n classes. Since R^M refines R_L and R_L is coarser, $\mathrm{rk}(R_L)\le\mathrm{rk}(R^M)\le n<\infty$.

($\Leftarrow$) If R_L has finite rank k, construct the minimal machine $A_{R_L}=\langle\Sigma,[\cdot]_{R_L},[\lambda]_{R_L},\delta_{R_L},F_{R_L}\rangle$ with k states (shown well-defined in Exercises 2.11–2.12). Show $L(A_{R_L})=L$: $x\in L(A_{R_L})\Leftrightarrow\overline{\delta}_{R_L}([\lambda],x)\in F_{R_L}\Leftrightarrow[x]\in F_{R_L}\Leftrightarrow x\in L$.

2.18. **Find a language giving rise to I (identity on $\{\mathbf{a}\}^*$); show it cannot be FAD.**

(a) Take

$$L=\{\mathbf{a}^{2^m}\mid m\ge 0\}\subseteq\{\mathbf{a}\}^*.$$

We claim $R_L=I$. Let $i<j$ and $d=j-i\ge 1$. Choose m so that $2^m>d$, and set $z=\mathbf{a}^{2^m-i}$. Then

$$\mathbf{a}^i z=\mathbf{a}^{2^m}\in L,$$

but

$$\mathbf{a}^j z=\mathbf{a}^{2^m+d}$$

with

$$2^m<2^m+d<2^{m+1},$$

so 2^m+d is not a power of 2. Hence $\mathbf{a}^j z\notin L$. Thus $\mathbf{a}^i\not R_L\,\mathbf{a}^j$ for all $i\ne j$, so every class is a singleton and $R_L=I$.

(b) Such a language cannot be FAD: $R_L=I$ has infinite rank (one class per string in $\{\mathbf{a}\}^*$), so by Nerode's theorem L is not FAD. □

2.19. **Prove Theorem 2.4 from Theorem 2.3.**

Theorem 2.3: If L is FAD then R_L has finite rank. Theorem 2.4 (Nerode): L is FAD $\Leftrightarrow$ R_L has finite rank.

We need the converse: finite rank $\Rightarrow$ FAD. Given R_L has k classes, the machine A_{R_L} has k states and accepts L (by Exercises 2.11–2.13). Hence L is FAD.

2.20. **Prove Theorem 2.5.**

Theorem 2.5: L is FAD $\Leftrightarrow$ there exists a right congruence R of finite rank such that L is a union of R-classes.

($\Rightarrow$) Take $R = R^M$ for a DFA M accepting L: finite rank (at most $|S|$), and $L = \bigcup_{s\in F}[s]_{R^M}$.

($\Leftarrow$) Given such R, build A_R with states = R-classes, F_R = classes $\subseteq L$. Then $L = L(A_R)$, so L is FAD.

2.21. **Prove Theorem 2.6.**

Theorem 2.6 states:

$$(\exists n)(\forall x \notin L,\ |x| \geq n)(\exists u, v, w)\big[x = uvw,\ |uv| \leq n,\ |v| \geq 1,\ (\forall i \geq 0)\ uv^i w \notin L\big].$$

Assume L is FAD, accepted by DFA $M = \langle \Sigma, S, s_0, \delta, F\rangle$, with $|S| = n$. Take any $x \notin L$ with $|x| \geq n$. Look at the first $n+1$ states on the run of x:

$$s_0,\ \overline{\delta}(s_0, a_1),\ \overline{\delta}(s_0, a_1a_2),\ \ldots,\ \overline{\delta}(s_0, a_1\cdots a_n).$$

By pigeonhole, two are equal. So for some decomposition $x = uvw$ with $|uv| \leq n$, $|v| \geq 1$,

$$\overline{\delta}(s_0, u) = \overline{\delta}(s_0, uv) = q.$$

Hence v is a loop at q, so for every $i \geq 0$,

$$\overline{\delta}(s_0, uv^i w) = \overline{\delta}(q, w) = \overline{\delta}(s_0, uvw) = \overline{\delta}(s_0, x).$$

Since $x \notin L$, $\overline{\delta}(s_0, x) \notin F$. Therefore $\overline{\delta}(s_0, uv^i w) \notin F$, i.e., $uv^i w \notin L$ for all $i \geq 0$. This is exactly Theorem 2.6. □

2.22. **Prove Theorem 2.7.**

Theorem 2.7 states: if M is an n-state DFA accepting L, then every $x = a_1a_2\cdots a_m \in L$ with $m \geq n$ has a subsequence $a_{i_1}a_{i_2}\cdots a_{i_j} \in L$ with $j < n$.

Let $x \in L$, $|x| = m \geq n$. By Theorem 2.3 (or Exercise 2.16), we can write

$$x = u_1v_1w_1, \quad |u_1v_1| \leq n,\ |v_1| \geq 1, \text{ and } u_1v_1^i w_1 \in L\ \forall i \geq 0.$$

In particular, with $i = 0$, $x_1 = u_1w_1 \in L$ and $|x_1| < |x|$.

If $|x_1| < n$, stop. Otherwise apply the same argument to x_1. Each step removes at least one symbol and keeps membership in L, so after finitely many steps we obtain

$$y \in L, \quad |y| < n,$$

and y is obtained from x by deleting some blocks. Therefore y is a subsequence of x:

$$y = a_{i_1}a_{i_2}\cdots a_{i_j}$$

for an increasing index sequence $i_1 < i_2 < \cdots < i_j$, with $j = |y| < n$. This is exactly Theorem 2.7. □

2.23. **Show $L = \{x \mid |x|_{\mathbf{a}} < |x|_{\mathbf{b}}\}$ is not FAD.**

For any n, take $w = \mathbf{a}^n\mathbf{b}^{n+1} \in L$, $|w| = 2n+1 \geq n$. Any decomposition $w = uvw'$ with $|uv| \leq n$ and $|v| \geq 1$ has $v = \mathbf{a}^k$ for some $k \geq 1$ (since $|uv| \leq n$). Then $uv^0w' = uw' = \mathbf{a}^{n-k}\mathbf{b}^{n+1}$ which has $n-k$ **a**s and $n+1$ **b**s. Since $n-k < n+1$, this is in L. And $uv^2w' = \mathbf{a}^{n+k}\mathbf{b}^{n+1}$: need $n+k < n+1$, i.e., $k < 1$, impossible since $k \geq 1$. So $uv^2w' \notin L$. The pumping lemma fails, so L is not FAD. □

2.24. **Show $G = \{x \mid |x|_{\mathbf{a}} \geq |x|_{\mathbf{b}}\}$ is not FAD.**

For any n, take $w = \mathbf{a}^n\mathbf{b}^n \in G$. Any $v = \mathbf{a}^k$ ($k \geq 1$, since $|uv| \leq n$). Then $uv^0w' = \mathbf{a}^{n-k}\mathbf{b}^n$: need $n-k \geq n$, i.e., $k \leq 0$. Contradiction. So pumping down takes us out of G. Not FAD. □

2.25. **Show $P = \{\mathbf{d}^p \mid p$ prime$\}$ is not FAD.**

Suppose P is FAD with n states. Take a prime $p > n$ and $w = \mathbf{d}^p \in P$. By pumping: $w = uvw'$ with $|uv| \leq n$, $|v| = k \geq 1$, so $v = \mathbf{d}^k$ and $u = \mathbf{d}^j$, $w' = \mathbf{d}^{p-j-k}$. Then $uv^iw' = \mathbf{d}^{j+ik+(p-j-k)} = \mathbf{d}^{p+(i-1)k}$ must be in P for all $i \geq 0$. So $p+(i-1)k$ must be prime for all $i \geq 0$. Take $i = p+1$: $p+pk = p(1+k)$, which is composite (both factors > 1 since $p \geq 2$ and $k \geq 1$). Contradiction. □

2.26. **Show $\Gamma = \{w \cdot \mathbf{2} \cdot w \mid w \in \{\mathbf{0},\mathbf{1}\}^*\}$ is not FAD.**

For any n, consider $w_n = \mathbf{0}^n\mathbf{2}\mathbf{0}^n \in \Gamma$. The strings $\mathbf{0}^n$ and $\mathbf{0}^m$ ($n \neq m$) are in different R_Γ-classes: $\mathbf{0}^n \cdot (\mathbf{2}\mathbf{0}^n) \in \Gamma$ but $\mathbf{0}^m \cdot (\mathbf{2}\mathbf{0}^n) \notin \Gamma$ (since $m \neq n$). So R_Γ has infinitely many classes. Not FAD. □

2.27. **Show $\Psi = \{ww \mid w \in \{\mathbf{0},\mathbf{1}\}^*\}$ is not FAD.**

For each $n \geq 0$, let $x_n = \mathbf{0}^n\mathbf{1}$. We show the x_n are pairwise in distinct R_Ψ-classes.

Fix $i \neq j$ and take suffix $z_i = \mathbf{0}^i\mathbf{1}$. Then

$$x_i z_i = \mathbf{0}^i\mathbf{1}\mathbf{0}^i\mathbf{1} = ww \in \Psi$$

with $w = \mathbf{0}^i\mathbf{1}$.

Now consider

$$x_j z_i = \mathbf{0}^j\mathbf{1}\mathbf{0}^i\mathbf{1}.$$

Suppose this were in Ψ, so $x_j z_i = tt$ for some t. The string has exactly two **1**s, so each half must contain exactly one **1**. Hence the position of the second **1** must be "half-length" after the first. Its first **1** is at position $j+1$, second at $j+i+2$, so the half-length must be $i+1$. But half-length is also $\frac{i+j+2}{2}$, giving $\frac{i+j+2}{2} = i+1$, hence $j = i$, contradiction. Therefore $x_j z_i \notin \Psi$.

So $x_i \not R_\Psi x_j$ for $i \neq j$, giving infinitely many Nerode classes. Thus Ψ is not FAD. □

2.28. **Show $K = \{w \mid w = w^r\}$ (palindromes) is not FAD.**

For any n, the strings $\mathbf{0}, \mathbf{0}^2, \ldots, \mathbf{0}^n$ are in distinct R_K-classes: $\mathbf{0}^i \, R_K \, \mathbf{0}^j$ requires $(\forall z)(\mathbf{0}^i z \in K \Leftrightarrow \mathbf{0}^j z \in K)$. Take $z = \mathbf{0}^i$: $\mathbf{0}^{2i}$ is a palindrome (yes). $\mathbf{0}^{j+i}$: palindrome (yes, any string of one letter is a palindrome). So we need a subtler argument. Take $z = \mathbf{1}\mathbf{0}^i$: $\mathbf{0}^i\mathbf{1}\mathbf{0}^i \in K$ (palindrome), but $\mathbf{0}^j\mathbf{1}\mathbf{0}^i \in K$ iff $\mathbf{0}^j\mathbf{1}\mathbf{0}^i = \mathbf{0}^i\mathbf{1}\mathbf{0}^j$, i.e., $i = j$. So distinct $i \neq j$ give distinct classes. Infinite rank. Not FAD. □

2.29. **Show $\Phi = \{\mathbf{a}^j\mathbf{b}^k\mathbf{c}^m \mid j \geq 3, k = m\}$ is not FAD (second pumping lemma).**

Use the Myhill-Nerode approach. For any n, consider $\mathbf{a}^3\mathbf{b}^i$ for $i = 0, 1, \ldots, n$. These are in distinct R_Φ-classes: $\mathbf{a}^3\mathbf{b}^i \, R_\Phi \, \mathbf{a}^3\mathbf{b}^j$ requires $(\forall z)(\mathbf{a}^3\mathbf{b}^i z \in \Phi \Leftrightarrow \mathbf{a}^3\mathbf{b}^j z \in \Phi)$. Take $z = \mathbf{c}^i$: $\mathbf{a}^3\mathbf{b}^i\mathbf{c}^i \in \Phi$ (yes, $j = 3 \geq 3$, $k = m = i$). But $\mathbf{a}^3\mathbf{b}^j\mathbf{c}^i \in \Phi$ iff $j = i$. So for $i \neq j$, distinct classes. □

Why first pumping lemma is hard: Any long word in Φ starts with ≥ 3 **a**s. Pumping within the **a**-block may still satisfy $j \geq 3$; pumping within **b** or **c** block destroys $k = m$.

2.30. **Show $C = \{\mathbf{d}^q \mid q$ non-prime$\}$ is not FAD.**

Over $\{\mathbf{d}\}^*$, the complement of C is exactly

$$\sim C = \{\mathbf{d}^p \mid p \text{ prime}\} = P.$$

Exercise 2.25 already proved P is not FAD. If C were FAD, then $\sim C$ would be FAD (closure under complement), contradicting that result. Therefore C is not FAD. □

2.31. **R_L with 3 classes: $\{\lambda\}$, odd-length, even-length$\backslash\{\lambda\}$.**

(a) Why $L = \{x \mid |x| \text{ odd}\}$ can't give this R_L: Compute $R_{\{x:|x| \text{ odd}\}}$. We need $x \, R_L \, y \Leftrightarrow (\forall z)(|xz| \text{ odd} \Leftrightarrow |yz| \text{ odd})$, i.e., $|x| \equiv |y| \pmod 2$. So R_L has only 2 classes (even-length, odd-length), not 3. λ is not distinguished from other even-length strings.

(b) Let

$$A = \{\lambda\}, \quad B = \{x : |x| \text{ odd}\}, \quad C = \{x : |x| \text{ even and } x \neq \lambda\}.$$

Checking all 8 unions of A, B, C, exactly two give this same 3-class relation:

$$L = C \quad \text{and} \quad L = A \cup B.$$

For each of these, A, B, C are pairwise distinguishable:

- A vs C: suffix $z = \lambda$.
- A vs B: suffix z of odd length.
- B vs C: suffix $z = \lambda$.

All other unions merge at least one pair of classes.

2.32. **Describe R_Ψ and draw machine for $\Psi = \{x \mid |x|_{\mathbf{a}}$ even, ends with **b**$\}$.**

R_Ψ classes: we need to determine when x, y have the same future w.r.t. Ψ. The future of x depends on: (1) parity of $|x|_{\mathbf{a}}$, and (2) whether x ends with **b**.

Thus R_Ψ has four classes:

- $C_{e,b} = \{x \mid |x|_{\mathbf{a}}$ even and x ends with **b**$\}$.
- $C_{e,\neg b} = \{x \mid |x|_{\mathbf{a}}$ even and x does not end with **b**$\}$.
- $C_{o,b} = \{x \mid |x|_{\mathbf{a}}$ odd and x ends with **b**$\}$.
- $C_{o,\neg b} = \{x \mid |x|_{\mathbf{a}}$ odd and x does not end with **b**$\}$.

Here $\lambda \in C_{e,\neg b}$ (even number of **a**'s, and it does not end with **b**).

So the minimal DFA has 4 states, start state $C_{e,\neg b}$, and unique final state $C_{e,b}$.

2.33. **Show $\Xi = \{\lambda, \mathbf{a}, \mathbf{a}^4, \mathbf{a}^9, \ldots\}$ (perfect square lengths) is not FAD.**

For distinct $i < j$, we show $\mathbf{a}^i \not R_\Xi \mathbf{a}^j$. Let $d = j - i \geq 1$. Choose n with $2n + 1 > d$, and set

$$k = n^2 - i.$$

Then

$$i + k = n^2$$

is a perfect square, so $\mathbf{a}^{i+k} \in \Xi$. But

$$j + k = n^2 + d,$$

and since $0 < d < 2n + 1$, we have

$$n^2 < n^2 + d < (n+1)^2.$$

So $n^2 + d$ is not a perfect square, hence $\mathbf{a}^{j+k} \notin \Xi$. Therefore $\mathbf{a}^i$ and $\mathbf{a}^j$ are distinguishable; infinitely many classes follow, so Ξ is not FAD. □

2.34. **Show $\Phi = \{\mathbf{b}, \mathbf{b}^2, \mathbf{b}^4, \mathbf{b}^8, \ldots\}$ (powers of 2) is not FAD.**

For distinct $i < j$, let $d = j - i \geq 1$. Choose m with $2^m > d$, and set

$$k = 2^m - i.$$

Then

$$i + k = 2^m$$

is a power of 2, so $\mathbf{b}^{i+k} \in \Phi$. But

$$j + k = 2^m + d$$

satisfies

$$2^m < 2^m + d < 2^{m+1}$$

because $0 < d < 2^m$. So $2^m + d$ is not a power of 2, hence $\mathbf{b}^{j+k} \notin \Phi$. Thus $\mathbf{b}^i$ and $\mathbf{b}^j$ are distinguishable; therefore R_Φ has infinite rank and Φ is not FAD. □

2.35. **Five-class R_L over $\{\mathbf{a}, \mathbf{b}\}$.**

(a) Let

$$C_0 = \{\lambda\},\ C_1 = \{\mathbf{a}\},\ C_2 = \{\mathbf{aa}\},\ C_3 = \{\mathbf{a}^k : k \geq 3\},\ C_4 = \{x : x \text{ contains a } \mathbf{b}\}.$$

If L is exactly one class and still has R_L equal to this 5-class partition, the only possibility is

$$L = C_3 = \{\mathbf{a}^3, \mathbf{a}^4, \ldots\}.$$

Choosing any other single class merges some classes under R_L.

(b) The other languages L that still give this same R_L are exactly:

$$C_0 \cup C_3, \quad C_1 \cup C_3, \quad C_0 \cup C_1 \cup C_3,$$

$$C_2 \cup C_4, \quad C_0 \cup C_2 \cup C_4, \quad C_1 \cup C_2 \cup C_4, \quad C_0 \cup C_1 \cup C_2 \cup C_4.$$

Together with part (a), there are 8 such languages.

2.36. **Find R_Ω for $\Omega = \{y \mid (y \text{ has exactly one } \mathbf{0}) \vee (y \text{ has even \#1s})\}$.**

$xz \in \Omega$ iff $(|x|_{\mathbf{0}} + |z|_{\mathbf{0}} = 1)$ or $(|x|_{\mathbf{1}} + |z|_{\mathbf{1}} \equiv 0 \pmod 2)$. The future of x (i.e., which z satisfy $xz \in \Omega$) depends only on $c_0 = |x|_{\mathbf{0}}$ (capped at 2) and $p = |x|_{\mathbf{1}} \bmod 2$. Writing (c_0, p) for these two pieces of information, the six cases yield the following future descriptions:

- $(0,0)$: $xz \in \Omega \Leftrightarrow |z|_{\mathbf{0}} = 1$ or $|z|_{\mathbf{1}}$ even.
- $(0,1)$: $xz \in \Omega \Leftrightarrow |z|_{\mathbf{0}} = 1$ or $|z|_{\mathbf{1}}$ odd.
- $(1,0)$: $xz \in \Omega \Leftrightarrow |z|_{\mathbf{0}} = 0$ or $|z|_{\mathbf{1}}$ even.
- $(1,1)$: $xz \in \Omega \Leftrightarrow |z|_{\mathbf{0}} = 0$ or $|z|_{\mathbf{1}}$ odd.
- $(2,0)$: $xz \in \Omega \Leftrightarrow |z|_{\mathbf{1}}$ even. ($|xz|_{\mathbf{0}} \geq 2$ so the first disjunct can never hold.)
- $(2,1)$: $xz \in \Omega \Leftrightarrow |z|_{\mathbf{1}}$ odd.

All six futures are distinct (for example, $(0,0)$ and $(2,0)$ differ on $z = \mathbf{01}$: the first accepts it since $|z|_{\mathbf{0}} = 1$, the second rejects since $|z|_{\mathbf{1}} = 1$ is odd; and so on for the other pairs). Hence R_Ω has exactly **6** equivalence classes:

$$C_{c_0,p} = \{x \in \{\mathbf{0},\mathbf{1}\}^* \mid |x|_{\mathbf{0}} = c_0 \text{ (or } \geq 2 \text{ if } c_0 = 2) \text{ and } |x|_{\mathbf{1}} \equiv p \pmod 2\},$$

for $(c_0, p) \in \{0,1,2\} \times \{0,1\}$. The minimal DFA for Ω has 6 states. Final states are those in Ω: $(1,0), (1,1), (0,0), (2,0)$ — i.e., strings with exactly one **0** (states $(1,\cdot)$) or with even #**1**s (states $(\cdot,0)$).

2.37. **Which of $L_1, L_2, L_1 \cap L_2, \sim L_2, L_1 \cup L_2$ are FAD?**

$L_1 = \{x \mid |x|_{\mathbf{a}} > |x|_{\mathbf{b}}\}$: Not FAD (Myhill-Nerode: strings $\mathbf{a}^n$ for each n are in distinct classes).

$L_2 = \{x \mid |x|_{\mathbf{a}} < 3\}$: FAD (finite DFA with states tracking $\min(|x|_{\mathbf{a}}, 3)$).

$L_1 \cap L_2$: $\{x \mid |x|_{\mathbf{a}} \in \{0,1,2\}$ and $|x|_{\mathbf{a}} > |x|_{\mathbf{b}}\}$. FAD (intersection of FAD with non-FAD can be FAD if the FAD constraint dominates – here L_2 restricts to finitely many $|x|_{\mathbf{a}}$ values). FAD.

$\sim L_2 = \{x \mid |x|_{\mathbf{a}} \geq 3\}$: FAD (complement of FAD is FAD).

$L_1 \cup L_2$: Not FAD. If it were, then $L_1 = (L_1 \cup L_2) \cap \sim L_2$ would be FAD (intersection of two FAD), contradiction.

2.38. **R_m defined by $|x| - |y|$ is a multiple of m.**

(a) *Right congruence*: If $m \mid (|x| - |y|)$, then $m \mid (|xz| - |yz|)$ since $|xz| - |yz| = |x| - |y|$.

(b) $R_2 \cap R_3 = R_6$: $x\, R_2 \cap R_3\, y \Leftrightarrow 2 \mid |x| - |y|$ and $3 \mid |x| - |y| \Leftrightarrow 6 \mid |x| - |y| \Leftrightarrow x\, R_6\, y$.

(c) In general: if $x\, R \cap S\, y$, then $x\, R\, y$ and $x\, S\, y$. For any z: $x\, R\, y \Rightarrow xz\, R\, yz$, and $x\, S\, y \Rightarrow xz\, S\, yz$. So $xz\, R \cap S\, yz$.

(d) $R_2 \cup R_3$: $\lambda\, R_2$ **aa** and **a** R_3 **aaaa**. But is $\lambda\, R_2 \cup R_3$ **a**? $|\lambda| - |\mathbf{a}| = -1$, not divisible by 2 or 3. No. Is $\lambda\, R_2 \cup R_3$ **aaa**? $0 - 3 = -3$, divisible by 3. Yes. Now $\lambda\, R_2 \cup R_3$ **aaa** and **aaa** $R_2 \cup R_3$ **aaaaa** (length diff=2). But is $\lambda\, R_2 \cup R_3$ **aaaaa**? Length diff=5, not divisible by 2 or 3. No. So transitivity fails: not an equivalence relation.

(e) If $R \cup S$ is an equivalence relation: for any x, y with $x\, R \cup S\, y$, we have $x\, R\, y$ or $x\, S\, y$. Either way, for any z: $xz\, R\, yz$ or $xz\, S\, yz$ (by right congruence of R or S), hence $xz\, R \cup S\, yz$.

2.39. **Example of two right congruences whose union is not a right congruence.**

Let $R = R_2$ and $S = R_3$. As shown above, $R_2 \cup R_3$ is not even an equivalence relation, hence not a right congruence.

2.40. **Analysis of $\Gamma = \{\lambda, \mathbf{a}, \mathbf{ab}, \mathbf{ba}, \mathbf{bb}, \mathbf{bbb}\} \cup \{x : |x| \geq 4\}$.**

(a) $\mathbf{ab}\, R_\Gamma\, \mathbf{ba}$: For all z: $\mathbf{ab}z \in \Gamma \Leftrightarrow \mathbf{ba}z \in \Gamma$. If $|z| = 0$: $\mathbf{ab}, \mathbf{ba} \in \Gamma$ (both listed). ✓If $|z| = 1$: $|\mathbf{ab}z| = 3$. Since Γ contains only **bbb** at length 3, and **ab** cannot be extended to **bbb** by appending one letter, $\mathbf{ab}z \notin \Gamma$; likewise $\mathbf{ba}z \notin \Gamma$ (since $\mathbf{baa}, \mathbf{bab} \neq \mathbf{bbb}$). Both outside Γ. ✓If $|z| \geq 2$: $|\mathbf{ab}z|, |\mathbf{ba}z| \geq 4$, so both in Γ. ✓Hence $\mathbf{ab}\, R_\Gamma\, \mathbf{ba}$.

(b) **ab** is not related to **bb**: take $z = \mathbf{b}$. Then

$$\mathbf{ab}z = \mathbf{abb} \notin \Gamma$$

because among length-3 strings only **bbb** is in Γ. But

$$\mathbf{bb}z = \mathbf{bbb} \in \Gamma.$$

So $\mathbf{ab}\ \not R_\Gamma\ \mathbf{bb}$. □

(c) The 8 classes are exactly:

$$C_0 = \{\lambda\},\ C_1 = \{\mathbf{a}\},\ C_2 = \{\mathbf{b}\},\ C_3 = \{\mathbf{aa}\},\ C_4 = \{\mathbf{bb}\},$$

$$C_5 = \{\mathbf{ab}, \mathbf{ba}\},\ C_6 = \{x : |x| = 3,\ x \neq \mathbf{bbb}\},\ C_7 = \{\mathbf{bbb}\} \cup \{x : |x| \geq 4\}.$$

These are pairwise distinct by suitable suffixes (for instance, C_4 vs C_5 by suffix **b**: $\mathbf{bbb} = \mathbf{bbb} \in \Gamma$ but $\mathbf{abb} = \mathbf{abb} \notin \Gamma$).

(d) Minimal DFA has these 8 states, start C_0, final states

$$F = \{C_0, C_1, C_4, C_5, C_7\}.$$

Transition function:

$$\delta(C_0, \mathbf{a}) = C_1,\ \delta(C_0, \mathbf{b}) = C_2,$$

$$\delta(C_1, \mathbf{a}) = C_3,\ \delta(C_1, \mathbf{b}) = C_5,$$

$$\delta(C_2, \mathbf{a}) = C_5,\ \delta(C_2, \mathbf{b}) = C_4,$$

$$\delta(C_3, \mathbf{a}) = C_6,\ \delta(C_3, \mathbf{b}) = C_6,$$

$$\delta(C_4, \mathbf{a}) = C_6,\ \delta(C_4, \mathbf{b}) = C_7,$$

$$\delta(C_5, \mathbf{a}) = C_6,\ \delta(C_5, \mathbf{b}) = C_6,$$

$$\delta(C_6, \mathbf{a}) = C_7,\ \delta(C_6, \mathbf{b}) = C_7,\ \delta(C_7, \mathbf{a}) = C_7,\ \delta(C_7, \mathbf{b}) = C_7.$$

2.41. **Prove R^M is a right congruence.**

R^M: $x\, R^M\, y \Leftrightarrow \overline{\delta}(s_0, x) = \overline{\delta}(s_0, y)$.

Since this relation is defined by equality of destination state, it is immediately an equivalence relation: reflexive, symmetric, and transitive all follow from equality.

To check the right-congruence property, assume $x\, R^M\, y$, so $\overline{\delta}(s_0, x) = \overline{\delta}(s_0, y) = s$. Then for any $z \in \Sigma^*$,

$$\overline{\delta}(s_0, xz) = \overline{\delta}(s, z) = \overline{\delta}(s_0, yz),$$

so $xz\, R^M\, yz$. Therefore R^M is a right congruence. □

2.42. **Is Pascal (set of all valid Pascal programs) FAD?**

No. We use a Chapter 2 distinct-futures argument: matching nested `begin`/`end` blocks requires arbitrarily many different continuations.

For each $n \geq 1$, let x_n be the ASCII string

```
program p; begin begin ...  begin
```

with exactly n occurrences of the keyword `begin`, and let z_n be the ASCII string

```
end end ...  end.
```

with exactly n occurrences of the keyword `end`.

Then $x_n z_n$ is a valid Pascal program: after the header `program p;`, the body is a nested compound statement with n occurrences of `begin` followed by the matching n occurrences of `end`, and then the final period.

If $m \neq n$, then $x_m z_n$ is not a valid Pascal program. If $m < n$, there are too many `end`s; if $m > n$, there are unmatched `begin`s left over before the final period.

Therefore x_n and x_m have different futures with respect to the language Pascal, since

$$x_n z_n \in \text{Pascal} \qquad \text{but} \qquad x_m z_n \notin \text{Pascal} \ \ (m \neq n).$$

So the strings $x_1, x_2, x_3, \ldots$ lie in distinct R_{Pascal}-classes. Hence R_{Pascal} has infinitely many equivalence classes, and Pascal is not FAD.

2.43. **Is Short Pascal (programs with $< 10^6$ characters) FAD?**

Yes. Short Pascal is a finite language (finitely many programs of length $< 10^6$ over the ASCII alphabet). Every finite language is FAD. The DFA would have a huge but finite number of states (roughly exponential in 10^6, but finite).

2.44. $L = \{\mathbf{a}^n \mathbf{b}^k \mathbf{c}^j \mid n < 3$ **or** $(n \geq 3$ **and** $k = j)\}$**.**

(a) **Hypothesis false, conclusion true.**
First, the hypothesis "L is FAD" is false. Intersect with the regular language $R = \mathbf{a}^3\mathbf{b}^*\mathbf{c}^*$:

$$L \cap R = \{\mathbf{a}^3 \mathbf{b}^k \mathbf{c}^k \mid k \geq 0\},$$

which is not regular (standard $\mathbf{b}^k\mathbf{c}^k$ argument). Hence L is not FAD.
Now show the pumping *conclusion* of Theorem 2.3 does hold for L. Take pumping length $N = 3$. Let

$$x = \mathbf{a}^n \mathbf{b}^k \mathbf{c}^j \in L, \quad |x| \geq 3.$$

We choose a decomposition $x = uvw$ with $|uv| \leq 3$, $|v| \geq 1$, and show $uv^m w \in L$ for all $m \geq 0$.
Case 1: $n \geq 3$ (so $k = j$). Choose

$$u = \lambda, \ v = \mathbf{a}, \ w = \mathbf{a}^{n-1}\mathbf{b}^k\mathbf{c}^k.$$

Then $uv^m w = \mathbf{a}^{n-1+m}\mathbf{b}^k\mathbf{c}^k$. If $n - 1 + m < 3$, it is in L by the $n < 3$ clause. If $n - 1 + m \geq 3$, it is in L because $k = j$ still holds.

Case 2: $n < 3$. Since $|x| \geq 3$, there is at least one non-**a** symbol, and its position is at most 3. Let v be that first non-**a** symbol (either **b** or **c**), with u the preceding prefix. Then $|uv| \leq 3$, $|v| = 1$. Pumping changes only the number of **b**'s or **c**'s, not the number of **a**'s, so $n < 3$ remains true for all m. Therefore $uv^m w \in L$ for all $m \geq 0$.

So the pumping conclusion holds even though L is not FAD.

(b) The contrapositive of Theorem 2.3: "if pumping fails then L is not FAD." This is trivially true (it's logically equivalent to Theorem 2.3). But here pumping doesn't fail, so the contrapositive doesn't give us information. The converse ("if pumping holds then L is FAD") is *false*, as this example shows.

2.45. **Show F_Q in Definition 2.5 is well defined.**

$F_Q = \{[x]_Q \mid x \in L\}$.

First, this is defined everywhere it needs to be: if $x \in L$, then certainly $x \in \Sigma^*$, so its equivalence class $[x]_Q$ exists and is an element of

$$S_Q = \{[y]_Q \mid y \in \Sigma^*\}.$$

Second, it is not multiply defined. Suppose $[x]_Q = [y]_Q$ and $x \in L$. Then x and y lie in the same Q-class. Since L is assumed to be a union of Q-classes, the whole class $[x]_Q = [y]_Q$ is either contained in L or disjoint from L. Because $x \in L$, that class must be contained in L, so $y \in L$ as well.

Thus the choice of representative does not matter, and F_Q is a well-defined subset of S_Q. □

2.46. **Show δ_Q in Definition 2.5 is well defined.**

$\delta_Q([x]_Q, a) = [xa]_Q$.

First, it is defined for every state and input letter: if $[x]_Q \in S_Q$ and $a \in \Sigma$, then $xa \in \Sigma^*$, so $[xa]_Q \in S_Q$.

Second, it is independent of the chosen representative of the class. If $[x]_Q = [y]_Q$, then $x\,Q\,y$. Since Q is a right congruence, $xa\,Q\,ya$, and therefore

$$[xa]_Q = [ya]_Q.$$

So δ_Q is well defined. □

2.47. **Prove $\overline{\delta}_Q([x]_Q, y) = [xy]_Q$ by induction.**

Base: $\overline{\delta}_Q([x]_Q, \lambda) = [x]_Q = [x\lambda]_Q$. ✓

Step: $y = wa$. $\overline{\delta}_Q([x]_Q, wa) = \delta_Q(\overline{\delta}_Q([x]_Q, w), a) = \delta_Q([xw]_Q, a) = [xwa]_Q = [xy]_Q$. □

2.48. **Prove $L(A_Q) = L$.**

$x \in L(A_Q) \Leftrightarrow \overline{\delta}_Q([\lambda]_Q, x) \in F_Q \Leftrightarrow [x]_Q \in F_Q \Leftrightarrow x \in L$ (by definition of F_Q). □

2.49. **Prove or disprove $Q = R^{A_Q}$.**

R^{A_Q}: $x\,R^{A_Q}\,y \Leftrightarrow \overline{\delta}_Q([\lambda], x) = \overline{\delta}_Q([\lambda], y) \Leftrightarrow [x]_Q = [y]_Q \Leftrightarrow x\,Q\,y$. So $R^{A_Q} = Q$. □

2.50. **Prove $R_L = R^{A_{R_L}}$.**

By Exercise 2.43 applied with $Q = R_L$: $R^{A_{R_L}} = R_L$. □

2.51. **Show the converse of Theorem 2.3 is false.**

The converse would say: if a language satisfies the conclusion of Theorem 2.3, then it must be FAD. The immediately preceding exercise already provides a counterexample:

$$L = \{\mathbf{a}^n\mathbf{b}^k\mathbf{c}^j \mid n < 3 \text{ or } (n \geq 3 \text{ and } k = j)\}.$$

Part (a) showed that the pumping conclusion of Theorem 2.3 does hold for this language, while the hypothesis fails because L is not FAD.

So a language can satisfy the pumping conclusion without being FAD. Therefore the converse of Theorem 2.3 is false. □

2.52. **Show $L = \{x : |x|_{\mathbf{a}} = 2|x|_{\mathbf{b}}\}$ is not FAD.**

For any n: strings $\mathbf{a}^0, \mathbf{a}^2, \mathbf{a}^4, \ldots, \mathbf{a}^{2n}$ are in distinct R_L-classes. Indeed: $\mathbf{a}^{2i}\, R_L\, \mathbf{a}^{2j}$ requires for all z: $\mathbf{a}^{2i}z \in L \Leftrightarrow \mathbf{a}^{2j}z \in L$. Take $z = \mathbf{b}^i$: $\mathbf{a}^{2i}\mathbf{b}^i \in L$ (yes). $\mathbf{a}^{2j}\mathbf{b}^i \in L$ iff $2j = 2i$, i.e., $j = i$. So distinct $i \neq j$ give distinct classes. □

2.53. **K palindromes: $[\mathbf{110}]_{R_K}$ and description of R_K.**

Let $K = \{w \mid w = w^r\}$.

To describe R_K, suppose $x\, R_K\, y$. Take suffixes

$$z_0 = \mathbf{0}x^r, \qquad z_1 = \mathbf{1}x^r.$$

Both $xz_0 = x\mathbf{0}x^r$ and $xz_1 = x\mathbf{1}x^r$ are palindromes, so by $x\, R_K\, y$, both yz_0 and yz_1 must be palindromes.

Now:

- If a palindrome ends with $\mathbf{0}x^r$, it begins with $x\mathbf{0}$.
- If a palindrome ends with $\mathbf{1}x^r$, it begins with $x\mathbf{1}$.

Hence if $|y| \geq |x| + 1$, the first $|x| + 1$ symbols of y would have to be both $x\mathbf{0}$ and $x\mathbf{1}$, impossible. So $|y| \leq |x|$. By symmetry (since equivalence relations are symmetric), also $|x| \leq |y|$. Therefore $|x| = |y|$. With equal lengths, from yz_0 palindrome we get that the first $|x|$ symbols of yz_0 equal x; those first $|x|$ symbols are exactly y. So $y = x$.

Thus $x\, R_K\, y \Rightarrow x = y$, and the converse is trivial. Therefore R_K is the identity relation on $\{\mathbf{0}, \mathbf{1}\}^*$.

So

$$[\mathbf{110}]_{R_K} = \{\mathbf{110}\}.$$

2.54. **Prove or disprove: $R_{L_1} \cap R_{L_2} = R_{L_1 \cap L_2}$.**

This is false in general.

However, $R_{L_1} \cap R_{L_2}$ always refines $R_{L_1 \cap L_2}$. Indeed, if $x\, R_{L_1}\, y$ and $x\, R_{L_2}\, y$, then for every z,

$$xz \in L_1 \cap L_2 \Leftrightarrow (xz \in L_1 \text{ and } xz \in L_2) \Leftrightarrow (yz \in L_1 \text{ and } yz \in L_2) \Leftrightarrow yz \in L_1 \cap L_2,$$

so $x\, R_{L_1 \cap L_2}\, y$.

Sometimes equality does happen. For example, over $\{\mathbf{a},\mathbf{b}\}$ let

$$L_1 = \{x : |x| \text{ even}\}, \qquad L_2 = \{x : |x|_{\mathbf{a}} \text{ even}\}.$$

Then both $R_{L_1} \cap R_{L_2}$ and $R_{L_1 \cap L_2}$ have the same four classes, determined by the parity of $|x|$ and the parity of $|x|_{\mathbf{a}}$.

But in general the equality can fail. Let $L_1 = \{\lambda\}$ and $L_2 = \{\mathbf{a}\}$ over $\{\mathbf{a}\}^*$. Then R_{L_1} has 2 classes, R_{L_2} has 3 classes, and so $R_{L_1} \cap R_{L_2}$ has 3 classes. On the other hand,

$$L_1 \cap L_2 = \emptyset,$$

so $R_{L_1 \cap L_2} = R_\emptyset$ has just one class, namely all of Σ^*.

Therefore $R_{L_1} \cap R_{L_2} = R_{L_1 \cap L_2}$ is false in general. □

2.55. **Prove: each R-class has a representative of length $< n$.**

R is a right congruence of rank n. Consider the DFA A_R with n states. By the pigeonhole principle, the BFS tree of a DFA with n states reaches every state by a path of length at most $n-1$: any path of length $\geq n$ must revisit a state. Hence every reachable state has a witness string of length $\leq n-1 < n$, so every R-class has a representative of length $< n$. □

2.56. **Find all L with $R_L = R^N$ for R^N from Example 2.6.**

Let the four R^N-classes from Example 2.6 be

$$A_0 = [\lambda], \quad A_1 = [\mathbf{1}], \quad A_2 = [\mathbf{11}], \quad A_3 = [\mathbf{000}].$$

Not every union of these classes preserves $R_L = R^N$; many choices collapse states. The unions that keep all four classes pairwise Nerode-distinct are exactly:

$$L = A_0 \cup A_1, \quad L = A_1 \cup A_2, \quad L = A_0 \cup A_3, \quad L = A_2 \cup A_3.$$

These are precisely the languages with $R_L = R^N$.

2.57. **Right congruences for languages of Exercise 1.6.**

In Exercise 1.6(a), let

$$p(x) = \big(|x|_{\mathbf{a}} \bmod 2,\ |x|_{\mathbf{b}} \bmod 2\big) \in \{0,1\}^2.$$

Appending a suffix adds parities mod 2, so each R_L is determined by which parity information must be remembered.

(a) $L_1 = \{x : (|x|_{\mathbf{a}} \text{ odd}) \wedge (|x|_{\mathbf{b}} \text{ even})\}$.

$$x \, R_{L_1} \, y \iff p(x) = p(y).$$

So there are 4 classes (one per parity pair).

(b) $L_2 = \{x : (|x|_{\mathbf{a}} \text{ even}) \vee (|x|_{\mathbf{b}} \text{ odd})\}$. Again all 4 parity pairs are distinguishable, so

$$x \, R_{L_2} \, y \iff p(x) = p(y).$$

(c) $L_3 = \{x : (|x|_{\mathbf{a}} \text{ even}) \overline{\vee} (|x|_{\mathbf{b}} \text{ even})\}$. This depends only on whether the two parities are equal or different:

$$x R_{L_3} y \iff \big(|x|_{\mathbf{a}} + |x|_{\mathbf{b}} \bmod 2\big) = \big(|y|_{\mathbf{a}} + |y|_{\mathbf{b}} \bmod 2\big).$$

So there are 2 classes: "same parity" and "different parity".

(d) $L_4 = \{x : |x|_{\mathbf{a}} \text{ even}\}$. Only parity of $\mathbf{a}$ matters:

$$x R_{L_4} y \iff |x|_{\mathbf{a}} \equiv |y|_{\mathbf{a}} \pmod 2.$$

So there are 2 classes.

2.58. $L \subseteq \{\mathbf{a}\}^*$**,** $\lambda R_L \mathbf{a}$**.**

$\lambda R_L \mathbf{a}$ means: for all $z \in \{\mathbf{a}\}^*$, $z \in L \Leftrightarrow \mathbf{a}z \in L$. This means L is closed under left-prepending $\mathbf{a}$ and left-removing $\mathbf{a}$ (when applicable). Combined, $\mathbf{a}^n \in L \Leftrightarrow \mathbf{a}^{n+1} \in L$ for all n. So either $L = \emptyset$ or $L = \{\mathbf{a}\}^*$.

2.59. $\mathbf{a} R_L \mathbf{aa}$ **over** $\{\mathbf{a}\}^*$**.**

$\mathbf{a} R_L \mathbf{aa}$: $(\forall z)(\mathbf{a}z \in L \Leftrightarrow \mathbf{aa}z \in L)$. So $\mathbf{a}^{n+1} \in L \Leftrightarrow \mathbf{a}^{n+2} \in L$. This means membership in L is the same at all lengths ≥ 1 except possibly at length 0 (where λ may or may not be in L). Also $\mathbf{a} R_L \mathbf{aa}$ means $[\mathbf{a}] = [\mathbf{aa}]$, so $L \cap \{\mathbf{a}^n : n \geq 1\}$ is either $\emptyset$ or $\{\mathbf{a}^n : n \geq 1\}$. With the possible inclusion of λ: 4 possibilities: $\emptyset$, $\{\lambda\}$, $\{\mathbf{a}^n : n \geq 1\}$, $\{\mathbf{a}\}^*$.

2.60. $\lambda R_L \mathbf{aa}$ **over** $\{\mathbf{a}\}^*$**.**

$(\forall z)(z \in L \Leftrightarrow \mathbf{aa}z \in L)$. So $\mathbf{a}^n \in L \Leftrightarrow \mathbf{a}^{n+2} \in L$: membership depends on parity of length. Possibilities: $\emptyset$, $\{\mathbf{a}^{2k} : k \geq 0\}$ (even lengths), $\{\mathbf{a}^{2k+1} : k \geq 0\}$ (odd lengths), $\{\mathbf{a}\}^*$.

2.61. **Examples with** $R^M = R_{L(M)}$ **or** $R^M \neq R_{L(M)}$**.**

(a) $R^M = R_{L(M)}$: any minimal DFA. E.g., the 2-state DFA for $\{\mathbf{a}^{2k} : k \geq 0\}$.

(b) $R^M \neq R_{L(M)}$: any non-minimal DFA. E.g., a 3-state DFA for the same language, where two states are equivalent. R^M has 3 classes (one per state) but $R_{L(M)}$ has 2.

(c) For every DFA M, R^M refines $R_{L(M)}$: if $\overline{\delta}(s_0, x) = \overline{\delta}(s_0, y) = s$, then for any z: $xz \in L(M) \Leftrightarrow \overline{\delta}(s_0, xz) \in F \Leftrightarrow \overline{\delta}(s, z) \in F \Leftrightarrow \overline{\delta}(s_0, yz) \in F \Leftrightarrow yz \in L(M)$.

2.62. **Find** R^M **and** $R_{L(M)}$ **for machine of Example 1.5.**

In Example 1.5 (vending machine), states track deposited amount in

$$T = \{0, 5, 10, 15, 20, 25, 30, 35, 40, 45, 50\}$$

(capped at 50), and final states are amounts ≥ 30.

R^M**.**

$$x R^M y \iff \overline{\delta}(s_0, x) = \overline{\delta}(s_0, y).$$

So R^M has one class for each amount in T (11 classes).

$R_{L(M)}$**.** All classes for amounts < 30 remain distinct, while all amounts ≥ 30 merge into one Nerode class.

Why amounts < 30 are distinct: if $t_1 < t_2 < 30$, append coins totaling $30 - t_2$ (possible with $\mathbf{n}, \mathbf{d}, \mathbf{q}$); then from t_2 the result is accepted (reaches 30), while from t_1 it is still < 30 and rejected.

Why amounts ≥ 30 are equivalent: once at least 30 cents has been deposited, any further input keeps the machine in accepting territory.

Hence $R_{L(M)}$ has 7 classes:

$$[0], [5], [10], [15], [20], [25], \ [\geq 30].$$

So R^M is strictly finer than $R_{L(M)}$.

2.63. **Is $A_{R_{L(A)}}$ always equivalent to A?**

Yes. $A_{R_{L(A)}}$ is the minimal DFA for $L(A)$, hence $L(A_{R_{L(A)}}) = L(A)$. So they are always equivalent (accept the same language). They need not be isomorphic (if A is not minimal).

2.64. **$L_4 = \{y : |y|_{\mathbf{0}} = |y|_{\mathbf{1}}\}$: does the example u, v, w show L_4 is FAD?**

No. The pumping lemma says: for each sufficiently long word in a regular language, there exists *some* decomposition that pumps. Finding one convenient decomposition for one word does not prove regularity. To prove non-regularity, we choose one long word and show that *every* allowed decomposition fails. For L_4, a standard choice is $y = \mathbf{0}^n\mathbf{1}^n$: any v with $|uv| \leq n$ lies in the $\mathbf{0}$-block, and pumping changes the $\mathbf{0}/\mathbf{1}$ balance, leaving L_4.

2.65. **Find R_{L_4}.**

$L_4 = \{y : |y|_{\mathbf{0}} = |y|_{\mathbf{1}}\}$. R_{L_4} is determined by: $x\,R_{L_4}\,y \Leftrightarrow |x|_{\mathbf{0}} - |x|_{\mathbf{1}} = |y|_{\mathbf{0}} - |y|_{\mathbf{1}}$ (the "balance" of 0s over 1s must be equal). This gives infinitely many classes (one per integer balance $\in \mathbb{Z}$), confirming L_4 is not FAD.

2.66. **Show postfix expressions over $\{\mathbf{A}, \mathbf{B}, +, -\}$ are not FAD.**

For any n: $\mathbf{A}^n$ (n copies of A) is a prefix of postfix expressions. The strings $\mathbf{A}^i$ for $i = 0, 1, \ldots, n$ are in distinct R_L-classes: $\mathbf{A}^i$ needs i more operands consumed (by operators) to complete a valid postfix expression. Specifically: $\mathbf{A}^i z \in L$ iff z is a sequence of $i-1$ binary operators (plus the right structure). So the future of $\mathbf{A}^i$ differs from $\mathbf{A}^j$ for $i \neq j$. Infinite rank. Not FAD. □

2.67. **Show parenthesized infix expressions are not FAD.**

Similar: $(^n$ is a prefix; each n gives a distinct future (requiring n closing parens eventually). Infinite rank. Not FAD. □

2.68. **R_L vs. $R_{\sim L}$.**

$R_L = R_{\sim L}$. Proof: $x\,R_L\,y \Leftrightarrow (\forall z)(xz \in L \Leftrightarrow yz \in L) \Leftrightarrow (\forall z)(xz \notin L \Leftrightarrow yz \notin L) \Leftrightarrow (\forall z)(xz \in \sim L \Leftrightarrow yz \in \sim L) \Leftrightarrow x\,R_{\sim L}\,y$. □

2.69. **Show no L has $R_L = Q$ where Q has classes $\{\lambda\}, \{\mathbf{a}\}, \{\mathbf{b}\}, \{x : |x| \geq 2\}$.**

Q is a right congruence: every class maps consistently under each letter (e.g. $\{\mathbf{a}\} \cdot \sigma$ and $\{\mathbf{b}\} \cdot \sigma$ both land in $\{|x| \geq 2\}$ for any σ). Nevertheless, no language L satisfies $R_L = Q$.

Proof. If $R_L = Q$, then $\mathbf{a}$ and $\mathbf{b}$ must lie in different R_L-classes. For $R_L = Q$ to hold, any candidate L must be a union of Q-classes. Consider $L = \{x : |x| \geq 2\}$. Then $\mathbf{a}\,R_L\,\mathbf{b}$: for all z, $\mathbf{a}z \in L \Leftrightarrow |\mathbf{a}z| \geq 2 \Leftrightarrow |z| \geq 1$, and $\mathbf{b}z \in L \Leftrightarrow |z| \geq 1$; the futures are identical. So R_L merges $\{\mathbf{a}\}$ and $\{\mathbf{b}\}$ into one class — contradicting Q. A case analysis over all unions of Q-classes shows the same merging occurs in every case. Hence $R_L \neq Q$ for any L. □

2.70. **Q with classes $\{\lambda, \mathbf{a}, \mathbf{aa}\}$ and $\{\mathbf{a}^3, \mathbf{a}^4, \ldots\}$ is not a right congruence.**

(a) $\mathbf{aa}\, Q\, \lambda$ (same class). Append $\mathbf{a}$: $\mathbf{aaa} \in$ second class, but $\lambda \cdot \mathbf{a} = \mathbf{a} \in$ first class. So $\mathbf{aaa} \not Q\, \mathbf{a}$, violating right congruence.

(b) Building A_Q: states are the two Q-classes. $\delta([\lambda]_Q, \mathbf{a}) = [?]$. For λ: $\lambda \cdot \mathbf{a} = \mathbf{a} \in [\lambda]_Q$. For $\mathbf{a}$: $\mathbf{a} \cdot \mathbf{a} = \mathbf{aa} \in [\lambda]_Q$. For $\mathbf{aa}$: $\mathbf{aa} \cdot \mathbf{a} = \mathbf{aaa} \in [\mathbf{a}^3]_Q$. So from the same class $[\lambda]_Q$, different elements lead to different successor classes. δ is not well-defined. This is the failure.

(c) Part (a) says Q is not a right congruence. Part (b) shows δ fails to be a function, which is precisely because right congruence is the condition needed for δ to be well-defined.

2.71. **Show $\{\mathbf{a}^i \mathbf{b}^j \mathbf{c}^k \mid i, j, k \in \mathbb{N}, i + j = k\}$ is not FAD.**

For any n: consider $\mathbf{a}^m$ for $m = 0, 1, \ldots, n$. $\mathbf{a}^m R_L \mathbf{a}^{m'}$: need $(\forall z)\mathbf{a}^m z \in L \Leftrightarrow \mathbf{a}^{m'} z \in L$. Take $z = \mathbf{b}^0 \mathbf{c}^m = \mathbf{c}^m$: $\mathbf{a}^m \mathbf{c}^m \in L \Leftrightarrow m + 0 = m$ (yes). $\mathbf{a}^{m'} \mathbf{c}^m \in L \Leftrightarrow m' + 0 = m \Leftrightarrow m' = m$. So for $m \neq m'$, $\mathbf{a}^m \not R_L \mathbf{a}^{m'}$. Infinite rank. Not FAD. □

2.72. **Show Q with classes $[\lambda] = \{\lambda\}$, $[\mathbf{a}] = \{\mathbf{a}\}$, $[\mathbf{aa}] = \{\mathbf{a}\}^{\geq 2}$ is a right congruence on $\{\mathbf{a}\}^*$.**

Case $x, y \in [\lambda]$: $x = y = \lambda$, $xa = ya = \mathbf{a} \in [\mathbf{a}]$.

Case $x, y \in [\mathbf{a}]$: $x = y = \mathbf{a}$, $xa = ya = \mathbf{aa} \in [\mathbf{aa}]$.

Case $x, y \in [\mathbf{aa}]$: $|x|, |y| \geq 2$, so $|xa|, |ya| \geq 3 \geq 2$, thus $xa, ya \in [\mathbf{aa}]$.

In each case $xa\, Q\, ya$. □

2.73. **R_2 on $\{\mathbf{a}, \mathbf{b}, \mathbf{c}\}^*$: same last letter.**

(a) Equivalence: Reflexive (x ends with same letter as x). Symmetric (same). Transitive (same). But for λ: λ has no last letter. By convention, λ is in its own class (or we restrict to Σ^+). Assuming λ is its own class: $\lambda\, R_2\, \lambda$, and for $x, y \in \Sigma^+$, $x\, R_2\, y \Leftrightarrow \text{last}(x) = \text{last}(y)$.

(b) Right congruence: If $x\, R_2\, y$ (same last letter, say c), then for any $z \in \Sigma^+$: $\text{last}(xz) = \text{last}(z) = \text{last}(yz)$, so $xz\, R_2\, yz$. For $z = \lambda$: $x\, R_2\, y$. □

2.74. **R_3 on $\{\mathbf{a}, \mathbf{b}, \mathbf{c}\}^*$: same first letter.**

(a) Equivalence: reflexive, symmetric, transitive by same argument (λ its own class).

(b) Right congruence: Suppose $x\, R_3\, y$, i.e., x and y have the same first letter (or both are λ). For any $z \in \Sigma^*$: if $z = \lambda$ then $xz = x$ and $yz = y$, so $xz\, R_3\, yz$ trivially. If $z \neq \lambda$ then $\text{first}(xz) = \text{first}(x) = \text{first}(y) = \text{first}(yz)$, so $xz\, R_3\, yz$. Hence R_3 is a right congruence. □

2.75. **Which of $L_1, \ldots, L_5$ are FAD?**

(a) L_1 = last letter matches first letter: FAD. Use 5 states: q_ε (no input yet), and $(a, a), (a, b), (b, a), (b, b)$ where the first component is the first symbol seen and the second is the current last symbol. Final states are (a, a) and (b, b) (and include/exclude q_ε depending on the convention for λ).

(b) L_2 = odd-length words whose first letter equals the center letter: Not FAD. Intersect with the regular language $R = \mathbf{ab}^*\mathbf{ab}^*$. Then

$$L_2 \cap R = \{\mathbf{ab}^n\mathbf{ab}^{n+1} \mid n \geq 0\}.$$

(In $\mathbf{ab}^i\mathbf{ab}^j$, being odd-length with center letter **a** forces $j = i+1$.) This language is not regular by pumping: for pumping length p, take

$$w = \mathbf{ab}^p\mathbf{ab}^{p+1}.$$

Any split $w = uvw'$ with $|uv| \le p+1$, $|v| \ge 1$ has v in the initial $\mathbf{ab}^p$ segment. If v includes the first **a**, pumping down removes that initial form. If $v = \mathbf{b}^t$ in the first **b**-block, pumping down gives

$$\mathbf{ab}^{p-t}\mathbf{ab}^{p+1},$$

which no longer has second block length = first block length + 1. So $L_2 \cap R$ is not regular, hence L_2 is not FAD.

(c) L_3 = last letter does not appear in any earlier position: **FAD**. The DFA tracks (i) the set of characters seen in positions $1,\ldots,n-1$ and (ii) the current last character. Since Σ is fixed and finite, there are at most $2^{|\Sigma|} \times |\Sigma|$ states (plus a start state for λ). Accept when the last character is not in the set of characters seen before it. Both components update in one step: reading a sets last-char = a and adds the previous last-char to the "seen" set. This is a finite machine.

(d) L_4 = even-length, two center letters match: Not FAD. Similar to L_2.

(e) L_5 = odd-length, center letter matches none of the others: Not FAD. Similar.

2.76. **Complete proof of case 1 in (2)⇒(3) of Nerode's theorem.**

(a) Case 1: $x = \lambda$. We need $\lambda\, R_L\, y$ (for some y with $\overline{\delta}(s_0, y) = \overline{\delta}(s_0, \lambda) = s_0$). Since $\overline{\delta}(s_0, \lambda) = s_0 = \overline{\delta}(s_0, y)$, for all z: $\overline{\delta}(s_0, \lambda z) = \overline{\delta}(s_0, z) = \overline{\delta}(s_0, yz)$ (using $\overline{\delta}(s_0, y) = s_0$ and the homomorphism property of $\overline{\delta}$). So $\lambda z \in L \Leftrightarrow yz \in L$, i.e., $\lambda\, R_L\, y$.

(b) Case 1 *can* be included under case 2 if we allow x to be a prefix of y with $|x| = 0$: the proof of case 2 (induction on $|x|$) can start at $|x| = 0$ (the λ case), so yes.

2.77. **Right congruence property can use ⇔ instead of ⇒.**

The original right congruence (RC): $xRy \Rightarrow xuRyu$.

Claim: $xRy \Leftrightarrow xuRyu$ (for all u).

The ⇒ direction is given. For ⇐: assume $xuRyu$ for all u; take $u = \lambda$: $x\lambda\, R\, y\lambda$, i.e., xRy (since $x\lambda = x$ and $y\lambda = y$). □

2.78. **Show $\overline{\delta}(s, uv^i) = q$ by induction.**

Given $\overline{\delta}(s, u) = q$ and $\overline{\delta}(q, v) = q$.

Base ($i = 0$): $\overline{\delta}(s, uv^0) = \overline{\delta}(s, u) = q$. ✓

Step: $\overline{\delta}(s, uv^{i+1}) = \overline{\delta}(\overline{\delta}(s, uv^i), v) = \overline{\delta}(q, v) = q$ (by IH). □

2.79. **L = strings agreeing with $\mathbf{0}^1\mathbf{1}0^2\mathbf{1}0^3\mathbf{1}\cdots$ is not FAD.**

The strings in L are all prefixes of this infinite sequence. Consider prefix $\mathbf{0}^1\mathbf{1}0^2\mathbf{1}\cdots\mathbf{0}^n\mathbf{1}$ (length $n(n+3)/2$) and the prefix $\mathbf{0}^1\mathbf{1}0^2\mathbf{1}\cdots\mathbf{0}^n$ (length $n(n+3)/2 - n = n(n+1)/2$). The futures of these two strings are different (one expects a **1** next, the other expects $n+1$ zeros followed by **1**). So different-length prefixes that cut at different points in the pattern have different futures. An unbounded number of distinct futures exist. Not FAD. □

2.80. **Show the "simple" BNF language is not FAD.**

⟨simple⟩ ::= **a** | (⟨simple⟩). The language is $L = \{({}^n\mathbf{a})^n \mid n \geq 0\}$. By Myhill-Nerode: the strings ${}^0, ({}^1, \ldots, ({}^n$ are pairwise in distinct R_L-classes. The distinguishing suffix for $({}^j$ vs. $({}^k$ (with $j \neq k$) is $z = \mathbf{a})^j$: $({}^j\mathbf{a})^j \in L$ but $({}^k\mathbf{a})^j \notin L$ (since $k \neq j$). Hence R_L has infinite rank and L is not FAD. □

2.81. L_2 **(binary powers of 2) is FAD;** L_{10} **(decimal powers of 2) is not.**

(a) $L_2 = \{\mathbf{1}, \mathbf{10}, \mathbf{100}, \mathbf{1000}, \ldots\}$: strings that are **1** followed by any number of **0**s. Regular expression: $\mathbf{10}^*$. FAD (2-state DFA: $q_0 \xrightarrow{\mathbf{1}} q_1 \xrightarrow{\mathbf{0}} q_1$, $F = \{q_1\}$, dead state for other transitions).

(b) $L_{10} = \{1, 2, 4, 8, 16, 32, \ldots\}$ (decimal powers of 2) is not FAD. *Proof (by pumping lemma and modular arithmetic).* Assume L_{10} is FAD with pumping length p. Choose n so large that if w is the decimal representation of 2^n, then

$$|w| = d > p + 2$$

and

$$4 \cdot 5^{d-p-1} > (p+1)\log_2 10.$$

Write $w = xyz$ with $|xy| \leq p$, $|y| = t \geq 1$, and let $|z| = r$. Then $r = d - |xy| \geq d - p$.
By pumping, $xy^2z \in L_{10}$, so for some $m > n$,

$$\mathrm{val}(xy^2z) = 2^m, \qquad \mathrm{val}(xyz) = 2^n.$$

Because pumping duplicates a block before the final z, both numbers end with the same r digits z. Hence

$$2^m \equiv 2^n \pmod{10^r},$$

so

$$2^{m-n} \equiv 1 \pmod{5^r}. \tag{*}$$

Lemma. If $2^q \equiv 1 \pmod{5^r}$, then $4 \cdot 5^{r-1} \mid q$.
Why: the order of 2 modulo 5^r is $4 \cdot 5^{r-1}$. Base $r = 1$: order mod 5 is 4. Induction step: if the order mod 5^r is $4 \cdot 5^{r-1}$, then

$$2^{4\cdot 5^{r-1}} = 1 + u5^r, \quad 5 \nmid u,$$

so

$$2^{4\cdot 5^r} = (1 + u5^r)^5 \equiv 1 + u5^{r+1} \not\equiv 1 \pmod{5^{r+2}},$$

which shows the order at the next level is multiplied by 5. Thus $\mathrm{order}(2 \bmod 5^r) = 4 \cdot 5^{r-1}$ for all $r \geq 1$.
Applying the lemma to $(*)$ gives

$$m - n \geq 4 \cdot 5^{r-1} \geq 4 \cdot 5^{d-p-1}. \tag{1}$$

Now get an upper bound for $m - n$ from decimal length. The pumped string xy^2z has length $d + t \leq d + p$, so

$$2^m = \mathrm{val}(xy^2z) < 10^{d+t} \leq 10^{d+p}.$$

Also $2^n = \mathrm{val}(w) \geq 10^{d-1}$. Therefore

$$2^{m-n} < 10^{p+1},$$

so

$$m - n < (p+1)\log_2 10. \tag{2}$$

(1) and (2) contradict our choice of n. Hence L_{10} is not FAD. □

Chapter 3

Minimization — Solutions

3.1. **Prove $\mu(\overline{\delta}(s,x)) = \overline{\delta}_{R_L}(\mu(s),x)$ by induction for the mapping μ in Theorem 3.1.**

$\mu(s) = [s]_{R^M}$ (the R^M-class of state s, identified with an R_L-class via the homomorphism). Let $P(n)$: $(\forall s \in S)(\forall x \in \Sigma^n)(\mu(\overline{\delta}(s,x)) = \overline{\delta}_{R_L}(\mu(s),x))$.

Base ($n=0$): $\mu(\overline{\delta}(s,\lambda)) = \mu(s) = \overline{\delta}_{R_L}(\mu(s),\lambda)$. ✓

Step: $x = ya$, $|y| = n$. Using the homomorphism property $\mu(\delta(t,a)) = \delta_{R_L}(\mu(t),a)$:

$$\mu(\overline{\delta}(s,ya)) = \mu(\delta(\overline{\delta}(s,y),a)) = \delta_{R_L}(\mu(\overline{\delta}(s,y)),a) = \delta_{R_L}(\overline{\delta}_{R_L}(\mu(s),y),a) = \overline{\delta}_{R_L}(\mu(s),ya). \quad \square$$

3.2. **Show $\overline{\delta}_{E_A}([s]_{E_A},x) = [\overline{\delta}(s,x)]_{E_A}$ by induction.**

Base ($x=\lambda$): $\overline{\delta}_{E_A}([s]_{E_A},\lambda) = [s]_{E_A} = [\overline{\delta}(s,\lambda)]_{E_A}$. ✓

Step: $x = ya$.

$$\overline{\delta}_{E_A}([s]_{E_A},ya) = \delta_{E_A}(\overline{\delta}_{E_A}([s]_{E_A},y),a) = \delta_{E_A}([\overline{\delta}(s,y)]_{E_A},a) = [\delta(\overline{\delta}(s,y),a)]_{E_A} = [\overline{\delta}(s,ya)]_{E_A}. \quad \square$$

3.3. **Prove $E_A = \bigcap_{j=0}^{\infty} E_{jA}$.**

Define E_{0A}: $s\,E_{0A}\,t \Leftrightarrow (s \in F \Leftrightarrow t \in F)$.

Define $E_{i+1,A}$: $s\,E_{i+1,A}\,t \Leftrightarrow s\,E_{iA}\,t \wedge (\forall a \in \Sigma)(\delta(s,a)\,E_{iA}\,\delta(t,a))$.

E_A (state equivalence): $s\,E_A\,t \Leftrightarrow L(A^s) = L(A^t)$ where A^s is A with start state s.

($\supseteq$): $E_A \subseteq E_{iA}$ for all i (by induction on i). $i=0$: $s\,E_A\,t \Rightarrow L(A^s) = L(A^t) \Rightarrow (\lambda \in L(A^s) \Leftrightarrow \lambda \in L(A^t)) \Rightarrow (s \in F \Leftrightarrow t \in F)$. Step: if $E_A \subseteq E_{iA}$ and $s\,E_A\,t$, then $\delta(s,a)\,E_A\,\delta(t,a)$ (since E_A is a right congruence on states – actually prove this: $L(A^{\delta(s,a)}) = \{x : ax \in L(A^s)\} = \{x : ax \in L(A^t)\} = L(A^{\delta(t,a)})$). So $\delta(s,a)\,E_A\,\delta(t,a)$, and by IH $\delta(s,a)\,E_{iA}\,\delta(t,a)$; also $s\,E_{iA}\,t$ by IH. Hence $s\,E_{i+1,A}\,t$. So $E_A \subseteq \bigcap_j E_{jA}$.

($\subseteq$): Show $\bigcap_j E_{jA} \subseteq E_A$. Suppose $s\,E_{jA}\,t$ for all j. By induction on $|x|$: $\overline{\delta}(s,x)\,E_{0A}\,\overline{\delta}(t,x)$ for all x, which means $\overline{\delta}(s,x) \in F \Leftrightarrow \overline{\delta}(t,x) \in F$, i.e., $x \in L(A^s) \Leftrightarrow x \in L(A^t)$. Proof: $|x| = 0$: $s\,E_{0A}\,t$ (given). $|x| = n+1$, $x = ay$: $\overline{\delta}(s,ay) = \overline{\delta}(\delta(s,a),y)$. Since $s\,E_{jA}\,t$ for all j (given), in particular $s\,E_{n+1,A}\,t$, so $\delta(s,a)\,E_{nA}\,\delta(t,a)$ and $\delta(s,a)\,E_{jA}\,\delta(t,a)$ for all $j \geq 1$. By IH: $\overline{\delta}(\delta(s,a),y)\,E_{0A}\,\overline{\delta}(\delta(t,a),y)$. So $L(A^s) = L(A^t)$, i.e., $s\,E_A\,t$. $\square$

3.4. **Show δ_{E_A} in Definition 3.8 is well defined.**

$\delta_{E_A}([s]_{E_A},a) = [\delta(s,a)]_{E_A}$. If $[s]_{E_A} = [t]_{E_A}$, i.e., $s\,E_A\,t$, then $\delta(s,a)\,E_A\,\delta(t,a)$ (proved in Exercise 3.3, $\supseteq$ direction): $L(A^{\delta(s,a)}) = \{x : ax \in L(A^s)\} = \{x : ax \in L(A^t)\} = L(A^{\delta(t,a)})$. So $[\delta(s,a)]_{E_A} = [\delta(t,a)]_{E_A}$. $\square$

3.5. **Show F_{E_A} is well defined.**

$F_{E_A} = \{[s]_{E_A} \mid s \in F\}$. If $[s]_{E_A} = [t]_{E_A}$ and $s \in F$: $sE_A t \Rightarrow L(A^s) = L(A^t) \Rightarrow \lambda \in L(A^s) \Leftrightarrow \lambda \in L(A^t) \Rightarrow s \in F \Leftrightarrow t \in F$. So $t \in F$. □

3.6. **Show δ^c maps into S^c.**

S^c = connected states = $\{\overline{\delta}(s_0, x) \mid x \in \Sigma^*\}$. δ^c is δ restricted to S^c. For any $s \in S^c$ and $a \in \Sigma$: $s = \overline{\delta}(s_0, x)$ for some x, so $\delta(s, a) = \delta(\overline{\delta}(s_0, x), a) = \overline{\delta}(s_0, xa) \in S^c$. □

3.7. **Prove Lemma 3.1.**

Suppose $sE_{m+1A}t$. By definition, this means

$$(\forall x \in \Sigma^* \ni |x| \leq m+1)(\overline{\delta}(s, x) \in F \Leftrightarrow \overline{\delta}(t, x) \in F).$$

In particular, the same equivalence holds for every x with $|x| \leq m$. Hence

$$(\forall x \in \Sigma^* \ni |x| \leq m)(\overline{\delta}(s, x) \in F \Leftrightarrow \overline{\delta}(t, x) \in F),$$

which is exactly $sE_{mA}t$. Therefore $E_{m+1A} \subseteq E_{mA}$. □

3.8. **Prove Lemma 3.3.**

By definition,

$$sE_{0A}t \Leftrightarrow (\forall x \in \Sigma^* \ni |x| \leq 0)(\overline{\delta}(s, x) \in F \Leftrightarrow \overline{\delta}(t, x) \in F).$$

The only string of length 0 is λ, so

$$sE_{0A}t \Leftrightarrow \overline{\delta}(s, \lambda) \in F \Leftrightarrow \overline{\delta}(t, \lambda) \in F \Leftrightarrow s \in F \Leftrightarrow t \in F.$$

Therefore the equivalence classes of E_{0A} are exactly F and $S - F$, unless one of these sets is empty, in which case there is only the single class S. □

3.9. **Prove Theorem 3.9.**

Assume $E_{mA} = E_{m+1A}$. We prove by induction on k that $E_{m+k,A} = E_{mA}$.

Base case ($k = 0$): $E_{m+0,A} = E_{mA}$ trivially.

Inductive step: assume $E_{m+k,A} = E_{mA}$. Let $s, t \in S$. By Theorem 3.8,

$$\begin{aligned} sE_{m+k+1,A}t &\Leftrightarrow sE_{m+k,A}t \wedge (\forall a \in \Sigma)(\delta(s, a)E_{m+k,A}\delta(t, a)) \\ &\Leftrightarrow sE_{mA}t \wedge (\forall a \in \Sigma)(\delta(s, a)E_{mA}\delta(t, a)) \end{aligned}$$

by the induction hypothesis. Applying Theorem 3.8 again,

$$sE_{m+k+1,A}t \Leftrightarrow sE_{m+1,A}t.$$

Since $E_{m+1A} = E_{mA}$ by hypothesis, this gives

$$sE_{m+k+1,A}t \Leftrightarrow sE_{mA}t.$$

Therefore $E_{m+k+1,A} = E_{mA}$. By induction, $(\forall k \in \mathbb{N})(E_{m+k,A} = E_{mA})$. □

3.10. **Prove $\mu(\overline{\delta}_A(s,x)) = \overline{\delta}_B(\mu(s),x)$ by induction for a homomorphism μ.**

Base: $\mu(\overline{\delta}_A(s,\lambda)) = \mu(s) = \overline{\delta}_B(\mu(s),\lambda)$. ✓

Step: $x = ya$. $\mu(\overline{\delta}_A(s,ya)) = \mu(\delta_A(\overline{\delta}_A(s,y),a)) = \delta_B(\mu(\overline{\delta}_A(s,y)),a)$ (homomorphism property) $= \delta_B(\overline{\delta}_B(\mu(s),y),a)$ (IH) $= \overline{\delta}_B(\mu(s),ya)$. □

3.11. **Prove $L(A) = L(B)$ for a homomorphism $\mu : A \to B$.**

For any $x \in \Sigma^*$,

$$\begin{aligned} x \in L(A) &\Leftrightarrow \overline{\delta}_A(s_{0A},x) \in F_A \\ &\Leftrightarrow \mu(\overline{\delta}_A(s_{0A},x)) \in F_B \qquad \text{(because } \mu(s) \in F_B \Leftrightarrow s \in F_A\text{)} \\ &\Leftrightarrow \overline{\delta}_B(\mu(s_{0A}),x) \in F_B \qquad \text{(by Exercise 3.10)} \\ &\Leftrightarrow \overline{\delta}_B(s_{0B},x) \in F_B \qquad \text{(since } \mu(s_{0A}) = s_{0B}\text{)} \\ &\Leftrightarrow x \in L(B). \end{aligned}$$

Therefore $L(A) = L(B)$. □

3.12. **Examples with connectivity and $A/_{E_A}$.**

(a) A not connected, $A/_{E_A}$ not connected: Let A have states $\{s_0, s_1, s_2\}$ where s_2 is unreachable, $s_0 E_A s_1$ (both accepting) and s_2 is distinct. Then $A/_{E_A}$ has $\{[s_0],[s_2]\}$, and $[s_2]$ is unreachable (since s_2 was unreachable in A).

(b) A not connected, $A/_{E_A}$ connected: Let s_0, s_1 be reachable, s_2 unreachable, and $s_0 E_A s_2$ (equivalent). Then $[s_0] = [s_2]$ and $A/_{E_A}$ only has reachable classes, so it's connected.

3.13. **Show there exists a homomorphism from A to $A/_{E_A}$.**

Define $\mu : S_A \to S_{E_A}$ by $\mu(s) = [s]_{E_A}$. Then:

- $\mu(s_{0A}) = [s_{0A}]_{E_A} = s_{0,E_A}$. ✓
- $\mu(\delta_A(s,a)) = [\delta_A(s,a)]_{E_A} = \delta_{E_A}([s]_{E_A},a) = \delta_{E_A}(\mu(s),a)$. ✓
- $s \in F_A \Leftrightarrow [s]_{E_A} \in F_{E_A}$, i.e., $\mu(s) \in F_{E_A} \Leftrightarrow s \in F_A$. ✓

So μ is a homomorphism. □

3.14. **Show there exists a homomorphism from connected A to $A_{R_{L(A)}}$.**

(a) Define $\psi : S \to \{R_{L(A)}\text{-classes}\}$ by $\psi(s) = [x_s]_{R_{L(A)}}$ where x_s is any string with $\overline{\delta}(s_{0A},x_s) = s$ (well-defined by part (b)).
More precisely: since A is connected, for each $s \in S$ there exists $x_s \in \Sigma^*$ with $\overline{\delta}(s_{0A},x_s) = s$. Define $\psi(s) = [x_s]_{R_{L(A)}}$.

(b) Well-definedness: if $\overline{\delta}(s_0,x) = \overline{\delta}(s_0,y) = s$, then $x R^A y$ (same state), and R^A refines $R_{L(A)}$, so $x R_{L(A)} y$, giving $[x]_{R_{L(A)}} = [y]_{R_{L(A)}}$. □

(c) Homomorphism:

- $\psi(s_{0A}) = [\lambda]_{R_{L(A)}} = s_{0,R_{L(A)}}$. ✓
- $\psi(\delta(s,a)) = [x_s a]_{R_{L(A)}} = \delta_{R_{L(A)}}([x_s]_{R_{L(A)}},a) = \delta_{R_{L(A)}}(\psi(s),a)$. ✓

- $s \in F \Leftrightarrow \lambda \in L(A^s) \Leftrightarrow x_s \in L(A) \Leftrightarrow [x_s]_{R_{L(A)}} \in F_{R_{L(A)}} \Leftrightarrow \psi(s) \in F_{R_{L(A)}}$. ✓

3.15. **Show there may not be a homomorphism $A \to A_{R_{L(A)}}$ if A is not connected.**

Take $\Sigma = \{\mathbf{0}\}$ and let

$$A = \langle \{\mathbf{0}\}, \{s_0, s_1\}, s_0, \delta, \{s_1\} \rangle$$

where $\delta(s_0, \mathbf{0}) = s_0$ and $\delta(s_1, \mathbf{0}) = s_1$. The state s_1 is unreachable, so A is not connected. Since the reachable part is just the nonfinal loop at s_0, we have $L(A) = \emptyset$. Therefore $A_{R_{L(A)}}$ is the one-state DFA for $\emptyset$, with a single nonfinal state q and $\delta(q, \mathbf{0}) = q$.

If there were a homomorphism $\psi : A \to A_{R_{L(A)}}$, then $\psi(s_0) = q$ because s_0 is the start state. But a homomorphism must also preserve finality:

$$s_1 \in F_A \Leftrightarrow \psi(s_1) \in F_{R_{L(A)}}.$$

This is impossible, because q is the only state of $A_{R_{L(A)}}$ and q is not final. Hence no such homomorphism exists.

3.16. **Show there may still be a homomorphism $A \to A_{R_{L(A)}}$ if A is not connected.**

Take $\Sigma = \{\mathbf{0}\}$ and let

$$A = \langle \{\mathbf{0}\}, \{s_0, s_1\}, s_0, \delta, \{s_0, s_1\} \rangle$$

where $\delta(s_0, \mathbf{0}) = s_0$ and $\delta(s_1, \mathbf{0}) = s_1$. The state s_1 is unreachable, so A is not connected, but $L(A) = \Sigma^*$ because every string keeps the start state s_0 in a final state.

Therefore $A_{R_{L(A)}}$ is the one-state DFA

$$\langle \{\mathbf{0}\}, \{q\}, q, \delta', \{q\} \rangle$$

with $\delta'(q, \mathbf{0}) = q$. Define $\psi : A \to A_{R_{L(A)}}$ by

$$\psi(s_0) = q, \qquad \psi(s_1) = q.$$

This is a homomorphism: the start state is preserved, finality is preserved because all three states involved are final, and

$$\psi(\delta(s_i, \mathbf{0})) = \psi(s_i) = q = \delta'(q, \mathbf{0}) = \delta'(\psi(s_i), \mathbf{0})$$

for $i = 0, 1$.

3.17. **Show there need not be a homomorphism from A_{R_L} to A_R.**

Let $\Sigma = \{\boldsymbol{a}\}$ and let Q be the right congruence with two classes

$$[\lambda]_Q = \{\lambda\}, \qquad [\boldsymbol{a}]_Q = \{x \in \Sigma^* : |x| \geq 1\}.$$

Let R refine Q by splitting the nonempty strings according to parity:

$$[\lambda]_R = \{\lambda\}, \qquad [\boldsymbol{a}]_R = \{x : |x| \text{ is odd}\}, \qquad [\boldsymbol{aa}]_R = \{x : |x| \text{ is even and } |x| \geq 2\}.$$

Take $L = \{\lambda\}$, which is a union of Q-classes. Then A_Q has two states,

$$q_0 = [\lambda]_Q \quad \text{(start and final)}, \qquad q_1 = [\boldsymbol{a}]_Q$$

with $\delta_Q(q_0, \boldsymbol{a}) = q_1$ and $\delta_Q(q_1, \boldsymbol{a}) = q_1$.

The DFA A_R has three states,

$$r_0 = [\lambda]_R \quad \text{(start and final)}, \qquad r_1 = [\boldsymbol{a}]_R, \qquad r_2 = [\boldsymbol{aa}]_R,$$

with transitions

$$\delta_R(r_0, \boldsymbol{a}) = r_1, \qquad \delta_R(r_1, \boldsymbol{a}) = r_2, \qquad \delta_R(r_2, \boldsymbol{a}) = r_1.$$

Assume there is a homomorphism $\psi : A_Q \to A_R$. Since start states must map to start states, $\psi(q_0) = r_0$. Then

$$\psi(q_1) = \psi(\delta_Q(q_0, \boldsymbol{a})) = \delta_R(\psi(q_0), \boldsymbol{a}) = \delta_R(r_0, \boldsymbol{a}) = r_1.$$

But now

$$\psi(q_1) = \psi(\delta_Q(q_1, \boldsymbol{a})) = \delta_R(\psi(q_1), \boldsymbol{a}) = \delta_R(r_1, \boldsymbol{a}) = r_2,$$

which says $r_1 = r_2$, a contradiction. Therefore there need not be a homomorphism from A_Q to A_R.

3.18. **Is $\cong$ appropriate to ask if it's a right congruence?**

$\cong$ is a relation on DFAs, not on Σ^*. Right congruences are relations on Σ^*. So asking whether $\cong$ (between DFAs) is a right congruence (on strings) is a category error – it makes no sense. One could ask whether the relation induced on strings by $\cong$ is a right congruence, but the question itself is ill-formed in the original sense.

3.19. **Is E_A a right congruence?**

E_A is a relation on states S, not on Σ^*. So strictly speaking, E_A is not a right congruence (which is a relation on Σ^*). However, R^A (induced on strings) is a right congruence, and E_A induces R^A via $x\,R^A\,y \Leftrightarrow \overline{\delta}(s_0, x)\,E_A\,\overline{\delta}(s_0, y)$.

3.20. **If A is reduced then A^c is reduced.**

A^c = connected part of A. States of A^c are $S^c \subseteq S$. If A is reduced, no two distinct states in S are equivalent; in particular, no two distinct states in $S^c \subseteq S$ are equivalent. So A^c is reduced. □

3.21. **For a homomorphism $\mu : S_A \to S_B$, prove $\mu(s)\,E_B\,\mu(t) \Leftrightarrow s\,E_A\,t$.**

($\Rightarrow$) Suppose $\mu(s)\,E_B\,\mu(t)$, i.e., $L(B^{\mu(s)}) = L(B^{\mu(t)})$. By Exercise 3.10: $L(A^s) = L(B^{\mu(s)})$ and $L(A^t) = L(B^{\mu(t)})$ (since μ is a homomorphism from A^s to $B^{\mu(s)}$). So $L(A^s) = L(A^t)$, i.e., $s\,E_A\,t$.

($\Leftarrow$) Suppose $s\,E_A\,t$. Then $L(A^s) = L(A^t)$. By the homomorphism property: $L(B^{\mu(s)}) = L(A^s) = L(A^t) = L(B^{\mu(t)})$, so $\mu(s)\,E_B\,\mu(t)$. □

3.22. **Define $\psi : M^c \to A_{R^M}$ and prove it is an isomorphism.**

(a) $\psi(s) = [x_s]_{R^M}$ where x_s is any string with $\overline{\delta}(s_0, x_s) = s$.

(b) Well-defined: as in Exercise 3.14(b), since R^M equates all x reaching the same state.

(c) Homomorphism: $\psi(s_0) = [\lambda]_{R^M}$; $\psi(\delta(s, a)) = [x_s a]_{R^M} = \delta_{R^M}(\psi(s), a)$; $s \in F \Leftrightarrow \psi(s) \in F_{R^M}$.

(d) Bijection: Injective: if $\psi(s) = \psi(t)$, then $[x_s]_{R^M} = [x_t]_{R^M}$, i.e., $x_s\,R^M\,x_t$, meaning $\overline{\delta}(s_0, x_s) = \overline{\delta}(s_0, x_t)$, i.e., $s = t$. Surjective: every $[x]_{R^M}$ is $\psi(\overline{\delta}(s_0, x))$.

(e) $M^c \cong A_{R^M}$. □

3.23. **For the machine A in Figure 3.10(a), find E_A, $L(A)$, $A_{R_{L(A)}}$, $R_{L(A)}$, and $A/_{E_A}$.**

(a)

$$E_{0A} = \{\{q_0, q_1, q_3, q_6\}, \{q_2, q_4, q_5\}\}.$$

Refining once gives

$$E_{1A} = \{\{q_0, q_1\}, \{q_3, q_6\}, \{q_2\}, \{q_4, q_5\}\}.$$

Refining again gives

$$E_{2A} = \{\{q_0\}, \{q_1\}, \{q_2\}, \{q_3, q_6\}, \{q_4, q_5\}\}.$$

This is already stable, so $E_A = E_{2A}$.

(b) The language is

$$L(A) = \{\lambda, 1\} \cup 00\Sigma^* \cup 11\Sigma^*.$$

Indeed, λ is accepted at q_0, the string 1 is accepted at q_1, and every string of length at least 2 is accepted exactly when its first two symbols are equal.

(c) $A_{R_{L(A)}}$ has five states, represented by the classes of λ, 1, 0, 00, and 01. Writing these as $[\lambda]$, $[1]$, $[0]$, $[00]$, $[01]$, the transitions are

	0	1
$[\lambda]$	$[0]$	$[1]$
$[1]$	$[01]$	$[00]$
$[0]$	$[00]$	$[01]$
$[00]$	$[00]$	$[00]$
$[01]$	$[01]$	$[01]$

with final states $[\lambda], [1], [00]$. Here $[11] = [00]$ and $[10] = [01]$.

(d) The right-congruence classes of $R_{L(A)}$ may be written as

$$[\lambda], \qquad [1], \qquad [0], \qquad [00] = [11], \qquad [01] = [10].$$

Equivalently, they are

$$\{\lambda\}, \quad \{1\}, \quad \{0\}, \quad \{x : |x| \geq 2 \text{ and the first two symbols are equal}\},$$

and

$$\{x : |x| \geq 2 \text{ and the first two symbols are different}\}.$$

(e) $A/_{E_A}$ has states

$$[q_0],\ [q_1],\ [q_2],\ [q_3] = [q_6],\ [q_4] = [q_5],$$

with $[q_0]$ initial and final states $[q_0], [q_1], [q_3] = [q_6]$. Its transition table is

	0	1
$[q_0]$	$[q_2]$	$[q_1]$
$[q_1]$	$[q_4] = [q_5]$	$[q_3] = [q_6]$
$[q_2]$	$[q_3] = [q_6]$	$[q_4] = [q_5]$
$[q_3] = [q_6]$	$[q_3] = [q_6]$	$[q_3] = [q_6]$
$[q_4] = [q_5]$	$[q_4] = [q_5]$	$[q_4] = [q_5]$

so $A/_{E_A} \cong A_{R_{L(A)}}$.

3.24. **Exercises 3.24–3.26.**

(a) **Machine B of Figure 3.10(b).**

i.
$$E_{0B} = \{\{t_0\}, \{t_1, t_2, t_3\}\}.$$
All three nonfinal states have the same 0- and 1-transitions into the nonfinal class, so $E_{1B} = E_{0B}$. Hence $E_B = E_{0B}$.

ii. $L(B) = \{\lambda\}$.

iii. $A_{R_{L(B)}}$ is the two-state DFA with states $[\lambda]$ and $[\mathbf{0}]$ (equivalently, $[\mathbf{1}]$), where $[\lambda]$ is initial and final, $[\mathbf{0}]$ is nonfinal, and both letters send $[\mathbf{0}]$ to itself.

iv. $R_{L(B)}$ has exactly two classes:
$$[\lambda] = \{\lambda\}, \qquad [\mathbf{0}] = \Sigma^+.$$

v. $B/_{E_B}$ has states $[t_0]$ and $[t_1] = [t_2] = [t_3]$, with $[t_0]$ initial and final, and
$$\delta([t_0], 0) = \delta([t_0], 1) = [t_1] = [t_2] = [t_3],$$
while both letters fix $[t_1] = [t_2] = [t_3]$. Thus $B/_{E_B} \cong A_{R_{L(B)}}$.

(b) **Machine C of Figure 3.10(c).**

i.
$$E_{0C} = \{\{q_1, q_4\}, \{q_0, q_2, q_3, q_5\}\}.$$
Refining once gives
$$E_{1C} = \{\{q_1\}, \{q_4\}, \{q_0, q_2\}, \{q_3, q_5\}\}.$$
This is already stable, so $E_C = E_{1C}$.

ii. The language is
$$L(C) = 1(1 \cup 01)^*,$$
that is, the strings that begin with 1, end with 1, and never contain the substring 00.

iii. $A_{R_{L(C)}}$ has three states, represented by the classes of λ, 1, and 0. Writing them as $[\lambda], [1], [0]$, the transition table is

	0	1
$[\lambda]$	$[0]$	$[1]$
$[1]$	$[\lambda]$	$[1]$
$[0]$	$[0]$	$[0]$

with only $[1]$ final. Here $[10] = [\lambda]$, while $[11] = [1]$ and $[00] = [01] = [0]$.

iv. The classes of $R_{L(C)}$ can be represented by $[\lambda], [1], [0]$. Concretely,
$$[1] = 1(1 \cup 01)^*,$$
$$[\lambda] = \{\lambda\} \cup 1(1 \cup 01)^* 0,$$
and
$$[0] = \Sigma^* - \big([\lambda] \cup [1]\big).$$

v. $C/_{E_C}$ has the four states

$$[q_1],\ [q_4],\ [q_0]=[q_2],\ [q_3]=[q_5].$$

The initial state is $[q_0]=[q_2]$, the final states are $[q_1]$ and $[q_4]$, and the transition table is

	0	1
$[q_0]=[q_2]$	$[q_3]=[q_5]$	$[q_1]$
$[q_1]$	$[q_0]=[q_2]$	$[q_1]$
$[q_3]=[q_5]$	$[q_3]=[q_5]$	$[q_3]=[q_5]$
$[q_4]$	$[q_3]=[q_5]$	$[q_3]=[q_5]$

The state $[q_4]$ is disconnected, so $C/_{E_C}$ is not connected; its connected part is isomorphic to $A_{R_{L(C)}}$.

(c) **Machine D of Figure 3.10(d).**

i.

$$E_{0D}=\{\{S_1,S_4,S_7\},\{S_0,S_2,S_3,S_5,S_6,S_8,S_9\}\}.$$

Refining once gives

$$E_{1D}=\{\{S_1,S_4\},\{S_7\},\{S_0,S_3\},\{S_2,S_5,S_6\},\{S_8,S_9\}\}.$$

Refining again gives

$$E_{2D}=\{\{S_1,S_4\},\{S_7\},\{S_0,S_3\},\{S_2,S_5\},\{S_6\},\{S_8\},\{S_9\}\}.$$

This is already stable, so $E_D=E_{2D}$.

ii. From the initial state S_0, the reachable classes are $[S_0]=[S_3]$, $[S_1]=[S_4]$, and $[S_2]=[S_5]$. Therefore $L(D)$ consists of exactly those strings over $\{a,b\}$ that contain at least one a and do not end with two or more consecutive a's.

iii. $A_{R_{L(D)}}$ has three states represented by $[\lambda]$, $[a]$, and $[aa]$, with transition table

	a	b
$[\lambda]$	$[a]$	$[\lambda]$
$[a]$	$[aa]$	$[a]$
$[aa]$	$[aa]$	$[a]$

and only $[a]$ final.

iv. The classes of $R_{L(D)}$ may be described as

$$[\lambda]=b^*,$$

$$[a]=L(D),$$

and

$$[aa]=\{x\in\{a,b\}^* : x \text{ ends with at least two consecutive } a\text{'s}\}.$$

v. $D/_{E_D}$ has the seven states

$$[S_0] = [S_3],\ [S_1] = [S_4],\ [S_2] = [S_5],\ [S_6],\ [S_7],\ [S_8],\ [S_9].$$

Its initial state is $[S_0] = [S_3]$, its final states are $[S_1] = [S_4]$ and $[S_7]$, and its transition table is

	a	b
$[S_0] = [S_3]$	$[S_1] = [S_4]$	$[S_0] = [S_3]$
$[S_1] = [S_4]$	$[S_2] = [S_5]$	$[S_1] = [S_4]$
$[S_2] = [S_5]$	$[S_2] = [S_5]$	$[S_1] = [S_4]$
$[S_6]$	$[S_6]$	$[S_7]$
$[S_7]$	$[S_7]$	$[S_7]$
$[S_8]$	$[S_0] = [S_3]$	$[S_9]$
$[S_9]$	$[S_2] = [S_5]$	$[S_2] = [S_5]$

The connected part of this quotient is the three-state machine $A_{R_{L(D)}}$ above.

3.25. **Prove: if A, B are reduced, connected, equivalent DFAs then $A \cong B$.**

(a) Since A is connected, for each $s \in S_A$ choose a string x_s with $\overline{\delta}_A(s_{0A}, x_s) = s$. Define

$$\psi(s) = \overline{\delta}_B(s_{0B}, x_s).$$

(b) To show this is well defined, suppose x and y both satisfy $\overline{\delta}_A(s_{0A}, x) = \overline{\delta}_A(s_{0A}, y) = s$. Then for every $z \in \Sigma^*$,

$$xz \in L(A) \Leftrightarrow yz \in L(A),$$

because xz and yz continue from the same state s in A. Since $L(A) = L(B)$, it follows that

$$xz \in L(B) \Leftrightarrow yz \in L(B)$$

for every z. Therefore the states $\overline{\delta}_B(s_{0B}, x)$ and $\overline{\delta}_B(s_{0B}, y)$ have the same future language in B. Because B is reduced, these states must be equal. Hence ψ is well defined.

(c) First,

$$\psi(s_{0A}) = \overline{\delta}_B(s_{0B}, \lambda) = s_{0B}.$$

Now let $s \in S_A$ and $a \in \Sigma$. Since

$$\overline{\delta}_A(s_{0A}, x_s a) = \delta_A(\overline{\delta}_A(s_{0A}, x_s), a) = \delta_A(s, a),$$

we may use $x_s a$ as a string reaching $\delta_A(s, a)$. Therefore

$$\psi(\delta_A(s, a)) = \overline{\delta}_B(s_{0B}, x_s a) = \delta_B(\overline{\delta}_B(s_{0B}, x_s), a) = \delta_B(\psi(s), a).$$

Also,

$$s \in F_A \Leftrightarrow x_s \in L(A) \Leftrightarrow x_s \in L(B) \Leftrightarrow \psi(s) \in F_B.$$

So ψ is a homomorphism.

(d) *Injective*: suppose $\psi(s) = \psi(t)$. Then for every $z \in \Sigma^*$,

$$x_s z \in L(B) \Leftrightarrow x_t z \in L(B),$$

because x_s and x_t reach the same state in B. Since $L(A) = L(B)$, we also have

$$x_s z \in L(A) \Leftrightarrow x_t z \in L(A)$$

for every z. Thus s and t have the same future language in A. Because A is reduced, $s = t$.

Surjective: let $u \in S_B$. Since B is connected, there exists $y \in \Sigma^*$ such that $\overline{\delta}_B(s_{0B}, y) = u$. Let $s = \overline{\delta}_A(s_{0A}, y)$. Then y is one of the strings reaching s, so by the definition of ψ and well-definedness,

$$\psi(s) = \overline{\delta}_B(s_{0B}, y) = u.$$

Hence ψ is surjective.

3.26. **Show the transition from line 3 to 4 in Theorem 3.1(6) is not $\Leftrightarrow$.**

In that step, we used $\overline{\delta}_A(s,x) \in F_A \Rightarrow \mu(\overline{\delta}_A(s,x)) \in F_B$. The converse would require $\mu(s) \in F_B \Rightarrow s \in F_A$, which is the definition of a homomorphism (both directions). If μ is only required to satisfy $s \in F_A \Rightarrow \mu(s) \in F_B$ (not $\Leftarrow$), then we only get $L(A) \subseteq L(B)$, not equality. Example: A has states $\{q_0, q_1\}$, $F_A = \{q_1\}$; B has $\{p_0\}$, $F_B = \{p_0\}$ (all final); $\mu(q_i) = p_0$. Then $L(A) \subset L(B) = \Sigma^*$.

3.27. **Supply reasons for equivalences in proof of Theorem 3.8.**

Reading down the proof of Theorem 3.8:

(a)
$$sE_{i+1A}t \Leftrightarrow (\forall x \in \Sigma^* \ni |x| \le i+1)(\overline{\delta}(s,x) \in F \Leftrightarrow \overline{\delta}(t,x) \in F)$$
is just the definition of E_{i+1A}.

(b) Splitting this into the strings of length at most i and the strings of length exactly $i+1$ is valid because those two cases exhaust the condition $|x| \le i+1$.

(c) Replacing "$|y| = i+1$" by "$1 \le |y| \le i+1$" is valid because the strings of lengths $1, \ldots, i$ are already handled by the first conjunct.

(d) Passing from nonempty y with $|y| \le i+1$ to strings of the form $\mathbf{a}x$ with $\mathbf{a} \in \Sigma$ and $|x| \le i$ uses the fact that every nonempty string can be written uniquely in that form.

(e)
$$(\forall x \in \Sigma^* \ni |x| \le i)(\overline{\delta}(s,x) \in F \Leftrightarrow \overline{\delta}(t,x) \in F) \Leftrightarrow sE_{iA}t$$
is the definition of E_{iA}.

(f)
$$\overline{\delta}(s, \mathbf{a}x) = \overline{\delta}(\delta(s,\mathbf{a}), x)$$
and similarly for t is the definition of the extended transition function $\overline{\delta}$.

(g) Finally,
$$(\forall x \in \Sigma^* \ni |x| \le i)(\overline{\delta}(\delta(s,\mathbf{a}),x) \in F \Leftrightarrow \overline{\delta}(\delta(t,\mathbf{a}),x) \in F) \Leftrightarrow \delta(s,\mathbf{a})E_{iA}\delta(t,\mathbf{a})$$
is again the definition of E_{iA}.

3.28. **Minimize the machine of Figure 3.3.**

Let

$$E_{0A} = \{\{S_0, S_5\}, \{S_1, S_2, S_3, S_4\}\}.$$

Refining once gives

$$E_{1A} = \{\{S_0, S_5\}, \{S_1, S_2\}, \{S_3, S_4\}\},$$

and this is already stable. Thus the minimal machine has the three states

$$A = [S_0] = [S_5], \qquad B = [S_1] = [S_2], \qquad C = [S_3] = [S_4],$$

with A initial and final, and transition table

	a	b
A	B	B
B	C	C
C	A	A

So the machine minimizes to a 3-state DFA in which both letters move one step forward around the cycle $A \to B \to C \to A$. □

3.29. **Examples where A not reduced, A^c not/is reduced.**

(a) A not reduced, A^c not reduced: let A have two equivalent reachable states q_0, q_1 (both reachable, both final). $A^c = A$ is not reduced.

(b) A not reduced, A^c is reduced: let A have two equivalent states q_0 (reachable) and q_2 (unreachable). $A^c = \{q_0, q_1\}$ may be reduced if q_0 and q_1 are inequivalent.

3.30. **Prove $\cong$ is an equivalence relation on DFAs.**

(a) If $g : A \to B$ and $f : B \to C$ are isomorphisms, then $f \circ g : A \to C$ satisfies all isomorphism conditions (since both are bijective homomorphisms and composition of bijections is bijective, composition of homomorphisms is a homomorphism).

(b) Symmetry: if $f : A \to B$ is an isomorphism (bijective homomorphism), then $f^{-1} : B \to A$ is also an isomorphism: $f^{-1}(s_{0B}) = s_{0A}$ (since $f(s_{0A}) = s_{0B}$); $f^{-1}(\delta_B(t,a)) = f^{-1}(\delta_B(f(s),a)) = f^{-1}(f(\delta_A(s,a))) = \delta_A(s,a) = \delta_A(f^{-1}(t),a)$; $t \in F_B \Leftrightarrow f^{-1}(t) \in F_A$.

(c) Reflexivity: $\mathrm{id} : A \to A$ is an isomorphism.

(d) From (a),(b),(c): $\cong$ is an equivalence relation. □

3.31. **Homomorphism is not an equivalence relation on DFAs.**

Homomorphism is not symmetric. Let $\Sigma = \{a\}$, and let A be the one-state DFA with state p (initial, nonfinal) and $\delta_A(p,a) = p$, so $L(A) = \emptyset$. Let B have two states q_0, q_1, with q_0 initial and nonfinal, q_1 final and unreachable, and both states looping on a.

There is a homomorphism $\mu : A \to B$ given by $\mu(p) = q_0$: initials match, transitions are preserved, and $p \in F_A \Leftrightarrow q_0 \in F_B$.

However, there is no homomorphism $\nu : B \to A$, because $q_1 \in F_B$ would have to imply $\nu(q_1) \in F_A$, and A has no final state. Therefore homomorphism is not an equivalence relation on DFAs. □

3.32. **For R and R_L from Theorem 2.2, show homomorphism from A_R to A_{R_L}.**

R refines R_L: each R-class is contained in some R_L-class. Define

$$\mu : S_{A_R} \to S_{A_{R_L}} \qquad \text{by} \qquad \mu([x]_R) = [x]_{R_L}.$$

This is well defined: if $[x]_R = [y]_R$, then xRy, and since R refines R_L, also xR_Ly, so $[x]_{R_L} = [y]_{R_L}$.

Now

$$\mu(s_{0,R}) = \mu([\lambda]_R) = [\lambda]_{R_L} = s_{0,R_L}.$$

Also, for any $a \in \Sigma$,

$$\mu(\delta_R([x]_R, a)) \;=\; \mu([xa]_R) \;=\; [xa]_{R_L} \;=\; \delta_{R_L}([x]_{R_L}, a) \;=\; \delta_{R_L}(\mu([x]_R), a).$$

Finally,

$$[x]_R \in F_R \Leftrightarrow x \in L \Leftrightarrow [x]_{R_L} \in F_{R_L} \Leftrightarrow \mu([x]_R) \in F_{R_L}.$$

Therefore μ is a homomorphism from A_R to A_{R_L}. □

3.33. **Prove: if there is a homomorphism $A \to B$ then R^A refines R^B.**

$\mu : A \to B$ homomorphism. If $x\,R^A\,y$: $\overline{\delta}_A(s_0, x) = \overline{\delta}_A(s_0, y)$. Then $\overline{\delta}_B(s_0, x) = \mu(\overline{\delta}_A(s_0, x)) = \mu(\overline{\delta}_A(s_0, y)) = \overline{\delta}_B(s_0, y)$, so $x\,R^B\,y$. □

3.34. **Prove: if $A \cong B$ then $R^A = R^B$.**

(a) Via Exercise 3.35: an isomorphism $\mu : A \to B$ is in particular a homomorphism, so R^A refines R^B (Exercise 3.35). By symmetry ($\mu^{-1} : B \to A$ is also a homomorphism), R^B refines R^A. So $R^A = R^B$.

(b) Directly: $x\,R^A\,y \Leftrightarrow \overline{\delta}_A(s_0, x) = \overline{\delta}_A(s_0, y) \Leftrightarrow \mu(\overline{\delta}_A(s_0, x)) = \mu(\overline{\delta}_A(s_0, y))$ (μ injective) $\Leftrightarrow \overline{\delta}_B(s_0, x) = \overline{\delta}_B(s_0, y)$ (Exercise 3.10) $\Leftrightarrow x\,R^B\,y$.

3.35. **A not homomorphic to B, $R^A = R^B$.**

(a) $L(A) = L(B)$: let $\Sigma = \{a\}$. Let B be the minimal 2-state DFA for even-length strings: q_0 is initial and final, q_1 is nonfinal, and $\delta(q_0, a) = q_1$, $\delta(q_1, a) = q_0$. Let A have the same reachable part, plus an unreachable final state r with $\delta(r, a) = r$.
Then $L(A) = L(B)$ and $R^A = R^B$, because R^A only depends on the states reached from the initial state, and A and B have the same reachable part. However, there is no homomorphism $A \to B$: since r is final, its image would have to be the final state q_0, but then

$$\mu(\delta_A(r, a)) = \mu(r) = q_0$$

while

$$\delta_B(\mu(r), a) = \delta_B(q_0, a) = q_1,$$

a contradiction.

(b) $L(A) \neq L(B)$: let A again be the even-length machine, and let B be the same transition graph but with final state q_1 instead of q_0, so $L(B)$ is the odd-length strings. Then $R^A = R^B$ (both relations simply partition Σ^* into even and odd lengths), but A is not homomorphic to B because the initial state of A is final while the initial state of B is not.

(c) No. Suppose A and B are connected, $L(A) = L(B)$, and $R^A = R^B$. Define

$$\psi(\overline{\delta}_A(s_{0A}, x)) = \overline{\delta}_B(s_{0B}, x).$$

Because $R^A = R^B$, this is well defined: if $\overline{\delta}_A(s_{0A}, x) = \overline{\delta}_A(s_{0A}, y)$, then xR^Ay, hence xR^By, and therefore $\overline{\delta}_B(s_{0B}, x) = \overline{\delta}_B(s_{0B}, y)$. Since both machines are connected, the same argument as in Exercise 3.27 shows that ψ is a bijection preserving transitions, and because $L(A) = L(B)$ it also preserves finality. Thus $A \cong B$, so in particular there is a homomorphism from A to B. Such examples therefore cannot exist.

(d) Yes. The example from part (a) already works if we note that both machines are reduced: B is minimal and therefore reduced, and the extra state r in A has language Σ^*, which is different from the state languages of q_0 and q_1. So A and B are both reduced, $L(A) = L(B)$, and $R^A = R^B$, but A is still not homomorphic to B.

3.36. **Disprove: if A homomorphic to B then $R^A = R^B$.**

Counterexample: A = 2-state DFA for $\{a^{2k}\}$, B = 1-state DFA for Σ^* (all states final). There is a homomorphism $\mu : A \to B$ mapping both states to the one state of B. R^A has 2 classes (even/odd), R^B has 1 class. $R^A \neq R^B$. □

3.37. **Prove or give counterexamples (homomorphisms involving A_{R^M}).**

(a) Homomorphism $\psi : A_{R^M} \to M$: Yes, such ψ exists (Exercise 3.22(a)). $A_{R^M} \cong M^c$, and there's a natural inclusion $M^c \hookrightarrow M$... but M might have disconnected states. Define $\psi([x]_{R^M}) = \overline{\delta}_M(s_0, x)$: well-defined and a homomorphism.

(b) Isomorphism $\psi : A_{R^M} \to M$: Not always. Only if M is connected (since $A_{R^M} \cong M^c$, isomorphism only if $M = M^c$, i.e., M connected).

(c) Homomorphism $\psi : M \to A_{R^M}$: Not always. The reachable part of M maps naturally to A_{R^M} (send each reachable state s to $[x_s]_{R^M}$ where x_s is any string reaching s), but unreachable states may obstruct extension.

Counterexample. Let $\Sigma = \{\mathbf{a}\}$, $S = \{s_0, s_1, s_d\}$, s_0 initial, $F = \{s_1\}$, with

$$\delta(s_0, \mathbf{a}) = s_1, \quad \delta(s_1, \mathbf{a}) = s_1, \quad \delta(s_d, \mathbf{a}) = s_0.$$

State s_d is unreachable. The accessible quotient A_{R^M} has two states q_0, q_1 with

$$\delta'(q_0, \mathbf{a}) = q_1, \quad \delta'(q_1, \mathbf{a}) = q_1,$$

q_0 initial, $F' = \{q_1\}$.

A homomorphism $\psi : M \to A_{R^M}$ must satisfy $\psi(s_0) = q_0$ (initial states correspond) and $\delta'(\psi(s), \mathbf{a}) = \psi(\delta(s, \mathbf{a}))$ for every s. From s_d: $\delta'(\psi(s_d), \mathbf{a}) = \psi(\delta(s_d, \mathbf{a})) = \psi(s_0) = q_0$. But $\delta'(q_0, \mathbf{a}) = q_1 \neq q_0$ and $\delta'(q_1, \mathbf{a}) = q_1 \neq q_0$, so no state in A_{R^M} can serve as $\psi(s_d)$. The homomorphism does not exist. □

3.38. **Prove: if A minimal then $R^A = R_{L(A)}$.**

If A is minimal, A is both connected and reduced. Connected: R^A is defined on reachable strings, and every class of R^A corresponds to a state. Reduced: no two states are equivalent, so no two R^A-classes map to the same $R_{L(A)}$-class. Since R^A refines $R_{L(A)}$ (Exercise 2.57c) and $|R^A| = |S| = |R_{L(A)}|$ (minimality means same number of states as minimal machine), they must be equal. □

3.39. **Example where Exercise 3.40 fails if A not minimal.**

Any non-minimal DFA: A = 3-state DFA for $\{a^{2k}\}$ with one extra state. R^A has 3 classes but $R_{L(A)}$ has 2. They differ.

3.40. **Example where Exercise 3.40 holds even if A not minimal.**

Take the minimal 2-state DFA for the language of even-length strings over $\{a\}$, with states q_0 (initial and final) and q_1 (nonfinal), and add an extra unreachable state q_2 with $\delta(q_2, a) = q_2$. The resulting machine A is not minimal, because it is not connected. However, R^A is unchanged by the presence of q_2, since no string from the initial state ever reaches q_2. Therefore R^A still has exactly the two classes "even length" and "odd length," which are precisely the classes of $R_{L(A)}$. So in this example A is not minimal but $R^A = R_{L(A)}$. □

3.41. **Quotient by arbitrary relation R.**

(a) $R = E_{0A}$: $A/_{E_{0A}}$ partitions into final and nonfinal. $\delta_{E_{0A}}$ maps $[s]_{E_{0A}} \times \Sigma \to S_{E_{0A}}$: need $[s] \ni s'$ with $\delta(s, a)$ and $\delta(s', a)$ in the same E_{0A}-class... not necessarily. So $A/_{E_{0A}}$ is not always well-defined. Example: A = 3-state DFA, $F = \{q_1\}$, $\delta(q_0, a) = q_1$, $\delta(q_2, a) = q_0$. E_{0A}-classes: $\{q_0, q_2\}$ (nonfinal) and $\{q_1\}$ (final). $\delta_{E_{0A}}(\{q_0, q_2\}, a)$: $\delta(q_0, a) = q_1 \in \{q_1\}$ but $\delta(q_2, a) = q_0 \in \{q_0, q_2\}$: two different classes! Not well-defined.

(b) If R refines E_A, then $A/_R$ is well defined exactly when R is stable under the transitions:

$$sRt \Rightarrow \delta(s, a) R \delta(t, a)$$

for every $a \in \Sigma$.

For such well-defined refinements R, the analogues of Theorems 3.4, 3.5, and 3.6 behave as follows.

Analogue of Theorem 3.4: false in general. Take R to be the identity relation on the states of a DFA A that is not reduced. Then R certainly refines E_A, and $A/_R = A$, which is not reduced.

Analogue of Theorem 3.5: true. If A is connected and $[s]_R$ is a state of $A/_R$, choose x with $\overline{\delta}(s_0, x) = s$. Since $A/_R$ is well defined,

$$\overline{\delta}_R([s_0]_R, x) = [\overline{\delta}(s_0, x)]_R = [s]_R,$$

so every state class is reachable.

Analogue of Theorem 3.6: true. Because R refines E_A, every R-class is contained in a single E_A-class, so no R-class mixes final and nonfinal states; hence F_R is well defined. Then the proof of Theorem 3.6 goes through verbatim:

$$x \in L(A/_R) \Leftrightarrow \overline{\delta}_R([s_0]_R, x) \in F_R \Leftrightarrow [\overline{\delta}(s_0, x)]_R \in F_R \Leftrightarrow \overline{\delta}(s_0, x) \in F \Leftrightarrow x \in L(A).$$

Thus $L(A/_R) = L(A)$.

3.42. **Homomorphisms between $M/_{E_M}$ and $A_{R_{L(M)}}$.**

(a) Not always. Consider the DFA

$$M = \langle \{a\}, \{s_0, s_1, s_d\}, s_0, \delta, \{s_1\} \rangle$$

with

$$\delta(s_0, a) = s_1, \qquad \delta(s_1, a) = s_1, \qquad \delta(s_d, a) = s_0.$$

The state s_d is unreachable. The three states have distinct future languages:

$$L(M^{s_0}) = a^+, \qquad L(M^{s_1}) = a^*, \qquad L(M^{s_d}) = a^{\geq 2},$$

so E_M is the identity relation and $M/_{E_M} = M$.

Now $L(M) = a^+$, and $A_{R_{L(M)}}$ is the usual 2-state DFA with initial nonfinal state q_0, final state q_1, and transitions $\delta(q_0, a) = q_1$, $\delta(q_1, a) = q_1$.

If there were a homomorphism $\psi : M/_{E_M} \to A_{R_{L(M)}}$, then $\psi(s_0) = q_0$ and $\psi(s_1) = q_1$. Since s_d is nonfinal, $\psi(s_d)$ would have to be q_0. But then

$$\psi(\delta(s_d, a)) = \psi(s_0) = q_0$$

while

$$\delta(\psi(s_d), a) = \delta(q_0, a) = q_1,$$

a contradiction. So the statement is false.

(b) Yes. Define

$$\psi([x]_{R_{L(M)}}) = [\overline{\delta}_M(s_0, x)]_{E_M}.$$

This is well defined: if $xR_{L(M)}y$, then for every $z \in \Sigma^*$,

$$xz \in L(M) \Leftrightarrow yz \in L(M),$$

so the states $\overline{\delta}_M(s_0, x)$ and $\overline{\delta}_M(s_0, y)$ have the same future language and are therefore E_M-equivalent.

Now

$$\psi([\lambda]_{R_{L(M)}}) = [s_0]_{E_M},$$

and for any $a \in \Sigma$,

$$\psi(\delta_{R_{L(M)}}([x]_{R_{L(M)}}, a)) = \psi([xa]_{R_{L(M)}}) = [\overline{\delta}_M(s_0, xa)]_{E_M} = [\delta(\overline{\delta}_M(s_0, x), a)]_{E_M} = \delta_{E_M}(\psi([x]_{R_{L(M)}}), a).$$

Finally,

$$[x]_{R_{L(M)}} \in F_{R_{L(M)}} \Leftrightarrow x \in L(M) \Leftrightarrow \overline{\delta}_M(s_0, x) \in F \Leftrightarrow [\overline{\delta}_M(s_0, x)]_{E_M} \in F_{E_M}.$$

So ψ is a homomorphism from $A_{R_{L(M)}}$ to $M/_{E_M}$. □

3.43. **Prove: $\exists m < \|S\|$ with $E_{mA} = E_{m+1,A}$.**

The sequence $E_{0A} \supseteq E_{1A} \supseteq \cdots$ is decreasing (each refinement removes some equivalences). Since S is finite, the number of equivalence classes is bounded by $|S|$. Each strict refinement step increases the number of classes by at least 1. Starting from E_{0A} (which has at most 2 classes: final/nonfinal), we can have at most $|S| - 1$ strict refinement steps before reaching E_A. So $E_{|S|-1,A} = E_{|S|,A} = E_A$, meaning $m \leq |S| - 1 < |S|$. □

3.44. **Equivalence vs. isomorphism.**

(a) False: A and B can be equivalent ($L(A) = L(B)$) without being isomorphic (different number of states, or same number but different structure). E.g., A = 3-state non-minimal DFA, B = 2-state minimal DFA for the same language.

(b) True: if $A \cong B$ then in particular $L(A) = L(B)$ (by Exercise 3.11).

3.45. **Prove $C_i \subseteq C_{i+1}$.**

$C_0 = \{s_0\}$, $C_{i+1} = C_i \cup \{\delta(s,a) : s \in C_i, a \in \Sigma\}$. Clearly $C_i \subseteq C_{i+1}$ by definition. □

3.46. **Prove C_i = states reachable from s_0 by strings of length $\leq i$.**

Let $R_i = \{\overline{\delta}(s_0, x) : |x| \leq i\}$.

Base: $R_0 = \{s_0\} = C_0$. ✓

Step: $R_{i+1} = \{\overline{\delta}(s_0,x) : |x| \leq i+1\} = R_i \cup \{\delta(\overline{\delta}(s_0,x),a) : |x| \leq i, a \in \Sigma\} = C_i \cup \{\delta(s,a) : s \in C_i, a \in \Sigma\} = C_{i+1}$. □

3.47. **Prove: $C_i = C_{i+1} \Rightarrow (\forall k)(C_i = C_{i+k})$.**

Base $k = 0$: trivial. $k = 1$: given.

Step: Assume $C_i = C_{i+k}$. Then $C_{i+k+1} = C_{i+k} \cup \{\delta(s,a) : s \in C_{i+k}\} = C_i \cup \{\delta(s,a) : s \in C_i\} = C_{i+1} = C_i$. □

3.48. **Prove: for a DFA A, $C_i = C_{i+1} \Rightarrow C_i = S^c$.**

$S^c = \bigcup_{k \geq 0} C_k$. If $C_i = C_{i+1}$, then by Exercise 3.49, $C_i = C_{i+k}$ for all $k \geq 0$. So $S^c = \bigcup_k C_k = C_i$ (since $C_k \subseteq C_i$ for $k \leq i$ by $C_i \supseteq C_{i-1} \supseteq \cdots$, and $C_k = C_i$ for $k \geq i$). □

3.49. **Prove: $\exists i$ with $C_i = C_{i+1}$.**

Since S is finite, the sequence $C_0 \subseteq C_1 \subseteq \cdots$ is non-decreasing and bounded above by S. So it must stabilize: $\exists i$ with $C_i = C_{i+1}$. □

3.50. **Prove: $\exists i \leq \|S\|$ with $C_i = S^c$.**

Each strict inclusion $C_i \subsetneq C_{i+1}$ adds at least one new state. Starting from $|C_0| = 1$, after at most $|S|-1$ strict inclusions we reach $C_{|S|-1} \supseteq \cdots$, which must equal $C_{|S|}$ (no room to grow beyond $|S|$ states). So $i \leq |S| - 1 \leq |S|$ and $C_i = S^c$. □

3.51. **Argue the C_i algorithm is correct and terminates.**

Correctness: by Exercise 3.48, C_i is exactly the reachable states within depth i. By Exercise 3.51, $C_i = S^c$ when stable. Termination: by Exercise 3.51 and 3.52, stability occurs by step $|S|$. □

3.52. **Give two DFAs with homomorphisms in both directions but no isomorphism.**

Let $L = \{a^{2k} : k \geq 0\}$. Let A be the minimal 2-state DFA for L: states q_0, q_1 cycling on a, $F_A = \{q_0\}$. Let B be a non-minimal 3-state DFA for L: states p_0, p_1, p_2 with $p_0 \sim p_2$ (both accepting, both cycling with step 2), $F_B = \{p_0, p_2\}$.

There is a homomorphism $\mu : B \to A$ collapsing $p_0, p_2 \mapsto q_0$ and $p_1 \mapsto q_1$: this satisfies $\delta_A(\mu(p), a) = \mu(\delta_B(p,a))$ and $p \in F_B \Leftrightarrow \mu(p) \in F_A$.

There is also a homomorphism $\nu : A \to B$ sending $q_0 \mapsto p_0$ and $q_1 \mapsto p_1$ (the natural inclusion into the connected part of B): $\nu(\delta_A(q,a)) = \delta_B(\nu(q),a)$ since $\delta_A(q_0,a) = q_1$ and $\delta_B(p_0,a) = p_1 = \nu(q_1)$, and $q_0 \in F_A \Leftrightarrow p_0 \in F_B$.

But $A \not\cong B$ since they have different numbers of states.

3.53. **R and Q right congruences of finite rank, R refines Q, L union of Q-classes.**

(a) L is a union of R-classes: since R refines Q, each Q-class is a union of R-classes. So $L = \bigcup$(some Q-classes) $= \bigcup$(corresponding R-classes).

(b) Homomorphism $A_R \to A_Q$: Define $\mu([x]_R) = [x]_Q$ (well-defined since R refines Q: $[x]_R = [y]_R \Rightarrow x\,R\,y \Rightarrow x\,Q\,y \Rightarrow [x]_Q = [y]_Q$). Check: $\mu(s_{0,R}) = \mu([\lambda]_R) = [\lambda]_Q = s_{0,Q}$. $\mu(\delta_R([x]_R, a)) = \mu([xa]_R) = [xa]_Q = \delta_Q([x]_Q, a) = \delta_Q(\mu([x]_R), a)$. $[x]_R \in F_R \Leftrightarrow x \in L \Leftrightarrow [x]_Q \in F_Q \Leftrightarrow \mu([x]_R) \in F_Q$.

(c) Not in general. Use the same style of example as in the earlier exercise. Take $\Sigma = \{\boldsymbol{a}\}$, let

$$[\lambda]_Q = \{\lambda\}, \qquad [\boldsymbol{a}]_Q = \{x \in \Sigma^* : |x| \geq 1\},$$

and let R refine Q by splitting the nonempty strings according to parity:

$$[\lambda]_R = \{\lambda\}, \qquad [\boldsymbol{a}]_R = \{x : |x| \text{ is odd}\}, \qquad [\boldsymbol{aa}]_R = \{x : |x| \text{ is even and } |x| \geq 2\}.$$

Let $L = \{\lambda\}$, which is a union of Q-classes. Then A_Q has two states, $q_0 = [\lambda]_Q$ (start and final) and $q_1 = [\boldsymbol{a}]_Q$, with

$$\delta_Q(q_0, \boldsymbol{a}) = q_1, \qquad \delta_Q(q_1, \boldsymbol{a}) = q_1.$$

The DFA A_R has three states, $r_0 = [\lambda]_R$ (start and final), $r_1 = [\boldsymbol{a}]_R$, and $r_2 = [\boldsymbol{aa}]_R$, with

$$\delta_R(r_0, \boldsymbol{a}) = r_1, \qquad \delta_R(r_1, \boldsymbol{a}) = r_2, \qquad \delta_R(r_2, \boldsymbol{a}) = r_1.$$

Assume there is a homomorphism $\psi : A_Q \to A_R$. Since start states must map to start states, $\psi(q_0) = r_0$. Then

$$\psi(q_1) = \psi(\delta_Q(q_0, \boldsymbol{a})) = \delta_R(\psi(q_0), \boldsymbol{a}) = \delta_R(r_0, \boldsymbol{a}) = r_1.$$

But also

$$\psi(q_1) = \psi(\delta_Q(q_1, \boldsymbol{a})) = \delta_R(\psi(q_1), \boldsymbol{a}) = \delta_R(r_1, \boldsymbol{a}) = r_2,$$

which is impossible. Therefore there need not be a homomorphism from A_Q to A_R.

3.54. **Prove A_Q is connected.**

Every state $[x]_Q$ of A_Q is reachable: $\overline{\delta}_Q([\lambda]_Q, x) = [x]_Q$. □

3.55. **Prove: if $A \cong B$ and A connected then B connected.**

Let $\mu : A \to B$ be an isomorphism. μ is surjective (bijection), so every state of B is $\mu(s)$ for some $s \in S_A$. Since A is connected, $s = \overline{\delta}_A(s_0, x)$ for some x. Then $\mu(s) = \mu(\overline{\delta}_A(s_0, x)) = \overline{\delta}_B(\mu(s_0), x) = \overline{\delta}_B(s_{0B}, x)$ (by Exercise 3.10). So every state of B is reachable. □

3.56. **Prove: if $A \cong B$ and B connected then A connected.**

$\mu^{-1} : B \to A$ is also an isomorphism. By Exercise 3.56, A is connected. □

3.57. **Disprove: homomorphism $A \to B$, A connected $\Rightarrow$ B connected.**

Take

$$A = \langle \{\mathbf{0}\}, \{s\}, s, \delta_A, \{s\} \rangle$$

with $\delta_A(s, \mathbf{0}) = s$. This DFA is connected. Let

$$B = \langle \{\mathbf{0}\}, \{t, u\}, t, \delta_B, \{t\} \rangle$$

where $\delta_B(t,\mathbf{0}) = t$ and $\delta_B(u,\mathbf{0}) = u$. The state u is unreachable from t, so B is not connected.

Define $\mu : A \to B$ by $\mu(s) = t$. Then μ preserves the start state, it preserves finality because s and t are both final, and

$$\mu(\delta_A(s,\mathbf{0})) = \mu(s) = t = \delta_B(t,\mathbf{0}) = \delta_B(\mu(s),\mathbf{0}).$$

Hence μ is a homomorphism even though B is not connected.

3.58. **Disprove: homomorphism $A \to B$, B connected $\Rightarrow$ A connected.**

Take

$$A = \langle\{\mathbf{0}\}, \{s_0, s_1\}, s_0, \delta_A, \{s_0, s_1\}\rangle$$

with $\delta_A(s_0,\mathbf{0}) = s_0$ and $\delta_A(s_1,\mathbf{0}) = s_1$. The state s_1 is unreachable, so A is not connected. Let

$$B = \langle\{\mathbf{0}\}, \{t\}, t, \delta_B, \{t\}\rangle$$

with $\delta_B(t,\mathbf{0}) = t$; this DFA is connected.

Define $\mu : A \to B$ by

$$\mu(s_0) = t, \qquad \mu(s_1) = t.$$

Then μ preserves the start state, it preserves finality because all these states are final, and

$$\mu(\delta_A(s_i,\mathbf{0})) = \mu(s_i) = t = \delta_B(t,\mathbf{0}) = \delta_B(\mu(s_i),\mathbf{0})$$

for $i = 0, 1$. So there is a homomorphism from A to connected B even though A is not connected.

3.59. $A_{R^A} \cong A^c$**: define ψ and prove it is an isomorphism.**

(a) $\psi([x]_{R^A}) = \overline{\delta}(s_0, x)$: sends each R^A-class (= set of strings reaching the same state) to that state.

(b) Well-defined: $[x]_{R^A} = [y]_{R^A} \Rightarrow \overline{\delta}(s_0, x) = \overline{\delta}(s_0, y)$. ✓

(c) Homomorphism: $\psi([\lambda]_{R^A}) = s_0$. $\psi([xa]_{R^A}) = \overline{\delta}(s_0, xa) = \delta(\overline{\delta}(s_0, x), a) = \delta(\psi([x]_{R^A}), a)$. $[x] \in F_{R^A} \Leftrightarrow x \in L(A) \Leftrightarrow \overline{\delta}(s_0, x) \in F = F^c$. ✓

(d) Isomorphism: Injective: $\psi([x]) = \psi([y]) \Rightarrow \overline{\delta}(s_0, x) = \overline{\delta}(s_0, y) \Rightarrow [x] = [y]$. Surjective (onto S^c): any $s \in S^c$ has $s = \overline{\delta}(s_0, x_s) = \psi([x_s])$.

3.60. **Prove $\psi = \mu^{-1}$ when A, B connected and $\psi : A \to B$, $\mu : B \to A$ isomorphisms.**

$(\mu \circ \psi)(s) = \mu(\psi(s))$. For connected A: $s = \overline{\delta}_A(s_0, x)$ for some x. $\psi(s) = \overline{\delta}_B(s_{0B}, x)$ (by Exercise 3.10 applied to ψ). $\mu(\psi(s)) = \mu(\overline{\delta}_B(s_{0B}, x)) = \overline{\delta}_A(s_{0A}, x) = s$ (by Exercise 3.10 applied to μ). So $\mu \circ \psi = \mathrm{id}_A$, i.e., $\psi = \mu^{-1}$. □

3.61. **Give example for which $\psi \neq \mu^{-1}$ (not connected).**

Take $A = B$ to be the DFA

$$\langle\{\mathbf{0}\}, \{q_0, q_1, q_2\}, q_0, \delta, \{q_0\}\rangle$$

where

$$\delta(q_0,\mathbf{0}) = q_0, \qquad \delta(q_1,\mathbf{0}) = q_1, \qquad \delta(q_2,\mathbf{0}) = q_2.$$

This machine is not connected because q_1 and q_2 are unreachable.

Let $\psi : A \to B$ be the identity isomorphism:

$$\psi(q_i) = q_i \qquad (i = 0, 1, 2).$$

Let $\mu : B \to A$ swap the two unreachable states:

$$\mu(q_0) = q_0, \qquad \mu(q_1) = q_2, \qquad \mu(q_2) = q_1.$$

Because q_1 and q_2 have exactly the same final/nonfinal status and the same loop transition on $\mathbf{0}$, μ is also an isomorphism. But

$$\mu^{-1} = \mu \neq \psi,$$

since, for example, $\mu^{-1}(q_1) = q_2$ while $\psi(q_1) = q_1$.

3.62. **Three-state DFA with E_{0A} having one class. Can $E_{0A} \neq E_{1A}$?**

E_{0A} has one class iff all states are equivalent under E_{0A}, i.e., all are final or all are nonfinal. Example: $F = S = \{q_0, q_1, q_2\}$ (all final). Then $E_{0A} = S \times S$ (one class). Can $E_{1A} \neq E_{0A}$? E_{1A}: $q\, E_{1A}\, r \Leftrightarrow q\, E_{0A}\, r \wedge (\forall a)\delta(q,a)\, E_{0A}\, \delta(r,a)$. Since $E_{0A} = S \times S$, all δ-targets are in the same class, so $E_{1A} = E_{0A}$. So no, if E_{0A} has one class then $E_{1A} = E_{0A}$.

3.63. **If A, B reduced and connected and $\psi : A \to B$ is a homomorphism, must ψ be an isomorphism?**

Yes. Since ψ is a homomorphism, it already satisfies the start-state, final-state, and transition conditions from Definition 3.5. We only need to show that ψ is one-to-one and onto.

To prove injectivity, assume $\psi(s) = \psi(t)$. Then certainly $\psi(s)\, E_B\, \psi(t)$, and Exercise 3.20 gives

$$\psi(s)\, E_B\, \psi(t) \Leftrightarrow s\, E_A\, t.$$

Since A is reduced, $s\, E_A\, t$ implies $s = t$. Thus ψ is one-to-one.

To prove surjectivity, let $u \in S_B$. Because B is connected, there exists $x \in \Sigma^*$ such that $\overline{\delta}_B(s_{0B}, x) = u$. Let $s = \overline{\delta}_A(s_{0A}, x) \in S_A$. By Exercise 3.10,

$$\psi(s) = \psi(\overline{\delta}_A(s_{0A}, x)) = \overline{\delta}_B(\psi(s_{0A}), x) = \overline{\delta}_B(s_{0B}, x) = u.$$

So every state of B lies in the image of ψ.

Therefore ψ is a bijective homomorphism, hence an isomorphism. □

3.64. **Prove Corollary 3.2.**

By Theorem 2.2, A_{R_L} accepts L. The discussion immediately preceding the corollary shows that if $M = \langle \Sigma, S, s_0, \delta, F \rangle$ is any DFA with $L(M) = L$, then

$$\|S\| \geq rk(R^M) \geq rk(R_L) = \|S_{R_L}\|.$$

Thus every DFA accepting L has at least as many states as A_{R_L}, so A_{R_L} is a minimal DFA for L. □

3.65. **Prove $S^c = \{\overline{\delta}(s_0, x) : |x| \leq \|S\|\}$.**

By Exercise 3.52: $\exists i \leq |S|$ with $C_i = S^c$. So $S^c = C_i = \{\overline{\delta}(s_0, x) : |x| \leq i\} \subseteq \{\overline{\delta}(s_0, x) : |x| \leq |S|\}$. The other inclusion is trivial (all strings of length $\leq |S|$ reach states in S^c). □

3.66. **Prove $s\,E_A\,t \Leftrightarrow L(A^s) = L(A^t)$.**

This is the definition of E_A. Formally: $s\,E_A\,t$ is defined as $L(A^s) = L(A^t)$ where $A^s = \langle \Sigma, S, s, \delta, F\rangle$. The equivalence is by definition. (The substantive content is showing E_A satisfies the right congruence property and the iterative characterization from Exercises 3.3–3.4.) □

3.67. **Properties of initial and terminal sets.**

(a) $\{I(A,t) \mid t \in S\}$ partitions Σ^*: Each $x \in \Sigma^*$ reaches exactly one state $\overline{\delta}(s_0, x) = t$. So $x \in I(A,t)$ for exactly one t. The sets are disjoint and cover Σ^*. □

(b) Terminal sets need not partition Σ^*. For example, let $A = \langle\{\boldsymbol{a}\}, \{q_0, q_1\}, q_0, \delta, \{q_0, q_1\}\rangle$ with $\delta(q_0, \boldsymbol{a}) = q_0$ and $\delta(q_1, \boldsymbol{a}) = q_1$. Then

$$T(A, q_0) = \Sigma^* = T(A, q_1).$$

Thus the terminal sets are not disjoint, so they do not form a partition.

(c) Terminal sets can partition Σ^*. Let $A = \langle\{\boldsymbol{a}, \boldsymbol{b}\}, \{q_0, q_1\}, q_0, \delta, \{q_0\}\rangle$ where both letters toggle the states:

$$\delta(q_0, \boldsymbol{a}) = \delta(q_0, \boldsymbol{b}) = q_1, \qquad \delta(q_1, \boldsymbol{a}) = \delta(q_1, \boldsymbol{b}) = q_0.$$

Then $T(A, q_0)$ is the set of even-length strings, while $T(A, q_1)$ is the set of odd-length strings. These two sets are disjoint and their union is all of Σ^*, so in this example the terminal sets do form a partition.

Chapter 4

Nondeterministic Finite Automata — Solutions

4.1. Using the local Figure 4.25, the connected deterministic equivalents are as follows. In each table, $\emptyset$ is the dead state and all missing subsets from $\wp(S)$ are disconnected.

(a) Figure 4.25(a): start state $\{t\}$, final state $\{t\}$.

	a	***b***	***c***
$\{t\}$	$\{t\}$	$\emptyset$	$\{t\}$
$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$

(b) Figure 4.25(b): start state $\{q_1\}$, final state $\{q_1\}$.

	a	***b***	***c***
$\{q_1\}$	$\{q_0\}$	$\{q_1\}$	$\emptyset$
$\{q_0\}$	$\emptyset$	$\{q_1\}$	$\{q_0\}$
$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$

(c) Figure 4.25(c): start state $\{r_0\}$, final states $\{r_0\}$ and $\{r_0, r_1\}$.

	a	***b***	***c***
$\{r_0\}$	$\{r_0, r_1\}$	$\emptyset$	$\{r_1\}$
$\{r_0, r_1\}$	$\{r_0, r_1\}$	$\{r_0\}$	$\{r_1\}$
$\{r_1\}$	$\emptyset$	$\{r_0\}$	$\emptyset$
$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$

(d) Figure 4.25(d): start state $\{s_0\}$, final state $\{s_2\}$.

	a	***b***	***c***
$\{s_0\}$	$\{s_1\}$	$\emptyset$	$\emptyset$
$\{s_1\}$	$\{s_2\}$	$\{s_3\}$	$\emptyset$
$\{s_2\}$	$\emptyset$	$\emptyset$	$\emptyset$
$\{s_3\}$	$\emptyset$	$\{s_1\}$	$\emptyset$
$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$

(e) Figure 4.25(e): start state $\{s_1\}$, final state $\{s_1, s_2\}$.

	$\boldsymbol{a}$	$\boldsymbol{b}$	$\boldsymbol{c}$
$\{s_1\}$	$\{s_1, s_2\}$	$\emptyset$	$\emptyset$
$\{s_1, s_2\}$	$\{s_1, s_2\}$	$\{s_1\}$	$\emptyset$
$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$

(f) Figure 4.25(f): after taking the λ-closures, the connected DFA has start state $\{s_0\}$ and final state $\{s_0, s_1, s_3\}$.

	$\boldsymbol{a}$	$\boldsymbol{b}$	$\boldsymbol{c}$
$\{s_0\}$	$\{s_0, s_1\}$	$\emptyset$	$\emptyset$
$\{s_0, s_1\}$	$\{s_0, s_1\}$	$\{s_2\}$	$\emptyset$
$\{s_2\}$	$\emptyset$	$\emptyset$	$\{s_0, s_1, s_3\}$
$\{s_0, s_1, s_3\}$	$\{s_0, s_1\}$	$\{s_2\}$	$\emptyset$
$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$

(g) Figure 4.25(g): start state $\{s_0\}$, final state $\{s_0\}$.

	$\boldsymbol{a}$	$\boldsymbol{b}$	$\boldsymbol{c}$
$\{s_0\}$	$\{s_1\}$	$\emptyset$	$\emptyset$
$\{s_1\}$	$\{s_0\}$	$\{s_3\}$	$\{s_2\}$
$\{s_2\}$	$\emptyset$	$\emptyset$	$\{s_1\}$
$\{s_3\}$	$\emptyset$	$\{s_1\}$	$\emptyset$
$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$

4.2. Consider Example 4.17, that is, the automaton E of Figure 4.23. Reading the figure gives

$$E = \langle \{\boldsymbol{a}, \boldsymbol{b}\}, \{s_1, s_2, s_3\}, \{s_1\}, \delta_\lambda, \{s_2\} \rangle$$

with ordinary transitions

$$\delta_\lambda(s_1, \boldsymbol{a}) = \{s_1, s_2\}, \qquad \delta_\lambda(s_1, \boldsymbol{b}) = \{s_3\},$$
$$\delta_\lambda(s_2, \boldsymbol{a}) = \{s_1\}, \qquad \delta_\lambda(s_2, \boldsymbol{b}) = \{s_1\},$$
$$\delta_\lambda(s_3, \boldsymbol{a}) = \emptyset, \qquad \delta_\lambda(s_3, \boldsymbol{b}) = \{s_1\},$$

and one λ-transition $\delta_\lambda(s_1, \lambda) = \{s_2\}$.

(a) *Remove λ-transitions (Definition 4.9).* The λ-closures are:

$$\Lambda(s_1) = \{s_1, s_2\}, \qquad \Lambda(s_2) = \{s_2\}, \qquad \Lambda(s_3) = \{s_3\}.$$

Since $\lambda \in L(E)$, Definition 4.9 adds the isolated start state q_0 and also makes q_0 final. Thus

$$E^\sigma_\lambda = \langle \{\boldsymbol{a}, \boldsymbol{b}\}, \{q_0, s_1, s_2, s_3\}, \{q_0, s_1\}, \delta^\sigma_\lambda, \{q_0, s_2\} \rangle$$

with

$$\delta^\sigma_\lambda(q_0, \boldsymbol{a}) = \delta^\sigma_\lambda(q_0, \boldsymbol{b}) = \emptyset,$$
$$\delta^\sigma_\lambda(s_1, \boldsymbol{a}) = \{s_1, s_2\}, \qquad \delta^\sigma_\lambda(s_1, \boldsymbol{b}) = \{s_1, s_2, s_3\},$$
$$\delta^\sigma_\lambda(s_2, \boldsymbol{a}) = \{s_1, s_2\}, \qquad \delta^\sigma_\lambda(s_2, \boldsymbol{b}) = \{s_1, s_2\},$$
$$\delta^\sigma_\lambda(s_3, \boldsymbol{a}) = \emptyset, \qquad \delta^\sigma_\lambda(s_3, \boldsymbol{b}) = \{s_1, s_2\}.$$

(b) *Convert this NDFA into a DFA (Definition 4.5).* The full deterministic machine has $2^4 = 16$ subset states, but only three are reachable from the start set $\{q_0, s_1\}$:

$$Q_0 = \{q_0, s_1\}, \qquad Q_1 = \{s_1, s_2\}, \qquad Q_2 = \{s_1, s_2, s_3\}.$$

All three are final, because $q_0 \in Q_0$ and $s_2 \in Q_1 \cap Q_2$. Their transitions are

	$\boldsymbol{a}$	$\boldsymbol{b}$
Q_0	Q_1	Q_2
Q_1	Q_1	Q_2
Q_2	Q_1	Q_2

so the connected part of the DFA is a 3-state machine.

4.3. Consider the NDFA A from Example 4.4: $A = \langle\{\boldsymbol{a}, \boldsymbol{b}\}, \{r, s, t\}, \{r, s\}, \delta, \{t\}\rangle$ with

$$\delta(r, \boldsymbol{a}) = \{s\}, \quad \delta(r, \boldsymbol{b}) = \emptyset, \quad \delta(s, \boldsymbol{a}) = \emptyset, \quad \delta(s, \boldsymbol{b}) = \{r, t\},$$

$$\delta(t, \boldsymbol{a}) = \emptyset, \qquad \delta(t, \boldsymbol{b}) = \{t\}.$$

(a) *Circuit diagram (no <SOS> or <EOS>).* Using one activity bit for each state, the next-state equations are

$$r' = s \wedge \boldsymbol{b}, \qquad s' = r \wedge \boldsymbol{a}, \qquad t' = (s \wedge \boldsymbol{b}) \vee (t \wedge \boldsymbol{b}).$$

Initially the active states are r and s, and t is inactive.

(b) *Circuit diagram (with <SOS> and <EOS>).* <SOS> resets the activity bits to $r = s = 1,\ t = 0$. The accept line is $t \wedge$ <EOS>.

(c) *DFA via subset construction (connected portion only).* The start state of A^d is $\{r, s\}$. The reachable states are

$$\{r, s\}, \quad \{s\}, \quad \{r, t\}, \quad \{t\}, \quad \emptyset.$$

Their transition table is

State	$\boldsymbol{a}$	$\boldsymbol{b}$
$\{r, s\}$	$\{s\}$	$\{r, t\}$
$\{s\}$	$\emptyset$	$\{r, t\}$
$\{r, t\}$	$\{s\}$	$\{t\}$
$\{t\}$	$\emptyset$	$\{t\}$
$\emptyset$	$\emptyset$	$\emptyset$

The final states are $\{r, t\}$ and $\{t\}$.

(d) *DFA including disconnected portion.* The full state set is

$$\wp(\{r, s, t\}) = \{\emptyset, \{r\}, \{s\}, \{t\}, \{r, s\}, \{r, t\}, \{s, t\}, \{r, s, t\}\}.$$

Its transition table is

State	$\boldsymbol{a}$	$\boldsymbol{b}$
$\emptyset$	$\emptyset$	$\emptyset$
$\{r\}$	$\{s\}$	$\emptyset$
$\{s\}$	$\emptyset$	$\{r,t\}$
$\{t\}$	$\emptyset$	$\{t\}$
$\{r,s\}$	$\{s\}$	$\{r,t\}$
$\{r,t\}$	$\{s\}$	$\{t\}$
$\{s,t\}$	$\emptyset$	$\{r,t\}$
$\{r,s,t\}$	$\{s\}$	$\{r,t\}$

Final states are exactly those subsets containing t.

4.4. Consider the NDFA from Example 4.2, which has multiple transitions on **0**: from s_0 to both s_1 and s_2.

(a) *Circuit diagram (with <SOS> and <EOS>).* Use one activity flip-flop for each state $s_0, \ldots, s_5$. The next-state equations are

$$s_0' = \text{<SOS>}, \qquad s_1' = (s_0 \wedge \mathbf{0}) \vee (s_1 \wedge \mathbf{0}) \vee (s_5 \wedge \mathbf{1}),$$

$$s_2' = (s_0 \wedge \mathbf{0}) \vee (s_0 \wedge \mathbf{1}) \vee (s_4 \wedge \mathbf{0}) \vee (s_4 \wedge \mathbf{1}),$$

$$s_3' = (s_0 \wedge \mathbf{1}) \vee (s_1 \wedge \mathbf{1}) \vee (s_3 \wedge \mathbf{0}),$$

$$s_4' = (s_2 \wedge \mathbf{0}) \vee (s_2 \wedge \mathbf{1}), \qquad s_5' = (s_3 \wedge \mathbf{1}) \vee (s_5 \wedge \mathbf{0}).$$

Since the final states in Figure 4.3 are s_0, s_1, and s_4, the accept line is

$$(s_0 \vee s_1 \vee s_4) \wedge \text{<EOS>}.$$

(b) *DFA via subset construction.* Starting from the set of start state $\{s_0\}$, the connected portion of the subset DFA has the following reachable states:

$$Q_0 = \{s_0\}, \quad Q_1 = \{s_1, s_2\}, \quad Q_2 = \{s_2, s_3\}, \quad Q_3 = \{s_1, s_4\}, \quad Q_4 = \{s_3, s_4\},$$

$$Q_5 = \{s_4, s_5\}, \quad Q_6 = \{s_2, s_5\}.$$

Their transitions are

	0	**1**
Q_0	Q_1	Q_2
Q_1	Q_3	Q_4
Q_2	Q_4	Q_5
Q_3	Q_1	Q_2
Q_4	Q_2	Q_6
Q_5	Q_6	Q_3
Q_6	Q_5	Q_3

The final states are exactly those subsets containing one of the original final states s_0, s_1, or s_4, namely

$$Q_0,\ Q_1,\ Q_3,\ Q_4,\ Q_5.$$

4.5. Consider the NDFA from Example 4.3, which accepts the same language as Example 4.2 but uses multiple start states instead of multiple transitions.

(a) *Circuit diagram (no <SOS> or <EOS>).* Again use one activity flip-flop per state. For the automaton of Figure 4.4,

$$s_1' = (s_1 \wedge \mathbf{0}) \vee (s_5 \wedge \mathbf{0}), \qquad s_2' = (s_4 \wedge \mathbf{0}) \vee (s_4 \wedge \mathbf{1}),$$
$$s_3' = (s_1 \wedge \mathbf{1}) \vee (s_3 \wedge \mathbf{0}), \qquad s_4' = (s_2 \wedge \mathbf{0}) \vee (s_2 \wedge \mathbf{1}),$$
$$s_5' = s_3 \wedge \mathbf{1}.$$

The initial configuration has $s_1 = s_4 = 1$ and the other activity bits equal to 0. Since the final states are s_1 and s_4, the accept line is $s_1 \vee s_4$.

(b) *Circuit diagram (with <SOS> and <EOS>).* Use the same next-state equations as in part (a), but OR the start pulse into the two start-state inputs:

$$s_1' = \text{<SOS>} \vee (s_1 \wedge \mathbf{0}) \vee (s_5 \wedge \mathbf{0}), \qquad s_4' = \text{<SOS>} \vee (s_2 \wedge \mathbf{0}) \vee (s_2 \wedge \mathbf{1}),$$

with the equations for s_2', s_3', and s_5' unchanged. The accept output is

$$(s_1 \vee s_4) \wedge \text{<EOS>}.$$

(c) *DFA (connected portion only).* The start state of A^d is the set of all start states, namely $\{s_1, s_4\}$. The reachable subsets are

$$P_0 = \{s_1, s_4\}, \quad P_1 = \{s_1, s_2\}, \quad P_2 = \{s_2, s_3\}, \quad P_3 = \{s_3, s_4\}, \quad P_4 = \{s_4, s_5\}, \quad P_5 = \{s_2, s_5\},$$
$$P_6 = \{s_2\}, \qquad P_7 = \{s_4\}.$$

Their transitions are

	0	**1**
P_0	P_1	P_2
P_1	P_0	P_3
P_2	P_3	P_4
P_3	P_2	P_5
P_4	P_1	P_6
P_5	P_0	P_7
P_6	P_7	P_7
P_7	P_6	P_6

The final states are

$$P_0,\ P_1,\ P_3,\ P_4,\ P_7,$$

since these are exactly the reachable subsets containing s_1 or s_4.

(d) *Is this DFA isomorphic to any of those in Exercise 4.4?* No. The DFA from Exercise 4.4(b) has seven reachable states

$$Q_0, \ldots, Q_6,$$

while the DFA from part (c) above has eight reachable states

$$P_0, \ldots, P_7.$$

Since isomorphic automata must have the same number of states, these two subset-construction DFAs are not isomorphic. They do accept the same language, but that only implies that their *minimal* DFAs are isomorphic, not these unreduced machines.

4.6. Consider the λ-NDFA from Example 4.14, with states $\{s_0, s_1, s_2, s_3\}$ and λ-transitions $s_0 \to s_1$ and $s_1 \to s_2$.

(a) *Circuit diagram for the original λ-NDFA (no SOS/EOS).* Figure 4.19 uses four activity bits, one for each state. Ignoring the λ-moves for a moment, the ordinary transitions give

$$s_0' = 0, \qquad s_1' = (s_0 \wedge \boldsymbol{a}) \vee (s_3 \wedge \boldsymbol{b}),$$

$$s_2' = (s_1 \wedge \boldsymbol{c}) \vee (s_2 \wedge \boldsymbol{d}), \qquad s_3' = s_1 \wedge \boldsymbol{b}.$$

The λ-moves $s_0 \to s_1$ and $s_1 \to s_2$ are implemented exactly as in Example 4.17 by OR-ing the source activity line directly into the destination input. Thus the circuit can be specified by

$$D_0 = 0, \qquad D_1 = s_0 \vee (s_0 \wedge \boldsymbol{a}) \vee (s_3 \wedge \boldsymbol{b}),$$

$$D_2 = s_1 \vee (s_1 \wedge \boldsymbol{c}) \vee (s_2 \wedge \boldsymbol{d}), \qquad D_3 = s_1 \wedge \boldsymbol{b}.$$

Initially only s_0 is active; the accept output is simply s_2.

(b) *Circuit diagram for the NDFA from Definition 4.9 (part (b) of Example 4.14, no SOS/EOS).* The corresponding λ-free automaton is the one shown in Figure 4.20. Use one activity bit for each of q_0, s_0, s_1, s_2, s_3; then

$$q_0' = 0, \qquad s_0' = 0,$$

$$s_1' = (s_0 \wedge \boldsymbol{a}) \vee (s_3 \wedge \boldsymbol{b}),$$

$$s_2' = (s_0 \wedge \boldsymbol{a}) \vee (s_0 \wedge \boldsymbol{c}) \vee (s_0 \wedge \boldsymbol{d}) \vee (s_1 \wedge \boldsymbol{c}) \vee (s_1 \wedge \boldsymbol{d}) \vee (s_3 \wedge \boldsymbol{b}) \vee (s_2 \wedge \boldsymbol{d}),$$

$$s_3' = (s_0 \wedge \boldsymbol{b}) \vee (s_1 \wedge \boldsymbol{b}).$$

The initial activity bits are $q_0 = s_0 = 1$ and $s_1 = s_2 = s_3 = 0$, and the accept output is $q_0 \vee s_2$.

4.7. Consider the second NDFA of Example 4.16, which accepts the language $L^+ = \{x_1 x_2 \cdots x_k \mid k \geq 1,\ x_i \in L\}$ where L is the language of strings over $\{\boldsymbol{a}, \boldsymbol{b}, \boldsymbol{c}\}$ containing exactly one $\boldsymbol{b}$ immediately followed by $\boldsymbol{c}$.

(a) *Circuit diagram for A_λ^σ (with EOS but no SOS).* For the λ-NDFA of Figure 4.22, use one activity bit for each of s_0, s_1, s_2. The ordinary transitions give

$$s_0' = s_0 \wedge (\boldsymbol{a} \vee \boldsymbol{c}), \qquad s_1' = s_0 \wedge \boldsymbol{b},$$

$$s_2' = (s_1 \wedge \boldsymbol{c}) \vee (s_2 \wedge (\boldsymbol{a} \vee \boldsymbol{c})).$$

Because Figure 4.22 also has the λ-transition $s_2 \to s_0$, OR the activity line for s_2 directly into the input for s_0. Thus

$$D_0 = s_2 \vee (s_0 \wedge (\boldsymbol{a} \vee \boldsymbol{c})), \qquad D_1 = s_0 \wedge \boldsymbol{b},$$

$$D_2 = (s_1 \wedge \boldsymbol{c}) \vee (s_2 \wedge (\boldsymbol{a} \vee \boldsymbol{c})).$$

The accept line is $s_2 \wedge$ <EOS>.

(b) *NDFA without λ-transitions (Definition 4.9).* For Figure 4.22 the λ-closures are

$$\Lambda(s_0) = \{s_0\}, \qquad \Lambda(s_1) = \{s_1\}, \qquad \Lambda(s_2) = \{s_0, s_2\}.$$

Since $\lambda \notin L(A_\lambda)$, Definition 4.9 adds q_0 as an extra disconnected start state but does not make it final. Thus

$$A'_\lambda = \langle \{\boldsymbol{a}, \boldsymbol{b}, \boldsymbol{c}\}, \{q_0, s_0, s_1, s_2\}, \{q_0, s_0\}, \delta'_\lambda, \{s_2\} \rangle,$$

where

$$\delta'_\lambda(q_0, \boldsymbol{a}) = \delta'_\lambda(q_0, \boldsymbol{b}) = \delta'_\lambda(q_0, \boldsymbol{c}) = \emptyset,$$
$$\delta'_\lambda(s_0, \boldsymbol{a}) = \{s_0\}, \qquad \delta'_\lambda(s_0, \boldsymbol{b}) = \{s_1\}, \qquad \delta'_\lambda(s_0, \boldsymbol{c}) = \{s_0\},$$
$$\delta'_\lambda(s_1, \boldsymbol{a}) = \emptyset, \qquad \delta'_\lambda(s_1, \boldsymbol{b}) = \emptyset, \qquad \delta'_\lambda(s_1, \boldsymbol{c}) = \{s_0, s_2\},$$
$$\delta'_\lambda(s_2, \boldsymbol{a}) = \{s_0, s_2\}, \qquad \delta'_\lambda(s_2, \boldsymbol{b}) = \{s_1\}, \qquad \delta'_\lambda(s_2, \boldsymbol{c}) = \{s_0, s_2\}.$$

(c) *Circuit diagram for the λ-free NDFA (with EOS, no SOS).* Use one activity bit for each of q_0, s_0, s_1, s_2. The next-state equations are

$$q'_0 = 0,$$
$$s'_0 = (s_0 \wedge \boldsymbol{a}) \vee (s_0 \wedge \boldsymbol{c}) \vee (s_1 \wedge \boldsymbol{c}) \vee (s_2 \wedge \boldsymbol{a}) \vee (s_2 \wedge \boldsymbol{c}),$$
$$s'_1 = (s_0 \wedge \boldsymbol{b}) \vee (s_2 \wedge \boldsymbol{b}),$$
$$s'_2 = (s_1 \wedge \boldsymbol{c}) \vee (s_2 \wedge \boldsymbol{a}) \vee (s_2 \wedge \boldsymbol{c}).$$

The initial configuration has $q_0 = s_0 = 1$ and $s_1 = s_2 = 0$, and the accept line is $s_2 \wedge$ <EOS>.

(d) *DFA (connected portion only).* Starting from $\{q_0, s_0\}$, the reachable states are

$$Q_0 = \{q_0, s_0\}, \qquad Q_1 = \{s_0\}, \qquad Q_2 = \{s_1\}, \qquad Q_3 = \emptyset, \qquad Q_4 = \{s_0, s_2\}.$$

Their transitions are

	$\boldsymbol{a}$	$\boldsymbol{b}$	$\boldsymbol{c}$
Q_0	Q_1	Q_2	Q_1
Q_1	Q_1	Q_2	Q_1
Q_2	Q_3	Q_3	Q_4
Q_3	Q_3	Q_3	Q_3
Q_4	Q_4	Q_2	Q_4

The only final state is Q_4.

4.8. **Complementing the final states of an NDFA does not yield a machine accepting the complement language.**

For a DFA, $L(\overline{A}) = \Sigma^* \setminus L(A)$ because every word follows exactly one path and is accepted iff it ends in a final state. Flipping final and nonfinal states reverses acceptance on that unique path.

For an NDFA, a word w may follow *multiple* paths, some ending in final states and some not. Under Definition 4.4, $w \in L(A)$ iff *at least one* path ends in a final state. If we complement the final states, a word is accepted by the modified machine iff at least one path ends in a *formerly nonfinal* state. This is *not* the same as $w \notin L(A)$, because a word can simultaneously have a path to a (formerly) final state and a path to a (formerly) nonfinal state; the complemented machine would accept it, but it was already in $L(A)$.

Example. Let $\Sigma = \{\boldsymbol{a}\}$, $S = \{s_0, s_1\}$, $S_0 = \{s_0\}$, $F = \{s_1\}$, $\delta(s_0, \boldsymbol{a}) = \{s_0, s_1\}$, $\delta(s_1, \boldsymbol{a}) = \emptyset$. Then $\boldsymbol{a} \in L(A)$ because $s_1 \in \delta(s_0, \boldsymbol{a})$. After complementing, $F' = \{s_0\}$, and $\boldsymbol{a}$ is still accepted (via the path $s_0 \to s_0 \in F'$), yet $\boldsymbol{a} \notin \Sigma^* \setminus L(A)$ since $\boldsymbol{a} \in L(A)$. Hence the complement-of-final-states trick fails for NDFAs.

4.9. **Given an NDFA A without λ-transitions, construct a λ-NDFA A' with exactly one start state, exactly one final state, and $L(A') = L(A)$.**

Construction. Let $A = \langle \Sigma, S, S_0, \delta, F \rangle$. Define $A' = \langle \Sigma, S', q_0, \delta', \{q_f\} \rangle$ where:

- $S' = S \cup \{q_0, q_f\}$ (two new states).
- q_0 is the unique start state.
- $\delta'(q_0, \lambda) = S_0$ (a λ-transition from q_0 to every start state of A).
- For all $s \in S$ and $\boldsymbol{a} \in \Sigma$: $\delta'(s, \boldsymbol{a}) = \delta(s, \boldsymbol{a})$.
- $\delta'(f, \lambda) \ni q_f$ for every $f \in F$ (λ-transitions from each old final state to the new unique final state q_f).
- $\delta'(q_f, \boldsymbol{a}) = \emptyset$ for all $\boldsymbol{a}$ (and no transitions out of q_f).

Correctness. A word $w \in \Sigma^+$ is accepted by A iff there is a path $q_0 \xrightarrow{\lambda} s_i \xrightarrow{w} f$ for some $s_i \in S_0$ and $f \in F$; by the λ-transition $f \xrightarrow{\lambda} q_f$, this same path exists in A' ending at q_f. Conversely, any accepting path in A' uses the initial λ-step to some $s_i \in S_0$, processes w within S (by identical transitions), then uses a final λ-step to q_f, which corresponds to an accepting path in A.

If $\lambda \in L(A)$, add the extra λ-transition $\delta'(q_0, \lambda) \ni q_f$. Then λ is accepted while A' still has exactly one start state (q_0) and exactly one final state (q_f). Hence $L(A') = L(A)$ in all cases.

4.10. **Can part (ii) of Definition 4.8 be deduced from parts (i) and (iii)?**

Definition 4.8 defines $\overline{\delta}$ for NDFAs:

(i) $\overline{\delta}(s, \lambda) = \{s\}$,

(ii) $(\forall \boldsymbol{a} \in \Sigma)\, \overline{\delta}(s, \boldsymbol{a}) = \delta(s, \boldsymbol{a})$,

(iii) $(\forall x \in \Sigma^*)(\forall \boldsymbol{a} \in \Sigma)\, \overline{\delta}(s, x\boldsymbol{a}) = \bigcup_{t \in \overline{\delta}(s,x)} \delta(t, \boldsymbol{a})$.

Part (ii) concerns single-letter strings. Setting $x = \lambda$ in part (iii):

$$\overline{\delta}(s, \lambda\boldsymbol{a}) = \bigcup_{t \in \overline{\delta}(s,\lambda)} \delta(t, \boldsymbol{a}) = \bigcup_{t \in \{s\}} \delta(t, \boldsymbol{a}) = \delta(s, \boldsymbol{a}),$$

where we used part (i) to obtain $\overline{\delta}(s, \lambda) = \{s\}$ and the fact that $\lambda\boldsymbol{a} = \boldsymbol{a}$. Since $\lambda\boldsymbol{a} = \boldsymbol{a}$, this gives $\overline{\delta}(s, \boldsymbol{a}) = \delta(s, \boldsymbol{a})$, which is exactly part (ii).

Conclusion. Yes, part (ii) can be derived from parts (i) and (iii). It is included explicitly to make the base case of inductive proofs more transparent, but it is logically redundant.

4.11. **Alternative construction for removing λ-transitions.**

Define $S' = S$, $S'_0 = \Lambda(S_0)$, $F' = F$, and $\delta'(s, \boldsymbol{a}) = \overline{\delta}_\lambda(s, \boldsymbol{a})$ for all $s \in S$, $\boldsymbol{a} \in \Sigma$.

This construction does work. Let

$$A' = \langle \Sigma, S, \Lambda(S_0), \delta', F \rangle.$$

We show $L(A') = L(A_\lambda)$.

First, for every $s \in S$ and every nonempty word $x \in \Sigma^+$,

$$\overline{\delta'}(s,x) = \overline{\delta}_\lambda(s,x).$$

This is proved by the same induction as Lemma 4.2: for a single letter $\boldsymbol{a}$, it is just the definition $\delta'(s,\boldsymbol{a}) = \overline{\delta}_\lambda(s,\boldsymbol{a})$, and the inductive step extends in the usual nondeterministic manner.

Case 1: $x = \lambda$**.** Then

$$\lambda \in L(A') \iff \Lambda(S_0) \cap F \neq \emptyset.$$

But this says exactly that some start state of A_λ can reach a final state by λ-moves alone, which is the condition $\lambda \in L(A_\lambda)$.

Case 2: $x \in \Sigma^+$**.** Using the identity above,

$$x \in L(A') \iff \Big(\bigcup_{t \in \Lambda(S_0)} \overline{\delta'}(t,x)\Big) \cap F \neq \emptyset \iff \Big(\bigcup_{t \in \Lambda(S_0)} \overline{\delta}_\lambda(t,x)\Big) \cap F \neq \emptyset.$$

If $t \in \Lambda(S_0)$, then $t \in \Lambda(s_0)$ for some $s_0 \in S_0$, so the initial λ-moves taking s_0 to t may be prefixed to any path counted in $\overline{\delta}_\lambda(t,x)$. Hence

$$\bigcup_{t \in \Lambda(S_0)} \overline{\delta}_\lambda(t,x) = \bigcup_{s_0 \in S_0} \overline{\delta}_\lambda(s_0,x),$$

and therefore

$$x \in L(A') \iff \Big(\bigcup_{s_0 \in S_0} \overline{\delta}_\lambda(s_0,x)\Big) \cap F \neq \emptyset \iff x \in L(A_\lambda).$$

So this is a valid alternative to Definition 4.9: instead of adding q_0, it enlarges the start-state set to $\Lambda(S_0)$.

4.12. **Algorithm for union of two FAD languages using λ-NDFAs.**

Let $L_1 = L(A_1)$ and $L_2 = L(A_2)$ where A_1 and A_2 are λ-NDFAs. By Exercise 4.9 we may assume each A_i has exactly one start state q_0^i and exactly one final state q_f^i. Construct A' as follows:

(1) Introduce a new start state q_0 and a new final state q_f.

(2) Add λ-transitions: $q_0 \xrightarrow{\lambda} q_0^1$ and $q_0 \xrightarrow{\lambda} q_0^2$.

(3) Add λ-transitions: $q_f^1 \xrightarrow{\lambda} q_f$ and $q_f^2 \xrightarrow{\lambda} q_f$.

(4) Keep all existing transitions of A_1 and A_2.

Then $L(A') = L(A_1) \cup L(A_2)$: a word is accepted iff it is accepted by A_1 or by A_2, because the λ-transitions route the computation into one of the two machines and collect the result at q_f.

4.13. **Example of NDFA A such that A^d is not connected, not reduced, but is minimal.**

(a) A^d is not connected. Any NDFA in which some subsets of states are never reachable from the start set under the transition function will produce an A^d with unreachable states. For example, let $\Sigma = \{\boldsymbol{a}\}$, $S = \{s_0, s_1\}$, $S_0 = \{s_0\}$, $F = \{s_1\}$, $\delta(s_0,\boldsymbol{a}) = \{s_1\}$, $\delta(s_1,\boldsymbol{a}) = \emptyset$. Then A^d has states $\{\emptyset, \{s_0\}, \{s_1\}, \{s_0,s_1\}\}$ but $\{s_0,s_1\}$ and $\{s_1\}$ with $\delta^d(\{s_1\},\boldsymbol{a}) = \emptyset$ are reachable, while $\{s_0,s_1\}$ is not reached from $\{s_0\}$.

(b) A^d **is not reduced.** Use a different example:

$$A = \langle \{\boldsymbol{a}, \boldsymbol{b}\}, \{p, u, v\}, \{p\}, \delta, \{u, v\} \rangle$$

where

$$\delta(p, \boldsymbol{a}) = \{u\}, \qquad \delta(p, \boldsymbol{b}) = \{v\},$$

and all other transitions are empty. Then the reachable subset states of A^d are

$$\{p\}, \qquad \{u\}, \qquad \{v\}, \qquad \emptyset.$$

Both $\{u\}$ and $\{v\}$ are final, and from either one every input leads to $\emptyset$. Therefore

$$\{u\} \; E_{A^d} \; \{v\},$$

so A^d is not reduced.

(c) A^d **is minimal.** Here we use a different example. Let $A = \langle \{\boldsymbol{a}, \boldsymbol{b}\}, \{s\}, \{s\}, \delta, \{s\} \rangle$ where

$$\delta(s, \boldsymbol{a}) = \{s\}, \qquad \delta(s, \boldsymbol{b}) = \emptyset.$$

Then $L(A) = \boldsymbol{a}^*$. The deterministic machine A^d has exactly two states, $\{s\}$ and $\emptyset$, with

$$\delta^d(\{s\}, \boldsymbol{a}) = \{s\}, \quad \delta^d(\{s\}, \boldsymbol{b}) = \emptyset, \quad \delta^d(\emptyset, \boldsymbol{a}) = \delta^d(\emptyset, \boldsymbol{b}) = \emptyset.$$

This is the standard 2-state DFA for $\boldsymbol{a}^*$ over the alphabet $\{\boldsymbol{a}, \boldsymbol{b}\}$, so A^d is minimal.

4.14. **Why was the extra state q_0 included in A^σ_λ?**

The role of q_0 is to preserve acceptance of λ while leaving the original start states and all ordinary transitions unchanged. In Definition 4.9 the extra state q_0 is disconnected from the rest of the machine, and

$$q_0 \in F^\sigma \iff \lambda \in L(A_\lambda).$$

Thus q_0 serves as a separate witness for the empty word.

Example. Let

$$A_\lambda = \langle \{\boldsymbol{a}\}, \{s_0, f\}, \{s_0\}, \delta_\lambda, \{f\} \rangle$$

where $\delta_\lambda(s_0, \lambda) = \{f\}$ and all ordinary transitions are empty. Then $\lambda \in L(A_\lambda)$ because $f \in \Lambda(s_0)$.

If we try to remove λ-moves *without* adding q_0 and keep the same start state s_0, then the new machine has no way to accept λ: no input is read, and s_0 is not final. The disconnected state q_0 fixes exactly this problem. We keep s_0 for all nonempty words, and we make q_0 final so that the empty word is still accepted.

4.15. **Union construction using NDFAs *without* λ-transitions.**

(a) **Algorithm.** Let $A_1 = \langle \Sigma, S_1, S_0^1, \delta_1, F_1 \rangle$ and $A_2 = \langle \Sigma, S_2, S_0^2, \delta_2, F_2 \rangle$ (assume $S_1 \cap S_2 = \emptyset$). Construct $A' = \langle \Sigma, S_1 \cup S_2, S_0^1 \cup S_0^2, \delta', F_1 \cup F_2 \rangle$ where $\delta'(s, \boldsymbol{a}) = \delta_i(s, \boldsymbol{a})$ for $s \in S_i$. The machine A' starts in the combined set of initial states and accepts any word accepted by either A_1 or A_2. Thus $L(A') = L(A_1) \cup L(A_2)$.

(b) **Comparison.** This is actually *easier* than the λ-transition approach (Exercise 4.12): no new states are needed at all, since we simply merge the state sets and initial-state sets.

(c) **Can we ensure exactly one start state and one final state?** Not in full generality. The disjoint-union construction in part (a) gives a correct union machine, but it usually has several start states and several final states. If the resulting language does *not* contain λ, then Exercise 4.22 shows how to convert a no-λ NDFA into another no-λ NDFA with exactly one start state and exactly one final state. But Exercise 4.23 shows that this can fail when λ is in the language. So the correct conclusion is: one start state and one final state can be forced under the hypothesis $\lambda \notin L$, but not uniformly in general.

4.16. Consider the λ-NDFA A_λ^σ from Example 4.15 (the machine without λ-transitions corresponding to the automaton of Example 4.14).

(a) *Circuit diagram (no SOS or EOS).* Example 4.15 gives the λ-free machine of Figure 4.20, so we can use one activity bit for each of q_0, s_0, s_1, s_2, s_3. The next-state equations are

$$q_0' = 0, \qquad s_0' = 0,$$

$$s_1' = (s_0 \wedge \boldsymbol{a}) \vee (s_3 \wedge \boldsymbol{b}),$$

$$s_2' = (s_0 \wedge \boldsymbol{a}) \vee (s_0 \wedge \boldsymbol{c}) \vee (s_0 \wedge \boldsymbol{d}) \vee (s_1 \wedge \boldsymbol{c}) \vee (s_1 \wedge \boldsymbol{d}) \vee (s_3 \wedge \boldsymbol{b}) \vee (s_2 \wedge \boldsymbol{d}),$$

$$s_3' = (s_0 \wedge \boldsymbol{b}) \vee (s_1 \wedge \boldsymbol{b}).$$

The initial configuration has $q_0 = s_0 = 1$ and $s_1 = s_2 = s_3 = 0$. Since q_0 and s_2 are final states in Example 4.15, the accept output is $q_0 \vee s_2$.

(b) *Convert to $A_\lambda^{\sigma d}$ (connected portion only).* In Example 4.15, the start states of A_λ^σ are

$$\{q_0, s_0\},$$

and the final states are q_0 and s_2. The connected part of the deterministic equivalent has the reachable states

$$R_0 = \{q_0, s_0\}, \quad R_1 = \{s_1, s_2\}, \quad R_2 = \{s_3\}, \quad R_3 = \{s_2\}, \quad R_4 = \emptyset.$$

Their transitions are

	$\boldsymbol{a}$	$\boldsymbol{b}$	$\boldsymbol{c}$	$\boldsymbol{d}$
R_0	R_1	R_2	R_3	R_3
R_1	R_4	R_2	R_3	R_3
R_2	R_4	R_1	R_4	R_4
R_3	R_4	R_4	R_4	R_3
R_4	R_4	R_4	R_4	R_4

The final states are

$$R_0,\ R_1,\ R_3,$$

because these are exactly the reachable subsets containing q_0 or s_2.

4.17. (a) **Proof that $\overline{\delta}(s, \boldsymbol{a}) = \delta(s, \boldsymbol{a})$ for any NDFA without λ-transitions.**
By Definition 4.8(ii), $\overline{\delta}(s, \boldsymbol{a}) = \delta(s, \boldsymbol{a})$ for all $s \in S$ and $\boldsymbol{a} \in \Sigma$. This is immediate from the definition; alternatively, apply part (iii) with $x = \lambda$:

$$\overline{\delta}(s, \lambda\boldsymbol{a}) = \bigcup_{t \in \overline{\delta}(s,\lambda)} \delta(t, \boldsymbol{a}) = \bigcup_{t \in \{s\}} \delta(t, \boldsymbol{a}) = \delta(s, \boldsymbol{a}),$$

using part (i): $\overline{\delta}(s, \lambda) = \{s\}$. Since $\lambda\boldsymbol{a} = \boldsymbol{a}$, we conclude $\overline{\delta}(s, \boldsymbol{a}) = \delta(s, \boldsymbol{a})$. □

(b) **Counterexample for λ-NDFAs.**
Let $\Sigma = \{\boldsymbol{a}\}$, $S = \{s_0, s_1\}$, $S_0 = \{s_0\}$, $F = \{s_1\}$, $\delta_\lambda(s_0, \lambda) = \{s_1\}$, $\delta_\lambda(s_0, \boldsymbol{a}) = \emptyset$, $\delta_\lambda(s_1, \boldsymbol{a}) = \emptyset$.
Then $\Lambda(s_0) = \{s_0, s_1\}$. So

$$\overline{\delta}_\lambda(s_0, \boldsymbol{a}) = \Lambda\Big(\bigcup_{t \in \Lambda(s_0)} \delta_\lambda(t, \boldsymbol{a})\Big) = \Lambda(\emptyset \cup \emptyset) = \emptyset,$$

but $\delta_\lambda(s_0, \boldsymbol{a}) = \emptyset$ also. Let us try a different example: $\delta_\lambda(s_0, \lambda) = \{s_1\}$, $\delta_\lambda(s_1, \boldsymbol{a}) = \{s_1\}$. Then $\delta_\lambda(s_0, \boldsymbol{a}) = \emptyset$, yet

$$\overline{\delta}_\lambda(s_0, \boldsymbol{a}) = \Lambda\Big(\bigcup_{t \in \{s_0, s_1\}} \delta_\lambda(t, \boldsymbol{a})\Big) = \Lambda(\{s_1\}) = \{s_1\} \neq \emptyset = \delta_\lambda(s_0, \boldsymbol{a}).$$

This demonstrates that $\overline{\delta}_\lambda(s, \boldsymbol{a}) \neq \delta_\lambda(s, \boldsymbol{a})$ is possible for λ-NDFAs.

4.18. Consider the NDFA accepting the original language L from Example 4.10: $L = \{x \in \{\boldsymbol{a}, \boldsymbol{b}\}^* \mid x$ begins with $\boldsymbol{a} \vee$ x contains $\boldsymbol{ba}\}$.

(a) *Circuit diagram (no SOS/EOS).* The NDFA has a start state q_0, two final states q_1, q_3, and the transitions of Figure 4.10. Using one activity bit per state, the next-state equations are

$$q_0' = 0, \qquad q_1' = (q_0 \wedge \boldsymbol{a}) \vee (q_1 \wedge \boldsymbol{a}) \vee (q_1 \wedge \boldsymbol{b}),$$

$$q_2' = (q_0 \wedge \boldsymbol{b}) \vee (q_2 \wedge \boldsymbol{b}), \qquad q_3' = (q_2 \wedge \boldsymbol{a}) \vee (q_3 \wedge \boldsymbol{a}) \vee (q_3 \wedge \boldsymbol{b}).$$

Initially only q_0 is active, and the accept line is $q_1 \vee q_3$.

(b) *DFA (connected portion only).* Apply the subset construction to Figure 4.10. Starting from $\{q_0\}$, the reachable states are

$$Q_0 = \{q_0\}, \qquad Q_1 = \{q_1\}, \qquad Q_2 = \{q_2\}, \qquad Q_3 = \{q_3\}.$$

Their transitions are

	$\boldsymbol{a}$	$\boldsymbol{b}$
Q_0	Q_1	Q_2
Q_1	Q_1	Q_1
Q_2	Q_3	Q_2
Q_3	Q_3	Q_3

The final states are Q_1 and Q_3.

4.19. Consider the NDFA accepting $L^r = \{x \in \{\boldsymbol{a}, \boldsymbol{b}\}^* \mid x$ ends with $\boldsymbol{a} \vee x$ contains $\boldsymbol{ab}\}$ from Example 4.10.

(a) *Circuit diagram (no SOS/EOS).* This NDFA is obtained by reversing the arrows in Figure 4.10 and swapping the start and final states. Using activity bits q_0, q_1, q_2, q_3, the next-state equations are

$$q_0' = (q_1 \wedge \boldsymbol{a}) \vee (q_2 \wedge \boldsymbol{b}),$$

$$q_1' = (q_1 \wedge \boldsymbol{a}) \vee (q_1 \wedge \boldsymbol{b}), \qquad q_2' = (q_3 \wedge \boldsymbol{a}) \vee (q_2 \wedge \boldsymbol{b}),$$

$$q_3' = (q_3 \wedge \boldsymbol{a}) \vee (q_3 \wedge \boldsymbol{b}).$$

The initial configuration has $q_1 = q_3 = 1$ and $q_0 = q_2 = 0$, and the accept line is q_0.

(b) *DFA (connected portion only).* From Figure 4.11, the NDFA has start states $\{q_1, q_3\}$ and only one final state, q_0. Its nonempty transitions are

$$\delta(q_1, \boldsymbol{a}) = \{q_1, q_0\}, \qquad \delta(q_1, \boldsymbol{b}) = \{q_1\},$$

$$\delta(q_2, \boldsymbol{b}) = \{q_2, q_0\}, \qquad \delta(q_3, \boldsymbol{a}) = \{q_3, q_2\}, \qquad \delta(q_3, \boldsymbol{b}) = \{q_3\}.$$

Starting from $Q_0 = \{q_1, q_3\}$, Definition 4.5 gives

$$\delta^d(Q_0, \boldsymbol{a}) = \{q_0, q_1, q_2, q_3\} = Q_1, \qquad \delta^d(Q_0, \boldsymbol{b}) = \{q_1, q_3\} = Q_0.$$

From Q_1 we get

$$\delta^d(Q_1, \boldsymbol{a}) = Q_1, \qquad \delta^d(Q_1, \boldsymbol{b}) = Q_1.$$

Therefore the connected DFA has just two states:

$$Q_0 = \{q_1, q_3\} \quad \text{(nonfinal)}, \qquad Q_1 = \{q_0, q_1, q_2, q_3\} \quad \text{(final)},$$

with transition table

	$\boldsymbol{a}$	$\boldsymbol{b}$
Q_0	Q_1	Q_0
Q_1	Q_1	Q_1

4.20. **Are the pumping lemma arguments still valid for NDFAs?**

Yes. The pumping lemma (Chapter 2) applies to any language accepted by a DFA (or equivalently, any FAD language). Since every NDFA A is equivalent to a DFA A^d (Theorem 4.1), $L(A) = L(A^d)$ is FAD, and the pumping lemma applies to $L(A)$.

More directly: the pumping lemma is a property of the *language*, not the machine representation. If L is FAD (accepted by some NDFA), then all the arguments from Chapter 2 go through unchanged. In particular, if A has n states, then A^d has at most 2^n states; words of length at least 2^n must cause a repeated state in A^d, yielding the pumping decomposition.

4.21. **Does the conclusion of Theorem 2.7 still hold for NDFAs?**

Yes. Theorem 2.7 characterizes FAD languages via the Myhill–Nerode theorem: L is FAD iff $\equiv_L$ has finitely many equivalence classes. This is a property of the language L, not of the particular automaton (DFA or NDFA) accepting it. Since NDFAs and DFAs accept exactly the same class of languages (the FAD languages, by Theorem 4.1), the conclusion holds: a language is accepted by some NDFA iff it is accepted by some DFA iff $\equiv_L$ has finitely many classes iff the minimal DFA is finite.

4.22. **Later no-λ exercise: given an NDFA A without λ-transitions with $\lambda \notin L(A)$, construct A'' with exactly one start state, one final state, and $L(A'') = L(A)$.**

Construction. Let $A = \langle \Sigma, S, S_0, \delta, F \rangle$ with $\lambda \notin L(A)$, so $S_0 \cap F = \emptyset$. Define

$$A'' = \langle \Sigma,\ S \cup \{q_0, q_f\},\ \{q_0\},\ \delta'',\ \{q_f\} \rangle$$

where q_0 and q_f are new states and:

1. For each $\boldsymbol{a} \in \Sigma$, let

$$\delta''(q_0, \boldsymbol{a}) = \Big(\bigcup_{s \in S_0} \delta(s, \boldsymbol{a}) \Big) \cup \begin{cases} \{q_f\} & \text{if } (\bigcup_{s \in S_0} \delta(s, \boldsymbol{a})) \cap F \neq \emptyset, \\ \emptyset & \text{otherwise.} \end{cases}$$

2. For each old state $s \in S$ and each $\boldsymbol{a} \in \Sigma$, let

$$\delta''(s,\boldsymbol{a}) = \delta(s,\boldsymbol{a}) \cup \begin{cases} \{q_f\} & \text{if } \delta(s,\boldsymbol{a}) \cap F \neq \emptyset, \\ \emptyset & \text{otherwise.} \end{cases}$$

3. $\delta''(q_f,\boldsymbol{a}) = \emptyset$ for every $\boldsymbol{a} \in \Sigma$.

This machine has exactly one start state (q_0) and exactly one final state (q_f).

Why this preserves the language. Since $\lambda \notin L(A)$, every accepted word has positive length. Suppose $w = \boldsymbol{a}_1 \cdots \boldsymbol{a}_n \in L(A)$. Then there is a path in A from some start state to some final state that reads all of w. In A'', use the same path for the first $n-1$ letters. On the last letter, when the old path would move into a final state, choose the extra target q_f instead. Thus $w \in L(A'')$.

Conversely, any accepting path in A'' must end in q_f. The only way to enter q_f is on some actual input symbol $\boldsymbol{a}$ from q_0 or from an old state s, and this happens only when the corresponding old transition set contains at least one state of F. Replacing that last step by a move into one of those original final states yields an accepting path in A. Hence $L(A'') = L(A)$.

4.23. **If $\lambda \in L(A)$, the three conditions of Exercise 4.22 may be incompatible (without λ-transitions).**

Example. Let

$$A = \langle \{\boldsymbol{a}\}, \{s_0, s_1\}, \{s_0\}, \delta, \{s_0, s_1\} \rangle$$

where $\delta(s_0,\boldsymbol{a}) = \{s_1\}$ and $\delta(s_1,\boldsymbol{a}) = \emptyset$. Then

$$L(A) = \{\lambda, \boldsymbol{a}\}.$$

We claim there is no NDFA without λ-transitions that has exactly one start state, exactly one final state, and also accepts exactly $\{\lambda, \boldsymbol{a}\}$.

Assume to the contrary that

$$B = \langle \{\boldsymbol{a}\}, S, \{q\}, \delta_B, \{f\} \rangle$$

has those three properties and $L(B) = \{\lambda, \boldsymbol{a}\}$. Since $\lambda \in L(B)$, the unique start state must also be the unique final state:

$$q = f.$$

Since $\boldsymbol{a} \in L(B)$ and there are no λ-transitions, there must be a transition on $\boldsymbol{a}$ from q to the only final state, namely to q itself. Thus $q \in \delta_B(q,\boldsymbol{a})$.

But then the same move can be used repeatedly, so every positive power of $\boldsymbol{a}$ is accepted:

$$\boldsymbol{a}^n \in L(B) \qquad \text{for all } n \geq 1.$$

In particular, $\boldsymbol{aa} \in L(B)$, contradicting $L(B) = \{\lambda, \boldsymbol{a}\}$. Therefore the three conditions may indeed be incompatible.

4.24. **Prove Lemma 4.2.**

Lemma 4.2: For any $s \in S$ and any $x \in \Sigma^+$, $\overline{\delta^\sigma_\lambda}(s,x) = \overline{\delta}_\lambda(s,x)$.

Proof. We proceed by induction on $|x| = k \geq 1$.

Base case ($k = 1$). Let $x = \boldsymbol{a} \in \Sigma$. By definition of δ^σ_λ:

$$\delta^\sigma_\lambda(s, \boldsymbol{a}) = \Lambda\Big(\bigcup_{t \in \Lambda(s)} \delta_\lambda(t, \boldsymbol{a})\Big) = \overline{\delta}_\lambda(s, \boldsymbol{a}).$$

So $\overline{\delta^\sigma_\lambda}(s, \boldsymbol{a}) = \delta^\sigma_\lambda(s, \boldsymbol{a}) = \overline{\delta}_\lambda(s, \boldsymbol{a})$.

Inductive step. Assume the result holds for all words of length k. Let $y = x\boldsymbol{a}$ with $|x| = k$ and $\boldsymbol{a} \in \Sigma$. Then:

$$\begin{aligned}
\overline{\delta^\sigma_\lambda}(s, x\boldsymbol{a}) &= \bigcup_{t \in \overline{\delta^\sigma_\lambda}(s,x)} \delta^\sigma_\lambda(t, \boldsymbol{a}) \\
&= \bigcup_{t \in \overline{\delta}_\lambda(s,x)} \delta^\sigma_\lambda(t, \boldsymbol{a}) \quad \text{(induction hypothesis)} \\
&= \bigcup_{t \in \overline{\delta}_\lambda(s,x)} \overline{\delta}_\lambda(t, \boldsymbol{a}) \quad \text{(base case)} \\
&= \overline{\delta}_\lambda(s, x\boldsymbol{a}). \quad \text{(definition of } \overline{\delta}_\lambda)
\end{aligned}$$

By induction, the result holds for all $x \in \Sigma^+$. □ □

4.25. **A DFA can be viewed as an NDFA, and $L(A^n) = L(A)$.**

Construction. Given a DFA $A = \langle \Sigma, S, s_0, \delta, F \rangle$ (with $\delta : S \times \Sigma \to S$ a total function), define the NDFA $A^n = \langle \Sigma, S, \{s_0\}, \delta^n, F \rangle$ where $\delta^n : S \times \Sigma \to \wp(S)$ is given by $\delta^n(s, \boldsymbol{a}) = \{\delta(s, \boldsymbol{a})\}$.

A^n is a valid NDFA: S is finite, $S^n_0 = \{s_0\}$ is a nonempty set of start states, and δ^n maps to subsets of S.

Claim: $L(A^n) = L(A)$.

Proof. By induction on $|x|$, we show $\overline{\delta}^n(s_0, x) = \{\overline{\delta}(s_0, x)\}$ (i.e., the nondeterministic extended transition function applied to the singleton start always yields the singleton $\{\overline{\delta}(s_0, x)\}$).

Base: $\overline{\delta}^n(s_0, \lambda) = \{s_0\} = \{\overline{\delta}(s_0, \lambda)\}$.

Step: $\overline{\delta}^n(s_0, x\boldsymbol{a}) = \bigcup_{t \in \overline{\delta}^n(s_0,x)} \delta^n(t, \boldsymbol{a}) = \bigcup_{t \in \{\overline{\delta}(s_0,x)\}} \{\delta(t, \boldsymbol{a})\} = \{\delta(\overline{\delta}(s_0, x), \boldsymbol{a})\} = \{\overline{\delta}(s_0, x\boldsymbol{a})\}$.

Hence $x \in L(A^n)$ iff $\overline{\delta}^n(s_0, x) \cap F \neq \emptyset$ iff $\overline{\delta}(s_0, x) \in F$ iff $x \in L(A)$. □ □

4.26. **Show that $\overline{\delta^\sigma_\lambda}(s, \lambda) \neq \overline{\delta}_\lambda(s, \lambda)$ in general.**

By definition:

- $\overline{\delta^\sigma_\lambda}(s, \lambda) = \{s\}$ (definition of the extended transition function for zero-length input).
- $\overline{\delta}_\lambda(s, \lambda) = \Lambda(s)$ (definition of $\overline{\delta}_\lambda$ for λ).

These differ whenever $\Lambda(s) \neq \{s\}$, i.e., whenever there exists at least one λ-transition out of s.

Example. In Example 4.14, $\Lambda(s_0) = \{s_0, s_1, s_2\}$ but $\overline{\delta^\sigma_\lambda}(s_0, \lambda) = \{s_0\}$. Hence $\overline{\delta^\sigma_\lambda}(s_0, \lambda) \neq \overline{\delta}_\lambda(s_0, \lambda)$.

This is why Lemma 4.2 is stated only for $x \in \Sigma^+$ (words of length ≥ 1) and cannot be extended to λ.

4.27. **Prove:** $(\forall A, B \in \wp(S))(\forall \boldsymbol{a} \in \Sigma)\, \delta^d(A \cup B, \boldsymbol{a}) = \delta^d(A, \boldsymbol{a}) \cup \delta^d(B, \boldsymbol{a})$.

Proof. By Definition 4.5, $\delta^d(Q, \boldsymbol{a}) = \bigcup_{q \in Q} \delta(q, \boldsymbol{a})$ for any $Q \in \wp(S)$. Therefore:

$$\begin{aligned}\delta^d(A \cup B, \boldsymbol{a}) &= \bigcup_{q \in A \cup B} \delta(q, \boldsymbol{a}) \\ &= \Big(\bigcup_{q \in A} \delta(q, \boldsymbol{a})\Big) \cup \Big(\bigcup_{q \in B} \delta(q, \boldsymbol{a})\Big) \\ &= \delta^d(A, \boldsymbol{a}) \cup \delta^d(B, \boldsymbol{a}). \quad \square\end{aligned}$$

□

4.28. **Acceptance requiring *every* reachable state to be final.**

(a) **New acceptance definition.** Define $L_\forall(A) = \{w \in \Sigma^* \mid \overline{\delta}(S_0, w) \neq \emptyset \wedge \overline{\delta}(S_0, w) \subseteq F\}$. That is, w is accepted iff (i) at least one state is reachable and (ii) *every* reachable state is final.

(b) **Modified A^d.** Define $A^\forall = A^d$ with the same state set $\wp(S)$ and transition function δ^d as before, but with the new final-state condition:

$$F^\forall = \{Q \in \wp(S) \mid Q \neq \emptyset \wedge Q \subseteq F\}.$$

That is, Q is a final state of $A^\forall$ iff Q is nonempty and all states in Q are final states of A. By the same argument as Theorem 4.1 (modified for this new acceptance criterion), $L_\forall(A) = L(A^\forall)$. Note: this "universal" acceptance is still FAD (the modified $A^\forall$ is a DFA), so $L_\forall(A)$ is always a FAD language.

4.29. **Draw the connected part of T^d, the deterministic equivalent of the NDFA T in Example 4.7.**

Let

$$\begin{gathered}Q_0 = \{s_0\}, \quad Q_1 = \{s_0, s_1\}, \quad Q_2 = \{s_0, s_2\}, \quad Q_3 = \{s_0, s_1, s_3\}, \\ Q_4 = \{s_0, s_1, s_4\}, \quad Q_5 = \{s_0, s_2, s_5\}, \quad Q_6 = \{s_0, s_1, s_3, s_6\}, \\ Q_7 = \{s_0, s_1, s_4, s_6\}, \quad Q_8 = \{s_0, s_2, s_6\}, \quad Q_9 = \{s_0, s_1, s_6\}, \\ Q_{10} = \{s_0, s_2, s_5, s_6\}, \qquad Q_{11} = \{s_0, s_6\}.\end{gathered}$$

These are exactly the reachable subsets from $\{s_0\}$ under Definition 4.5. The accepting states are those containing s_6, namely

$$Q_6, Q_7, Q_8, Q_9, Q_{10}, Q_{11}.$$

Their transitions are:

	0	**1**
Q_0	Q_1	Q_0
Q_1	Q_1	Q_2
Q_2	Q_3	Q_0
Q_3	Q_4	Q_2
Q_4	Q_1	Q_5
Q_5	Q_6	Q_0
Q_6	Q_7	Q_8
Q_7	Q_9	Q_{10}
Q_8	Q_6	Q_{11}
Q_9	Q_9	Q_8
Q_{10}	Q_6	Q_{11}
Q_{11}	Q_9	Q_{11}

This is the connected part of T^d.

4.30. **Modify T to revert to a nonfinal state when 000111 (EOT) is detected.**

Keep the original start-of-transmission arm for **010010**, but replace the single final sink s_6 by six final "recording" states $r_0, \ldots, r_5$ that track the longest suffix of the data seen so far that is also a prefix of the EOT string **000111**. The only nonfinal states are the original $s_0, \ldots, s_5$.

	0	**1**
s_0	$\{s_0, s_1\}$	$\{s_0\}$
s_1	$\emptyset$	$\{s_2\}$
s_2	$\{s_3\}$	$\emptyset$
s_3	$\{s_4\}$	$\emptyset$
s_4	$\emptyset$	$\{s_5\}$
s_5	$\{r_0\}$	$\emptyset$
r_0	$\{r_1\}$	$\{r_0\}$
r_1	$\{r_2\}$	$\{r_0\}$
r_2	$\{r_3\}$	$\{r_0\}$
r_3	$\{r_3\}$	$\{r_4\}$
r_4	$\{r_1\}$	$\{r_5\}$
r_5	$\{r_1\}$	$\{s_0\}$

Here r_i means that recording is on and the last i scanned bits match the first i bits of **000111**. Thus $r_5 \xrightarrow{\mathbf{1}} s_0$ turns the recorder off exactly when the full EOT signal **000111** has just been seen, while every other input keeps the machine in one of the final recording states.

4.31. Consider the NDFA A from Example 4.14.

(a) **Diagram of A_λ^σ.** For Example 4.14, $\lambda \in L(A_\lambda)$, so Definition 4.9 yields

$$A_\lambda^\sigma = \langle \{\boldsymbol{a}, \boldsymbol{b}, \boldsymbol{c}, \boldsymbol{d}\}, \{q_0, s_0, s_1, s_2, s_3\}, \{q_0, s_0\}, \delta_\lambda^\sigma, \{q_0, s_2\} \rangle$$

with transition table

	$\boldsymbol{a}$	$\boldsymbol{b}$	$\boldsymbol{c}$	$\boldsymbol{d}$
q_0	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$
s_0	$\{s_1, s_2\}$	$\{s_3\}$	$\{s_2\}$	$\{s_2\}$
s_1	$\emptyset$	$\{s_3\}$	$\{s_2\}$	$\{s_2\}$
s_2	$\emptyset$	$\emptyset$	$\emptyset$	$\{s_2\}$
s_3	$\emptyset$	$\{s_1, s_2\}$	$\emptyset$	$\emptyset$

This table is exactly the state-transition diagram of Figure 4.20.

(b) **Diagram of $A_\lambda^{\sigma d}$ (connected portion).** Starting from the actual start set $\{q_0, s_0\}$, the reachable subsets are

$$R_0 = \{q_0, s_0\}, \quad R_1 = \{s_1, s_2\}, \quad R_2 = \{s_3\}, \quad R_3 = \{s_2\}, \quad R_4 = \emptyset.$$

Their transitions are

	$\boldsymbol{a}$	$\boldsymbol{b}$	$\boldsymbol{c}$	$\boldsymbol{d}$
R_0	R_1	R_2	R_3	R_3
R_1	R_4	R_2	R_3	R_3
R_2	R_4	R_1	R_4	R_4
R_3	R_4	R_4	R_4	R_3
R_4	R_4	R_4	R_4	R_4

The final states are R_0, R_1, and R_3.

4.32. **What is wrong with the given "proof" of Lemma 4.2?**

The flaw is in the inductive step. The "proof" starts the induction at $k = 1$ (base case for single letters), but the inductive step uses $\overline{\delta^\sigma_\lambda}(s,x)$ for $x \in \Sigma^k$ and then applies the "induction hypothesis" $\overline{\delta^\sigma_\lambda}(s,x) = \overline{\delta}_\lambda(s,x)$.

The critical error is on the line:

$$\delta^\sigma_\lambda(\overline{\delta^\sigma_\lambda}(s,x), \boldsymbol{a}) = \overline{\delta^\sigma_\lambda}(\overline{\delta}_\lambda(s,x), \boldsymbol{a}).$$

Here, $\overline{\delta^\sigma_\lambda}(s,x)$ is a *set* of states (since we're in an NDFA), but δ^σ_λ is defined on individual states, not sets. The expression $\delta^\sigma_\lambda(Q, \boldsymbol{a})$ where Q is a set is not directly the same as $\overline{\delta^\sigma_\lambda}(Q, \boldsymbol{a})$ for a set-valued argument: the extended function $\overline{\delta^\sigma_\lambda}$ on a set-argument must take the union over all states in the set.

More specifically, the step

$$\delta^\sigma_\lambda\big(\overline{\delta^\sigma_\lambda}(s,x), \boldsymbol{a}\big) = \overline{\delta^\sigma_\lambda}\big(\overline{\delta}_\lambda(s,x), \boldsymbol{a}\big)$$

conflates applying δ^σ_λ to a *set* with applying the set-union version of $\overline{\delta^\sigma_\lambda}$ to a set argument. These are not the same: the left side applies δ^σ_λ to the entire set at once (which is not how the function is defined), while the right side correctly takes the union over individual states. This makes the step invalid without justification.

4.33. Consider the NDFA T from Example 4.7 (detecting substring **010010**).

(a) *Circuit diagram (no SOS/EOS).* T has 8 states (one start state that loops on all input plus the six states that track the pattern **010010**). Using one activity bit per state, the next-state equations are

$$s'_0 = s_0, \qquad s'_1 = s_0 \wedge \mathbf{0}, \qquad s'_2 = s_1 \wedge \mathbf{1},$$
$$s'_3 = s_2 \wedge \mathbf{0}, \qquad s'_4 = s_3 \wedge \mathbf{0}, \qquad s'_5 = s_4 \wedge \mathbf{1},$$
$$s'_6 = (s_5 \wedge \mathbf{0}) \vee (s_6 \wedge \mathbf{0}) \vee (s_6 \wedge \mathbf{1}).$$

Initially only s_0 is active, and the accept line is s_6.

(b) *DFA T^d (connected portion).* See Exercise 4.29 for the subset construction. The connected DFA is exactly the 12-state machine listed there, with states $Q_0, \ldots, Q_{11}$ and the transition table given in Exercise 4.29.

4.34. Consider the NDFA from Example 4.11, which accepts all words containing **10110**, **1010**, or **01101** as substrings.

(a) *Circuit diagram (no SOS/EOS).* The NDFA has a single initial state s_0 that loops on Σ^*, three pattern-matching arms, and the final sink s_{18} from Figure 4.12. Since s_1, s_7, and s_{12} are only shown for clarity in the text, they can be omitted from the actual circuit. With one activity bit per remaining state, the nonzero next-state equations are

$$s_0' = s_0,$$

$$s_2' = s_0 \wedge \mathbf{1}, \quad s_3' = s_2 \wedge \mathbf{0}, \quad s_4' = s_3 \wedge \mathbf{1}, \quad s_5' = s_4 \wedge \mathbf{1}, \quad s_6' = s_5 \wedge \mathbf{0},$$

$$s_8' = s_0 \wedge \mathbf{1}, \quad s_9' = s_8 \wedge \mathbf{0}, \quad s_{10}' = s_9 \wedge \mathbf{1}, \quad s_{11}' = s_{10} \wedge \mathbf{0},$$

$$s_{13}' = s_0 \wedge \mathbf{0}, \quad s_{14}' = s_{13} \wedge \mathbf{1}, \quad s_{15}' = s_{14} \wedge \mathbf{1}, \quad s_{16}' = s_{15} \wedge \mathbf{0}, \quad s_{17}' = s_{16} \wedge \mathbf{1},$$

$$s_{18}' = s_6 \vee s_{11} \vee s_{17} \vee s_{18}.$$

The accept line is $s_6 \vee s_{11} \vee s_{17} \vee s_{18}$.

(b) *DFA (connected portion only).* Since all three patterns are started from the looping state s_0, the subset construction yields the following 23 reachable states:

$Q_0 = \{s_0\}$,
$Q_1 = \{s_0, s_{13}\}$,
$Q_2 = \{s_0, s_2, s_8\}$,
$Q_3 = \{s_0, s_{14}, s_2, s_8\}$,
$Q_4 = \{s_0, s_{13}, s_3, s_9\}$,
$Q_5 = \{s_0, s_{15}, s_2, s_8\}$,
$Q_6 = \{s_0, s_{10}, s_{14}, s_2, s_4, s_8\}$,
$Q_7 = \{s_0, s_{13}, s_{16}, s_3, s_9\}$,
$Q_8 = \{s_0, s_{11}, s_{13}, s_3, s_9\}$,
$Q_9 = \{s_0, s_{15}, s_2, s_5, s_8\}$,
$Q_{10} = \{s_0, s_{10}, s_{14}, s_{17}, s_2, s_4, s_8\}$,
$Q_{11} = \{s_0, s_{13}, s_{18}\}$,
$Q_{12} = \{s_0, s_{10}, s_{14}, s_{18}, s_2, s_4, s_8\}$,
$Q_{13} = \{s_0, s_{13}, s_{16}, s_3, s_6, s_9\}$,
$Q_{14} = \{s_0, s_{11}, s_{13}, s_{18}, s_3, s_9\}$,
$Q_{15} = \{s_0, s_{15}, s_{18}, s_2, s_5, s_8\}$,
$Q_{16} = \{s_0, s_{14}, s_{18}, s_2, s_8\}$,
$Q_{17} = \{s_0, s_{10}, s_{14}, s_{17}, s_{18}, s_2, s_4, s_8\}$,
$Q_{18} = \{s_0, s_{13}, s_{16}, s_{18}, s_3, s_6, s_9\}$,
$Q_{19} = \{s_0, s_{18}, s_2, s_8\}$,
$Q_{20} = \{s_0, s_{13}, s_{18}, s_3, s_9\}$,
$Q_{21} = \{s_0, s_{15}, s_{18}, s_2, s_8\}$,
$Q_{22} = \{s_0, s_{13}, s_{16}, s_{18}, s_3, s_9\}$.

Their transition table is

	0	**1**
Q_0	Q_1	Q_2
Q_1	Q_1	Q_3
Q_2	Q_4	Q_2
Q_3	Q_4	Q_5
Q_4	Q_1	Q_6
Q_5	Q_7	Q_2
Q_6	Q_8	Q_9
Q_7	Q_1	Q_{10}
Q_8	Q_{11}	Q_{12}
Q_9	Q_{13}	Q_2
Q_{10}	Q_{14}	Q_{15}
Q_{11}	Q_{11}	Q_{16}
Q_{12}	Q_{14}	Q_{15}
Q_{13}	Q_{11}	Q_{17}
Q_{14}	Q_{11}	Q_{12}
Q_{15}	Q_{18}	Q_{19}
Q_{16}	Q_{20}	Q_{21}
Q_{17}	Q_{14}	Q_{15}
Q_{18}	Q_{11}	Q_{17}
Q_{19}	Q_{20}	Q_{19}
Q_{20}	Q_{11}	Q_{12}
Q_{21}	Q_{22}	Q_{19}
Q_{22}	Q_{11}	Q_{17}

The final states are exactly those containing one of s_6, s_{11}, s_{17}, or s_{18}:

$$Q_8, Q_{10}, Q_{11}, Q_{12}, Q_{13}, Q_{14}, Q_{15}, Q_{16}, Q_{17}, Q_{18}, Q_{19}, Q_{20}, Q_{21}, Q_{22}.$$

4.35. Consider the NDFA B from Example 4.8 and its deterministic equivalent B^d.

(a) *Circuit diagram for B (no SOS/EOS).* Encode the states of B using one activity bit for r and one for s. From Figure 4.7,

$$r' = (r \wedge \boldsymbol{a}) \vee (s \wedge \boldsymbol{a}), \qquad s' = (r \wedge \boldsymbol{a}) \vee (r \wedge \boldsymbol{b}),$$

with initial state $r = 1$, $s = 0$. Since s is the only final state, the accept output is s.

(b) *Circuit diagram for B^d (no SOS/EOS).* The states of B^d are

$$\varnothing, \qquad \{r\}, \qquad \{s\}, \qquad \{r, s\}.$$

Using the binary state assignment

$$\varnothing = 00, \qquad \{r\} = 01, \qquad \{s\} = 10, \qquad \{r, s\} = 11,$$

with present-state bits $x_1 x_0$, the transition table is

$x_1 x_0$	$\boldsymbol{a}$	$\boldsymbol{b}$
00	00	00
01	11	10
10	01	00
11	11	10

so the DFA circuit is given by

$$x_1' = x_0, \qquad x_0' = \boldsymbol{a} \wedge (x_1 \vee x_0).$$

The start state is 01 and the accept output is x_1, since the final subset states are $\{s\}$ and $\{r, s\}$.

4.36. **Circuit diagrams for each NDFA in Figure 4.25 (no SOS/EOS).**

For each NDFA A_i in Figure 4.25, with $\Sigma = \{\boldsymbol{a}, \boldsymbol{b}, \boldsymbol{c}\}$:

1. **(a)** With activity bit t for the single state:

$$t' = t \wedge (\boldsymbol{a} \vee \boldsymbol{c}), \qquad \text{accept} = t.$$

2. **(b)** With activity bits q_1, q_0:

$$q_1' = (q_1 \wedge \boldsymbol{b}) \vee (q_0 \wedge \boldsymbol{b}), \qquad q_0' = (q_1 \wedge \boldsymbol{a}) \vee (q_0 \wedge \boldsymbol{c}),$$

and the accept output is q_1.

3. **(c)** With activity bits r_0, r_1:

$$r_0' = (r_0 \wedge \boldsymbol{a}) \vee (r_1 \wedge \boldsymbol{b}), \qquad r_1' = (r_0 \wedge \boldsymbol{a}) \vee (r_0 \wedge \boldsymbol{c}),$$

and the accept output is r_0.

4. **(d)** With activity bits s_0, s_1, s_2, s_3:

$$s_0' = 0, \qquad s_1' = (s_0 \wedge \boldsymbol{a}) \vee (s_3 \wedge \boldsymbol{b}),$$

$$s_2' = s_1 \wedge \boldsymbol{a}, \qquad s_3' = s_1 \wedge \boldsymbol{b},$$

and the accept output is s_2.

5. **(e)** With activity bits s_1, s_2:

$$s_1' = (s_1 \wedge \boldsymbol{a}) \vee (s_2 \wedge \boldsymbol{a}) \vee (s_2 \wedge \boldsymbol{b}), \qquad s_2' = s_1 \wedge \boldsymbol{a},$$

and the accept output is s_2.

6. **(f)** The λ-machine has states s_0, s_1, s_2, s_3 with ordinary transitions $s_0 \xrightarrow{\boldsymbol{a}} s_1$, $s_1 \xrightarrow{\boldsymbol{b}} s_2$, $s_2 \xrightarrow{\boldsymbol{c}} s_3$ and λ-moves $s_1 \to s_0$, $s_3 \to s_1$. The standard circuit therefore uses

$$D_0 = s_1, \qquad D_1 = (s_0 \wedge \boldsymbol{a}) \vee s_3, \qquad D_2 = s_1 \wedge \boldsymbol{b}, \qquad D_3 = s_2 \wedge \boldsymbol{c},$$

with accept output s_3. Adding the transitive λ-connection $s_3 \to s_0$ as well is also consistent with the text's discussion.

7. **(g)** With activity bits s_0, s_1, s_2, s_3:

$$s_0' = s_1 \wedge \boldsymbol{a}, \qquad s_1' = (s_0 \wedge \boldsymbol{a}) \vee (s_2 \wedge \boldsymbol{c}) \vee (s_3 \wedge \boldsymbol{b}),$$

$$s_2' = s_1 \wedge \boldsymbol{c}, \qquad s_3' = s_1 \wedge \boldsymbol{b},$$

and the accept output is s_0.

4.37. **Circuit diagrams for each NDFA in Figure 4.25 (with SOS and EOS).**

Extend each circuit from Exercise 4.36 by forcing the start state(s) active on <SOS> and gating the final-state output with <EOS>:

- **(a)** $t' =$ <SOS> $\vee\, (t \wedge (\boldsymbol{a} \vee \boldsymbol{c}))$ and accept $= t \wedge$ <EOS>.
- **(b)** OR <SOS> into the input for q_1:

$$q_1' = \text{<SOS>} \vee (q_1 \wedge \boldsymbol{b}) \vee (q_0 \wedge \boldsymbol{b}), \qquad q_0' = (q_1 \wedge \boldsymbol{a}) \vee (q_0 \wedge \boldsymbol{c}),$$

  and accept $= q_1 \wedge$ <EOS>.
- **(c)** OR <SOS> into the input for r_0 and gate the output:

$$r_0' = \text{<SOS>} \vee (r_0 \wedge \boldsymbol{a}) \vee (r_1 \wedge \boldsymbol{b}), \qquad r_1' = (r_0 \wedge \boldsymbol{a}) \vee (r_0 \wedge \boldsymbol{c}),$$

  with accept $= r_0 \wedge$ <EOS>.
- **(d)** OR <SOS> into s_0' and use accept $= s_2 \wedge$ <EOS>.
- **(e)** OR <SOS> into the corrected first equation:

$$s_1' = \text{<SOS>} \vee (s_1 \wedge \boldsymbol{a}) \vee (s_2 \wedge \boldsymbol{a}) \vee (s_2 \wedge \boldsymbol{b}), \qquad s_2' = s_1 \wedge \boldsymbol{a},$$

  and use accept $= s_2 \wedge$ <EOS>.
- **(f)** OR <SOS> into D_0 and use accept $= s_3 \wedge$ <EOS>.
- **(g)** OR <SOS> into s_0' and use accept $= s_0 \wedge$ <EOS>.

4.38. **Adapting Definition 3.10 for NDFAs and proving that an algorithm exists to find the connected portion.**

(a) **Adapted definition.** The connected portion of an NDFA $A = \langle \Sigma, S, S_0, \delta, F \rangle$ is defined as follows. Let

$$\text{Reach}_0 = S_0, \qquad \text{Reach}_{k+1} = \text{Reach}_k \cup \bigcup_{\boldsymbol{a} \in \Sigma} \bigcup_{s \in \text{Reach}_k} \delta(s, \boldsymbol{a}).$$

The connected portion is $\text{Reach} = \bigcup_{k \geq 0} \text{Reach}_k$, together with all transitions among states in Reach and the restriction of F to Reach.

(b) **Proof that the algorithm terminates.**

Proof. The sets Reach_k form a non-decreasing chain $\text{Reach}_0 \subseteq \text{Reach}_1 \subseteq \cdots \subseteq S$. Since S is finite, the chain must stabilize: there exists k^* such that $\text{Reach}_{k^*} = \text{Reach}_{k^*+1}$. At that point, no new states are added. The algorithm terminates in at most $|S|$ iterations (since at least one new state must be added in each non-stable step, and there are only $|S|$ states).

To compute this algorithmically:

1. Initialize $R := S_0$, New $:= S_0$.
2. While New $\neq \emptyset$:
   (a) Next $:= \bigcup_{\boldsymbol{a} \in \Sigma} \bigcup_{s \in \text{New}} \delta(s, \boldsymbol{a}) \setminus R$.
   (b) $R := R \cup$ Next; New $:=$ Next.
3. Output R as the connected portion.

Each iteration of the while loop adds at least one state to R (otherwise New $= \emptyset$ and we stop). Thus the loop executes at most $|S|$ times, and the algorithm runs in $O(|S| \cdot |\Sigma|)$ time. □ □

Chapter 5

Closure Properties — Solutions

5.1. Let F_Σ be the collection of all finite languages over Σ.

(a) **Complementation: No.** Σ^* is infinite, so for any finite L, $\Sigma^* \setminus L$ is infinite (assuming $|\Sigma| \geq 1$), hence not in F_Σ.

(b) **Union: Yes.** If L_1, L_2 are finite, then $|L_1 \cup L_2| \leq |L_1| + |L_2| < \infty$.

(c) **Intersection: Yes.** $|L_1 \cap L_2| \leq |L_1| < \infty$.

(d) **Concatenation: Yes.** $|L_1 \cdot L_2| \leq |L_1| \cdot |L_2| < \infty$.

(e) **Kleene closure: No.** Let $L = \{\boldsymbol{a}\}$. Then $L^* = \{\boldsymbol{a}\}^* = \{\lambda, \boldsymbol{a}, \boldsymbol{aa}, \ldots\}$ is infinite.

(f) **Relative complement: Yes.** $L_1 \setminus L_2 \subseteq L_1$, so $|L_1 \setminus L_2| \leq |L_1| < \infty$.

5.2. Let C_Σ be the collection of all cofinite languages (complements of finite languages).

(a) **Complementation: No.** Let $L = \Sigma^* \setminus \{\boldsymbol{a}\}$. Then $L \in C_\Sigma$, but $\sim L = \{\boldsymbol{a}\}$ is finite, hence not in C_Σ.

(b) **Union: Yes.** If $L_1 = \Sigma^* \setminus F_1$ and $L_2 = \Sigma^* \setminus F_2$, then $L_1 \cup L_2 = \Sigma^* \setminus (F_1 \cap F_2)$. Since $F_1 \cap F_2$ is finite, $L_1 \cup L_2 \in C_\Sigma$.

(c) **Intersection: Yes.** $L_1 \cap L_2 = \Sigma^* \setminus (F_1 \cup F_2)$. Since F_1 and F_2 are finite, so is $F_1 \cup F_2$, hence $L_1 \cap L_2 \in C_\Sigma$.

(d) **Concatenation: Yes.** If $L_1, L_2 \in C_\Sigma$ (both infinite), then $L_1 \cdot L_2$ contains arbitrarily long words, but let us verify cofiniteness. Since $\Sigma^* \setminus (L_1 \cdot L_2) \subseteq$ the set of words not writable as a product from L_1 and L_2; this set is finite because L_1 and L_2 each omit only finitely many words. More precisely: there exists N_1 such that all words of length $\geq N_1$ are in L_1, and similarly N_2 for L_2. Then any word of length $\geq N_1 + N_2$ can be split so that both parts are in L_1 and L_2, so $L_1 \cdot L_2$ is cofinite.

(e) **Kleene closure: Yes.** Since L is cofinite, there is an N such that every word of length at least N lies in L. Hence every word of length at least N also lies in L^* (just take it as a one-factor product). Therefore $\Sigma^* \setminus L^*$ contains only words of length $< N$, so it is finite and $L^* \in C_\Sigma$.

(f) **Relative complement: No.** $L_1 \setminus L_2 = L_1 \cap \sim L_2$. Here $\sim L_2$ is finite, so $L_1 \setminus L_2 \subseteq \sim L_2$ is finite, not cofinite in general. Counterexample: $L_1 = \Sigma^* \setminus \emptyset = \Sigma^*$ and $L_2 = \Sigma^* \setminus \{\boldsymbol{a}\}$. Then $L_1 \setminus L_2 = \{\boldsymbol{a}\}$, which is finite, not cofinite.

5.3. $B_\Sigma = F_\Sigma \cup C_\Sigma$.

(a) **Complementation: Yes.** The complement of a finite language is cofinite and vice versa.

(b) **Union: Yes.** If both L_1, L_2 are finite, $L_1 \cup L_2$ is finite. If one is cofinite, $L_1 \cup L_2$ is cofinite. If both are cofinite, $L_1 \cup L_2$ is cofinite (by 5.2b). **Closed.**

(c) **Intersection: Yes.** If both finite: finite. If one cofinite and one finite: finite. If both cofinite: cofinite (by 5.2c).

(d) **Concatenation: No.** Let $L_1 = C_\Sigma$ element and $L_2 = C_\Sigma$ element; concatenation is cofinite (by 5.2d). Let $L_1, L_2 \in F_\Sigma$; product is finite. Let $L_1 \in F_\Sigma$ and $L_2 \in C_\Sigma$ or vice versa: if $L_1 = \emptyset$, product is $\emptyset \in F_\Sigma$. If L_1 is finite and nonempty: the product may be neither finite nor cofinite. **Counterexample:** $L_1 = \{\boldsymbol{a}\}$ (finite), $L_2 = \Sigma^* \setminus \{\boldsymbol{aa}\}$ (cofinite). Then $L_1 \cdot L_2$ = all words of the form $\boldsymbol{a}w$ where $w \neq \boldsymbol{aa}$, i.e., all words starting with $\boldsymbol{a}$ except $\boldsymbol{aaa}$. This is neither finite nor cofinite (it's infinite and its complement is infinite). So **not closed**.

(e) **Kleene closure: No.** $\{\boldsymbol{a}\}^*$ is infinite but $\{\boldsymbol{a}\} \in F_\Sigma$; $\{\boldsymbol{a}\}^*$ is not in F_Σ. Is it cofinite? $\{\boldsymbol{a}\}^* = \{\lambda, \boldsymbol{a}, \boldsymbol{aa}, \ldots\}$, and its complement contains all words with any $\boldsymbol{b}$ (if $|\Sigma| \geq 2$), which is infinite, so $\{\boldsymbol{a}\}^*$ is not cofinite. **Not closed.**

(f) **Relative complement: Yes.** If $L_1 \in C_\Sigma$ and $L_2 \in C_\Sigma$, then $L_1 \setminus L_2 = L_1 \cap \sim L_2$ where $\sim L_2$ is finite, so the result is finite and therefore belongs to $F_\Sigma \subseteq B_\Sigma$. If $L_1 \in F_\Sigma$, then $L_1 \setminus L_2 \subseteq L_1$ is finite. Hence B_Σ is closed under relative complement.

5.4. $I_\Sigma = \wp(\Sigma^*) \setminus F_\Sigma$, the infinite languages.

(a) **Complementation: No.** $\Sigma^* \setminus L$ may be finite. E.g., $L = \Sigma^* \setminus \{\lambda\}$ is infinite; $\sim L = \{\lambda\}$ is finite.

(b) **Union: Yes.** Union of two infinite sets is infinite.

(c) **Intersection: No.** $L_1 = \{\boldsymbol{a}^{2n} \mid n \geq 0\}$ and $L_2 = \{\boldsymbol{a}^{2n+1} \mid n \geq 0\}$ are both infinite, but $L_1 \cap L_2 = \emptyset$, which is finite.

(d) **Concatenation: Yes.** If L_1, L_2 are infinite, then $L_1 \cdot L_2$ is infinite: it contains $\{xy \mid x \in L_1, y \in L_2\}$, which is infinite since L_1 (and L_2) contribute infinitely many distinct words.

(e) **Kleene closure: Yes.** $L^* \supseteq L$ when $\lambda \in L^*$, and L^* is always infinite if L contains any nonempty word.

(f) **Relative complement: No.** $L_1 \setminus L_2$ could be finite even if L_1, L_2 are infinite (e.g., $L_1 = L_2$).

5.5. $J_\Sigma = \wp(\Sigma^*) \setminus C_\Sigma$, languages with infinite complements.

(a) **Complementation: No.** $L \in J_\Sigma$ means $\sim L$ is infinite, i.e., $\sim L \in I_\Sigma$. Then $\sim (\sim L) = L$; is $\sim L \in J_\Sigma$? $\sim (\sim L) = L$ is infinite (since $L \in J_\Sigma$ is not cofinite, its complement is infinite, so L itself need not be infinite). Counterexample: $L = \emptyset \in J_\Sigma$ (complement is Σ^*, infinite), but $\sim L = \Sigma^* \notin J_\Sigma$ (complement is $\emptyset$, finite). **Not closed.**

(b) **Union: No.** Over $\Sigma = \{\boldsymbol{a}\}$, let $L_1 = \{\boldsymbol{a}^{2n} \mid n \geq 0\}$ and $L_2 = \{\boldsymbol{a}^{2n+1} \mid n \geq 0\}$. Both have infinite complements, so both belong to J_Σ. But $L_1 \cup L_2 = \Sigma^*$, whose complement is finite, so $L_1 \cup L_2 \notin J_\Sigma$.

(c) **Intersection: Yes.** If $\sim L_1$ and $\sim L_2$ are both infinite, then $\sim (L_1 \cap L_2) = \sim L_1 \cup \sim L_2 \supseteq \sim L_1$, which is infinite.

(d) **Concatenation: No.** Over $\Sigma = \{\boldsymbol{a}\}$, let $L_1 = \{\boldsymbol{a}^n \mid n \text{ odd or } n = 2\}$ and $L_2 = \{\boldsymbol{a}^{2k} \mid k \geq 1\}$. Then $\sim L_1 = \{\lambda, \boldsymbol{a}^4, \boldsymbol{a}^6, \ldots\}$ and $\sim L_2 = \{\lambda, \boldsymbol{a}, \boldsymbol{a}^3, \boldsymbol{a}^5, \ldots\}$ are both infinite, so $L_1, L_2 \in J_\Sigma$. But $L_1 \cdot L_2 = \{\boldsymbol{a}^n \mid n \geq 3\}$ (any $n \geq 3$ is expressible as odd+even or 2+even), whose complement $\{\lambda, \boldsymbol{a}, \boldsymbol{a}^2\}$ is finite—so $L_1 \cdot L_2 \notin J_\Sigma$.

(e) **Kleene closure: No.** Over $\Sigma = \{\boldsymbol{a}\}$, let $L = \{\boldsymbol{a}^n \mid n \text{ odd}\}$. Then $\sim L = \{\lambda, \boldsymbol{a}^2, \boldsymbol{a}^4, \ldots\}$ is infinite, so $L \in J_\Sigma$. But odd+odd=even, even+odd=odd, so L^k contains all $\boldsymbol{a}$-strings of the right parity at length $\geq k$; inductively $L^* = \{\boldsymbol{a}\}^* = \Sigma^*$. Then $\sim L^* = \emptyset$ is finite, so $L^* \notin J_\Sigma$.

(f) **Relative complement: Yes.** $\sim (L_1 \setminus L_2) = \sim (L_1 \cap \sim L_2) = \sim L_1 \cup L_2 \supseteq \sim L_1$. Since $L_1 \in J_\Sigma$, $\sim L_1$ is infinite, so $\sim (L_1 \setminus L_2)$ is infinite, hence $L_1 \setminus L_2 \in J_\Sigma$. □

5.6. $\boldsymbol{E}$: all languages over $\{\boldsymbol{a}, \boldsymbol{b}\}$ that contain $\boldsymbol{abba}$.

(a) **Complementation: No.** $\sim L$ contains all words *not* in L; since L contains $\boldsymbol{abba}$, $\sim L$ does not contain $\boldsymbol{abba}$, so $\sim L \notin \boldsymbol{E}$.

(b) **Union: Yes.** If $\boldsymbol{abba} \in L_1$, then $\boldsymbol{abba} \in L_1 \cup L_2$.

(c) **Intersection: Yes.** If $L_1, L_2 \in \boldsymbol{E}$ then both contain $\boldsymbol{abba}$, so $L_1 \cap L_2$ also contains $\boldsymbol{abba}$. **Closed.**

(d) **Concatenation: No.** Here $\boldsymbol{E}$ means languages that have the specific word $\boldsymbol{abba}$ as an element. Let $L_1 = L_2 = \{\boldsymbol{abba}\}$. Then $L_1, L_2 \in \boldsymbol{E}$, but $L_1 \cdot L_2 = \{\boldsymbol{abbaabba}\}$, which does not contain $\boldsymbol{abba}$ as a member. So $\boldsymbol{E}$ is not closed under concatenation.

(e) **Kleene closure: Yes.** L^* contains $L^1 = L$, so $\boldsymbol{abba} \in L \subseteq L^*$.

(f) **Relative complement: No.** $L_1 \setminus L_2$ might not contain $\boldsymbol{abba}$ if $\boldsymbol{abba} \in L_2$. E.g., $L_1 = L_2 = \{\boldsymbol{abba}\}$: $L_1 \setminus L_2 = \emptyset \notin \boldsymbol{E}$.

5.7. **Intersection closed does not imply union closed.**

Counterexample. Let $\mathscr{C} = \{\emptyset\} \cup \{\{x\} \mid x \in \Sigma^*\}$ (the empty language together with all singleton languages).

Closed under intersection: $\{x\} \cap \{y\} = \{x\}$ if $x = y$, and $\emptyset$ if $x \neq y$; both are in $\mathscr{C}$.

Not closed under union: $\{x\} \cup \{y\} = \{x, y\}$ for $x \neq y$, which is a two-element set and not in $\mathscr{C}$. □

5.8. **If closed under intersection and complement, then closed under union (by De Morgan).**

If $\mathscr{C}$ is closed under intersection and complement, then for any $L_1, L_2 \in \mathscr{C}$: $\sim L_1, \sim L_2 \in \mathscr{C}$ (by closure under complement), so $\sim L_1 \cap \sim L_2 \in \mathscr{C}$ (by closure under intersection), so $\sim (\sim L_1 \cap \sim L_2) = L_1 \cup L_2 \in \mathscr{C}$ (by De Morgan and closure under complement). Hence $\mathscr{C}$ is also closed under union. □

5.9. **Closed under concatenation does not imply closed under Kleene closure.**

Let $\mathscr{C} = F_\Sigma$ (all finite languages). Closed under concatenation (5.1d). Not closed under Kleene closure: $\{\boldsymbol{a}\}^*$ is infinite (5.1e).

5.10. **Closed under Kleene closure does not imply closed under concatenation.**

Let $\mathscr{C}$ be the collection of all languages of the form $\{w\}^*$ for some word $w \in \Sigma^*$ (including $\Sigma^* = \Sigma^*$ and $\emptyset^* = \{\lambda\}$). Each $\{w\}^*$ is closed under Kleene closure trivially ($(\{w\}^*)^* = \{w\}^*$). But $\{w\}^* \cdot \{v\}^*$ is not generally of the form $\{u\}^*$ unless $w = v$ or one is λ. Counterexample: $\{a\}^* \cdot \{b\}^* = \{a^m b^n \mid m, n \geq 0\} \neq \{u\}^*$ for any u.

5.11. **Closed under complement does not imply closed under relative complement.**

Counterexample. Let

$$K = \{\emptyset, \Sigma^*\} \cup \{\{w\} \mid w \in \Sigma^*\} \cup \{\Sigma^* \setminus \{w\} \mid w \in \Sigma^*\}.$$

This collection is closed under complement by construction. But it is not closed under relative complement: if $u \neq v$, then

$$(\Sigma^* \setminus \{u\}) \setminus \{v\} = \Sigma^* \setminus \{u, v\} \notin K.$$

So closure under complement does not imply closure under relative complement.

5.12. **A finite set of numbers closed under $\sqrt{}$:**

$\{0,1\}$. $\sqrt{0} = 0 \in \{0,1\}$ and $\sqrt{1} = 1 \in \{0,1\}$.

5.13. **An infinite set of numbers closed under $\sqrt{}$:**

$[0,1] = \{x \in \mathbb{R} \mid 0 \leq x \leq 1\}$. For any $x \in [0,1]$, $\sqrt{x} \in [0,1]$ since $0 \leq x \leq 1 \Rightarrow 0 \leq \sqrt{x} \leq 1$.

5.14. **Algorithm for deterministic union machine A^U.**

Given $A_1 = \langle \Sigma, S_1, s_0^1, \delta_1, F_1 \rangle$ and $A_2 = \langle \Sigma, S_2, s_0^2, \delta_2, F_2 \rangle$ (both DFAs, $S_1 \cap S_2 = \emptyset$):

1. Build the NDFA $A^\cup = \langle \Sigma, S_1 \cup S_2, \{s_0^1, s_0^2\}, \delta^\cup, F_1 \cup F_2 \rangle$ (as in Theorem 5.2). 2. Apply Definition 4.5 (subset construction) to $A^\cup$ to produce a DFA A^U. Start state: $\{s_0^1, s_0^2\}$. For each reachable state-set $Q \subseteq S_1 \cup S_2$ and each $\boldsymbol{a} \in \Sigma$: $\delta^U(Q, \boldsymbol{a}) = \delta^d(Q, \boldsymbol{a}) = \bigcup_{q \in Q} \delta^\cup(q, \boldsymbol{a})$. Final states: all Q with $Q \cap (F_1 \cup F_2) \neq \emptyset$.

5.15. (a) **Direct product machine for union.** Define $A^u = \langle \Sigma, S_1 \times S_2, \langle s_0^1, s_0^2 \rangle, \delta^u, F^u \rangle$ where $\delta^u(\langle s, t \rangle, \boldsymbol{a}) = \langle \delta_1(s, \boldsymbol{a}), \delta_2(t, \boldsymbol{a}) \rangle$ and $F^u = \{\langle s, t \rangle \mid s \in F_1 \vee t \in F_2\}$.

(b) **Proof.** By induction, $\overline{\delta^u}(\langle s_0^1, s_0^2 \rangle, x) = \langle \overline{\delta^1}(s_0^1, x), \overline{\delta^2}(s_0^2, x) \rangle$. Then $x \in L(A^u)$ iff $\overline{\delta^u}(\langle s_0^1, s_0^2 \rangle, x) \in F^u$ iff $\overline{\delta^1}(s_0^1, x) \in F_1$ or $\overline{\delta^2}(s_0^2, x) \in F_2$ iff $x \in L(A_1) \cup L(A_2)$.

(c) **State counts (no minimization).** $A^\cup$ (NDFA union): $|S_1| + |S_2|$ states (plus the new start state if added separately, but in Theorem 5.2 no new state is needed). A^U (subset construction of $A^\cup$): up to $2^{|S_1|+|S_2|}$ states, but in practice far fewer. A^u (product): $|S_1| \cdot |S_2| = nm$ states. With $n = |S_1|$ and $m = |S_2|$: $A^\cup$ has $n + m$ states; A^U has up to 2^{n+m} states; A^u has nm states.

5.16. **$\mathfrak{D}_\Sigma$ is closed under the prefix operator P.**

Proof. Let $L \in \mathfrak{D}_\Sigma$ and $A = \langle \Sigma, S, s_0, \delta, F \rangle$ be a DFA with $L(A) = L$. Define $A' = \langle \Sigma, S, s_0, \delta, F' \rangle$ where $F' = \{s \in S \mid \exists y \in \Sigma^* \ni \overline{\delta}(s, y) \in F\}$. (A state is final in A' iff some continuation from it leads to acceptance in A.)

Claim: $L(A') = P(L)$.

$x \in P(L)$ iff $\exists y \in \Sigma^* \ni xy \in L$ iff $\exists y \in \Sigma^* \ni \overline{\delta}(s_0, xy) \in F$ iff $\exists y \in \Sigma^* \ni \overline{\delta}(\overline{\delta}(s_0, x), y) \in F$ iff $\overline{\delta}(s_0, x) \in F'$ iff $x \in L(A')$.

Since F' is computable (for each state s, check if any state in F is reachable from s—a finite graph problem), A' is a valid DFA and $P(L) = L(A') \in \mathfrak{D}_\Sigma$. □ □

5.17. **$\mathfrak{D}_\Sigma$ is closed under the suffix operator S.**

(a) **Construction.** Given DFA $A = \langle \Sigma, S, s_0, \delta, F\rangle$, build NDFA $A' = \langle \Sigma, S, S'_0, \delta, F\rangle$ where $S'_0 = \{\overline{\delta}(s_0, y) \mid y \in \Sigma^*\}$, the set of states reachable from s_0.

(b) **Proof.** $x \in S(L)$ iff $\exists y \in \Sigma^* \ni yx \in L$ iff $\exists y \in \Sigma^* \ni \overline{\delta}(s_0, yx) \in F$ iff $\exists s = \overline{\delta}(s_0, y) \in S'_0 \ni \overline{\delta}(s, x) \in F$ iff $\overline{\delta}'(\{s\}, x) \cap F \neq \emptyset$ for some $s \in S'_0$ iff $x \in L(A')$.

(c) Since A' is an NDFA over a finite state set, $L(A') \in \mathfrak{D}_\Sigma$ by Theorem 4.1. Hence $S(L) \in \mathfrak{D}_\Sigma$. □

5.18. **$\mathfrak{D}_\Sigma$ is closed under the center operator C.**

(a) **Construction.** Let $R = \{\overline{\delta}(s_0, y) \mid y \in \Sigma^*\}$ be the set of reachable states. Build an NDFA $A' = \langle \Sigma, S, R, \delta, F'\rangle$ where $F' = \{s \in S \mid \exists y \in \Sigma^* \ni \overline{\delta}(s, y) \in F\}$ (states from which F is reachable).

(b) **Proof.** $x \in C(L)$ iff $\exists y, z \ni yxz \in L$ iff $\exists y, z \ni \overline{\delta}(\overline{\delta}(\overline{\delta}(s_0, y), x), z) \in F$ iff $\exists s = \overline{\delta}(s_0, y) \in R$ and $\overline{\delta}(s, x) \in F'$ iff $x \in L(A')$.

(c) Since A' is an NDFA, $C(L) \in \mathfrak{D}_\Sigma$. □

5.19. **$\mathfrak{D}_\Sigma$ is closed under F (maximal words not extended in L).**

Let $A = \langle \Sigma, S, s_0, \delta, F\rangle$ be a minimal DFA for L. $F(L) = \{x \in L \mid \text{no nonempty word } y \text{ satisfies } xy \in L\}$. Define $A' = \langle \Sigma, S, s_0, \delta, F''\rangle$ where $F'' = \{s \in F \mid (\forall y \in \Sigma^+)(\overline{\delta}(s, y) \notin F)\}$. Then $x \in L(A')$ iff $\overline{\delta}(s_0, x) \in F''$ iff $x \in L$ and there is no nonempty y with $\overline{\delta}(s_0, xy) \in F$, iff $x \in F(L)$. So $F(L) = L(A') \in \mathfrak{D}_\Sigma$. □

5.20. **$\mathfrak{D}_\Sigma$ is closed under reversal.**

(a) **Construction.** Given DFA $A = \langle \Sigma, S, s_0, \delta, F\rangle$, build NDFA $A^r = \langle \Sigma, S, F, \delta^r, \{s_0\}\rangle$ where $\delta^r(s, \boldsymbol{a}) = \{t \in S \mid \delta(t, \boldsymbol{a}) = s\}$ (reverse all arrows).

(b) **Proof.** We show $x \in L(A^r)$ iff $x^r \in L(A)$.
By induction on $|x|$: a path $f \xrightarrow{a_1} s_1 \xrightarrow{a_2} \cdots \xrightarrow{a_k} s_0$ in A^r (from some $f \in F$ to s_0) corresponds to the path $s_0 \xrightarrow{a_k} \cdots \xrightarrow{a_1} f$ in A. Hence $a_1 a_2 \cdots a_k \in L(A^r)$ iff $a_k \cdots a_1 \in L(A)$, i.e., $x \in L(A^r)$ iff $x^r \in L(A)$.

(c) $L^r = L(A^r) \in \mathfrak{D}_\Sigma$ (since A^r is an NDFA which is equivalent to some DFA). □

5.21. (a) **$\mathfrak{D}_\Sigma$ is not closed under $G(L) = \{w^r \cdot w \mid w \in L\}$.**
Let $L = \Sigma^*$ over $\Sigma = \{\boldsymbol{a}, \boldsymbol{b}\}$. Then $G(L) = \{w^r w \mid w \in \Sigma^*\}$, the set of even-length palindromes (of the form $w^r w$). By the pumping lemma, this is not FAD: take $w = \boldsymbol{a}^n \boldsymbol{b}^n$; then $w^r w = \boldsymbol{b}^n \boldsymbol{a}^n \boldsymbol{a}^n \boldsymbol{b}^n \in G(L)$. If $G(L)$ were FAD with n states, pump the string $\boldsymbol{b}^n \boldsymbol{a}^n \boldsymbol{a}^n \boldsymbol{b}^n$ to get a contradiction (pumping the first block of $\boldsymbol{b}$s destroys the palindrome structure).

(b) **$\mathfrak{N}_\Sigma$ is closed under G.** Let $L \in \mathfrak{N}_\Sigma$. If $G(L)$ were FAD, then Theorem 5.11 would imply that $G(L)^b$ is FAD. But $G(L)^b = L^r$, and Exercise 5.20 shows that reversal preserves the FAD languages. Hence $L = (L^r)^r$ would be FAD, contradicting $L \in \mathfrak{N}_\Sigma$. Therefore $G(L) \in \mathfrak{N}_\Sigma$.

(c) **$\mathfrak{N}_\Sigma$ is not closed under b.** By Lemma 5.6, $L = \{\boldsymbol{a}^n \boldsymbol{b}^n \mid n \geq 0\} \in \mathfrak{N}_\Sigma$, but $L^b = \{\boldsymbol{a}^n \mid n \geq 0\}$ is FAD.

5.22. **$\mathfrak{N}_\Sigma$ is closed under reversal.**

Assume $L \in \mathfrak{N}_\Sigma$ but $L^r \notin \mathfrak{N}_\Sigma$. Then $L^r \in \mathfrak{D}_\Sigma$. By Exercise 5.20, $\mathfrak{D}_\Sigma$ is closed under reversal, so $L = (L^r)^r \in \mathfrak{D}_\Sigma$, a contradiction. Hence $L^r \in \mathfrak{N}_\Sigma$ whenever $L \in \mathfrak{N}_\Sigma$.

5.23. $\mathfrak{N}_\Sigma$ **is *not* closed under** P **(prefix operator).**

Let $K = \{x \in \{\boldsymbol{a}, \boldsymbol{b}\}^* \mid |x|_{\boldsymbol{a}} = |x|_{\boldsymbol{b}}\}$. This language belongs to $\mathfrak{N}_\Sigma$ (it is the language used in Lemma 5.4). But $P(K) = \{\boldsymbol{a}, \boldsymbol{b}\}^*$: given any word x, if $|x|_{\boldsymbol{a}} - |x|_{\boldsymbol{b}} = d$, append $|d|$ copies of the deficient symbol to obtain a balanced word. Thus $P(K)$ is FAD, so $\mathfrak{N}_\Sigma$ is not closed under P.

5.24. $\mathfrak{N}_\Sigma$ **is not closed under** S **(suffix operator).**

Using the same language K, we have $S(K) = \{\boldsymbol{a}, \boldsymbol{b}\}^*$: for any word x, prefix it by enough copies of the deficient symbol to balance the counts. Since $K \in \mathfrak{N}_\Sigma$ but $S(K)$ is FAD, $\mathfrak{N}_\Sigma$ is not closed under S.

5.25. $\mathfrak{N}_\Sigma$ **is not closed under** C **(center operator).**

Again let $K = \{x \mid |x|_{\boldsymbol{a}} = |x|_{\boldsymbol{b}}\}$. Then $C(K) = \{\boldsymbol{a}, \boldsymbol{b}\}^*$, because for any word x we can choose $y = \boldsymbol{a}^m$ and $z = \boldsymbol{b}^{m+|x|_{\boldsymbol{a}}-|x|_{\boldsymbol{b}}}$ with m large enough so that yxz has equally many $\boldsymbol{a}$s and $\boldsymbol{b}$s. Hence $C(K)$ is FAD while $K \in \mathfrak{N}_\Sigma$.

5.26. $\mathfrak{N}_\Sigma$ **is not closed under** F**.**

For the same language K, every word in K has a proper extension in K: if $x \in K$, then $x\boldsymbol{ab} \in K$. Therefore $F(K) = \emptyset$, which is FAD. So $\mathfrak{N}_\Sigma$ is not closed under F.

5.27. **Complementing final states of an NDFA does not give complement language.**

From Exercise 4.8: if A is an NDFA, $L(A^\sim) \neq\sim L(A)$ in general.

Example. Let A have $\Sigma = \{\boldsymbol{a}\}$, $S = \{s_0, s_1\}$, $S_0 = \{s_0\}$, $F = \{s_1\}$, $\delta(s_0, \boldsymbol{a}) = \{s_0, s_1\}$, $\delta(s_1, \boldsymbol{a}) = \{s_1\}$. Then $L(A) = \{\boldsymbol{a}\}^+ = \{\boldsymbol{a}, \boldsymbol{aa}, \ldots\}$. After complementing: $F^\sim = \{s_0\}$. $\boldsymbol{a}$: from s_0, reach $\{s_0, s_1\}$; $s_0 \in F^\sim$, so $\boldsymbol{a} \in L(A^\sim)$. But $\boldsymbol{a} \in L(A)$, so $\boldsymbol{a} \notin\sim L(A)$. Thus $\boldsymbol{a} \in L(A^\sim)$ but $\boldsymbol{a} \notin\sim L(A)$, so $L(A^\sim) \neq\sim L(A)$.

5.28. **Complete the proof of Lemma 5.7.**

Proof. Let $L = \{x \in \{\boldsymbol{a}, \boldsymbol{b}\}^* \mid |x|_{\boldsymbol{a}} \neq |x|_{\boldsymbol{b}}\}$, the language used in Lemma 5.7. We show that $L/L = \Sigma^*$.

Take any $x \in \Sigma^*$ and let $d = |x|_{\boldsymbol{a}} - |x|_{\boldsymbol{b}}$. Define $y = \boldsymbol{a}^{|d|+1}$. Then $y \in L$, because it has strictly more $\boldsymbol{a}$s than $\boldsymbol{b}$s. Also

$$|xy|_{\boldsymbol{a}} - |xy|_{\boldsymbol{b}} = d + (|d| + 1) \neq 0,$$

so $xy \in L$ as well. Hence $x \in L/L$. Since x was arbitrary, $L/L = \Sigma^*$. Therefore $\mathfrak{N}_\Sigma$ is not closed under quotient. □

5.29. **Collection closed under union, concatenation, Kleene closure, but not intersection.**

Take $\mathscr{C} = I_\Sigma$, the collection of all infinite languages from Exercise 5.4. Exercise 5.4 shows that I_Σ is closed under union, concatenation, and Kleene closure, but not under intersection. So I_Σ is the required example.

5.30. **Closed under union does not imply closed under intersection.**

Counterexample. Again use I_Σ, the infinite languages. Exercise 5.4(b) shows it is closed under union, while Exercise 5.4(c) shows it is not closed under intersection.

5.31. **Prove** $L(A^\bullet) = L_1 \cdot L_2$ **(Theorem 5.4 construction).**

Proof. If $x \in L_1 \cdot L_2$, write $x = uv$ with $u \in L_1$ and $v \in L_2$. Run A_1 on u from s_{0_1} to some state of F_1.

If $v = \lambda$, then $\lambda \in L_2$, so Theorem 5.4 makes every state of F_1 final in $A^\bullet$; hence the same path accepts x.

If $v = \boldsymbol{a}w$ is nonempty, then from the final state reached after u the definition of $\delta^\bullet$ allows the next input letter $\boldsymbol{a}$ to be processed exactly as it would be from s_{0_2} in A_2. The rest of v then follows the transitions of A_2, so some path in $A^\bullet$ accepts x. Thus $L_1 \cdot L_2 \subseteq L(A^\bullet)$.

Conversely, let $x \in L(A^\bullet)$. An accepting path either stays entirely in S_1 and ends in a state of F_1; this can happen only when $\lambda \in L_2$, so then $x \in L_1 = L_1 \cdot \lambda \subseteq L_1 \cdot L_2$. Otherwise there is a first transition that enters S_2. Just before that step, the path is at some state $f \in F_1$, so the already-read prefix u lies in L_1. The rest of the input, beginning with the symbol that caused the move into S_2, follows A_2 from s_{0_2} to a final state, so it lies in L_2. Hence $x = uv \in L_1 \cdot L_2$. □

5.32. **Prove $L(A_*) = L^*$ (Theorem 5.5 construction).**

Proof. Since $q_0 \in F_*$, the empty word is accepted, so $L^0 = \{\lambda\} \subseteq L(A_*)$.

Now let $x = x_1 x_2 \cdots x_k$ with each $x_i \in L$. Run the original machine A on x_1 from some start state in S_0 to a final state in F. Whenever one factor ends and the next factor begins with a letter $\boldsymbol{a}$, the definition of δ_* for final states adds the transitions $\bigcup_{t \in S_0} \delta(t, \boldsymbol{a})$, so the same input letter can also begin a new run of A on the next factor. Repeating this for all factors yields an accepting path for x, so $L^* \subseteq L(A_*)$.

Conversely, consider an accepting path of A_* on some nonempty word x. Because $\delta_*(q_0, \boldsymbol{a}) = \emptyset$, the path must begin in some state of S_0. Each time the path uses one of the extra transitions $\bigcup_{t \in S_0} \delta(t, \boldsymbol{a})$ from a final state, it starts a fresh copy of the original machine on the next factor of the input. Thus the input splits into finitely many consecutive blocks, each of which labels a path in A from a start state to a final state; hence each block lies in L. Therefore $x \in L^*$. □

5.33. **Amplify the proof of Theorem 5.2 (union).**

Theorem 5.2: $\mathfrak{D}_\Sigma$ is closed under union. The proof uses $A^\cup$ (NDFA combining A_1 and A_2 with joint start-state set $S_0^1 \cup S_0^2$). The key equivalences:

$x \in L(A^\cup)$ [Definition of $L(A^\cup)$] $\Leftrightarrow \overline{\delta}^\cup(S_0^1 \cup S_0^2, x) \cap (F_1 \cup F_2) \neq \emptyset$ [Def. 4.4] $\Leftrightarrow (\overline{\delta}^\cup(S_0^1, x) \cup \overline{\delta}^\cup(S_0^2, x)) \cap (F_1 \cup F_2) \neq \emptyset$ [Exercise 4.27] $\Leftrightarrow (\overline{\delta}_1(s_0^1, x) \cup \overline{\delta}_2(s_0^2, x)) \cap (F_1 \cup F_2) \neq \emptyset$ [paths stay in S_i] $\Leftrightarrow \overline{\delta}_1(s_0^1, x) \cap F_1 \neq \emptyset$ or $\overline{\delta}_2(s_0^2, x) \cap F_2 \neq \emptyset \Leftrightarrow x \in L_1 \cup L_2$.

5.34. **Given DFA A, define NDFA accepting $L(A)^+$.**

Define $A^+ = \langle \Sigma, S, s_0, \delta^+, F \rangle$ where $\delta^+(s, \boldsymbol{a}) = \{\delta(s, \boldsymbol{a})\} \cup \{s_0 \mid \delta(s, \boldsymbol{a}) \in F\}$. That is, whenever a transition would lead to a final state, simultaneously allow the machine to restart from s_0. The final states remain F. This accepts $L^+ = \bigcup_{k \geq 1} L^k$.

5.35. **Given NDFA A, define NDFA accepting $L(A)^+$.**

Same construction as 5.34 adapted to NDFAs: $A^+ = \langle \Sigma, S, S_0, \delta^+, F \rangle$ where for each $s \in S$ and $\boldsymbol{a} \in \Sigma$: $\delta^+(s, \boldsymbol{a}) = \delta(s, \boldsymbol{a}) \cup (S_0 \text{ if } \delta(s, \boldsymbol{a}) \cap F \neq \emptyset)$. This allows returning to any start state after reaching a final state.

5.36. **If L is FAD, must all subsets of L be FAD?**

No. Let $\Sigma = \{\boldsymbol{a}, \boldsymbol{b}\}$ and $L = \Sigma^*$ (FAD). The subset $\{a^n b^n \mid n \geq 0\}$ is not FAD (by the pumping lemma).

5.37. **If $L \in \mathfrak{D}_\Sigma$, must all supersets of L be in $\mathfrak{D}_\Sigma$?**

No. Let $L = \emptyset \in \mathfrak{D}_\Sigma$. The superset $\{a^n b^n \mid n \geq 0\}$ is not FAD.

5.38. **If $L \in \mathfrak{N}_\Sigma$, must all subsets be in $\mathfrak{N}_\Sigma$?**

No. Let $K = \{x \in \{\boldsymbol{a}, \boldsymbol{b}\}^* \mid |x|_{\boldsymbol{a}} = |x|_{\boldsymbol{b}}\} \in \mathfrak{N}_\Sigma$. Then $\{\lambda\} \subseteq K$, but $\{\lambda\} \in \mathfrak{D}_\Sigma$.

5.39. **If $L \in \mathfrak{N}_\Sigma$, must all supersets be in $\mathfrak{N}_\Sigma$?**

No. With the same language $K \in \mathfrak{N}_\Sigma$, we have $K \subseteq \{\boldsymbol{a}, \boldsymbol{b}\}^*$, but $\{\boldsymbol{a}, \boldsymbol{b}\}^* \in \mathfrak{D}_\Sigma$.

5.40. **Why is q_0 needed in the Kleene closure construction (Theorem 5.5)?**

The extra start state q_0 handles the case $\lambda \in L^*$ (which is always true) independently from the rest of the machine. Without q_0, making s_0 both the start and a final state could create spurious acceptance: words that loop back through the original start states might be accepted even when they are not valid concatenations of words from L. The new state q_0 is needed only to accept λ without changing the behavior of the original start states.

5.41. **Redesign Theorem 5.4 (concatenation) using λ-transitions.**

Given DFAs A_1 and A_2, construct $A^\bullet_\lambda = \langle \Sigma, S_1 \cup S_2, s_0^1, \delta^\bullet_\lambda, F_2 \rangle$ where: $\delta^\bullet_\lambda(s, \boldsymbol{a}) = \delta_1(s, \boldsymbol{a})$ for $s \in S_1$, $\delta^\bullet_\lambda(s, \boldsymbol{a}) = \delta_2(s, \boldsymbol{a})$ for $s \in S_2$, and $\delta^\bullet_\lambda(f, \lambda) \ni s_0^2$ for each $f \in F_1$. The λ-transition from each $f \in F_1$ to s_0^2 cleanly separates the two phases.

5.42. **Redesign Theorem 5.5 (Kleene closure) using λ-transitions, without extra q_0.**

Given DFA $A = \langle \Sigma, S, s_0, \delta, F \rangle$, make s_0 a final state (to accept λ) and add λ-transitions from each $f \in F$ back to s_0: $A_* = \langle \Sigma, S, s_0, \delta_*, F \cup \{s_0\} \rangle$ where $\delta_*(f, \lambda) \ni s_0$ for each $f \in F$ and $\delta_*(s, \boldsymbol{a}) = \delta(s, \boldsymbol{a})$ otherwise. This avoids the extra q_0 because the λ-back-edges from F to s_0 directly enable re-running from the start state after each accepted subword.

5.43. **Redesign Theorem 5.4 for NDFAs A_1 and A_2.**

Given NDFAs $A_1 = \langle \Sigma, S_1, S_0^1, \delta_1, F_1 \rangle$ and $A_2 = \langle \Sigma, S_2, S_0^2, \delta_2, F_2 \rangle$ with $S_1 \cap S_2 = \emptyset$: Define

$$A^\bullet = \langle \Sigma, S_1 \cup S_2, S_0^1, \delta^\bullet, F^\bullet \rangle,$$

where

$$F^\bullet = \begin{cases} F_2, & \lambda \notin L_2, \\ F_1 \cup F_2, & \lambda \in L_2, \end{cases}$$

and

$$\delta^\bullet(s, \boldsymbol{a}) = \begin{cases} \delta_1(s, \boldsymbol{a}), & s \in S_1 - F_1, \\ \delta_1(s, \boldsymbol{a}) \cup \bigcup\limits_{t \in S_0^2} \delta_2(t, \boldsymbol{a}), & s \in F_1, \\ \delta_2(s, \boldsymbol{a}), & s \in S_2. \end{cases}$$

This is the exact NDFA analogue of Theorem 5.4: when the current state is already in F_1, the same input symbol may also start a run of A_2 from any start state in S_0^2.

5.44. **Redesign Theorem 5.5 for DFA A.**

If $A = \langle \Sigma, S, s_0, \delta, F\rangle$ is deterministic, the redesigned machine is the NDFA

$$A_* = \langle \Sigma, S \cup \{q_0\}, \{q_0, s_0\}, \delta_*, F \cup \{q_0\}\rangle,$$

where

$$\delta_*(s, \boldsymbol{a}) = \begin{cases} \{\delta(s, \boldsymbol{a})\}, & s \in S - F, \\ \{\delta(s, \boldsymbol{a}), \delta(s_0, \boldsymbol{a})\}, & s \in F, \\ \emptyset, & s = q_0. \end{cases}$$

This is the DFA-specialized form of Theorem 5.5, with singleton outputs exactly because the old transition function δ is deterministic.

5.45. **Comparing R_L and $R_{\sim L}$.**

The right congruence $\equiv_L$ on Σ^* is defined by $x \equiv_L y$ iff $\forall z \in \Sigma^*, xz \in L \Leftrightarrow yz \in L$. The right congruence $\equiv_{\sim L}$ satisfies $x \equiv_{\sim L} y$ iff $\forall z, xz \in \sim L \Leftrightarrow yz \in \sim L$, i.e., $xz \notin L \Leftrightarrow yz \notin L$, i.e., $xz \in L \Leftrightarrow yz \in L$. So $\equiv_L = \equiv_{\sim L}$: the two right congruences are identical.

This makes sense: if A is a minimal DFA for L, then $A^\sim$ (with complemented final states) is a minimal DFA for $\sim L$, and both have the same state set and transition function. Thus $R_L = R_{\sim L}$.

5.46. (a) **Examples where $R_{L_1 \cap L_2} = R_{L_1} \cap R_{L_2}$.** Let $L_1 = L_2$ be any FAD language. Then $L_1 \cap L_2 = L_1$, so $R_{L_1 \cap L_2} = R_{L_1} = R_{L_1} \cap R_{L_1} = R_{L_1} \cap R_{L_2}$.

(b) **Examples where $R_{L_1 \cap L_2} \neq R_{L_1} \cap R_{L_2}$.** Let $L_1 = \{\boldsymbol{a}\}$ and $L_2 = \{\boldsymbol{b}\}$ over $\Sigma = \{\boldsymbol{a}, \boldsymbol{b}\}$. Then $L_1 \cap L_2 = \emptyset$, so $R_{L_1 \cap L_2} = R_\emptyset$ is the universal relation. But $\lambda \not R_{L_1} \boldsymbol{a}$, because with suffix λ the word $\boldsymbol{a}$ is in L_1 and λ is not. Hence

$$R_{L_1 \cap L_2} \neq R_{L_1} \cap R_{L_2}.$$

5.47. **$\mathfrak{D}_\Sigma$ is closed under relative complement.**

(a) **By theorems.** $L_1 \setminus L_2 = L_1 \cap \sim L_2$. Since $\mathfrak{D}_\Sigma$ is closed under complement (Theorem 5.1) and intersection (Lemma 5.1), $L_1 \setminus L_2 \in \mathfrak{D}_\Sigma$.

(b) **Machine definition.** Define A^- as the product of A_1 and $A_2^\sim$ (complement of A_2), using the intersection machine from Lemma 5.1 with $F^- = F_1 \times (S_2 \setminus F_2)$.

(c) **Proof.** By induction (as in Exercise 5.56/5.57), $\overline{\delta^-}(\langle s_0^1, s_0^2\rangle, x) = \langle \overline{\delta^1}(s_0^1, x), \overline{\delta^2}(s_0^2, x)\rangle$. Then $x \in L(A^-)$ iff $\overline{\delta^1}(s_0^1, x) \in F_1$ and $\overline{\delta^2}(s_0^2, x) \notin F_2$ iff $x \in L_1 \setminus L_2$. □

5.48. $\mathfrak{L}_\Sigma$ = languages recognized by λ-NDFAs. By Theorem 4.2, $\mathfrak{L}_\Sigma = \mathfrak{D}_\Sigma$. Hence $\mathfrak{L}_\Sigma$ has exactly the same closure properties as $\mathfrak{D}_\Sigma$ (Theorems 5.1–5.12).

5.49. (a) **Example of L with $\lambda \in L^+$.** Let $L = \{\lambda\}$. Then $L^+ = \bigcup_{k \geq 1} L^k = \{\lambda\}$, and $\lambda \in L^+$.

(b) **Three languages with $L^+ = L$.**

(i) $L = \Sigma^*$: $(\Sigma^*)^+ = \Sigma^*$.

(ii) $L = \{\boldsymbol{a}\}^*$: $(\{\boldsymbol{a}\}^*)^+ = \{\boldsymbol{a}\}^*$.

(iii) $L = \Sigma^+$: $(\Sigma^+)^+ = \Sigma^+$ (since $\Sigma^+ \cdot \Sigma^+ \subseteq \Sigma^+$).

5.50. (a) **Prove $\overline{\delta^\cup}(s, x) = \overline{\delta}_i(s, x)$ for $s \in S_i$.**

Proof. By induction on $|x|$.

Base ($|x| = 0$): $\overline{\delta^{\cup}}(s,\lambda) = \{s\} = \overline{\delta}_i(s,\lambda)$ (by definition of extended transition functions for single start-states, noting NDFAs).

Step: Assume $\overline{\delta^{\cup}}(s,x) = \overline{\delta}_i(s,x)$ for $s \in S_i$. For $x\boldsymbol{a}$:

$$\overline{\delta^{\cup}}(s,x\boldsymbol{a}) = \bigcup_{t \in \overline{\delta^{\cup}}(s,x)} \delta^{\cup}(t,\boldsymbol{a}) = \bigcup_{t \in \overline{\delta}_i(s,x)} \delta_i(t,\boldsymbol{a}) = \overline{\delta}_i(s,x\boldsymbol{a}),$$

where we used the fact that $\overline{\delta}_i(s,x) \subseteq S_i$ (paths from $s \in S_i$ stay in S_i, since $\delta_i : S_i \to \wp(S_i)$) and $\delta^{\cup}(t,\boldsymbol{a}) = \delta_i(t,\boldsymbol{a})$ for $t \in S_i$. □ □

(b) **Was this used in Theorem 5.2?** Yes, implicitly: the proof that $L(A^{\cup}) = L_1 \cup L_2$ relies on the fact that paths starting in S_1 stay in S_1 and paths starting in S_2 stay in S_2, which is exactly this lemma.

5.51. (a) $\mathfrak{N}_\Sigma$ **is not closed under relative complement.** Let $K = \{x \in \{\boldsymbol{a},\boldsymbol{b}\}^* \mid |x|_{\boldsymbol{a}} = |x|_{\boldsymbol{b}}\} \in \mathfrak{N}_\Sigma$. Then $K - K = \emptyset \in \mathfrak{D}_\Sigma$.

(b) $\mathfrak{N}_\Sigma$ **is not closed under union.** If $K \in \mathfrak{N}_\Sigma$, then $\sim K \in \mathfrak{N}_\Sigma$ by Theorem 5.8. But $K \cup \sim K = \Sigma^* \in \mathfrak{D}_\Sigma$.

(c) $\mathfrak{N}_\Sigma$ **is not closed under concatenation.** Let $E = \{\boldsymbol{a}^n\boldsymbol{b}^n \mid n \geq 1\}$ and define $L = \{\boldsymbol{a},\boldsymbol{b}\}^* - E$. Since $E \in \mathfrak{N}_\Sigma$, Theorem 5.8 gives $L \in \mathfrak{N}_\Sigma$. Also $\lambda \in L$, and if $x \notin L$ then $x = \boldsymbol{a}^n\boldsymbol{b}^n = \boldsymbol{a} \cdot \boldsymbol{a}^{n-1}\boldsymbol{b}^n$ with both factors in L. Hence $L \cdot L = \{\boldsymbol{a},\boldsymbol{b}\}^* \in \mathfrak{D}_\Sigma$.

(d) $\mathfrak{N}_\Sigma$ **is not closed under Kleene closure.** With the same language L, we have $\lambda \in L$ and $L \cdot L = \Sigma^*$, so $L^* = \Sigma^* \in \mathfrak{D}_\Sigma$.

(e) **If $L \in \mathfrak{N}_\Sigma$, then $L^+ \in \mathfrak{N}_\Sigma$: No.** For the same language L, because $\lambda \in L$ we have $L^+ = L^* = \Sigma^*$, which is FAD.

5.52. **Why require $S_1 \cap S_2 = \emptyset$ in Theorem 5.4?**

If $S_1 \cap S_2 \neq \emptyset$, the transition function $\delta^\bullet$ would be ambiguous: for a state $s \in S_1 \cap S_2$, which transitions should apply—those of A_1 or A_2? The definition of $\delta^\bullet$ would not be well-defined. Moreover, the proof that paths through the concatenation machine correctly simulate A_1 then A_2 relies on the fact that once the computation enters S_2, it stays in S_2 (and similarly for S_1). If the state sets overlap, a path might inadvertently "mix" transitions from A_1 and A_2.

5.53. $\mathfrak{D}_\Sigma$ **is closed under** $E(L) = \{z \mid \exists y \in \Sigma^+, \exists x \in L \ni z = yx\}$**.**

Here $E(L) = \Sigma^+ \cdot L$, the set of words obtained by prepending a nonempty prefix to a word of L.

(a) **Construction.** Let B be the two-state DFA for Σ^+: start state p_0 (nonfinal), final state p_1, and for every $\boldsymbol{a} \in \Sigma$, $\delta_B(p_0,\boldsymbol{a}) = p_1$ and $\delta_B(p_1,\boldsymbol{a}) = p_1$. If A accepts L, apply the concatenation construction of Theorem 5.4 to B and A.

(b) **Proof.** A word z is accepted by the new machine iff it can be written as $z = yx$ where y is accepted by B and x is accepted by A. That is equivalent to saying $y \in \Sigma^+$ and $x \in L$, so the new machine accepts exactly $E(L)$.

(c) Since $\Sigma^+ \in \mathfrak{D}_\Sigma$ and $\mathfrak{D}_\Sigma$ is closed under concatenation, $E(L) = \Sigma^+ \cdot L \in \mathfrak{D}_\Sigma$. □

5.54. $\mathfrak{D}_\Sigma$ **is closed under** $B(L) = \{z \mid \exists x \in L, \exists y \in \Sigma^* \ni z = xy\}$**.**

$B(L)$ is the set of all words that have a prefix in L.

(a) **Construction.** $B(L) = L \cdot \Sigma^*$ (the set of all extensions of words in L). Given DFA $A = \langle \Sigma, S, s_0, \delta, F \rangle$ for L, set $F' = \{s \in S \mid \overline{\delta}(s_0, x) \in F \text{ for some prefix } x \text{ of the input}\}$. Equivalently, let F' be the set of states reachable from any state in F (including F itself): $F' = \{s \mid \exists f \in F, \exists y \in \Sigma^*, \overline{\delta}(f, y) = s\} \cup F$. Then $A' = \langle \Sigma, S, s_0, \delta, F' \rangle$ accepts $B(L)$.

(b) **Proof:** $B(L) = L \cdot \Sigma^* \in \mathfrak{D}_\Sigma$ by closure under concatenation.

(c) □

5.55. $\mathfrak{D}_\Sigma$ **is closed under** $M(L) = \{z \mid \exists x \in L, \exists y \in \Sigma^+ \ni z = xy\}$**.**

$M(L) = \{z \mid z = xy, x \in L, y \in \Sigma^+\} = L \cdot \Sigma^+$. By closure under concatenation, $L \cdot \Sigma^+ \in \mathfrak{D}_\Sigma$.

(a) **Construction:** Concatenate A (for L) with a DFA for Σ^+ (which is a two-state machine: start nonfinal, self-loop on Σ to final, final self-loops on Σ).

(b) **Proof:** By Theorem 5.4, $M(L) = L(A^\bullet) \in \mathfrak{D}_\Sigma$.

(c) □

5.56. **Prove** $\overline{\delta^\cap}(\langle s, t \rangle, x) = \langle \overline{\delta}_1(s, x), \overline{\delta}_2(t, x) \rangle$**.**

Proof. By induction on $|x|$.

Base: $\overline{\delta^\cap}(\langle s, t \rangle, \lambda) = \langle s, t \rangle = \langle \overline{\delta}_1(s, \lambda), \overline{\delta}_2(t, \lambda) \rangle$.

Step: Assume the result for x. Then:

$$\begin{aligned}
\overline{\delta^\cap}(\langle s, t \rangle, x\boldsymbol{a}) &= \delta^\cap(\overline{\delta^\cap}(\langle s, t \rangle, x), \boldsymbol{a}) \\
&= \delta^\cap(\langle \overline{\delta}_1(s, x), \overline{\delta}_2(t, x) \rangle, \boldsymbol{a}) \\
&= \langle \delta_1(\overline{\delta}_1(s, x), \boldsymbol{a}), \delta_2(\overline{\delta}_2(t, x), \boldsymbol{a}) \rangle \\
&= \langle \overline{\delta}_1(s, x\boldsymbol{a}), \overline{\delta}_2(t, x\boldsymbol{a}) \rangle. \quad \square
\end{aligned}$$

□

5.57. **Prove** $L(A^\cap) = L_1 \cap L_2$**.**

Proof. By Exercise 5.56:

$$\begin{aligned}
x \in L(A^\cap) &\Leftrightarrow \overline{\delta^\cap}(\langle s_0^1, s_0^2 \rangle, x) \in F^\cap \\
&\Leftrightarrow \langle \overline{\delta}_1(s_0^1, x), \overline{\delta}_2(s_0^2, x) \rangle \in F_1 \times F_2 \\
&\Leftrightarrow \overline{\delta}_1(s_0^1, x) \in F_1 \wedge \overline{\delta}_2(s_0^2, x) \in F_2 \\
&\Leftrightarrow x \in L_1 \wedge x \in L_2 \\
&\Leftrightarrow x \in L_1 \cap L_2. \quad \square
\end{aligned}$$

□

5.58. **Prove Theorem 5.6 (**$\mathfrak{D}_\Sigma$ **closed under** Z**).**

Proof. Let $A = \langle \Sigma, S, s_0, \delta, F \rangle$ be a DFA for L. Build a λ-NDFA A_Z by adding, for every ordinary transition $s \xrightarrow{\boldsymbol{a}} t$ of A, a matching λ-transition $s \xrightarrow{\lambda} t$. Then a path in A_Z may either read a letter of the input or skip a letter that was present in some word of L.

Thus x is accepted by A_Z iff there exists some $w \in L$ from which x can be obtained by deleting zero or more letters, i.e., iff $x \in Z(L)$. Hence $Z(L) \in \mathfrak{D}_\Sigma$. □

5.59. **Prove Theorem 5.7 ($\mathfrak{D}_\Sigma$ closed under Y).**

Proof. Let $A = \langle \Sigma, S, s_0, \delta, F \rangle$ be a DFA for L. Following the text after Theorem 5.7, build two copies of A. Being in the first copy means that no letter has yet been deleted; being in the second copy means that exactly one letter has been deleted. Ordinary transitions stay within each copy, and for every transition $s \xrightarrow{\boldsymbol{a}} t$ in the first copy add a λ-transition from the first-copy state s to the second-copy state t.

An accepting path therefore behaves like a path of A on some word of L, with exactly one letter skipped at the moment when the path moves from the first copy to the second. So the resulting machine accepts exactly $Y(L)$, and therefore $Y(L) \in \mathfrak{D}_\Sigma$. □

5.60. (a) **Alternative construction for Theorem 5.7 without λ-moves.**
Let $A = \langle \Sigma, S, s_0, \delta, F \rangle$ be a DFA for L. Use state set $S \times \{0,1\}$, where the second component records whether the one deletion has already occurred. The start state is $\langle s_0, 0 \rangle$. Final states are

$$\{\langle s, 1\rangle \mid s \in F\} \cup \{\langle s, 0\rangle \mid (\exists \boldsymbol{a} \in \Sigma)\ \delta(s, \boldsymbol{a}) \in F\}.$$

On input $\boldsymbol{a}$, define

$$\delta'(\langle s, 0\rangle, \boldsymbol{a}) = \{\langle \delta(s, \boldsymbol{a}), 0\rangle\} \cup \{\langle \delta(\delta(s, \boldsymbol{b}), \boldsymbol{a}), 1\rangle \mid \boldsymbol{b} \in \Sigma\},$$

and

$$\delta'(\langle s, 1\rangle, \boldsymbol{a}) = \{\langle \delta(s, \boldsymbol{a}), 1\rangle\}.$$

(b) **Proof:** In the 0-copy the machine either reads the current input symbol normally, or guesses that one letter $\boldsymbol{b}$ of the original word was deleted immediately before it and moves into the 1-copy. Once in the 1-copy, the rest of the input is processed exactly as in A. The extra final states in the 0-copy handle the case where the deleted letter was the last letter of the original word. Hence this NDFA accepts exactly $Y(L)$.

5.61. **$\mathfrak{D}_\Sigma$ is closed under $W(L)$ (delete one or more letters).**

(a) **Construction.** Given DFA A for L, build λ-NDFA A' by adding, for each transition $s \xrightarrow{\boldsymbol{a}} t$, an optional λ-transition $s \xrightarrow{\lambda} t$ (allowing deletion of $\boldsymbol{a}$). Require at least one deletion by adding a flag bit.
More precisely: A' has state set $S \times \{0,1\}$ where the second component tracks whether at least one letter has been deleted. Initial state: $\langle s_0, 0\rangle$. Final states: $\{\langle f, 1\rangle \mid f \in F\}$. For each $c \in \Sigma$,

$$\delta'(\langle s, b\rangle, c) = \{\langle \delta(s, c), b\rangle\}, \qquad \delta'(\langle s, b\rangle, \lambda) = \{\langle \delta(s, c), 1\rangle \mid c \in \Sigma\}.$$

The ordinary transition reads the next input symbol, while the λ-transition simulates deleting one symbol c from the original word and moves into the "already deleted" copy.

(b) **Proof.** A word z is in $W(L)$ iff z can be obtained by deleting at least one letter from some $w \in L$. A path in A' simulates a path in A but uses λ-transitions to skip letters; the flag ensures at least one skip. Hence $L(A') = W(L)$.

(c) Since A' is a λ-NDFA, $W(L) \in \mathfrak{D}_\Sigma$. □

5.62. $\mathfrak{D}_\Sigma$ **is closed under** $V(L)$ **(delete odd-positioned letters).**

(a) **Construction.** Build A' from A with a parity bit added to the state: $S' = S \times \{0,1\}$ where 0 = currently at an odd position, 1 = at an even position. Transitions: on reading input letter $\boldsymbol{a}$ at position parity b: if $b = 0$ (odd position): delete $\boldsymbol{a}$ (take λ-transition, flip parity to 1); if $b = 1$ (even position): process $\boldsymbol{a}$ normally, flip parity to 0.

(b) **Proof.** The machine A' processes every even-positioned letter via A's transitions and skips every odd-positioned letter. An accepting path corresponds exactly to a word in $V(L)$.

(c) $L(A') \in \mathfrak{D}_\Sigma$. □

5.63. $\mathfrak{D}_\Sigma$ **is closed under** $U(L)$ **(delete even-positioned letters).**

Same construction as 5.62 but swap the roles of odd and even positions: delete even-positioned letters, process odd-positioned letters. The parity tracking is identical. □

5.64. $\mathfrak{D}_\Sigma$ **is closed under** $T(L)$ **(delete every 3rd, 6th, ... letter).**

(a) **Construction.** Add a counter modulo 3 to the state: $S' = S \times \{0,1,2\}$. Initial state: $\langle s_0, 1\rangle$ (position 1 counted from 1). Transition on reading $\boldsymbol{a}$ at position k: if $k \equiv 0 \pmod 3$: delete $\boldsymbol{a}$ (λ-transition), increment counter mod 3; else: process $\boldsymbol{a}$ via δ, increment counter mod 3.

(b) **Proof.** The counter tracks position modulo 3; every multiple-of-3 position is skipped.

(c) $L(A') \in \mathfrak{D}_\Sigma$. □

5.65. $I(L) = L \cap P$ **where** $P = \{x \mid |x|$ **is prime**$\}$**.**

(a) $\mathfrak{D}_\Sigma$ **not closed under** I**.** Let $\Sigma = \{\boldsymbol{a}\}$ and $L = \Sigma^*$. Then $I(L) = P = \{\boldsymbol{a}^p \mid p \text{ prime}\}$. By the pumping lemma (pump $\boldsymbol{a}^p$ to get $\boldsymbol{a}^{p+k}$ for any $k \geq 0$, most of which are not prime), $P \notin \mathfrak{D}_\Sigma$.

(b) F_Σ **closed under** I**.** $I(L) = L \cap P \subseteq L$; if L is finite, so is $I(L)$.

(c) C_Σ **closed under** I**? No.** $L = \Sigma^* \setminus F$ for finite F. $I(L) = P \setminus F'$ for some finite F'. If P is cofinite: P is not cofinite (infinitely many non-primes), so $P \setminus F'$ is not cofinite. **Not closed.**

(d) B_Σ **closed under** I**? No.** $I(\Sigma^*) = P \notin B_\Sigma$ (not finite, not cofinite).

(e) I_Σ **closed under** I**? No.** Let $\Sigma = \{\boldsymbol{a}\}$ and $L = \{\boldsymbol{a}^{2n} \mid n \geq 1\}$. Then $L \in I_\Sigma$, but

$$I(L) = L \cap P = \{\boldsymbol{aa}\},$$

since 2 is the only even prime. Thus $I(L) \notin I_\Sigma$.

(f) J_Σ **closed under** I**?** $I(L) = L \cap P$. If $\sim L$ is infinite, $\sim (L \cap P) \supseteq \sim P$ which is infinite (non-primes). **Yes, closed.**

(g) $\boldsymbol{E}$ **closed under** I**?** If $\boldsymbol{abba} \in L$ and $|\boldsymbol{abba}| = 4$ is not prime, then $\boldsymbol{abba} \notin I(L)$. So $I(L)$ might not contain $\boldsymbol{abba}$. **Not closed.**

(h) $\mathfrak{N}_\Sigma$ **closed under I? No.** Over $\Sigma = \{\boldsymbol{a}\}$, let $P = \{\boldsymbol{a}^p \mid p \text{ prime}\}$ and $L = \{\boldsymbol{a}\}^* - P$. Since $P \notin \mathfrak{D}_\Sigma$, Theorem 5.8 implies $L \in \mathfrak{N}_\Sigma$. But

$$I(L) = L \cap P = \emptyset \in \mathfrak{D}_\Sigma.$$

Hence $\mathfrak{N}_\Sigma$ is not closed under I.

5.66. C: all languages over $\{\boldsymbol{a}, \boldsymbol{b}\}$ not containing λ.

(a) **Complementation: No.** $\sim L$ contains λ (since $\lambda \notin L$ but $\lambda \in \Sigma^*$), so $\sim L \notin C$.

(b) **Union: Yes.** If $\lambda \notin L_1$ and $\lambda \notin L_2$, then $\lambda \notin L_1 \cup L_2$.

(c) **Intersection: Yes.** $\lambda \notin L_1 \cap L_2 \subseteq L_1$.

(d) **Concatenation: Yes.** If $\lambda \notin L_1$ and $\lambda \notin L_2$: every word in $L_1 \cdot L_2$ is of the form xy with $x \in L_1$, $y \in L_2$, and $x \neq \lambda$, $y \neq \lambda$, so $xy \neq \lambda$.

(e) **Kleene closure: No.** $\lambda \in L^*$ always.

(f) **Relative complement: Yes.** $\lambda \notin L_1 \setminus L_2 \subseteq L_1$.

(g) L^+**: Yes.** $L^+ = \bigcup_{k \geq 1} L^k$. Each L^k contains words of positive length (since $\lambda \notin L$), so $\lambda \notin L^+$.

5.67. (a) $\mathfrak{D}_\Sigma$ **closed under finite union.**

(i) *By theorems and induction:* Base: $\{L\} \subseteq \mathfrak{D}_\Sigma$ trivially. Step: if $L_1 \cup L_2 \cup \cdots \cup L_k \in \mathfrak{D}_\Sigma$ and $L_{k+1} \in \mathfrak{D}_\Sigma$, then $(L_1 \cup \cdots \cup L_k) \cup L_{k+1} \in \mathfrak{D}_\Sigma$ by Theorem 5.2.

(ii) *By construction:* Build $A^{(k)}$ iteratively by the union construction of Exercise 5.14; each step is deterministic.

(b) $\mathfrak{D}_\Sigma$ **is NOT closed under infinite union.** Each singleton $\{a^n b^n\} \in \mathfrak{D}_\Sigma$ (finite languages are regular), but $\bigcup_{n \geq 0} \{a^n b^n\} = \{a^n b^n \mid n \geq 0\} \notin \mathfrak{D}_\Sigma$ (not regular, by pumping).

5.68. (a) **Three homomorphisms under which $\mathfrak{N}_\Sigma$ is not closed** (over $\Sigma = \{\boldsymbol{a}, \boldsymbol{b}\}$): Use the non-FAD language $K = \{x \in \{\boldsymbol{a}, \boldsymbol{b}\}^* \mid |x|_{\boldsymbol{a}} = |x|_{\boldsymbol{b}}\}$ from Lemma 5.4.

(i) $\xi_1(\boldsymbol{a}) = \boldsymbol{a}$, $\xi_1(\boldsymbol{b}) = \boldsymbol{a}$. By Lemma 5.4, $\overline{\xi_1}(K)$ is the regular language of even-length $\boldsymbol{a}$-strings.

(ii) $\xi_2(\boldsymbol{a}) = \lambda$, $\xi_2(\boldsymbol{b}) = \lambda$. Then $\overline{\xi_2}(K) = \{\lambda\} \in \mathfrak{D}_\Sigma$.

(iii) $\xi_3(\boldsymbol{a}) = \boldsymbol{a}$, $\xi_3(\boldsymbol{b}) = \lambda$. For $L = \{\boldsymbol{a}^n \boldsymbol{b}^n \mid n \geq 0\} \in \mathfrak{N}_\Sigma$, we get $\overline{\xi_3}(L) = \{\boldsymbol{a}^n \mid n \geq 0\} \in \mathfrak{D}_\Sigma$.

(b) **Three homomorphisms under which $\mathfrak{N}_\Sigma$ is closed:**

(i) The identity map: $\psi_1(\boldsymbol{a}) = \boldsymbol{a}$, $\psi_1(\boldsymbol{b}) = \boldsymbol{b}$.

(ii) The swap map: $\psi_2(\boldsymbol{a}) = \boldsymbol{b}$, $\psi_2(\boldsymbol{b}) = \boldsymbol{a}$.

(iii) The doubling map: $\psi_3(\boldsymbol{a}) = \boldsymbol{aa}$, $\psi_3(\boldsymbol{b}) = \boldsymbol{bb}$.

For ψ_1 and ψ_2, if $\overline{\psi_i}(L)$ were FAD then applying the inverse letter permutation would make L FAD. For ψ_3, the map is injective, so if $\overline{\psi_3}(L)$ were FAD then $L = \overline{\psi_3^{-1}}(\overline{\psi_3}(L))$ would be FAD by Theorem 5.10.

5.69. Over $\Sigma = \{\boldsymbol{a}\}$ there is only one homomorphism under which $\mathfrak{N}_\Sigma$ is obviously not closed, namely the erasing homomorphism $\psi(\boldsymbol{a}) = \lambda$. For any non-FAD unary language L, $\overline{\psi}(L) = \{\lambda\} \in \mathfrak{D}_\Sigma$.

For every other unary homomorphism, $\psi(\boldsymbol{a}) = \boldsymbol{a}^k$ with $k \geq 1$. Such a homomorphism is injective, so if $\overline{\psi}(L)$ were FAD, then

$$L = \overline{\psi^{-1}}(\overline{\psi}(L))$$

would be FAD by Theorem 5.10. Therefore $\mathfrak{N}_\Sigma$ is closed under every nonerasing unary homomorphism. So the answer is **no**: there are not two different unary homomorphisms under which $\mathfrak{N}_\Sigma$ is not closed.

5.70. (a) **Prove $\overline{\delta^\psi}(s,x) = \overline{\delta}(s,\overline{\psi}(x))$.**

Proof. By induction on $|x|$.
Base: $\overline{\delta^\psi}(s,\lambda) = s = \overline{\delta}(s,\lambda) = \overline{\delta}(s,\overline{\psi}(\lambda))$ (since $\psi(\lambda)=\lambda$).
Step: $\overline{\delta^\psi}(s,x\boldsymbol{b}) = \delta^\psi(\overline{\delta^\psi}(s,x),\boldsymbol{b}) = \overline{\delta}(\overline{\delta}(s,\overline{\psi}(x)),\psi(\boldsymbol{b})) = \overline{\delta}(s,\overline{\psi}(x)\psi(\boldsymbol{b})) = \overline{\delta}(s,\overline{\psi}(x\boldsymbol{b}))$. □ □

(b) **Proof of Theorem 5.10:** $y \in L(A^\psi)$ iff $\overline{\delta^\psi}(s_0,y) \in F$ iff $\overline{\delta}(s_0,\psi(y)) \in F$ iff $\psi(y) \in L$ iff $y \in \psi^{-1}(L)$. □

5.71. (a) **Apparent contradiction resolved.** $\xi(L)$ is the set of even-length $\boldsymbol{a}$-strings; $\xi^{-1}(\xi(L))$ is the set of all strings x with $\xi(x) \in \xi(L)$, not L itself. In general, $\xi^{-1}(\xi(L)) \supseteq L$ and can be much larger; it is not necessarily L. The inverse image lands in $\mathfrak{D}_\Sigma$ by Theorem 5.10, but it's $\xi^{-1}(\xi(L))$, not L.

(b) **Example where $\overline{\psi}(\psi^{-1}(L)) \neq L$:** Take $\psi(\boldsymbol{a}) = \boldsymbol{aa}$, $L = \{\boldsymbol{a}\}$ (singleton). Then $\psi^{-1}(L) = \emptyset$ (no word maps to $\boldsymbol{a}$ under ψ, since ψ always produces even-length strings), and $\overline{\psi}(\emptyset) = \emptyset \neq L$.

(c) **Example where $\psi^{-1}(\overline{\psi}(L)) \neq L$:** Same: $\psi^{-1}(\overline{\psi}(\{\boldsymbol{a}\})) = \psi^{-1}(\{\boldsymbol{a}\}) = \emptyset \neq \{\boldsymbol{a}\} = L$.

(d) **Prove $\overline{\psi}(\psi^{-1}(L)) \subseteq L$:** Let $y \in \overline{\psi}(\psi^{-1}(L))$. Then $y = \psi(x)$ for some $x \in \psi^{-1}(L)$. Since $x \in \psi^{-1}(L)$, $\psi(x) \in L$, i.e., $y \in L$. □

(e) **Prove $L \subseteq \psi^{-1}(\overline{\psi}(L))$:** Let $x \in L$. Then $\psi(x) \in \overline{\psi}(L)$ (since $\psi(x)$ is the image of some element of L, namely x itself). Hence $x \in \psi^{-1}(\overline{\psi}(L))$. □

5.72. **$\mathfrak{D}_\Sigma$ is closed under L^e (last halves of even-length words in L).**

$L^e = \{x \mid \exists y \in \Sigma^* \ni yx \in L \wedge |x| = |y|\}$.

Proof. Observe that

$$x \in L^e \iff \exists y \in \Sigma^* \ (yx \in L \ \wedge \ |y| = |x|) \iff x^r \in (L^r)^b.$$

Indeed, $yx \in L$ with $|y| = |x|$ iff $x^r y^r \in L^r$ with $|x^r| = |y^r|$, which says exactly that x^r is a first half of some word in L^r.

Therefore

$$L^e = \left((L^r)^b\right)^r.$$

Exercise 5.20 shows that $\mathfrak{D}_\Sigma$ is closed under reversal, and Theorem 5.11 shows that it is closed under b. Hence $\mathfrak{D}_\Sigma$ is closed under e. □

5.73. **Show that redefining final states to merely "midway" states is incorrect.**

Example 5.25 gives a concrete counterexample. For the highlighted state q,

$$I(A,q) = \{(\boldsymbol{abc})^n \mid n \geq 1\} \qquad \text{and} \qquad T(A,q) = \{\boldsymbol{a}^{2n} \mid n \geq 1\}.$$

So a word can use q as a midpoint only when its first half has a length that occurs in both sets, namely a length divisible by 6. The proof of Theorem 5.11 therefore keeps only

$$I(A,q) \cap \overline{\psi^{-1}}(\overline{\psi}(I(A,q)) \cap \overline{\psi}(T(A,q))) = \{(\boldsymbol{abc})^{2n} \mid n \geq 1\}.$$

If we simply declared q to be a final state, the resulting automaton would accept all of $I(A,q)$, including $\boldsymbol{abc}$. But $\boldsymbol{abc}$ is not a first half of any word in L, because after reaching q there is no continuation of length 3 to a final state. Hence "make the midway states final" accepts too many words and is incorrect.

5.74. **Flaw in the argument that K is not FAD because M is not FAD.**

Theorem 5.9 states that $\mathfrak{D}_\Sigma$ is closed under homomorphism: if L is FAD and ψ is a homomorphism, then $\psi(L)$ is FAD. This does *not* mean that if L is not FAD, then $\psi(L)$ is not FAD. A homomorphism can collapse the structure that made M non-FAD, yielding a simpler FAD image $K = \psi(M)$. Hence the fact that $M \notin \mathfrak{D}_\Sigma$ says nothing by itself about whether $K = \psi(M)$ is FAD.

5.75. **Prove Theorem 5.9 ($\mathfrak{D}_\Sigma$ closed under homomorphism).**

Proof. Let $A = \langle \Sigma, S, s_0, \delta, F\rangle$ be a DFA for L, and let $\psi : \Sigma \to \Sigma^*$ be a homomorphism. For each transition $s \xrightarrow{\boldsymbol{a}} t$ of A, replace that edge by a path labeled $\psi(\boldsymbol{a})$. If $\psi(\boldsymbol{a}) = \lambda$, use a λ-transition from s to t; if $\psi(\boldsymbol{a}) = \boldsymbol{b}_1 \cdots \boldsymbol{b}_k$ with $k \geq 1$, insert $k-1$ new states so that the new path reads $\boldsymbol{b}_1, \boldsymbol{b}_2, \ldots, \boldsymbol{b}_k$.

The resulting finite automaton accepts exactly the homomorphic images of words in L, namely $\overline{\psi}(L)$. Hence $\overline{\psi}(L) \in \mathfrak{D}_\Sigma$, proving Theorem 5.9. □

5.76. **Homomorphism capitalizing lowercase ASCII letters.**

Define $\psi : \text{ASCII}^* \to \text{ASCII}^*$ by $\psi(c) = \begin{cases} \text{uppercase}(c) & \text{if } c \in \{a, \ldots, z\} \\ c & \text{otherwise} \end{cases}$ (extended letter-by-letter to words). This is a homomorphism since $\psi(xy) = \psi(x)\psi(y)$ follows from the letter-by-letter definition.

5.77. (a) **Prove $L(A') = L_1 / L_2$.**

$L_1/L_2 = \{x \mid \exists y \in L_2 \ni xy \in L_1\}$. $F' = \{t \in S_1 \mid \exists y \in \Sigma^* \ni y \in L_2 \wedge \overline{\delta}_1(t, y) \in F_1\}$.

Proof. $x \in L(A')$ iff $\overline{\delta}_1(s_{0_1}, x) \in F'$ iff $\exists y \in L_2 \ni \overline{\delta}_1(\overline{\delta}_1(s_{0_1}, x), y) \in F_1$ iff $\exists y \in L_2 \ni \overline{\delta}_1(s_{0_1}, xy) \in F_1$ iff $\exists y \in L_2 \ni xy \in L_1$ iff $x \in L_1/L_2$. □ □

(b) **Algorithm to compute F'.**

For each state $t \in S_1$, determine whether $\exists y \in L_2 \ni \overline{\delta}_1(t, y) \in F_1$:

1. Build the product machine $A_1 \times A_2$: state set $S_1 \times S_2$, start at $\langle t, s_{0_2}\rangle$ for each t.
2. In this product, run A_2 on the input and simultaneously advance A_1.
3. State $t \in F'$ iff some state $\langle f_1, f_2\rangle$ with $f_1 \in F_1$ and $f_2 \in F_2$ is reachable from $\langle t, s_{0_2}\rangle$ in the product.

This is a reachability computation in a finite graph, computable in $O(|S_1| \cdot |S_2|)$ time.

5.78. **Extend DFA over Σ_1 to handle $\Sigma_1 \cup \Sigma_2$.**

(a) Define $A' = \langle \Sigma_1 \cup \Sigma_2, S \cup \{d\}, s_0, \delta', F\rangle$ where d is a new dead state and $\delta'(s, \boldsymbol{a}) = \begin{cases} \delta(s, \boldsymbol{a}) & \boldsymbol{a} \in \Sigma_1 \\ d & \boldsymbol{a} \in \Sigma_2 \setminus \Sigma_1 \end{cases}$

and $\delta'(d, \boldsymbol{a}) = d$ for all $\boldsymbol{a}$. Final states: F (unchanged).

(b) **Proof.** If $w \in (\Sigma_1 \cup \Sigma_2)^*$ and w contains any letter from $\Sigma_2 \setminus \Sigma_1$, then A' goes to d and stays there, rejecting w. If $w \in \Sigma_1^*$, then A' behaves identically to A. Hence $L(A') = L(A)$ (viewed as a subset of $(\Sigma_1 \cup \Sigma_2)^*$). □

5.79. **If $\mathscr{S}$ is closed under union, concatenation, Kleene closure, and is infinite, must every language in $\mathscr{S}$ be FAD?**

No. The collection I_Σ of all infinite languages is infinite, and Exercise 5.4 shows that it is closed under union, concatenation, and Kleene closure. But I_Σ contains languages in $\mathfrak{N}_\Sigma$, so not every language in such a collection must be FAD.

5.80. **If $\mathscr{S}$ is closed under union, concatenation, Kleene closure, and is finite, must every language be FAD?**

No. Let

$$K = \{x \in \{\boldsymbol{a}, \boldsymbol{b}\}^* \mid |x|_{\boldsymbol{a}} = |x|_{\boldsymbol{b}}\},$$

which is in $\mathfrak{N}_\Sigma$ by Lemma 5.4, and define

$$\mathscr{S} = \{\emptyset, \{\lambda\}, K\}.$$

This collection is finite. It is closed under union because $\{\lambda\} \subseteq K$, so every union is still one of the three listed languages. It is closed under concatenation because

$$\emptyset \cdot L = \emptyset, \qquad \{\lambda\} \cdot L = L \cdot \{\lambda\} = L, \qquad K \cdot K = K$$

(the numbers of $\boldsymbol{a}$s and $\boldsymbol{b}$s add, and $\lambda \in K$). It is also closed under Kleene closure:

$$\emptyset^* = \{\lambda\}, \qquad \{\lambda\}^* = \{\lambda\}, \qquad K^* = K.$$

So a finite collection with these closure properties need not consist entirely of FAD languages.

5.81. **If $u \circ u = \mathbf{id}$ and $\mathfrak{D}_\Sigma$ is closed under u, then $\mathfrak{N}_\Sigma$ is closed under u.**

Proof. Let $L \in \mathfrak{N}_\Sigma$. Suppose, toward a contradiction, that $u(L) \in \mathfrak{D}_\Sigma$. Because $\mathfrak{D}_\Sigma$ is closed under u, applying u again gives

$$u(u(L)) \in \mathfrak{D}_\Sigma.$$

But $u \circ u = \mathrm{id}$, so $u(u(L)) = L$. Hence $L \in \mathfrak{D}_\Sigma$, contradicting $L \in \mathfrak{N}_\Sigma$. Therefore $u(L) \in \mathfrak{N}_\Sigma$ whenever $L \in \mathfrak{N}_\Sigma$, so $\mathfrak{N}_\Sigma$ is closed under u. □

Chapter 6

Regular Expressions — Solutions

6.1. Even-length words over $\{\boldsymbol{a}, \boldsymbol{b}\}$:

$$((\boldsymbol{a} \cup \boldsymbol{b})(\boldsymbol{a} \cup \boldsymbol{b}))^*$$

6.2. Words of length ≥ 2 over $\{\boldsymbol{a}, \boldsymbol{b}\}$:

$$(\boldsymbol{a} \cup \boldsymbol{b})(\boldsymbol{a} \cup \boldsymbol{b})(\boldsymbol{a} \cup \boldsymbol{b})^*$$

6.3. Words with $|x|_{\boldsymbol{a}} = |x|_{\boldsymbol{b}}$ over $\{\boldsymbol{a}, \boldsymbol{b}\}$: This language is *not* regular (not FAD). By the pumping lemma, if it were FAD with n states, take $w = \boldsymbol{a}^n \boldsymbol{b}^n$; any pump within the first n characters (all $\boldsymbol{a}$s) would unbalance the counts. Hence no regular expression exists.

6.4. Odd-length words over $\{\boldsymbol{a}, \boldsymbol{b}, \boldsymbol{c}\}$ not ending in $\boldsymbol{b}$:

A word of odd length not ending in $\boldsymbol{b}$ has an even-length prefix (possibly empty) followed by a non-$\boldsymbol{b}$ letter:

$$((\boldsymbol{a} \cup \boldsymbol{b} \cup \boldsymbol{c})(\boldsymbol{a} \cup \boldsymbol{b} \cup \boldsymbol{c}))^*(\boldsymbol{a} \cup \boldsymbol{c})$$

(When the Kleene star produces k pairs, the total length is $2k+1$, which is odd.)

6.5. Words over $\{\boldsymbol{a}, \boldsymbol{b}, \boldsymbol{c}\}$ not containing $\boldsymbol{cc}$: Words with no two consecutive $\boldsymbol{c}$s are those where every $\boldsymbol{c}$ is followed by a non-$\boldsymbol{c}$ (or is at the end). Let $N = \boldsymbol{a} \cup \boldsymbol{b}$ (non-$\boldsymbol{c}$ letters).

$$(N \cup \boldsymbol{c}N)^*(N \cup \boldsymbol{c} \cup \epsilon)$$

More precisely:

$$(N \cup \boldsymbol{c}N)^*(\epsilon \cup \boldsymbol{c})$$

6.6. Words over $\{\boldsymbol{a}, \boldsymbol{b}, \boldsymbol{c}\}$ containing $\boldsymbol{cc}$:

$$(\boldsymbol{a} \cup \boldsymbol{b} \cup \boldsymbol{c})^* \boldsymbol{cc} (\boldsymbol{a} \cup \boldsymbol{b} \cup \boldsymbol{c})^*$$

6.7. Words over $\{\boldsymbol{a}, \boldsymbol{b}, \boldsymbol{c}\}$ with no $\boldsymbol{c}$s:

$$(\boldsymbol{a} \cup \boldsymbol{b})^*$$

6.8. Let $\Sigma = \{\mathbf{0}, \mathbf{1}\}$.

(a) $L_1 = \{x \mid |x| \bmod 3 = 2\}$: words whose length is congruent to 2 modulo 3:

$$(\mathbf{0} \cup \mathbf{1})(\mathbf{0} \cup \mathbf{1})((\mathbf{0} \cup \mathbf{1})(\mathbf{0} \cup \mathbf{1})(\mathbf{0} \cup \mathbf{1}))^*$$

(b) $L_2 = \Sigma^* \setminus \{w \mid w \text{ ends in } \mathbf{1}\}$: words that do *not* end in $\mathbf{1}$, i.e., words that end in $\mathbf{0}$ or equal λ:

$$\epsilon \cup (\mathbf{0} \cup \mathbf{1})^*\mathbf{0}$$

(c) $L_3 = \{y \mid |y|_{\mathbf{0}} > |y|_{\mathbf{1}}\}$: words with more $\mathbf{0}$s than $\mathbf{1}$s. This language is *not* regular. By the pumping lemma, consider $\mathbf{0}^{n+1}\mathbf{1}^n$; pumping within the $\mathbf{0}$ block preserves the property, but the languages of this form require counting, which finite automata cannot do. (No regular expression exists.)

6.9. Let $\Sigma = \{\boldsymbol{a}, \boldsymbol{b}, \boldsymbol{c}\}$. Let $\Sigma^* = (\boldsymbol{a} \cup \boldsymbol{b} \cup \boldsymbol{c})^*$.

(a) $L_1 = \{x \mid |x|_{\boldsymbol{a}} \text{ odd} \wedge |x|_{\boldsymbol{b}} \text{ even}\}$:
Track the parity of $\boldsymbol{a}$s and $\boldsymbol{b}$s jointly. A word is in L_1 iff the number of $\boldsymbol{a}$s is odd and the number of $\boldsymbol{b}$s is even. This is a product of two conditions on counts modulo 2; the minimal DFA has 4 states. The corresponding regular expression (derived from the state equations) is:

$$\boldsymbol{c}^*\boldsymbol{a}(\boldsymbol{c} \cup \boldsymbol{bb})^*(\epsilon \cup \boldsymbol{b}\boldsymbol{c}^*\boldsymbol{a}(\boldsymbol{c} \cup \boldsymbol{bb})^*)^* \quad \text{(one valid form)}$$

(Words with an odd number of $\boldsymbol{a}$s interleaved with an even number of $\boldsymbol{b}$s; $\boldsymbol{c}$s can appear anywhere.)

(b) $L_2 = \{y \mid |y|_{\boldsymbol{c}} \text{ even} \vee |y|_{\boldsymbol{b}} \text{ odd}\}$: The complement is $\{y \mid |y|_{\boldsymbol{c}} \text{ odd} \wedge |y|_{\boldsymbol{b}} \text{ even}\}$, which is regular, so L_2 is regular. A regular expression can be derived similarly.

(c) $L_3 = \{z \mid |z|_{\boldsymbol{a}} \text{ even}\}$:

$$(\boldsymbol{b} \cup \boldsymbol{c})^*(\boldsymbol{aa}(\boldsymbol{b} \cup \boldsymbol{c})^* \cup \epsilon)^* = ((\boldsymbol{b} \cup \boldsymbol{c})^*\boldsymbol{a}(\boldsymbol{b} \cup \boldsymbol{c})^*\boldsymbol{a})^*(\boldsymbol{b} \cup \boldsymbol{c})^*$$

(d) $L_4 = \{z \mid |z|_{\boldsymbol{c}} \text{ prime}\}$: Not regular. The set of primes is not eventually periodic, so the language $\{c^p \mid p \text{ prime}\} \subseteq L_4$ restricted to $\Sigma = \{\boldsymbol{c}\}$ is not FAD (pumping lemma). Hence L_4 has no regular expression.

(e) $L_5 = \{x \mid \boldsymbol{abc} \text{ is a substring}\}$:

$$(\boldsymbol{a} \cup \boldsymbol{b} \cup \boldsymbol{c})^*\boldsymbol{abc}(\boldsymbol{a} \cup \boldsymbol{b} \cup \boldsymbol{c})^*$$

(f) $L_6 = \{x \mid \boldsymbol{acaba} \text{ is a substring}\}$:

$$(\boldsymbol{a} \cup \boldsymbol{b} \cup \boldsymbol{c})^*\boldsymbol{acaba}(\boldsymbol{a} \cup \boldsymbol{b} \cup \boldsymbol{c})^*$$

(g) $L_7 = \{x \mid |x|_{\boldsymbol{a}} \equiv 0 \pmod 3\}$:

$$(\boldsymbol{b} \cup \boldsymbol{c})^*(\boldsymbol{a}(\boldsymbol{b} \cup \boldsymbol{c})^*\boldsymbol{a}(\boldsymbol{b} \cup \boldsymbol{c})^*\boldsymbol{a}(\boldsymbol{b} \cup \boldsymbol{c})^*)^*$$

6.10. $\Psi = \{x \mid x \text{ begins with } \boldsymbol{d} \vee x \text{ contains } \boldsymbol{bb}\}$ over $\{\boldsymbol{a}, \boldsymbol{b}, \boldsymbol{d}\}$:

$$\boldsymbol{d}(\boldsymbol{a} \cup \boldsymbol{b} \cup \boldsymbol{d})^* \cup (\boldsymbol{a} \cup \boldsymbol{b} \cup \boldsymbol{d})^*\boldsymbol{bb}(\boldsymbol{a} \cup \boldsymbol{b} \cup \boldsymbol{d})^*$$

6.11. $\Phi = \{x \mid$ every $\boldsymbol{b}$ in x is immediately followed by $\boldsymbol{c}\}$ over $\{\boldsymbol{a},\boldsymbol{b},\boldsymbol{c}\}$: Words with no isolated $\boldsymbol{b}$ (every $\boldsymbol{b}$ must be followed immediately by $\boldsymbol{c}$). Let $S = \boldsymbol{a} \cup \boldsymbol{c} \cup \boldsymbol{bc}$ (safe building blocks):

$$(\boldsymbol{a} \cup \boldsymbol{c} \cup \boldsymbol{bc})^*$$

6.12. Numbers divisible by 3 over $\{0,\ldots,9\}$:

A number is divisible by 3 iff the sum of its digits is divisible by 3. This is recognized by a 3-state DFA tracking the running digit sum modulo 3. The corresponding regular expression (from the state equations) is complex but finite; one approach:

Let $D_k = \{$digits whose value$\equiv k \pmod 3\}$: $D_0 = \{0,3,6,9\}$, $D_1 = \{1,4,7\}$, $D_2 = \{2,5,8\}$. The language Γ is accepted by a DFA with states $\{q_0, q_1, q_2\}$ where $q_i =$ "running sum $\equiv i \pmod 3$", q_0 is the (only) final state. The state equations:

$$\begin{aligned} X_0 &= \epsilon \cup D_0 X_0 \cup D_1 X_2 \cup D_2 X_1 \\ X_1 &= D_1 X_0 \cup D_2 X_2 \cup D_0 X_1 \\ X_2 &= D_2 X_0 \cup D_0 X_2 \cup D_1 X_1 \end{aligned}$$

Solving (by Theorem 6.2): $X_0 = D_0^*(D_1X_2 + D_2X_1)^*$... the full solution is a regular expression in D_0, D_1, D_2; the simplest form is $\Gamma = (D_0 \cup D_1 D_2^* D_1 \cup D_2 D_1^* D_2)^*$ (an approximation; the exact expression requires the full Kleene computation).

6.13. $K = \{x \mid x$ is divisible by 5$\}$ over decimal digits:

A decimal number is divisible by 5 iff it ends in **0** or **5**.

$$K = \epsilon \cup (\mathbf{0} \cup \mathbf{1} \cup \cdots \cup \mathbf{9})^*(\mathbf{0} \cup \mathbf{5})$$

(Including λ which represents 0, which is divisible by 5.)

6.14. **Build NDFA for $\boldsymbol{b} \cup \boldsymbol{a}^*\boldsymbol{c}$ using exact constructs of Chapter 5.**

Step 1: $\boldsymbol{a}^*$ via Theorem 5.5 (Kleene closure of $\{\boldsymbol{a}\}$). Step 2: $\boldsymbol{a}^*\boldsymbol{c}$ via Theorem 5.4 (concatenation of $\boldsymbol{a}^*$ and $\{\boldsymbol{c}\}$). Step 3: $\boldsymbol{b} \cup \boldsymbol{a}^*\boldsymbol{c}$ via Theorem 5.2 (union of $\{\boldsymbol{b}\}$ and $\boldsymbol{a}^*\boldsymbol{c}$).

The NDFA:

- Machine for $\{\boldsymbol{a}\}$: one start state p_0, one final state p_1, $\delta(p_0, \boldsymbol{a}) = \{p_1\}$.
- Machine for $\boldsymbol{a}^*$ (Theorem 5.5): add start state q_0 (final); λ-transition $q_0 \to p_0$; λ-transition $p_1 \to q_0$.
- Machine for $\{\boldsymbol{c}\}$: start r_0, final r_1, $\delta(r_0, \boldsymbol{c}) = \{r_1\}$.
- Machine for $\boldsymbol{a}^*\boldsymbol{c}$ (Theorem 5.4): λ-transition $q_0 \to r_0$; final states $\{r_1\}$.
- Machine for $\{\boldsymbol{b}\}$: start s_0, final s_1, $\delta(s_0, \boldsymbol{b}) = \{s_1\}$.
- Union (Theorem 5.2): start state set $\{q_0, s_0\}$; final states $\{r_1, s_1\}$.

6.15. **Inequalities in Lemma 6.1:**

(a) $R_1 \cup \epsilon \neq R_1$: Let $R_1 = \boldsymbol{a}$. Then $R_1 \cup \epsilon = \{\lambda, \boldsymbol{a}\} \neq \{\boldsymbol{a}\} = R_1$.

(b) $R_1 \cdot R_2 \neq R_2 \cdot R_1$: Let $R_1 = \boldsymbol{a}$, $R_2 = \boldsymbol{b}$. $\boldsymbol{ab} \neq \boldsymbol{ba}$.

(c) $R_1 \cdot R_1 \neq R_1$: Let $R_1 = \boldsymbol{a}$. $\boldsymbol{aa} \neq \boldsymbol{a}$.

(d) $R_1 \cup (R_2 \cdot R_3) \neq (R_1 \cup R_2) \cdot (R_1 \cup R_3)$: Let $R_1 = \boldsymbol{a}$, $R_2 = \boldsymbol{b}$, $R_3 = \boldsymbol{c}$. LHS: $\{\boldsymbol{a}, \boldsymbol{bc}\}$. RHS: $\{\boldsymbol{a}, \boldsymbol{b}, \boldsymbol{c}\} \cdot \{\boldsymbol{a}, \boldsymbol{b}, \boldsymbol{c}\} = \{\boldsymbol{aa}, \boldsymbol{ab}, \boldsymbol{ac}, \boldsymbol{ba}, \boldsymbol{bb}, \boldsymbol{bc}, \boldsymbol{ca}, \boldsymbol{cb}, \boldsymbol{cc}\} \cup \ldots$. Not equal.

(e) $(R_1 \cdot R_2)^* \neq (R_1^* \cdot R_2^*)^*$: Let $R_1 = \boldsymbol{a}$, $R_2 = \boldsymbol{b}$. $(ab)^* = \{\lambda, \boldsymbol{ab}, \boldsymbol{abab}, \ldots\}$. $(a^* b^*)^* = (\boldsymbol{a} \cup \boldsymbol{b})^*$. Not equal.

(f) $(R_1 \cdot R_2)^* \neq (R_1^* \cup R_2^*)^*$: Same example. $(R_1^* \cup R_2^*)^* = (a^* \cup b^*)^* = (a \cup b)^*$. Not equal to $(\boldsymbol{ab})^*$.

Examples where equality holds:

(g) $R_1 \cup \epsilon = R_1$: Let $R_1 = \epsilon$. $\epsilon \cup \epsilon = \epsilon = R_1$.

(h) $R_1 \cdot R_2 = R_2 \cdot R_1$: Let $R_1 = R_2 = \boldsymbol{a}$. Both equal $\boldsymbol{aa}$. Or $R_1 = \boldsymbol{ab}$, $R_2 = \boldsymbol{ab}$.

(i) $R_1 \cdot R_1 = R_1$: Let $R_1 = \Sigma^*$. $\Sigma^* \cdot \Sigma^* = \Sigma^*$.

(j) $R_1 \cup (R_2 \cdot R_3) = (R_1 \cup R_2) \cdot (R_1 \cup R_3)$: Let $R_1 = \Sigma^*$, R_2, R_3 arbitrary. Both sides equal Σ^*.

(k) $(R_1 \cdot R_2)^* = (R_1^* \cdot R_2^*)^*$: Let $R_1 = \boldsymbol{a}^*$, $R_2 = \epsilon$. Both equal $(\boldsymbol{a}^*)^* = \boldsymbol{a}^*$.

(l) $(R_1 \cdot R_2)^* = (R_1^* \cup R_2^*)^*$: Let $R_1 = \boldsymbol{a}$, $R_2 = \boldsymbol{b}$... no, not equal. Let $R_1 = \epsilon$, $R_2 = \boldsymbol{a}$. $(R_1 R_2)^* = \boldsymbol{a}^*$. $(R_1^* \cup R_2^*)^* = (\epsilon \cup \boldsymbol{a}^*)^* = \boldsymbol{a}^*$. Equal.

6.16. **Prove the equalities of Lemma 6.1.**

Key equalities (Lemma 6.1 typically states algebraic laws for regular sets):

(i) $R \cup \emptyset = R$: $L \cup \emptyset = L$ for any language.

(ii) $R \cup R = R$: idempotency; $L \cup L = L$.

(iii) $R \cdot \emptyset = \emptyset$: concatenation with empty language is empty.

(iv) $R \cdot \epsilon = R$: $L \cdot \{\lambda\} = L$.

(v) $(R^*)^* = R^*$: L^* is already closed under concatenation and taking closures.

(vi) $R^* \cdot R^* = R^*$: $L^* \cdot L^* = L^*$.

(vii) $\emptyset^* = \epsilon$: $\emptyset^* = \{\lambda\}$.

(viii) $(R_1 \cup R_2) \cdot R_3 = (R_1 \cdot R_3) \cup (R_2 \cdot R_3)$: distributivity.

Each of these follows directly from the set-theoretic definitions of union, concatenation, and Kleene closure. For example, distributivity: $w \in (L_1 \cup L_2) \cdot L_3$ iff $w = xy$ with $x \in L_1 \cup L_2$ and $y \in L_3$ iff $(x \in L_1 \wedge y \in L_3) \vee (x \in L_2 \wedge y \in L_3)$ iff $w \in L_1 L_3 \cup L_2 L_3$. □

6.17. (a) **Non-uniqueness of $A^* E$ when $\lambda \in A$.**

Let $A = \{\lambda, \boldsymbol{a}\} = \epsilon \cup \boldsymbol{a}$ and $E = \{\boldsymbol{b}\}$. Then $A^* E = \{\boldsymbol{a}\}^* \{\boldsymbol{b}\} = \{\boldsymbol{a}^n \boldsymbol{b} \mid n \geq 0\}$. But $X = \Sigma^* \{\boldsymbol{b}\}$ also satisfies $X = A \cdot X \cup E$ since $\lambda \in A$ means $A^* E \subseteq X$ for any superset X that is a fixed point. Specifically, check $X = \Sigma^*$: $A \cdot \Sigma^* \cup \{\boldsymbol{b}\} = \Sigma^* \cup \{\boldsymbol{b}\} = \Sigma^* = X$. So Σ^* is also a solution, but $\Sigma^* \neq A^* E$. Hence the solution is not unique.

(b) $A^* E$ **as unique solution even if $\lambda \in A$.**

Let $A = \{\lambda\}$ and $E = \{\boldsymbol{b}\}$. Then $X = AX \cup E = \{\lambda\} X \cup \{\boldsymbol{b}\} = X \cup \{\boldsymbol{b}\}$. The unique solution is $X \supseteq \{\boldsymbol{b}\}$; the smallest such X is $\{\boldsymbol{b}\} = A^* E$. (The equation $X = X \cup E$ has $X = E$ as a solution, and any superset works; if we take the smallest fixed point, it equals $E = A^* E$.)

6.18. **Solve $X_1 = (\mathbf{0} \cup \mathbf{1})X_1$, $X_2 = \epsilon \cup \mathbf{1}X_0 \cup \mathbf{0}X_1$.**

The system has equations for X_1 and X_2, with X_0 appearing free (this appears to be a typo in the problem; we solve the given system and express X_2 in terms of X_0.)

From the first equation: $X_1 = (\mathbf{0} \cup \mathbf{1})X_1$. By Theorem 6.1 (since $\lambda \notin \mathbf{0} \cup \mathbf{1}$), $X_1 = (\mathbf{0} \cup \mathbf{1})^* \cdot \emptyset = \emptyset$.

So $X_1 = \emptyset$, and $X_2 = \epsilon \cup \mathbf{1}X_0 \cup \mathbf{0} \cdot \emptyset = \epsilon \cup \mathbf{1}X_0$.

This system involves X_0 which has no defining equation, so X_2 remains in terms of X_0. The connection to Example 3.4: the equations describe the state languages of a DFA, where X_i represents strings leading from state i to acceptance.

6.19. **Solve for X_1, X_2, X_3 from the equations.**

Simplifying (noting $\emptyset \cdot X_i = \emptyset$):

$$X_1 = (\mathbf{0} \cup \mathbf{1})X_2$$
$$X_2 = \epsilon \cup \mathbf{0}X_1 \cup \mathbf{1}X_2$$
$$X_3 = (\mathbf{0} \cup \mathbf{1})X_2$$

Note $X_3 = X_1$.

(a) **Eliminate X_3, then X_2:**
Since $X_3 = X_1$, eliminate X_3. From $X_2 = \epsilon \cup \mathbf{0}X_1 \cup \mathbf{1}X_2$: by Theorem 6.1 ($\lambda \notin \{\mathbf{1}\}$): $X_2 = \mathbf{1}^*(\epsilon \cup \mathbf{0}X_1) = \mathbf{1}^* \cup \mathbf{1}^*\mathbf{0}X_1$. Substitute into $X_1 = (\mathbf{0} \cup \mathbf{1})X_2 = (\mathbf{0} \cup \mathbf{1})(\mathbf{1}^* \cup \mathbf{1}^*\mathbf{0}X_1) = (\mathbf{0} \cup \mathbf{1})\mathbf{1}^* \cup (\mathbf{0} \cup \mathbf{1})\mathbf{1}^*\mathbf{0}X_1$. By Theorem 6.1: $X_1 = ((\mathbf{0} \cup \mathbf{1})\mathbf{1}^*\mathbf{0})^*(\mathbf{0} \cup \mathbf{1})\mathbf{1}^*$. Then $X_2 = \mathbf{1}^* \cup \mathbf{1}^*\mathbf{0}((\mathbf{0} \cup \mathbf{1})\mathbf{1}^*\mathbf{0})^*(\mathbf{0} \cup \mathbf{1})\mathbf{1}^* = \mathbf{1}^*((\mathbf{0}(\mathbf{0} \cup \mathbf{1})\mathbf{1}^*)^*$ (simplified). And $X_3 = X_1$.

(b) **Eliminate X_3, then X_1:**
Same $X_3 = X_1$. From $X_1 = (\mathbf{0} \cup \mathbf{1})X_2$, substitute into X_2: $X_2 = \epsilon \cup \mathbf{0}(\mathbf{0} \cup \mathbf{1})X_2 \cup \mathbf{1}X_2 = \epsilon \cup (\mathbf{0}(\mathbf{0} \cup \mathbf{1}) \cup \mathbf{1})X_2$. Let $A = \mathbf{0}(\mathbf{0} \cup \mathbf{1}) \cup \mathbf{1} = \mathbf{00} \cup \mathbf{01} \cup \mathbf{1}$; $\lambda \notin A$. $X_2 = A^*$. Then $X_1 = (\mathbf{0} \cup \mathbf{1})A^*$ and $X_3 = X_1$.

(c) **Comparison:** Both solutions are equivalent: $(\mathbf{0} \cup \mathbf{1})\mathbf{1}^*(\mathbf{0}(\mathbf{0} \cup \mathbf{1})\mathbf{1}^*)^* = (\mathbf{0} \cup \mathbf{1})(\mathbf{00} \cup \mathbf{01} \cup \mathbf{1})^*$. The second form is more concise.

6.20. **Prove Lemma 6.2: every regular expression describes a FAD language.**

Proof. Let $P(m)$: "Every regular expression with $\leq m$ operators represents a FAD language."

Base ($m = 0$): No operators: the expression is $\emptyset$, ϵ, or a single letter $\boldsymbol{a} \in \Sigma$. These represent $\emptyset$, $\{\lambda\}$, and $\{\boldsymbol{a}\}$ respectively—all finite, hence FAD.

Inductive step: Assume $P(m)$. Any regular expression with $m+1$ operators is either $R_1 \cup R_2$, $R_1 \cdot R_2$, or R_1^*, where R_1 and R_2 each have $\leq m$ operators. By $P(m)$, $L(R_1), L(R_2) \in \mathfrak{D}_\Sigma$. Then:

- $L(R_1 \cup R_2) = L(R_1) \cup L(R_2) \in \mathfrak{D}_\Sigma$ (Theorem 5.2).
- $L(R_1 \cdot R_2) = L(R_1) \cdot L(R_2) \in \mathfrak{D}_\Sigma$ (Theorem 5.4).
- $L(R_1^*) = L(R_1)^* \in \mathfrak{D}_\Sigma$ (Theorem 5.5).

So $P(m+1)$ holds. By induction, every regular expression represents a FAD language. □ □

6.21. **All solutions to $X = X \cup \{\boldsymbol{b}\}$ over $\{\boldsymbol{a},\boldsymbol{b},\boldsymbol{c}\}^*$.**

$X = X \cup \{\boldsymbol{b}\}$ means $\{\boldsymbol{b}\} \subseteq X$ and $X = X$ (which is always true). The solutions are all languages X containing $\boldsymbol{b}$: $X = \{L \mid \boldsymbol{b} \in L\}$. Every such language is a solution. The smallest solution is $\{\boldsymbol{b}\}$.

6.22. **Prove $A^*E = E \cup A \cdot (A^*E)$ for any languages A and E.**

Proof. $A^*E = (\epsilon \cup A \cup A^2 \cup \cdots)E = \epsilon E \cup AE \cup A^2E \cup \cdots = E \cup A(E \cup AE \cup A^2E \cup \cdots) = E \cup A(A^*E)$. □ □

6.23. **Regular expression for $(\boldsymbol{ab} \cup \boldsymbol{b})^*\boldsymbol{a} \cap (\boldsymbol{ba} \cup \boldsymbol{a})^*$.**

Both are FAD; their intersection is FAD (Lemma 5.1). We use the product DFA approach.

$L_1 = (\boldsymbol{ab} \cup \boldsymbol{b})^*\boldsymbol{a}$: strings over $\{\boldsymbol{a},\boldsymbol{b}\}$ of the form $w\boldsymbol{a}$ where $w \in (\boldsymbol{ab} \cup \boldsymbol{b})^*$. $L_2 = (\boldsymbol{ba} \cup \boldsymbol{a})^*$: strings over $\{\boldsymbol{a},\boldsymbol{b}\}$ that are concatenations of $\boldsymbol{ba}$ or $\boldsymbol{a}$.

L_1 ends in $\boldsymbol{a}$. $L_2 = (\boldsymbol{ba} \cup \boldsymbol{a})^*$ consists of all strings that decompose as concatenations of $\boldsymbol{a}$ and $\boldsymbol{ba}$.

$L_2 = (\boldsymbol{ba} \cup \boldsymbol{a})^*$ means every $\boldsymbol{b}$ is immediately followed by $\boldsymbol{a}$. A word in $L_1 = (\boldsymbol{ab} \cup \boldsymbol{b})^*\boldsymbol{a}$ has prefix in $(\boldsymbol{ab} \cup \boldsymbol{b})^*$, so every $\boldsymbol{a}$ in the prefix is immediately followed by $\boldsymbol{b}$. Combining both constraints on the prefix atoms: each atom (whether $\boldsymbol{ab}$ or $\boldsymbol{b}$) must be immediately followed by something starting with $\boldsymbol{a}$ (since the atom ends in $\boldsymbol{b}$, and every $\boldsymbol{b}$ must be followed by $\boldsymbol{a}$). Consequently, $\boldsymbol{b}$-atoms can appear only once, at the start (any $\boldsymbol{b}$-atom followed by another $\boldsymbol{b}$-atom would put $\boldsymbol{b}$ before $\boldsymbol{b}$, violating L_2); and $\boldsymbol{ab}$-atoms must be followed by $\boldsymbol{ab}$-atoms or the final $\boldsymbol{a}$. The intersection is therefore:

$$L_1 \cap L_2 = (\epsilon \cup \boldsymbol{b})(\boldsymbol{ab})^*\boldsymbol{a}$$

Verification: $(\epsilon \cup \boldsymbol{b})(\boldsymbol{ab})^k\boldsymbol{a}$ gives words $\boldsymbol{a}, \boldsymbol{aba}, \boldsymbol{ababa}, \ldots$ and $\boldsymbol{ba}, \boldsymbol{baba}, \boldsymbol{bababa}, \ldots$, all of which belong to both L_1 and L_2; and any word in $L_1 \cap L_2$ has this form by the above analysis.

6.24. **Algorithm for intersection of two regular expressions.**

Given regular expressions R_1 and R_2 (each describing a FAD language):

1. Convert R_1 to an NDFA A_1 (using the constructions of Chapter 5 / Theorem 6.5).
2. Convert R_2 to an NDFA A_2 similarly.
3. Apply the subset construction to get DFAs A_1^d and A_2^d.
4. Build the product DFA $A^\cap$ (Lemma 5.1) with $F^\cap = F_1 \times F_2$.
5. Apply Theorem 6.6 (DFA to regular expression) to $A^\cap$ to produce a regular expression for $L(A_1) \cap L(A_2)$.

6.25. **Verify that $X_1 = (\boldsymbol{a} \cup \boldsymbol{bb})^*$ and $X_2 = \boldsymbol{b} \cdot (\boldsymbol{a} \cup \boldsymbol{bb})^*$ satisfy:**

$$X_1 = \epsilon \cup \boldsymbol{a} \cdot X_1 \cup \boldsymbol{b} \cdot X_2$$
$$X_2 = \boldsymbol{b} \cdot X_1$$

Verification of $X_2 = \boldsymbol{b} \cdot X_1$:

$$\boldsymbol{b} \cdot X_1 = \boldsymbol{b} \cdot (\boldsymbol{a} \cup \boldsymbol{bb})^* = X_2. \quad \checkmark$$

Verification of $X_1 = \epsilon \cup \boldsymbol{a} \cdot X_1 \cup \boldsymbol{b} \cdot X_2$:

$$\begin{aligned} \epsilon \cup \boldsymbol{a} \cdot X_1 \cup \boldsymbol{b} \cdot X_2 &= \epsilon \cup \boldsymbol{a}(\boldsymbol{a} \cup \boldsymbol{bb})^* \cup \boldsymbol{b} \cdot \boldsymbol{b}(\boldsymbol{a} \cup \boldsymbol{bb})^* \\ &= \epsilon \cup (\boldsymbol{a} \cup \boldsymbol{bb}) \cdot (\boldsymbol{a} \cup \boldsymbol{bb})^* \\ &= (\boldsymbol{a} \cup \boldsymbol{bb})^* = X_1. \quad \checkmark \end{aligned}$$

6.26. (a) $L(D)$ **for Example 6.10:** From the state equations of machine D, solve the system (as in Theorem 6.3) to get $L(D) = X_{start}$ where X_{start} is the language variable for the start state. The specific answer depends on the machine in Example 6.10.

(b) **General formula:** For a machine A with start states $s_{i_1}, \ldots, s_{i_m}$:

$$L(A) = X_{i_1} \cup X_{i_2} \cup \cdots \cup X_{i_m}$$

where each X_j is the solution to the state equation for state s_j.

6.27. **Regular expression for the complement of** $(\boldsymbol{ab} \cup \boldsymbol{b})^* \boldsymbol{a}$**.**

$\mathfrak{R}_\Sigma$ is closed under complement. Algorithm:

1. Build NDFA for $(\boldsymbol{ab} \cup \boldsymbol{b})^* \boldsymbol{a}$.
2. Convert to DFA via subset construction.
3. Complement final states.
4. Apply Theorem 6.6 to get a regular expression.

The complement of $(\boldsymbol{ab} \cup \boldsymbol{b})^* \boldsymbol{a}$ consists of all words over $\{\boldsymbol{a}, \boldsymbol{b}\}$ that do NOT end with $\boldsymbol{a}$ after a prefix in $(\boldsymbol{ab} \cup \boldsymbol{b})^*$, i.e., strings that end in $\boldsymbol{b}$ or words not of this form. The complement includes λ, all words ending in $\boldsymbol{b}$, and words with substructure violating the pattern.

A regular expression: $\lambda \cup (\boldsymbol{a} \cup \boldsymbol{b})^* \boldsymbol{b} \cup$(words not matching the pattern). The exact expression requires the product DFA complement computation.

6.28. **Algorithm to complement a regular expression.**

Given regular expression R:

1. Convert R to a DFA A (via NDFA $\rightarrow$ subset construction).
2. Complement A by swapping F and $S \setminus F$.
3. Apply Theorem 6.6 to the complemented DFA to produce a regular expression.

6.29. $\mathfrak{R}_\Sigma$ **is closed under** $E(L)$ **using regular expressions.**

$E(L) = \{z \mid \exists y \in \Sigma^+, yx \in L\}$ = suffixes with at least one character removed from the front. If $L = L(R)$, then $E(L) = \Sigma^+ \setminus$ (words not in $E(L)$). More directly: $E(L) = \{z \mid \exists \boldsymbol{a} \in \Sigma, \exists y \in \Sigma^*, \boldsymbol{a}yz \in L\}$. This is the set of strings that can follow some nonempty prefix in L. From the regular expression perspective: $E(L) = \bigcup_{\boldsymbol{a} \in \Sigma} \overline{\psi_{\boldsymbol{a}}^{-1}}(L)$ where $\psi_{\boldsymbol{a}}(z) = \boldsymbol{a}z\ldots$ this is inverse image under left-concatenation, which is regular. Alternatively, $E(L) = \text{Suf}(L) \setminus \{\lambda\}$ restricted to at least one preceding letter, which can be expressed as a regular expression by the suffix construction.

6.30. $\mathfrak{R}_\Sigma$ **is closed under** $B(L)$**.**

$B(L) = L \cdot \Sigma^*$ (words starting with a word from L). If $L = L(R)$, then $B(L) = L(R \cdot \Sigma^*) = L(R \cdot (\boldsymbol{a} \cup \boldsymbol{b} \cup \boldsymbol{c})^*)$. This is a regular expression, so $B(L) \in \mathfrak{R}_\Sigma$.

6.31. $\mathfrak{R}_\Sigma$ **is closed under** $M(L)$**.**

$M(L) = L \cdot \Sigma^+$. If $L = L(R)$, then $M(L) = L(R \cdot (\boldsymbol{a} \cup \boldsymbol{b} \cup \boldsymbol{c})^+) = L(R \cdot (\boldsymbol{a} \cup \boldsymbol{b} \cup \boldsymbol{c}) \cdot (\boldsymbol{a} \cup \boldsymbol{b} \cup \boldsymbol{c})^*)$. Regular expression, hence $M(L) \in \mathfrak{R}_\Sigma$.

6.32. (a) **No unique solution to the given system:**

The equations: $X_1 = \boldsymbol{b} \cup \epsilon \cdot X_1 \cup \boldsymbol{a} \cdot X_2 = \boldsymbol{b} \cup X_1 \cup \boldsymbol{a}X_2$. Since $X_1 \subseteq X_1 \cup \boldsymbol{b} \cup \boldsymbol{a}X_2$, the equation $X_1 = \boldsymbol{b} \cup X_1 \cup \boldsymbol{a}X_2$ says $X_1 \supseteq \boldsymbol{b} \cup \boldsymbol{a}X_2$. Any X_1 that contains $\boldsymbol{b}$ and $\boldsymbol{a}X_2$ is a solution; there are infinitely many such X_1.

$X_2 = \boldsymbol{c} \cup \emptyset \cdot X_1 \cup \epsilon \cdot X_2 = \boldsymbol{c} \cup X_2$. Similarly, any $X_2 \supseteq \{\boldsymbol{c}\}$ is a solution.

(b) **No contradiction with Theorem 6.2:** Theorem 6.2 requires that the coefficients A_{ij} (of X_j in the equation for X_i) satisfy $\lambda \notin A_{ii}$ (i.e., the diagonal coefficients cannot contain λ). Here, $\epsilon \cdot X_1$ appears in the equation for X_1, meaning $A_{11} = \epsilon = \{\lambda\}$. Since $\lambda \in A_{11}$, the hypothesis of Theorem 6.2 is violated, so the theorem does not guarantee uniqueness.

6.33. **Solve for X_0 and X_1.**

$$X_0 = \mathbf{0}^*\mathbf{1} \cup (\mathbf{10})^* X_0 \cup \mathbf{0}(\mathbf{0} \cup \mathbf{1}) X_1$$
$$X_1 = \epsilon \cup \mathbf{1}^*\mathbf{01} X_0 \cup \mathbf{0} X_1$$

From the X_1 equation (by Theorem 6.1, $\lambda \notin \{\mathbf{0}\}$): $X_1 = \mathbf{0}^*(\epsilon \cup \mathbf{1}^*\mathbf{01} X_0) = \mathbf{0}^* \cup \mathbf{0}^*\mathbf{1}^*\mathbf{01} X_0$.

Substitute into X_0: $X_0 = \mathbf{0}^*\mathbf{1} \cup (\mathbf{10})^* X_0 \cup \mathbf{0}(\mathbf{0} \cup \mathbf{1})(\mathbf{0}^* \cup \mathbf{0}^*\mathbf{1}^*\mathbf{01} X_0)$.

Let $C = \mathbf{0}(\mathbf{0} \cup \mathbf{1})\mathbf{0}^*\mathbf{1}^*\mathbf{01}$. Then: $X_0 = ((\mathbf{10})^* \cup C) X_0 \cup (\mathbf{0}^*\mathbf{1} \cup \mathbf{0}(\mathbf{0} \cup \mathbf{1})\mathbf{0}^*)$.

By Theorem 6.1 (check $\lambda \notin (\mathbf{10})^* \cup C$... $\lambda \in (\mathbf{10})^*$!). If λ appears in the coefficient of X_0, uniqueness fails. The minimal solution is: $X_0 = ((\mathbf{10})^* \cup C)^*(\mathbf{0}^*\mathbf{1} \cup \mathbf{0}(\mathbf{0} \cup \mathbf{1})\mathbf{0}^*)$.

Then back-substitute to get X_1.

6.34. (a) Words ending with $\boldsymbol{c}$ with no $\boldsymbol{aa}$, $\boldsymbol{bb}$, $\boldsymbol{cc}$ substrings:

Build the DFA with states q_0 (start), q_a (last letter $\boldsymbol{a}$), q_b (last letter $\boldsymbol{b}$), q_c (last letter $\boldsymbol{c}$, accepting), and dead state q_d; transitions $q_x \xrightarrow{x} q_d$ and $q_x \xrightarrow{y} q_y$ for $y \neq x$. Write state equations for strings reaching q_c:

$$X_a = \boldsymbol{b} X_b \cup \boldsymbol{c} X_c$$
$$X_b = \boldsymbol{a} X_a \cup \boldsymbol{c} X_c$$
$$X_c = \epsilon \cup \boldsymbol{a} X_a \cup \boldsymbol{b} X_b$$

Eliminate X_a: substituting $X_b = \boldsymbol{a}X_a \cup \boldsymbol{c}X_c$ into X_a gives $X_a = \boldsymbol{ba}X_a \cup (\boldsymbol{bc} \cup \boldsymbol{c})X_c$, so $X_a = (\boldsymbol{ba})^*(\boldsymbol{b} \cup \epsilon)\boldsymbol{c}\, X_c$. Then $X_b = \boldsymbol{a}(\boldsymbol{ba})^*(\boldsymbol{b} \cup \epsilon)\boldsymbol{c}\, X_c \cup \boldsymbol{c}\, X_c$. Substituting into X_c:

$$X_c = \epsilon \cup \big((\boldsymbol{a} \cup \boldsymbol{ba})(\boldsymbol{ba})^*(\boldsymbol{b} \cup \epsilon)\boldsymbol{c} \cup \boldsymbol{bc}\big) X_c$$

Since λ is absent from the coefficient, $X_c = A^*$ where

$$A = (\boldsymbol{a} \cup \boldsymbol{ba})(\boldsymbol{ba})^*(\boldsymbol{b} \cup \epsilon)\boldsymbol{c} \cup \boldsymbol{bc}.$$

The language is $L = \boldsymbol{a}\, X_a \cup \boldsymbol{b}\, X_b \cup \boldsymbol{c}\, X_c = (\boldsymbol{a}(\boldsymbol{ba})^*(\boldsymbol{b} \cup \epsilon)\boldsymbol{c} \cup \boldsymbol{b}(\boldsymbol{a}(\boldsymbol{ba})^*(\boldsymbol{b} \cup \epsilon)\boldsymbol{c} \cup \boldsymbol{c}) \cup \boldsymbol{c})\, A^*$.

(b) Words beginning with $\boldsymbol{c}$ with no $\boldsymbol{aa}$, $\boldsymbol{bb}$, $\boldsymbol{cc}$ substrings: Similar constraint; the word starts with $\boldsymbol{c}$ and the next letter (if any) must not be $\boldsymbol{c}$:

$$\boldsymbol{c}((\boldsymbol{a} \cup \boldsymbol{b})(\boldsymbol{a} \cup \boldsymbol{b} \cup \boldsymbol{c}))^*$$

(where consecutive letters in the $(\boldsymbol{a} \cup \boldsymbol{b} \cup \boldsymbol{c})$ part still avoid doubles; this needs refinement.)

6.35. (a) Words with at most two $\boldsymbol{c}$s:

$$(\boldsymbol{a}\cup\boldsymbol{b})^*\cup(\boldsymbol{a}\cup\boldsymbol{b})^*\boldsymbol{c}(\boldsymbol{a}\cup\boldsymbol{b})^*\cup(\boldsymbol{a}\cup\boldsymbol{b})^*\boldsymbol{c}(\boldsymbol{a}\cup\boldsymbol{b})^*\boldsymbol{c}(\boldsymbol{a}\cup\boldsymbol{b})^*$$

(b) Words that do *not* have exactly one $\boldsymbol{c}$ (i.e., zero or ≥ 2 $\boldsymbol{c}$s):

$$(\boldsymbol{a}\cup\boldsymbol{b})^*\cup(\boldsymbol{a}\cup\boldsymbol{b})^*\boldsymbol{c}(\boldsymbol{a}\cup\boldsymbol{b})^*\boldsymbol{c}(\boldsymbol{a}\cup\boldsymbol{b}\cup\boldsymbol{c})^*$$

6.36. **Reversal of regular expressions.**

(a) Equivalences:

- $(R_1\cup R_2)^r = R_1^r\cup R_2^r$
- $(R_1\cdot R_2)^r = R_2^r\cdot R_1^r$
- $(R^*)^r = (R^r)^*$
- $\emptyset^r = \emptyset$
- $\epsilon^r = \epsilon$
- $\boldsymbol{a}^r = \boldsymbol{a}$ for each $\boldsymbol{a}\in\Sigma$

(b) To generate a regular expression for L^r from R: apply the reversal operator recursively to R using the rules above.

(c) **Proof by induction on the number of operators m in R:**

Base ($m=0$): $R\in\{\emptyset,\epsilon,\boldsymbol{a}\}$. $L(R)^r = L(R^r)$ by direct check.

Step: Assume the result for all expressions with $< m$ operators. If $R = R_1\cup R_2$: $(L(R_1)\cup L(R_2))^r = L(R_1)^r\cup L(R_2)^r = L(R_1^r)\cup L(R_2^r) = L(R_1^r\cup R_2^r) = L(R^r)$. Similarly for $\cdot$ and $*$. □

(d) Since every FAD language has a regular expression (Theorem 6.6), and the reversal of any regular expression (via the rules above) is also a regular expression, $\mathfrak{R}_\Sigma$ is closed under r.

6.37. **Complete details of Theorem 6.4.**

Theorem 6.4 establishes that every FAD language has a regular expression (one direction of Kleene's theorem). The proof uses the state-equation approach: for each state s_i of a DFA, define X_i = set of strings leading from s_i to acceptance. The system of linear equations $X_i = \bigcup_{\boldsymbol{a}\in\Sigma}\boldsymbol{a}\cdot X_{\delta(s_i,\boldsymbol{a})}\cup(\epsilon$ if $s_i\in F)$ is solved using Theorem 6.2 (Arden's lemma), yielding a regular expression for each X_i. The language of A is X_{s_0}. Full details: apply Theorem 6.2 repeatedly by variable elimination; the resulting expressions are regular by Lemma 6.2.

6.38. (a) Words with no $\boldsymbol{b}$ immediately preceded by $\boldsymbol{a}$ over $\{\boldsymbol{a},\boldsymbol{b},\boldsymbol{c}\}$:

Condition: $\boldsymbol{ab}$ never occurs as a substring. Let $S=\{\boldsymbol{a},\boldsymbol{b},\boldsymbol{c}\}$ and avoid $\boldsymbol{ab}$. Words built from $\boldsymbol{b}$s after non-$\boldsymbol{a}$s:

$$(\boldsymbol{c}\cup\boldsymbol{b})^*(\boldsymbol{a}(\boldsymbol{a}\cup\boldsymbol{c})^*(\boldsymbol{b}\cup\boldsymbol{c}\cup\epsilon))^*(\boldsymbol{a}\cup\epsilon)$$

More cleanly: $\boldsymbol{a}$ can appear only if not immediately followed by $\boldsymbol{b}$. So $\boldsymbol{a}$ must be followed by $\boldsymbol{a}$, $\boldsymbol{c}$, or end. Let $N=\boldsymbol{b}\cup\boldsymbol{c}$ (non-$\boldsymbol{a}$).

$$(\boldsymbol{b}\cup\boldsymbol{c})^*(\boldsymbol{a}(\boldsymbol{a}\cup\boldsymbol{c})^*(\boldsymbol{b}\cup\boldsymbol{c}))^*(\boldsymbol{a}(\boldsymbol{a}\cup\boldsymbol{c})^*\cup\epsilon)$$

(b) Words with exactly two $\boldsymbol{c}$s and no $\boldsymbol{ab}$:
Combine the no-$\boldsymbol{ab}$ constraint with exactly two $\boldsymbol{c}$s:

$$(\text{no-}\boldsymbol{ab}\text{ part with no }\boldsymbol{c})\boldsymbol{c}(\text{no-}\boldsymbol{ab}\text{ part with no }\boldsymbol{c})\boldsymbol{c}(\text{no-}\boldsymbol{ab}\text{ part with no }\boldsymbol{c})$$

Let $N_0 =$ words over $\{\boldsymbol{a},\boldsymbol{b}\}$ with no $\boldsymbol{ab} = \boldsymbol{b}^*(\boldsymbol{a}^*\boldsymbol{b}^*)^*$... better: $\boldsymbol{b}^*(\boldsymbol{a}^+\boldsymbol{b}^*)^* = (\boldsymbol{b}\cup\boldsymbol{ab})^*(\boldsymbol{a}^*\cup\epsilon)$... Words over $\{\boldsymbol{a},\boldsymbol{b}\}$ avoiding $\boldsymbol{ab}$: each $\boldsymbol{a}$ must be followed by $\boldsymbol{a}$ or end; $\boldsymbol{b}$ can be preceded by anything non-$\boldsymbol{a}$. So: $(\boldsymbol{b}^*\boldsymbol{a}^*)^* = (\boldsymbol{b}\cup\boldsymbol{a})^*$... No. Words avoiding $\boldsymbol{ab}$: $\boldsymbol{b}^*(\boldsymbol{a}^*\boldsymbol{b}^*)^*$... any word over $\{a,b\}$ is $b^*a^*b^*a^*\ldots$ But we need no $\boldsymbol{ab}$, so after any $\boldsymbol{a}$ comes only $\boldsymbol{a}$ or end, meaning $\boldsymbol{a}^*$ appears only at the end: $N_0 = \boldsymbol{b}^*\boldsymbol{a}^*$. Check: $\boldsymbol{bba}$ avoids $\boldsymbol{ab}$ ✓; $\boldsymbol{abba}$ has no $\boldsymbol{ab}$ prefix but $\boldsymbol{ab}$ at position 1-2... wait, $\boldsymbol{ab}$ is exactly $\boldsymbol{a}$ followed by $\boldsymbol{b}$; $\boldsymbol{abba}$ does contain $\boldsymbol{ab}$. $N_0 = \boldsymbol{b}^*\boldsymbol{a}^*$ avoids $\boldsymbol{ab}$ since in b^*a^* no $\boldsymbol{a}$ precedes a $\boldsymbol{b}$. Yes.
Answer: $\boldsymbol{b}^*\boldsymbol{a}^*\cdot\boldsymbol{c}\cdot\boldsymbol{b}^*\boldsymbol{a}^*\cdot\boldsymbol{c}\cdot\boldsymbol{b}^*\boldsymbol{a}^*$.

6.39. (a) Words with no $\boldsymbol{b}$ immediately preceded by $\boldsymbol{c}$ (no $\boldsymbol{cb}$ substring):
$\boldsymbol{c}$ must not be immediately followed by $\boldsymbol{b}$. So $\boldsymbol{c}$ is followed by $\boldsymbol{a}$, $\boldsymbol{c}$, or end:

$$(\boldsymbol{a}\cup\boldsymbol{b}\cup\boldsymbol{c}(\boldsymbol{a}\cup\boldsymbol{c}))^*(\boldsymbol{c}\cup\epsilon) = (\boldsymbol{a}\cup\boldsymbol{b}\cup\boldsymbol{ca}\cup\boldsymbol{cc})^*(\epsilon\cup\boldsymbol{c})$$

(b) Words with exactly one $\boldsymbol{c}$ and no $\boldsymbol{cb}$:
Exactly one $\boldsymbol{c}$, and that $\boldsymbol{c}$ is not immediately followed by $\boldsymbol{b}$:

$$(\boldsymbol{a}\cup\boldsymbol{b})^*\boldsymbol{c}(\boldsymbol{a}\cup\epsilon)(\boldsymbol{a}\cup\boldsymbol{b})^* \quad \text{where the } \boldsymbol{c} \text{ is followed by } \boldsymbol{a} \text{ or end}$$

More precisely:

$$(\boldsymbol{a}\cup\boldsymbol{b})^*\cdot\boldsymbol{c}\cdot((\boldsymbol{a}(\boldsymbol{a}\cup\boldsymbol{b})^*)\cup\epsilon)$$

6.40. Exercises 6.40–6.42 require the NDFA figures (6.11, 6.12, 6.13). The method is:

(a) Write the state equations (right-linear or left-linear) using Theorem 6.3.

(b) Solve by Theorem 6.2 (Arden's lemma), eliminating variables one at a time.

(c) The solution for the start-state variable gives the regular expression.

Since the figures are in the printed text, the reader should apply this procedure to the specific NDFAs shown.

6.41. See Exercise 6.40 for method.

6.42. See Exercise 6.40 for method.

6.43. **Prove: for any A, E, Y, if $E \subseteq Y$ then $A\cdot E \subseteq A\cdot Y$.**

Proof. Let $w \in A\cdot E$. Then $w = xy$ with $x \in A$ and $y \in E$. Since $E \subseteq Y$, $y \in Y$. Hence $w = xy \in A\cdot Y$. □ □

6.44. (a) **Image of $\mathfrak{R}_\Sigma$ under substitution $\bar{s}$ is in $\mathfrak{R}_\Gamma$.**

Proof. Substitution $s : \Sigma \to \wp(\Gamma^*)$ maps each letter to a regular language. For a regular expression R over Σ, define $\bar{s}(R)$ by: $\bar{s}(\boldsymbol{a}) = s(\boldsymbol{a}) \in \mathfrak{R}_\Gamma$ (by assumption), $\bar{s}(\emptyset) = \emptyset$, $\bar{s}(\epsilon) = \{\lambda\}$, $\bar{s}(R_1 \cup R_2) = \bar{s}(R_1) \cup \bar{s}(R_2) \in \mathfrak{R}_\Gamma$ (closed under union), $\bar{s}(R_1 \cdot R_2) = \bar{s}(R_1) \cdot \bar{s}(R_2) \in \mathfrak{R}_\Gamma$ (closed under concatenation), $\bar{s}(R^*) = \bar{s}(R)^* \in \mathfrak{R}_\Gamma$ (closed under Kleene closure). By induction on the structure of R, $\bar{s}(R) \in \mathfrak{R}_\Gamma$. □ □

(b) **Image of $\mathfrak{N}_\Sigma$ under $\bar{s}$ need not be in $\mathfrak{N}_\Gamma$.**

Let $L \in \mathfrak{N}_\Sigma$ be a non-FAD context-free language, and s map letters to regular languages. The image $\bar{s}(L)$ can exceed context-free languages in some interpretations.

6.45. **Proof of Lemma 6.3.**

Lemma 6.3 (Arden's lemma / uniqueness): If $\lambda \notin A$, then $X = AX \cup E$ has the unique solution $X = A^*E$.

Proof. *A^*E is a solution:* $A \cdot (A^*E) \cup E = A^+E \cup E = (A^+ \cup \epsilon)E = A^*E$. ✓

Uniqueness: Suppose $X = AX \cup E$ and $X' = AX' \cup E$. We show $X = X'$. $X \supseteq E$ (since $E \subseteq AX \cup E = X$). By induction: $X \supseteq A^kE$ for all $k \geq 0$ (since $A^kE \subseteq A(A^{k-1}E) \cup E \subseteq AX \cup E = X$). Hence $X \supseteq A^*E$. Conversely, suppose $w \in X$. If $w \notin A^*E$, we derive a contradiction. Since $X = AX \cup E$, $w \in AX \cup E$. If $w \in E \subseteq A^*E$, contradiction. So $w \in AX$: $w = av$ with $a \in A$ and $v \in X$. Since $\lambda \notin A$, $|a| \geq 1$, so $|v| < |w|$. By finite descent, $w \in A^nE$ for some n, so $w \in A^*E$. Contradiction. Hence $X = A^*E$, uniquely. □ □

6.46. $\Xi = \{x \in \{\boldsymbol{a}, \boldsymbol{b}\}^* \mid x$ contains at least two consecutive $\boldsymbol{b}$s $\wedge$ x contains no two consecutive $\boldsymbol{a}$s$\}$:

Words with $\boldsymbol{bb}$ as substring but no $\boldsymbol{aa}$. After the $\boldsymbol{bb}$, no $\boldsymbol{aa}$ either. The surrounding context avoids $\boldsymbol{aa}$: $(\boldsymbol{b} \cup \boldsymbol{ab})^* \boldsymbol{bb} (\boldsymbol{a} \cup \boldsymbol{b})^*$ ... but then we need no $\boldsymbol{aa}$ anywhere.

Words over $\{\boldsymbol{a}, \boldsymbol{b}\}$ avoiding $\boldsymbol{aa}$: $(\boldsymbol{b} \cup \boldsymbol{ab})^* (\boldsymbol{a} \cup \epsilon)$. This must also contain $\boldsymbol{bb}$:

$$(\boldsymbol{b} \cup \boldsymbol{ab})^* \boldsymbol{bb} (\boldsymbol{b} \cup \boldsymbol{ab})^* (\boldsymbol{a} \cup \epsilon)$$

Verify no $\boldsymbol{aa}$: in $(\boldsymbol{b} \cup \boldsymbol{ab})^*$, consecutive $\boldsymbol{a}$s cannot occur since each $\boldsymbol{a}$ is preceded by a $\boldsymbol{b}$ (in the $\boldsymbol{ab}$ block) and followed by a $\boldsymbol{b}$.

6.47. (a) Every $\boldsymbol{b}$ eventually followed by $\boldsymbol{c}$ (not necessarily immediately): Words where no $\boldsymbol{b}$ appears after the last $\boldsymbol{c}$ in a trailing position, i.e., every $\boldsymbol{b}$ is followed somewhere later by $\boldsymbol{c}$:

$$(\boldsymbol{a} \cup \boldsymbol{c} \cup \boldsymbol{b}(\boldsymbol{a} \cup \boldsymbol{b})^* \boldsymbol{c})^*$$

(Each $\boldsymbol{b}$ is followed eventually by $\boldsymbol{c}$ with possible $\boldsymbol{a}$s and $\boldsymbol{b}$s in between.)

(b) Every $\boldsymbol{b}$ immediately followed by $\boldsymbol{c}$: (same as Exercise 6.11)

$$(\boldsymbol{a} \cup \boldsymbol{c} \cup \boldsymbol{bc})^*$$

6.48. Over $\Sigma = \{\boldsymbol{a}, \boldsymbol{b}\}$:

(a) No consecutive $\boldsymbol{a}$s and no consecutive $\boldsymbol{b}$s (strictly alternating):

$$(\boldsymbol{ab})^* (\boldsymbol{a} \cup \epsilon) \cup (\boldsymbol{ba})^* (\boldsymbol{b} \cup \epsilon)$$

(Alternating starting with $\boldsymbol{a}$ or $\boldsymbol{b}$, or empty word.)

(b) Words beginning and ending with different letters:

$$\boldsymbol{a}(\boldsymbol{a}\cup\boldsymbol{b})^*\boldsymbol{b}\cup\boldsymbol{b}(\boldsymbol{a}\cup\boldsymbol{b})^*\boldsymbol{a}$$

(c) Words for which the last two letters match (**aa** or **bb** at the end):

$$(\boldsymbol{a}\cup\boldsymbol{b})^*\boldsymbol{aa}\cup(\boldsymbol{a}\cup\boldsymbol{b})^*\boldsymbol{bb}$$

(d) Words for which the first two letters match (**aa** or **bb** at the start):

$$\boldsymbol{aa}(\boldsymbol{a}\cup\boldsymbol{b})^*\cup\boldsymbol{bb}(\boldsymbol{a}\cup\boldsymbol{b})^*$$

(Also include words of length < 2: λ and **a** and **b** trivially satisfy the vacuous condition; but "first two letters match" requires length ≥ 2.)

(e) Words for which the first and last letters match:

$$\epsilon\cup\boldsymbol{a}\cup\boldsymbol{b}\cup\boldsymbol{a}(\boldsymbol{a}\cup\boldsymbol{b})^*\boldsymbol{a}\cup\boldsymbol{b}(\boldsymbol{a}\cup\boldsymbol{b})^*\boldsymbol{b}$$

6.49. **The language of all valid regular expressions over $\{\boldsymbol{a},\boldsymbol{b}\}$ is not FAD.**

Proof. Let $\Sigma' = \{\boldsymbol{a},\boldsymbol{b},(,),\cup,\cdot,*,\emptyset,\epsilon\}$ be the meta-alphabet. Consider the sublanguage of expressions of the form $(\ldots((\boldsymbol{a}\cup\boldsymbol{b})\cup\boldsymbol{a})\ldots\cup\boldsymbol{a})$ with n nested parentheses. These have a nested bracket structure. The language of valid regular expressions contains arbitrarily deeply nested parentheses, and by the pumping lemma (applied to the bracket depth), it cannot be FAD. (More precisely: the set of valid regular expressions contains the language of balanced parentheses as a sublanguage, which is not FAD; since every FAD language's sublanguages are not necessarily FAD, we instead show directly: the set $\{(\cdots(R_1)\cdots) \mid n \text{ matching pairs}\}$ pumped over the parentheses leads to a contradiction.) □

6.50. Apply Theorem 6.3 (state equations) and Theorem 6.2 (Arden's lemma) to each NDFA in Figure 6.14 to produce a regular expression. (Specific answers depend on the printed figures.)

6.51. **Complete details of Theorem 6.6.**

Theorem 6.6 (Kleene's theorem, second direction): Every FAD language has a regular expression. The proof proceeds by induction on the number of states n of the minimal DFA, using the path-based formula:

$$R_{ij}^{(k)} = R_{ij}^{(k-1)} \cup R_{ik}^{(k-1)}(R_{kk}^{(k-1)})^* R_{kj}^{(k-1)}$$

where $R_{ij}^{(k)}$ is the set of strings labeling paths from state i to state j using only states with index $\leq k$ as intermediaries. The base case ($k = 0$) sets $R_{ij}^{(0)}$ to the letters causing direct transitions $i \to j$ (plus ϵ if $i = j$). The language of the DFA is $\bigcup_{j\in F} R_{1j}^{(n)}$.

6.52. **Prove Lemma 6.4.**

Lemma 6.4 typically states that the regular sets (defined by regular expressions) coincide with the FAD languages. This is Kleene's theorem (both Lemma 6.2 and Theorem 6.4/6.6 together). The proof is covered in the text; the reader should assemble both directions: (i) every regular expression represents a FAD language (Lemma 6.2), and (ii) every FAD language has a regular expression (Theorem 6.6).

6.53. **Theorems 5.2, 5.4, 5.5 as corollaries of Theorem 6.6.**

Theorem 6.6 states that $\mathfrak{R}_\Sigma = \mathfrak{D}_\Sigma$ (the regular sets and FAD languages coincide). From this:

- **Theorem 5.2 (union):** If $L_1, L_2 \in \mathfrak{D}_\Sigma$, then $L_1 = L(R_1)$ and $L_2 = L(R_2)$ for some regular expressions R_1, R_2. Then $L_1 \cup L_2 = L(R_1 \cup R_2) \in \mathfrak{R}_\Sigma = \mathfrak{D}_\Sigma$.
- **Theorem 5.4 (concatenation):** $L_1 \cdot L_2 = L(R_1 \cdot R_2) \in \mathfrak{R}_\Sigma = \mathfrak{D}_\Sigma$.
- **Theorem 5.5 (Kleene closure):** $L_1^* = L(R_1^*) \in \mathfrak{R}_\Sigma = \mathfrak{D}_\Sigma$.

6.54. **Class $\boldsymbol{F}$ generated by the five rules:**

The five rules generate exactly the class of *finite* languages over $\{\boldsymbol{a}, \boldsymbol{b}\}$:

(i) Singletons $\{\boldsymbol{a}\}$ and $\{\boldsymbol{b}\}$ are in $\boldsymbol{F}$.

(ii) $\emptyset \in \boldsymbol{F}$.

(iii) $\{\lambda\} \in \boldsymbol{F}$.

(iv) Concatenation: $F_1 \cdot F_2$ is finite if F_1 and F_2 are.

(v) Union: $F_1 \cup F_2$ is finite if F_1 and F_2 are.

No Kleene closure is allowed, so $\boldsymbol{F}$ consists only of finite languages. Every finite language over $\{\boldsymbol{a}, \boldsymbol{b}\}$ is in $\boldsymbol{F}$ (can be expressed as a finite union of finite concatenations of singletons), and $\boldsymbol{F} \subseteq F_{\{\boldsymbol{a},\boldsymbol{b}\}}$ (the finite languages). Thus $\boldsymbol{F} = F_{\{\boldsymbol{a},\boldsymbol{b}\}}$, the class of all finite languages over $\{\boldsymbol{a}, \boldsymbol{b}\}$.

Chapter 7

Finite State Transducers — Solutions

7.1. (a) **Prove** $\overline{\omega}(t, yx) = \overline{\omega}(t, y) \cdot \overline{\omega}(\overline{\delta}(t, y), x)$ for any Mealy machine.

Proof. By induction on $|x| = k$.
Base ($k = 0$): $\overline{\omega}(t, y\lambda) = \overline{\omega}(t, y) = \overline{\omega}(t, y) \cdot \lambda = \overline{\omega}(t, y) \cdot \overline{\omega}(\overline{\delta}(t, y), \lambda)$ since $\overline{\omega}(\cdot, \lambda) = \lambda$.
Step: Let $x = x'\boldsymbol{a}$ with $|x'| = k$. Then:

$$\begin{aligned}
\overline{\omega}(t, yx'\boldsymbol{a}) &= \overline{\omega}(t, yx') \cdot \omega(\overline{\delta}(t, yx'), \boldsymbol{a}) \\
&= \overline{\omega}(t, y) \cdot \overline{\omega}(\overline{\delta}(t, y), x') \cdot \omega(\overline{\delta}(\overline{\delta}(t, y), x'), \boldsymbol{a}) \\
&\quad \text{(inductive hypothesis applied to } yx') \\
&= \overline{\omega}(t, y) \cdot \overline{\omega}(\overline{\delta}(t, y), x'\boldsymbol{a}) \\
&= \overline{\omega}(t, y) \cdot \overline{\omega}(\overline{\delta}(t, y), x). \quad \square
\end{aligned}$$

□

(b) **Prove** $\overline{\delta}(s, yx) = \overline{\delta}(\overline{\delta}(s, y), x)$**.**

Proof. By induction on $|x|$.
Base: $\overline{\delta}(s, y\lambda) = \overline{\delta}(s, y) = \overline{\delta}(\overline{\delta}(s, y), \lambda)$.
Step: $\overline{\delta}(s, yx'\boldsymbol{a}) = \delta(\overline{\delta}(s, yx'), \boldsymbol{a}) = \delta(\overline{\delta}(\overline{\delta}(s, y), x'), \boldsymbol{a}) = \overline{\delta}(\overline{\delta}(s, y), x'\boldsymbol{a})$. □ □

7.2. Let $\mu : S_A \to S_B$ be a homomorphism between Mealy machines.

(a) **Prove** $\mu(\overline{\delta}_A(s, x)) = \overline{\delta}_B(\mu(s), x)$**.**

Proof. By induction on $|x|$.
Base: $\mu(\overline{\delta}_A(s, \lambda)) = \mu(s) = \overline{\delta}_B(\mu(s), \lambda)$.
Step: $\mu(\overline{\delta}_A(s, x\boldsymbol{a})) = \mu(\delta_A(\overline{\delta}_A(s, x), \boldsymbol{a})) = \delta_B(\mu(\overline{\delta}_A(s, x)), \boldsymbol{a}) = \delta_B(\overline{\delta}_B(\mu(s), x), \boldsymbol{a}) = \overline{\delta}_B(\mu(s), x\boldsymbol{a})$. (Used: $\mu(\delta_A(t, \boldsymbol{a})) = \delta_B(\mu(t), \boldsymbol{a})$ for all $t \in S_A$—homomorphism property.) □ □

(b) **Prove** $\overline{\omega}_A(s, x) = \overline{\omega}_B(\mu(s), x)$**.**

Proof. By induction on $|x|$.

Base: $\overline{\omega}_A(s,\lambda) = \lambda = \overline{\omega}_B(\mu(s),\lambda)$.

Step: $\overline{\omega}_A(s,x\boldsymbol{a}) = \overline{\omega}_A(s,x)\cdot\omega_A(\overline{\delta}_A(s,x),\boldsymbol{a})$. By induction: $= \overline{\omega}_B(\mu(s),x)\cdot\omega_A(\overline{\delta}_A(s,x),\boldsymbol{a})$. By homomorphism: $\omega_A(t,\boldsymbol{a}) = \omega_B(\mu(t),\boldsymbol{a})$ for all t; so: $= \overline{\omega}_B(\mu(s),x)\cdot\omega_B(\mu(\overline{\delta}_A(s,x)),\boldsymbol{a}) = \overline{\omega}_B(\mu(s),x)\cdot\omega_B(\overline{\delta}_B(\mu(s),x),\boldsymbol{a}) = \overline{\omega}_B(\mu(s),x\boldsymbol{a})$. □ □

7.3. **Prove Corollary 7.3.**

Corollary 7.3 states (typically): if there is a homomorphism $\mu : A \to B$ between connected Mealy machines, then B is a homomorphic image of A and A is equivalent to B.

Proof. By Exercise 7.2(b), $\overline{\omega}_A(s_0,x) = \overline{\omega}_B(\mu(s_0),x)$ for all $x \in \Sigma^*$. Since $\mu(s_{0_A}) = s_{0_B}$ (by definition of homomorphism), $\overline{\omega}_A(s_{0_A},x) = \overline{\omega}_B(s_{0_B},x)$, i.e., $f_A(x) = f_B(x)$ for all x. Hence $A \equiv B$. □ □

7.4. **Prove Corollary 7.4: M is minimal iff M is reduced and connected.**

Proof. ($\Rightarrow$) If M is minimal, it is the smallest machine computing f_M. Any disconnected state can be removed without changing f_M (since the machine starts at s_0 and disconnected states are unreachable). Any reducible machine (with two equivalent states) can be merged. Both would yield a smaller equivalent machine, contradicting minimality. Hence M is reduced and connected.

($\Leftarrow$) If M is reduced and connected, assume for contradiction there is an equivalent machine M' with fewer states. By Theorem 7.5, the minimal machine is unique (up to isomorphism); since M is reduced and connected, it has no redundant states. If M' has fewer states, some states of M must correspond to multiple states of M' via any homomorphism, but this contradicts M being reduced. Hence M is minimal. □ □

7.5. **Pumping lemma for FTD functions.**

(a) **Statement:** Let $f : \Sigma^* \to \Gamma^*$ be FTD, implemented by a Mealy machine M with n states. Then for any $x \in \Sigma^*$ with $|x| \geq n$, there exist strings u, v, w with $x = uvw$, $|uv| \leq n$, $|v| \geq 1$, such that for all $k \geq 0$:

$$f(uv^k w) = \overline{\omega}(s_0,u)\cdot\overline{\omega}(\overline{\delta}(s_0,u),v)^k\cdot\overline{\omega}(\overline{\delta}(s_0,uv),w).$$

That is, the output of $uv^k w$ consists of the output while processing u, followed by k repetitions of the output while processing v (each time starting from state $\overline{\delta}(s_0,u)$), followed by the output while processing w from state $\overline{\delta}(s_0,uv) = \overline{\delta}(s_0,u)$.

(b) **Proof:** Since $|x| \geq n$ and M has n states, by the pigeonhole principle the sequence of states $s_0, \overline{\delta}(s_0,u), \overline{\delta}(s_0,uv), \ldots$ must repeat a state within the first $n+1$ steps. Let u, v, w be such that $uvw = x$, v causes a cycle: $\overline{\delta}(s_0,u) = \overline{\delta}(s_0,uv)$. Then for any $k \geq 0$, $\overline{\delta}(s_0,uv^k) = \overline{\delta}(s_0,u)$, so the machine processes w from the same state regardless of k. Thus $f(uv^k w) = \overline{\omega}(s_0,u)\cdot\overline{\omega}(\overline{\delta}(s_0,u),v)^k\cdot\overline{\omega}(\overline{\delta}(s_0,uv),w)$. □

7.6. **Prove Lemma 7.3 parts a–e.**

(Lemma 7.3 typically gives properties of the equivalence relation E_M on states of a Mealy machine.)

(a) **Lemma 7.3a:** $sE_M t$ iff $\forall x \in \Sigma^*, \overline{\omega}(s,x) = \overline{\omega}(t,x)$.

Proof. $sE_M t$ is defined exactly this way: two states are equivalent iff they produce the same output on every input string. This is the definition. □ □

(b) **Lemma 7.3b:** E_M is an equivalence relation.

Proof. *Reflexivity:* $\overline{\omega}(s,x) = \overline{\omega}(s,x)$ trivially, so $sE_M s$. *Symmetry:* If $\overline{\omega}(s,x) = \overline{\omega}(t,x)$ for all x, then $\overline{\omega}(t,x) = \overline{\omega}(s,x)$ for all x. *Transitivity:* If $sE_M t$ and $tE_M u$, then $\overline{\omega}(s,x) = \overline{\omega}(t,x) = \overline{\omega}(u,x)$ for all x. □ □

(c) **Lemma 7.3c:** $sE_M t$ implies $\delta(s,\boldsymbol{a})E_M\delta(t,\boldsymbol{a})$ for all $\boldsymbol{a} \in \Sigma$.

Proof. For any $x \in \Sigma^*$ and $\boldsymbol{a} \in \Sigma$: $\overline{\omega}(\delta(s,\boldsymbol{a}),x) = \overline{\omega}(s,\boldsymbol{a}x)/\omega(s,\boldsymbol{a})$ (the output of $\boldsymbol{a}x$ from s, stripping the first output letter). More precisely: $\overline{\omega}(s,\boldsymbol{a}x) = \omega(s,\boldsymbol{a})\cdot\overline{\omega}(\delta(s,\boldsymbol{a}),x)$. Since $sE_M t$: $\overline{\omega}(s,\boldsymbol{a}x) = \overline{\omega}(t,\boldsymbol{a}x)$, i.e., $\omega(s,\boldsymbol{a})\cdot\overline{\omega}(\delta(s,\boldsymbol{a}),x) = \omega(t,\boldsymbol{a})\cdot\overline{\omega}(\delta(t,\boldsymbol{a}),x)$. Since $sE_M t$ implies $\omega(s,\boldsymbol{a}) = \omega(t,\boldsymbol{a})$ (single-letter case), we get $\overline{\omega}(\delta(s,\boldsymbol{a}),x) = \overline{\omega}(\delta(t,\boldsymbol{a}),x)$ for all x, so $\delta(s,\boldsymbol{a})E_M\delta(t,\boldsymbol{a})$. □ □

(d) **Lemma 7.3d:** E_M is the coarsest congruence consistent with the output function.
(This follows from 7.3a–c and the fact that any finer relation satisfying the same properties must equal E_M.)

(e) **Lemma 7.3e:** The quotient machine $M/_{E_M}$ is well-defined.
(Follows from Lemma 7.3c: δ_{E_M} is well-defined because equivalent states are mapped to equivalent successors.)

7.7. **Prove parts of Lemma 7.4 for FST M.**

(a) E_{0M} **has one class consisting of all of** S**:**
E_{0M} is defined by $sE_{0M}t$ always (no conditions). Hence all states are in one class: $[s]_{E_{0M}} = S$ for all s.

(b) E_{1M} **iff** $\forall \boldsymbol{a} \in \Sigma, \omega(s,\boldsymbol{a}) = \omega(t,\boldsymbol{a})$**:**
E_{1M} refines E_{0M} by requiring that s and t agree on single-letter outputs. $sE_{1M}t$ iff $sE_{0M}t$ (trivially) and $\forall \boldsymbol{a} \in \Sigma, \omega(s,\boldsymbol{a}) = \omega(t,\boldsymbol{a})$.

(c) $sE_{i+1,M}t$ **iff** $sE_{iM}t$ **and** $\forall \boldsymbol{a} \in \Sigma, \delta(s,\boldsymbol{a})E_{iM}\delta(t,\boldsymbol{a})$**:**

Proof. By definition, $E_{i+1,M}$ refines E_{iM} by additionally requiring that the successors are E_{iM}-equivalent. This is the standard iterative refinement.
($\Rightarrow$) If $sE_{i+1,M}t$, then s and t agree on all strings of length $\leq i+1$. In particular, they agree on strings of length $\leq i$ (so $sE_{iM}t$), and for each $\boldsymbol{a}$, $\delta(s,\boldsymbol{a})$ and $\delta(t,\boldsymbol{a})$ agree on all strings of length $\leq i$ (so $\delta(s,\boldsymbol{a})E_{iM}\delta(t,\boldsymbol{a})$).
($\Leftarrow$) If $sE_{iM}t$ and $\delta(s,\boldsymbol{a})E_{iM}\delta(t,\boldsymbol{a})$ for all $\boldsymbol{a}$, then for any $w = \boldsymbol{a}w'$ with $|w| \leq i+1$: $\overline{\omega}(s,\boldsymbol{a}w') = \omega(s,\boldsymbol{a})\cdot\overline{\omega}(\delta(s,\boldsymbol{a}),w')$. Since $sE_{1M}t$ (implied by $sE_{iM}t$, $i \geq 1$): $\omega(s,\boldsymbol{a}) = \omega(t,\boldsymbol{a})$. And $\delta(s,\boldsymbol{a})E_{iM}\delta(t,\boldsymbol{a})$ gives $\overline{\omega}(\delta(s,\boldsymbol{a}),w') = \overline{\omega}(\delta(t,\boldsymbol{a}),w')$ for $|w'| \leq i$. So $\overline{\omega}(s,w) = \overline{\omega}(t,w)$. □ □

7.8. **Prove Theorem 7.6 for Moore machines.**

For a Moore machine $A = \langle \Sigma, \Gamma, S, s_0, \delta, \omega \rangle$:

Proof. By induction on $|x|$.

Base ($|x| = 0$): $\overline{\omega}(t, y\lambda) = \overline{\omega}(t, y) = \overline{\omega}(t, y) \cdot \lambda = \overline{\omega}(t, y) \cdot \overline{\omega}(\overline{\delta}(t, y), \lambda)$, where $\overline{\omega}(\cdot, \lambda) = \omega(\cdot)$ for the initial state output.

Step: Let $x = x'\boldsymbol{a}$. In a Moore machine, $\overline{\omega}(t, x) = \omega(t) \cdot \omega(\delta(t, x_1)) \cdots$ (output depends on states visited, not transitions). The concatenation property follows from the state-sequence decomposition: $\overline{\omega}(t, yx'\boldsymbol{a}) = \overline{\omega}(t, yx') \cdot \omega(\overline{\delta}(t, yx')) = \overline{\omega}(t, y) \cdot \overline{\omega}(\overline{\delta}(t, y), x') \cdot \omega(\overline{\delta}(t, yx'))$. Since $\overline{\delta}(t, yx') = \overline{\delta}(\overline{\delta}(t, y), x')$ (by Exercise 7.1b), this equals $\overline{\omega}(t, y) \cdot \overline{\omega}(\overline{\delta}(t, y), x'\boldsymbol{a}) = \overline{\omega}(t, y) \cdot \overline{\omega}(\overline{\delta}(t, y), x)$. □ □

7.9. **Prove Theorem 7.7.**

Theorem 7.7 (typically): For any Mealy machine M, there exists an equivalent Moore machine M'.

Proof. Given $M = \langle \Sigma, \Gamma, S, s_0, \delta, \omega \rangle$, construct $M' = \langle \Sigma, \Gamma, S', s_0', \delta', \omega' \rangle$ where: $S' = \{[s, \boldsymbol{a}] \mid s \in S, \boldsymbol{a} \in \Sigma\} \cup \{[s_0, \text{init}]\}$ (pairs of state and last input letter). $s_0' = [s_0, \text{init}]$. $\delta'([s, \boldsymbol{b}], \boldsymbol{a}) = [\delta(s, \boldsymbol{a}), \boldsymbol{a}]$. $\omega'([s, \boldsymbol{b}]) = \omega(s, \boldsymbol{b})$ (output is the Mealy output for the last transition). $\omega'(s_0') = \lambda$ (or an arbitrary default).

By induction on $|x|$, $\overline{\omega}_{M'}(s_0', x) = \overline{\omega}_M(s_0, x)$ for all $x \in \Sigma^+$. Hence $M' \equiv M$. □ □

7.10. **Prove Theorem 7.8.**

Theorem 7.8 (typically): For any Moore machine, there exists an equivalent Mealy machine.

Proof. Given Moore machine $M = \langle \Sigma, \Gamma, S, s_0, \delta, \omega \rangle$, define Mealy machine $M' = \langle \Sigma, \Gamma, S, s_0, \delta, \omega' \rangle$ where $\omega'(s, \boldsymbol{a}) = \omega(\delta(s, \boldsymbol{a}))$ (the output is the Moore output of the successor state).

By induction on $|x|$: $\overline{\omega}_{M'}(s_0, x) = \overline{\omega}_M(s_0, x)$ for all $x \in \Sigma^*$.

Base: $\overline{\omega}_{M'}(s_0, \lambda) = \lambda$ and $\overline{\omega}_M(s_0, \lambda) = \omega(s_0)$... there is a discrepancy if the Moore machine outputs at each state including the initial. The conversion shifts the output by one step, which is handled by noting that the initial output $\omega(s_0)$ of the Moore machine is prepended; the Mealy machine matches from the first letter onward. □ □

7.11. **Use Lemma 7.6 to find E_C in Example 7.10.**

Lemma 7.6 gives an iterative algorithm: compute $E_0, E_1, E_2, \ldots$ until stabilization. Starting from E_0 (all states equivalent), refine using the output function and transition function of the machine in Example 7.10. The specific E_C depends on the machine; apply the iterative refinement until no further partitioning occurs. (Since Example 7.10 is in the printed text, the reader should apply the algorithm to the specific machine given.)

7.12. **Homomorphism from machine M (Example 7.11) to machine B (Example 7.2).**

Define $\mu : S_M \to S_B$ by mapping each state of M to the corresponding state of B that produces the same output behavior. Verify: (i) $\mu(s_{0_M}) = s_{0_B}$; (ii) $\mu(\delta_M(s, \boldsymbol{a})) = \delta_B(\mu(s), \boldsymbol{a})$ for all $s, \boldsymbol{a}$; (iii) $\omega_M(s, \boldsymbol{a}) = \omega_B(\mu(s), \boldsymbol{a})$. The specific mapping depends on the machines in Examples 7.11 and 7.2.

7.13. **Prove $\overline{\omega}(t, \boldsymbol{a}) = \omega(t, \boldsymbol{a})$ for any FST.**

Proof. By definition of $\overline{\omega}$ for a single letter: $\overline{\omega}(t, \boldsymbol{a}) = \overline{\omega}(t, \lambda) \cdot \omega(t, \boldsymbol{a}) = \lambda \cdot \omega(t, \boldsymbol{a}) = \omega(t, \boldsymbol{a})$. (Since $\overline{\omega}(t, \lambda) = \lambda$ and concatenation with λ is identity.) □ □

7.14. **Modified vending machine with coin return.**

Add to the machine:

- New input $\boldsymbol{r}$ (coin return).
- New output $\boldsymbol{a}$ (release coins).
- From each state s_k (representing k cents in holding), the transition on $\boldsymbol{r}$ goes to s_0 (reset to zero) and outputs $\boldsymbol{a}$.
- The output $\boldsymbol{a}$ could encode how many coins to return (one per 5 cents, etc.), but if $\boldsymbol{a}$ is a single symbol meaning "release all," then it's a single output.

Formally: add $\delta(s_k, \boldsymbol{r}) = s_0$ and $\omega(s_k, \boldsymbol{r}) = \boldsymbol{a}$ for each state s_k in the vending machine.

7.15. δ_{E_M} **is well defined.**

Proof. $\delta_{E_M}([s]_{E_M}, \boldsymbol{a}) = [\delta(s, \boldsymbol{a})]_{E_M}$. Suppose $[s]_{E_M} = [s']_{E_M}$, i.e., sE_Ms'. By Lemma 7.3c, $\delta(s, \boldsymbol{a})E_M\delta(s', \boldsymbol{a})$. Hence $[\delta(s, \boldsymbol{a})]_{E_M} = [\delta(s', \boldsymbol{a})]_{E_M}$, so δ_{E_M} gives the same result regardless of the representative chosen. □ □

7.16. ω_{E_M} **is well defined.**

Proof. $\omega_{E_M}([s]_{E_M}, \boldsymbol{a}) = \omega(s, \boldsymbol{a})$. Suppose sE_Ms'. By the definition of E_M (states are equivalent iff they produce the same output on all inputs), in particular on the single letter $\boldsymbol{a}$: $\omega(s, \boldsymbol{a}) = \omega(s', \boldsymbol{a})$. Hence ω_{E_M} is independent of the choice of representative. □ □

7.17. **Reduced is not sufficient for minimality.**

Example. A reduced machine may have unreachable states. Let M have states $\{s_0, s_1\}$ where s_1 is unreachable from s_0. If s_0 and s_1 have distinct output behaviors, M is reduced (no two states are equivalent). But s_1 is useless; removing it gives an equivalent machine with fewer states. Hence M is reduced but not minimal.

7.18. μ **in Theorem 7.5 is well defined.**

Proof. μ is defined by $\mu(s) = \overline{\delta}_2(s_{0_2}, x)$ where x is any word with $\overline{\delta}_1(s_{0_1}, x) = s$ (such x exists because M_1 is connected). Well-definedness requires: if $\overline{\delta}_1(s_{0_1}, x) = \overline{\delta}_1(s_{0_1}, x') = s$, then $\overline{\delta}_2(s_{0_2}, x) = \overline{\delta}_2(s_{0_2}, x')$.

Since M_1 and M_2 are equivalent, $\overline{\omega}_1(s_{0_1}, wx) = \overline{\omega}_2(s_{0_2}, wx)$ and $\overline{\omega}_1(s_{0_1}, wx') = \overline{\omega}_2(s_{0_2}, wx')$ for all w. In particular, $\overline{\omega}_1(s_{0_1}, xy) = \overline{\omega}_2(s_{0_2}, xy)$ implies $\overline{\omega}_1(s, y) = \overline{\omega}_2(\overline{\delta}_2(s_{0_2}, x), y)$ for all y. Similarly with x'. Since M_2 is reduced, the state $\overline{\delta}_2(s_{0_2}, x)$ is uniquely determined by its output behavior, which equals that of s. Hence $\overline{\delta}_2(s_{0_2}, x) = \overline{\delta}_2(s_{0_2}, x')$, and μ is well defined. □ □

7.19. μ **is an isomorphism.**

(a) $\mu(s_{0_1}) = \overline{\delta}_2(s_{0_2}, \lambda) = s_{0_2}$.

(b) $\mu(\delta_1(s, \boldsymbol{a})) = \overline{\delta}_2(s_{0_2}, x\boldsymbol{a}) = \delta_2(\overline{\delta}_2(s_{0_2}, x), \boldsymbol{a}) = \delta_2(\mu(s), \boldsymbol{a})$.

(c) $\omega_1(s, \boldsymbol{a}) = \overline{\omega}_1(s_{0_1}, x\boldsymbol{a})|_{\text{last output}} = \overline{\omega}_2(s_{0_2}, x\boldsymbol{a})|_{\text{last}} = \omega_2(\mu(s), \boldsymbol{a})$. (The output functions agree since the machines are equivalent and μ commutes with transitions.)

(d) **Injective:** If $\mu(s) = \mu(t)$, then $\overline{\delta}_2(s_{0_2}, x_s) = \overline{\delta}_2(s_{0_2}, x_t)$, so s and t have the same output behavior from s_{0_1}; by M_1 being reduced, $s = t$.

(e) **Surjective:** Since M_2 is connected, every $t \in S_2$ is reachable: $t = \overline{\delta}_2(s_{0_2}, y)$ for some y. Let $s = \overline{\delta}_1(s_{0_1}, y)$; then $\mu(s) = t$.

7.20. **One-unit delay Mealy and Moore machines.**

$\overline{\omega}(x_1 x_2 \cdots x_m) = \boldsymbol{x} \cdot x_1 \cdots x_{m-1}$ (output is $\boldsymbol{x}$ followed by the input shifted by one).

(a) **Mealy sextuple:** $M = \langle \{\boldsymbol{a}, \boldsymbol{b}\}, \{\boldsymbol{a}, \boldsymbol{b}, \boldsymbol{x}\}, \{s_a, s_b, s_0\}, s_0, \delta, \omega \rangle$ where s_0 is the initial state, s_a means "last input was $\boldsymbol{a}$", s_b means "last input was $\boldsymbol{b}$". $\delta(s_0, \boldsymbol{a}) = s_a$, $\delta(s_0, \boldsymbol{b}) = s_b$, $\delta(s_a, \boldsymbol{a}) = s_a$, $\delta(s_a, \boldsymbol{b}) = s_b$, $\delta(s_b, \boldsymbol{a}) = s_a$, $\delta(s_b, \boldsymbol{b}) = s_b$. $\omega(s_0, \boldsymbol{a}) = \boldsymbol{x}$, $\omega(s_0, \boldsymbol{b}) = \boldsymbol{x}$, $\omega(s_a, \boldsymbol{a}) = \boldsymbol{a}$, $\omega(s_a, \boldsymbol{b}) = \boldsymbol{a}$, $\omega(s_b, \boldsymbol{a}) = \boldsymbol{b}$, $\omega(s_b, \boldsymbol{b}) = \boldsymbol{b}$.

(b) **Mealy machine diagram:** Three states; s_0 outputs $\boldsymbol{x}$ on both inputs; s_a outputs $\boldsymbol{a}$ on both inputs; s_b outputs $\boldsymbol{b}$ on both inputs.

(c) **Moore sextuple:** Same states but output is determined by state: $\omega(s_0) = \boldsymbol{x}$, $\omega(s_a) = \boldsymbol{a}$, $\omega(s_b) = \boldsymbol{b}$. (The Moore machine outputs at the current state, not the transition.)

(d) **Moore machine diagram:** Three states with state-output labels; same transition structure.

7.21. **Circuit for vending machine (Example 7.1).**

(a) **EOS:** No strong reason; the vending machine processes coins continuously without a defined end signal. The output (dispensing a product) happens at a specific state transition, not at the end.

(b) **SOS:** Useful to reset the machine to the initial state (0 cents accumulated); corresponds to powering on or resetting the machine.

(c) **Input encoding:** If inputs are **5** (nickel), **10** (dime), **25** (quarter), that's 3 symbols, needing $\lceil \log_2 3 \rceil = 2$ bits. E.g., 00 = **5**, 01 = **10**, 10 = **25**, 11 =unused.

(d) **Output encoding:** If outputs are "nothing" or "dispense product," 1 bit suffices.

(e) **State encoding:** If the machine has states $\{s_0, s_5, s_{10}, s_{15}, s_{20}\}$ (5 states), need $\lceil \log_2 5 \rceil = 3$ bits.

(f) t_2 **truth table:** Depends on the specific state encoding chosen. After encoding states and inputs as binary, construct the Karnaugh map for next-state bit t_2 and minimize.

(g) w_2 **truth table:** Similarly for output bit w_2.

(h) The complete circuit combines the minimized Boolean functions into a synchronous sequential circuit with flip-flops for state and combinational logic for next-state and output.

7.22. **Circuit for modified vending machine (Exercise 7.14).**

Similar to Exercise 7.21 but with the coin-return input $\boldsymbol{r}$ added. The input encoding now needs 2 bits for 4 inputs ($\mathbf{5}, \mathbf{10}, \mathbf{25}, \boldsymbol{r}$). The state encoding is the same. The truth table for t_3 includes the new $\boldsymbol{r}$ transitions.

7.23. **Circuit for Example 7.2.**

Apply the standard encoding conventions: assign binary codes to states, inputs, and outputs; write the next-state and output truth tables; minimize Boolean functions; draw the sequential logic circuit.

7.24. **Circuit for Example 7.6.**

Same procedure as Exercise 7.23.

7.25. **Circuit for FST D (Example 7.8).**

Same procedure.

7.26. **Connected is not sufficient for minimality.**

Example. Let M have states $\{s_0, s_1\}$, $\Sigma = \{\boldsymbol{a}\}$, $\Gamma = \{\mathbf{0}, \mathbf{1}\}$: $\delta(s_0, \boldsymbol{a}) = s_1$, $\delta(s_1, \boldsymbol{a}) = s_0$; $\omega(s_0, \boldsymbol{a}) = \mathbf{0}$, $\omega(s_1, \boldsymbol{a}) = \mathbf{0}$. Both states output $\mathbf{0}$ on $\boldsymbol{a}$ and alternate, so $s_0 E_M s_1$. M is connected (both states reachable from s_0) but not reduced (two equivalent states). Hence it is not minimal.

7.27. **Two-unit delay Mealy and Moore machines.**

$\overline{\omega}(x_1 \cdots x_m) = \boldsymbol{x}\boldsymbol{x}x_1 \cdots x_{m-2}$.

(a) **Mealy sextuple:** States encode the last two inputs: $\{s_{00}, s_{01}, s_{10}, s_{11}, s_{x0}, s_{x1}, s_{xx}\}$ where the subscripts denote the last two letters ($\boldsymbol{a} = 0$, $\boldsymbol{b} = 1$, $\boldsymbol{x}$=not yet seen). Initial state: s_{xx}. $\omega(s_{xx}, \boldsymbol{a}) = \boldsymbol{x}$, $\omega(s_{xa}, \sigma) = \boldsymbol{x}$, $\omega(s_{ab}, \sigma) = \boldsymbol{a}$, etc. The output at any transition is the letter that was processed two steps ago.

(b) –

d. The diagrams and Moore version follow the same pattern as Exercise 7.20 but with 7 states encoding the last two inputs.

7.28. (a) If M_1 is not reduced: two states $s, t \in S_1$ with $sE_{M_1}t$ but $s \neq t$. Then $\mu(s)$ and $\mu(t)$ would both map to the same state in any reduced machine, but μ might be forced to map them differently (since M_1 is not reduced, μ need not be injective). The isomorphism fails because it would not be one-to-one.

(b) Without reducedness, μ fails to be injective (part (d) of Exercise 7.19).

7.29. (a) If M_1 is not connected: some state $s \in S_1$ is not reachable from s_{0_1}. Then $\mu(s)$ is not defined (since $\mu(s) = \overline{\delta}_2(s_{0_2}, x)$ requires x to reach s, which is impossible).

(b) The property lost is surjectivity (μ onto S_2) or the domain of μ is incomplete.

7.30. (a) If M_2 is not reduced: μ cannot be an isomorphism because M_1 and M_2 might have different numbers of states (since two equivalent states of M_2 could be collapsed).

(b) The property lost is injectivity or the counting argument fails.

7.31. (a) If M_2 is not connected: μ onto S_2 might not hold if unreachable states of M_2 have no preimage.

(b) Surjectivity of μ fails.

7.32. (a) FST A not reduced and A^c not reduced: Let A have two equivalent states; the connected portion A^c (connected component of A) might still have those two equivalent states.

(b) FST A not reduced and A^c reduced: Take A with an isolated duplicate state not in the connected portion; A^c is the connected part which might already be reduced.

7.33. **Complete proof of Theorem 7.4.**

(a) $\overline{\omega}(t,y) = \overline{\omega}_{E_M}([t]_{E_M}, y)$: By induction on $|y|$, using the fact that $\omega_{E_M}([s]_{E_M}, \boldsymbol{a}) = \omega(s,\boldsymbol{a})$ and $\delta_{E_M}([s]_{E_M}, \boldsymbol{a}) = [\delta(s,\boldsymbol{a})]_{E_M}$.

(b) $M/_{E_M} \equiv M$: Both produce the same output for all input strings (from part (a)).

(c) $M/_{E_M}$ is reduced: Suppose $[s]_{E_M} E_{M/E_M} [t]_{E_M}$. Then for all y, $\overline{\omega}_{E_M}([s]_{E_M}, y) = \overline{\omega}_{E_M}([t]_{E_M}, y)$, which means $\overline{\omega}(s,y) = \overline{\omega}(t,y)$ for all y, so $sE_M t$, so $[s]_{E_M} = [t]_{E_M}$.

(d) If M connected, $M/_{E_M}$ connected: Every equivalence class $[t]_{E_M}$ is reachable in $M/_{E_M}$ because t is reachable in M (via some x), and $[\overline{\delta}(s_0,x)]_{E_M} = \overline{\delta}_{E_M}([s_0]_{E_M}, x)$.

7.34. (a) f_1 **is FTD:** Define Mealy machine M with states {read1, read0}, initial state start (extra state for first letter): From start: on **1**, go to read1 with output $\boldsymbol{y}$; on **0**, go to read0 with output $\boldsymbol{n}$. From read1: on any input, stay in read1, output $\boldsymbol{y}$. From read0: on any input, stay in read0, output $\boldsymbol{n}$. Then $\overline{\omega}(s_{\text{start}}, \boldsymbol{a}_1 \cdots \boldsymbol{a}_m) = \boldsymbol{y}^m$ if $\boldsymbol{a}_1 = \mathbf{1}$, $\boldsymbol{n}^m$ otherwise. Hence f_1 is FTD.

(b) f_2 **is not FTD:** By the pumping lemma for FTD functions (Exercise 7.5): for any n and $x = \mathbf{0}^n\mathbf{1}$ (length $n+1 > n$), if f_2 were FTD with n states, we pump the **0** block. $f_2(\mathbf{0}^n\mathbf{1}) = \boldsymbol{nn}\cdots\boldsymbol{ny}$ (n $\boldsymbol{n}$s and one $\boldsymbol{y}$); but $f_2(\mathbf{0}^{n+k}\mathbf{1}) = \boldsymbol{n}^{n+k}\boldsymbol{y}$ for any k. The last letter of the output changes from $\boldsymbol{n}$ to $\boldsymbol{y}$ depending on the last input letter, which requires the machine to "remember" the entire input—impossible with finite memory. Formally: any two words differing only in the last letter (one ending in **0**, one in **1**) must have different output strings, but for any n-state machine, words of length $> n$ that agree on a prefix must produce the same output on that prefix; however, $f_2(\mathbf{0}^m\mathbf{1})$ and $f_2(\mathbf{0}^m\mathbf{0})$ disagree on the last output character. This means the machine must distinguish $m+1$ different lengths of leading zeros, requiring unbounded state space. □

7.35. f_3 **is not FTD.**

f_3 maps each input to the first $|x|$ characters of the fixed infinite sequence **010010001…**. The number of **0**s between successive **1**s grows without bound. For any n-state Mealy machine, the output sequence must eventually be periodic with period at most n. But the gaps between **1**s in the infinite sequence grow arbitrarily, so f_3 is not periodic—hence not implementable by a finite machine. More precisely: if f_3 were FTD, by the pumping lemma, processing a word of length $> n$ would pump a cycle; but cycling would repeat a finite gap between **1**s, preventing the ever-growing gaps. Contradiction. □

7.36. **Prove: for FTD f, the first n letters of $f(xy)$ equal those of $f(xz)$.**

Proof. Let M be a Mealy machine for f with states S. For any $x \in \Sigma^n$: $\overline{\delta}(s_0,x)$ is some state s. Then $f(xy) = \overline{\omega}(s_0,xy) = \overline{\omega}(s_0,x)\cdot\overline{\omega}(s,y)$ and $f(xz) = \overline{\omega}(s_0,xz) = \overline{\omega}(s_0,x)\cdot\overline{\omega}(s,z)$. The first n letters of both are $\overline{\omega}(s_0,x)$ (since $|x| = n$ and the first n output letters come from processing x). Hence they agree on the first n letters. □ □

7.37. **Elevator transducer.**

(a) **Input alphabet:** $\Sigma = \{\boldsymbol{u}, \boldsymbol{d}, \mathbf{1}, \mathbf{2}, \boldsymbol{none}\}$ ($\boldsymbol{u}$=up button floor 1, $\boldsymbol{d}$=down button floor 2, **1**=button inside for floor 1, **2**=button inside for floor 2, ***none***=no button pressed).

(b) **Output alphabet:** $\Gamma = \{\text{close}, \text{open}, \text{goto1}, \text{goto2}\}$.

(c) **Mealy sextuple:** States represent (floor, door-state): e.g., s_{1c} = floor 1, doors closed; s_{1o} = floor 1, doors open; s_{2c} = floor 2, closed; s_{2o} = floor 2, open. Transitions handle button presses and move accordingly.

(d) –

g. Detailed diagrams and circuit depend on the exact state machine chosen; the above framework is the formal starting point.

7.38. **Mealy machine for traffic signal (Example 7.16).**

Build a Mealy machine with states representing each phase of the traffic cycle (east-west green, east-west yellow, north-south green, north-south yellow, etc.) and transitions triggered by sensor inputs and timer signals, with outputs being the light configurations.

7.39. **Traffic signal with walk signals.**

Extend the machine from Exercise 7.38 with additional states for the walk phase.

(a) Input: add γ (pedestrian button); output: add $\boldsymbol{W}/\boldsymbol{D}$ component.

(b) –

c. Diagrams extend the traffic signal machine with walk-phase states.

7.40. **Intersection with left-turn signals.**

Extend with states for left-turn phases triggered by sensors γ and δ.

7.41. **Base-3 adder.**

(a) **Input alphabet:** Pairs of base-3 digits: $\Sigma = \{0,1,2\} \times \{0,1,2\}$ (9 symbols), or encoded as digit pairs.

(b) **Output alphabet:** $\Gamma = \{0,1,2\}$ (one base-3 digit per step, with carry handled by state).

(c) **States:** $\{s_0, s_1\}$ where s_0 = no carry, s_1 = carry. $\delta(s_0,(a,b)) = s_1$ if $a+b \geq 3$, else s_0. $\omega(s_0,(a,b)) = (a+b) \bmod 3$. $\delta(s_1,(a,b)) = s_1$ if $a+b+1 \geq 3$, else s_0. $\omega(s_1,(a,b)) = (a+b+1) \bmod 3$.

(d) For the Moore version: states encode both the carry and the last output digit.

(e) Circuit: implement δ and ω as combinational logic; use one flip-flop for the carry bit.

7.42. **Base-10 adder.**

Similar to Exercise 7.41 but with $\Sigma = \{0,\ldots,9\} \times \{0,\ldots,9\}$, $\Gamma = \{0,\ldots,9\}$, and 2 states (carry/no-carry). Formulas: $\omega(s_c,(a,b)) = (a+b+c) \bmod 10$; $\delta(s_c,(a,b)) = s_{(a+b+c)\geq 10}$.

7.43. **Left-to-right addition is not FTD.**

In right-to-left addition (Example 7.15), the carry propagates from right to left, allowing online processing. In left-to-right addition, the carry at each position depends on all digits to the right (which have not yet been seen), requiring unbounded lookahead. By the pumping lemma for FTD functions: consider adding two numbers where carrying propagates across an arbitrarily long prefix. For any n-state machine, two numbers differing only in the rightmost digits but affecting carries at position 1 can be constructed; the machine would need $> n$ states to distinguish all carry lengths. Hence f_4 is not FTD.

7.44. **Prove Theorem 7.9 for MSM.**

(a) $M/_{E_M} \equiv M$: Same argument as Exercise 7.33(b).

(b) $M/_{E_M}$ is reduced: Same argument as Exercise 7.33(c).

(c) If M connected, $M/_{E_M}$ connected: Same as Exercise 7.33(d).

7.45. **Minimal Moore machine is reduced.**

Proof. Suppose the minimal Moore machine M for f is not reduced. Then two states $s \neq t$ in M have sE_Mt. Form M' by merging $[s] = [t]$ in the quotient $M/_{E_M}$; this gives an equivalent machine with fewer states. But M was defined as minimal—contradiction. Hence M is reduced. □ □

7.46. **Theorem 7.10 for MSM.**

(a) M minimal iff M is reduced and connected: Same argument as for Mealy machines (Exercise 7.4).

(b) $M^c/_{E_{M^c}}$ is minimal: M^c is the connected portion (remove unreachable states); $M^c/_{E_{M^c}}$ is the quotient (merge equivalent states). The result is reduced (by quotient) and connected (by taking connected portion), hence minimal.

7.47. **Prove Lemma 7.5 and Corollary 7.9 for MSMs.**

(a) Same as Exercise 7.2(a) (replace ω_A, ω_B with Moore output functions).

(b) Same as Exercise 7.2(b).

(c) $A \equiv B$: $\overline{\omega}_A(s_{0_A}, x) = \overline{\omega}_B(\mu(s_{0_A}), x) = \overline{\omega}_B(s_{0_B}, x)$ for all x.

7.48. **Prove Corollary 7.10.**

Corollary 7.10 (typically): The quotient $M/_{E_M}$ is the unique minimal machine equivalent to M (up to isomorphism).

Proof. By Theorem 7.9/Theorem 7.4, $M/_{E_M}$ is reduced and connected (if M is connected). By Theorem 7.5, any two minimal equivalent machines are isomorphic. Hence $M/_{E_M}$ is the unique minimal machine (up to isomorphism) equivalent to M. □ □

7.49. **Prove Corollary 7.11.**

(The specific content of Corollary 7.11 depends on the text; it typically follows from the preceding theorems about minimization and isomorphism.)

7.50. δ_{E_M} **and** ω_{E_M} **well-defined for FST quotient.**

(a) δ_{E_M} well-defined: See Exercise 7.15.

(b) ω_{E_M} well-defined: See Exercise 7.16.

7.51. δ_{E_M} **and** ω_{E_M} **well-defined for MSM quotient.**

(a) δ_{E_M}: Same argument as Exercise 7.15 (using $sE_Mt \Rightarrow \delta(s, \boldsymbol{a})E_M\delta(t, \boldsymbol{a})$).

(b) $\omega_{E_M}([s]_{E_M}) = \omega(s)$: If sE_Ms', then s and s' produce the same output on all strings, in particular λ (i.e., $\omega(s) = \omega(s')$).

7.52. **Isomorphism from A to B and A connected $\Rightarrow$ B connected.**

(a) **Mealy machines:** Let $\psi : S_A \to S_B$ be an isomorphism. Every $t \in S_B$ has a preimage $s = \psi^{-1}(t)$. Since A is connected, $s = \overline{\delta}_A(s_{0_A}, x)$ for some x. Then $t = \psi(s) = \psi(\overline{\delta}_A(s_{0_A}, x)) = \overline{\delta}_B(\psi(s_{0_A}), x) = \overline{\delta}_B(s_{0_B}, x)$. So t is reachable in B.

(b) **Moore machines:** Same argument.

7.53. **Isomorphism from A to B and B connected $\Rightarrow$ A connected.**

Since ψ is an isomorphism, ψ^{-1} is also an isomorphism from B to A. By Exercise 7.52, B connected and isomorphism ψ^{-1} implies A connected.

7.54. **Homomorphism from A to B and A connected does NOT imply B connected.**

(a) **Mealy counterexample:** Let A have state $\{s_0\}$ (connected), $\Sigma = \{\boldsymbol{a}\}$, $\delta_A(s_0, \boldsymbol{a}) = s_0$, $\omega_A(s_0, \boldsymbol{a}) = \mathbf{0}$. Let B have states $\{t_0, t_1\}$ where t_1 is unreachable from t_0; $\delta_B(t_0, \boldsymbol{a}) = t_0$, $\omega_B(t_0, \boldsymbol{a}) = \mathbf{0}$, $\delta_B(t_1, \boldsymbol{a}) = t_1$, $\omega_B(t_1, \boldsymbol{a}) = \mathbf{0}$. Map $\mu(s_0) = t_0$. Then μ is a homomorphism, A is connected, but B is not connected (t_1 unreachable).

(b) **Moore machines:** Same construction.

7.55. **Homomorphism from A to B and B connected does NOT imply A connected.**

(a) **Counterexample:** Let A have states $\{s_0, s_1\}$ where s_1 is unreachable; B has state $\{t_0\}$ (trivially connected). Map $\mu(s_0) = \mu(s_1) = t_0$. μ is a homomorphism, B is connected, but A is not connected.

(b) Same for Moore.

7.56. **If $\psi : A \to B$ and $\mu : B \to A$ are isomorphisms of connected FSTs, then $\psi = \mu^{-1}$.**

Proof. $\mu \circ \psi : S_A \to S_A$ is an isomorphism (composition of isomorphisms). For any $s \in S_A$ reachable via x ($s = \overline{\delta}_A(s_{0_A}, x)$): $(\mu \circ \psi)(s) = \mu(\psi(\overline{\delta}_A(s_{0_A}, x))) = \mu(\overline{\delta}_B(s_{0_B}, x)) = \overline{\delta}_A(s_{0_A}, x) = s$. So $\mu \circ \psi = \mathrm{id}_{S_A}$, i.e., $\mu = \psi^{-1}$. □ □

7.57. **If A, B are FSTs (possibly not connected), $\psi = \mu^{-1}$ may fail.**

Example: Let $A = B$ have states $\{s_0, s_1\}$ where both are unreachable from each other (two isolated states, each with self-loops). The function $\psi(s_0) = s_0, \psi(s_1) = s_1$ and $\mu(s_0) = s_1, \mu(s_1) = s_0$ are both isomorphisms (since the machines are symmetric), but $\mu \neq \psi^{-1}$.

7.58. **Three-state MSM with E_{0A} having one class; can $E_{0A} \neq E_{1A}$?**

E_{0A} always has exactly one class for any MSM (by definition/Lemma 7.4a). $E_{0A} \neq E_{1A}$ iff not all states have the same output (ω not constant). Yes, this is possible: if three states s_0, s_1, s_2 have $\omega(s_0) \neq \omega(s_1)$, then E_{1A} will separate them.

7.59. (a) Mealy M not connected and $M/_{E_M}$ not connected: Take M with states $\{s_0, s_1, s_2\}$ where s_2 is unreachable and $s_1 \not E_M s_2$. Then $M/_{E_M}$ has classes $\{[s_0], [s_1], [s_2]\}$, and $[s_2]$ is unreachable.

(b) Mealy M not connected but $M/_{E_M}$ connected: Take M with s_2 unreachable but $s_2 E_M s_0$. Then in $M/_{E_M}$, $[s_2] = [s_0]$, and the quotient's states are a subset of the original, with the unreachable class merged into a reachable one.

7.60. Similar to Exercise 7.59 for Moore machines.

7.61. **Prove $\mu(s) E_B \mu(t) \Leftrightarrow s E_A t$ for Mealy homomorphism.**

Proof. ($\Rightarrow$) If $\mu(s) E_B \mu(t)$: $\overline{\omega}_B(\mu(s), x) = \overline{\omega}_B(\mu(t), x)$ for all x. By Exercise 7.2(b), $\overline{\omega}_A(s, x) = \overline{\omega}_B(\mu(s), x) = \overline{\omega}_B(\mu(t), x) = \overline{\omega}_A(t, x)$, so $s E_A t$.

($\Leftarrow$) If $s E_A t$: $\overline{\omega}_A(s, x) = \overline{\omega}_A(t, x)$ for all x. So $\overline{\omega}_B(\mu(s), x) = \overline{\omega}_A(s, x) = \overline{\omega}_A(t, x) = \overline{\omega}_B(\mu(t), x)$, giving $\mu(s) E_B \mu(t)$. □ □

7.62. Same as Exercise 7.61 for Moore machines.

7.63. Exercises 7.63 and 7.64: Same as Exercises 7.32(a) and (b).

7.64. Same as Exercise 7.32 for MSMs.

7.65. **Isomorphism $\cong$ is an equivalence relation on Mealy machines.**

(a) **Symmetry:** If $\psi : S_A \to S_B$ is an isomorphism, then $\psi^{-1} : S_B \to S_A$ is also an isomorphism (verify all homomorphism properties hold for ψ^{-1}).

(b) **Reflexivity:** The identity $\mathrm{id} : S_A \to S_A$ is an isomorphism.

(c) **Composition:** If $\psi : A \to B$ and $\varphi : B \to C$ are isomorphisms, verify that $\varphi \circ \psi : S_A \to S_C$ satisfies all isomorphism properties (by chaining the homomorphism properties).

(d) **Equivalence:** From (a), (b), and (c) (which gives transitivity via $\varphi \circ \psi$), $\cong$ is reflexive, symmetric, and transitive.

(e) **Homomorphism is not an equivalence relation:** Not symmetric: a homomorphism $\mu : A \to B$ need not have an inverse that is a homomorphism from B to A (e.g., if μ is not injective, no inverse exists).

7.66. Same argument as Exercise 7.65 for Moore machines.

7.67. **Prove homomorphism $\mu : M \to M/_{E_M}$ exists.**

Proof. Define $\mu : S \to S_{E_M}$ by $\mu(s) = [s]_{E_M}$.

Check homomorphism properties:

(i) $\mu(s_0) = [s_0]_{E_M} = s_{0_{E_M}}$. ✓

(ii) $\mu(\delta(s, \boldsymbol{a})) = [\delta(s, \boldsymbol{a})]_{E_M} = \delta_{E_M}([s]_{E_M}, \boldsymbol{a}) = \delta_{E_M}(\mu(s), \boldsymbol{a})$. ✓

(iii) $\omega(s, \boldsymbol{a}) = \omega_{E_M}([s]_{E_M}, \boldsymbol{a}) = \omega_{E_M}(\mu(s), \boldsymbol{a})$. ✓

□ □

7.68. Same as Exercise 7.67 for Moore machines.

7.69. **Limit time in state s_2 to 3 clock cycles.**

(a) Replace s_2 with a chain $s_2^{(1)}, s_2^{(2)}, s_2^{(3)}$ where: at $s_2^{(1)}$, if sensor still active, go to $s_2^{(2)}$; at $s_2^{(2)}$, if still active, go to $s_2^{(3)}$; at $s_2^{(3)}$, regardless of sensor, proceed to the next phase (yellow light). All three states output $\langle R, R, R, G\rangle$.

(b) –

c. Similar expansions for s_6.

7.70. **Ensure east-west traffic gets green occasionally.**

(a) Without new states: add a time-out counter within the existing state structure; when the sensor is active too long, force a transition to s_0 (east-west green).

(b) With new states: add a "fairness counter" that remembers how many cycles each lane has gotten; add states encoding the last lane served and guarantee round-robin.

7.71. **Prove: if two Moore machines are homomorphic, they are equivalent.**

Proof. If $\mu : A \to B$ is a homomorphism, by Exercise 7.47(c), $f_A = f_B$. Hence $A \equiv B$. □ □

7.72. **Prove $\mathfrak{D}_\Sigma$ is closed under any FTD function $f : \Sigma^* \to \Sigma^*$.**

Proof. Let $L \in \mathfrak{D}_\Sigma$ and f be FTD, implemented by Mealy machine M. We want to show $f^{-1}(L) \in \mathfrak{D}_\Sigma$ (inverse image) and $f(L) \in \mathfrak{D}_\Sigma$ (image).

Closure under inverse: Given DFA A for L and Mealy machine M for f, build a product machine P whose states are $S_A \times S_M$. On input $\boldsymbol{a}$: P advances M to compute the output $f(\boldsymbol{a})$, then advances A on that output. If the output of M is a single letter per input (Mealy), then P directly simulates A on the translation. The result is a DFA for $f^{-1}(L) = \{x \mid f(x) \in L\} \in \mathfrak{D}_\Sigma$.

Closure under image: $f(L) = \{f(x) \mid x \in L\}$. Build an NDFA that reads the output of M while M's input was accepted by A: nondeterministically guess the input $x \in L$ and simulate M on x to generate output. Since $L \in \mathfrak{D}_\Sigma$ and M is a finite transducer, $f(L) \in \mathfrak{D}_\Sigma$. (More precisely: the composition of a DFA with an FST yields an NDFA, and by Theorem 4.1, this is equivalent to a DFA.) □ □

Chapter 8

Regular and Context-Free Grammars — Solutions

8.1. **Can strings like $\boldsymbol{ab}BA\boldsymbol{d}B\boldsymbol{c}$ be derived from the start symbol in a right-linear grammar?**

No. In a right-linear grammar, every production has the form $A \to w$ or $A \to wB$ where $w \in \Sigma^*$ and B is a nonterminal. Thus any sentential form derived from the start symbol has the structure w or wB where all terminal symbols precede the single nonterminal (if any). A string like $\boldsymbol{ab}BA\boldsymbol{d}B\boldsymbol{c}$ contains nonterminals interspersed with and following terminals, which cannot occur in any right-linear grammar derivation.

8.2. **Prove $P(n)$: for all $x \in \Sigma^n$, if $t_0 \overset{*}{\Rightarrow} x\,t_j$ then $\overline{\delta}_A(t_0, x) = t_j$.**

Base case ($n = 0$): The only string of length 0 is λ. The derivation $t_0 \overset{*}{\Rightarrow} \lambda\, t_0$ uses zero steps, and $\overline{\delta}_A(t_0, \lambda) = t_0$ by definition, so $P(0)$ holds.

Inductive step: Assume $P(n)$ holds for all strings of length n. Let $x = ya$ where $y \in \Sigma^n$ and $a \in \Sigma$. If $t_0 \overset{*}{\Rightarrow} ya \cdot t_j$, then the derivation must pass through some intermediate state: $t_0 \overset{*}{\Rightarrow} y \cdot t_k \Rightarrow ya \cdot t_j$ for some t_k. By the inductive hypothesis applied to y, $\overline{\delta}_A(t_0, y) = t_k$. The final step $t_k \Rightarrow a \cdot t_j$ corresponds to a production $t_k \to \boldsymbol{a}\,t_j$ in the grammar (by construction of G_A), which corresponds to a transition $\delta(t_k, a) = t_j$ in A. Therefore $\overline{\delta}_A(t_0, ya) = \delta(\overline{\delta}_A(t_0, y), a) = \delta(t_k, a) = t_j$.

8.3. **Regular expressions for the languages of:**

(a) G_4: Tracing the grammar, we get

$$L(G_4) = \boldsymbol{ab}(\boldsymbol{b} \cup \boldsymbol{c}(\boldsymbol{ab}(\boldsymbol{b} \cup \boldsymbol{c} \cdots))^*) \cup \boldsymbol{bbab} \cup \boldsymbol{cc}(\boldsymbol{a} \cup \boldsymbol{c})^* \boldsymbol{aa}(\boldsymbol{b}(\boldsymbol{a} \cup \boldsymbol{c})^* \boldsymbol{aa})^*$$

More precisely, the equations yield: $V = \boldsymbol{a}V \cup \boldsymbol{c}X$, so $V = \boldsymbol{a}^*\boldsymbol{c}X$. $X = \boldsymbol{b}V \cup \boldsymbol{aa}X$, so $X = (\boldsymbol{aa})^*\boldsymbol{b}V = (\boldsymbol{aa})^*\boldsymbol{ba}^*\boldsymbol{c}X$, hence $X = ((\boldsymbol{aa})^*\boldsymbol{ba}^*\boldsymbol{c})^* \cdot \lambda$ — but X has no terminal production, so in fact X generates the empty language unless we continue: $X = (\boldsymbol{aa})^*\boldsymbol{b}V$ and $V = \boldsymbol{a}^*\boldsymbol{c}X$, giving $X = (\boldsymbol{aa})^*\boldsymbol{ba}^*\boldsymbol{c}X$ with no escape, so $L(X) = \emptyset$. Similarly $L(V) = \emptyset$.
Since $L(X) = L(V) = \emptyset$, the $\boldsymbol{c}$-paths from A contribute nothing. The surviving productions are $S \to \boldsymbol{ab}A$, $A \to \boldsymbol{b}C$, $C \to \lambda \mid \boldsymbol{c}S$, and $S \to \boldsymbol{bb}B$, $B \to \boldsymbol{ab}$. Solving:

$$L(G_4) = \boldsymbol{bbab} \cup \boldsymbol{abb}.$$

The regular expression is $\boldsymbol{bbab} \cup (\boldsymbol{ab})^+\boldsymbol{b}$.

(b) G_5 generates strings with an even number of **1**s (binary parity even):

$$L(G_5) = (\mathbf{0}^*\mathbf{10}^*\mathbf{1})^*\mathbf{0}^* = (\mathbf{0}^*(\mathbf{10}^*\mathbf{1})^*\mathbf{0}^*).$$

The regular expression is $(\mathbf{0} \cup \mathbf{10}^*\mathbf{1})^*$.

(c) G_6: From the productions $Z \to \boldsymbol{a}Z|\boldsymbol{b}T$, $T \to \boldsymbol{a}Z$. Language equations: $Z = \boldsymbol{a}Z \cup \boldsymbol{b}T$ and $T = \boldsymbol{a}Z$. Substituting: $Z = \boldsymbol{a}Z \cup \boldsymbol{ba}Z = (\boldsymbol{a} \cup \boldsymbol{ba})Z$. Since there is no terminal production (no production $Z \to w$ or $T \to w$ for $w \in \Sigma^*$), $L(G_6) = \emptyset$.

Every derivation cycles: $Z \Rightarrow \boldsymbol{b}T \Rightarrow \boldsymbol{ba}Z \Rightarrow \cdots$, and neither Z nor T has a terminal production, confirming $L(G_6) = \emptyset$.

(d) G_7: $S \to \boldsymbol{a}S|\boldsymbol{ab}B|\boldsymbol{c}C$, $B \to \boldsymbol{ab}B|\lambda$, $C \to \boldsymbol{c}C|\boldsymbol{ca}$.

- From B: $B = \boldsymbol{ab}B \cup \lambda$, so $L(B) = (\boldsymbol{ab})^*$.
- From C: $C = \boldsymbol{c}C \cup \boldsymbol{ca}$, so $L(C) = \boldsymbol{c}^*\boldsymbol{ca} = \boldsymbol{c}^+\boldsymbol{a}$.
- From S: $S = \boldsymbol{a}S \cup \boldsymbol{ab}B \cup \boldsymbol{c}C = \boldsymbol{a}S \cup \boldsymbol{ab}(\boldsymbol{ab})^* \cup \boldsymbol{c}^+\boldsymbol{a}$. $S = \boldsymbol{a}S \cup \boldsymbol{a}(\boldsymbol{ab})^+ \cup \boldsymbol{c}^+\boldsymbol{a}$, so $L(S) = \boldsymbol{a}^*(\boldsymbol{a}(\boldsymbol{ab})^+ \cup \boldsymbol{c}^+\boldsymbol{a})$.

The regular expression is $\boldsymbol{a}^+(\boldsymbol{ab})^+ \cup \boldsymbol{a}^*\boldsymbol{c}^+\boldsymbol{a}$.

8.4. **Use Exercise 8.2 to formally prove Lemma 8.1.**

Lemma 8.1 states that for the grammar G_A constructed from DFA A, $L(G_A) = L(A)$.

By the inductive result of Exercise 8.2, for all $x \in \Sigma^*$ and all states t_i, t_j: $t_i \overset{*}{\Rightarrow} xt_j$ in G_A iff $\overline{\delta}_A(t_i, x) = t_j$. Taking $t_i = t_0$ (start state):

$x \in L(G_A)$ iff $t_0 \overset{*}{\Rightarrow} x$ in G_A iff $(\exists t_j \in F)$ $t_0 \overset{*}{\Rightarrow} xt_j \Rightarrow x$ iff $(\exists t_j \in F)$ $\overline{\delta}_A(t_0, x) = t_j$ iff $\overline{\delta}_A(t_0, x) \in F$ iff $x \in L(A)$. □

8.5. **Draw the automata for the grammars in Exercise 8.3.**

The automata are constructed by treating each nonterminal as a state, each terminal production $A \to w$ as a transition to the implicit final state, and each production $A \to wB$ as a transition labeled w from state A to state B.

(a) For G_4: States are $\{S, A, B, C, V, W, X, f\}$ where f is the (implicit) final/accepting state. Transitions include $S \xrightarrow{\boldsymbol{ab}} A$, $S \xrightarrow{\boldsymbol{bb}} B$, $S \xrightarrow{\boldsymbol{cc}} V$, $A \xrightarrow{\boldsymbol{b}} C$, $A \xrightarrow{\boldsymbol{c}} X$, $B \xrightarrow{\boldsymbol{ab}} f$, $C \xrightarrow{\boldsymbol{c}} S$ (loops), $C \xrightarrow{\lambda} f$, $V \xrightarrow{\boldsymbol{a}} V$ (self-loop), $V \xrightarrow{\boldsymbol{c}} X$, $W \xrightarrow{\boldsymbol{aa}} f$, $W \xrightarrow{\boldsymbol{a}} W$, $X \xrightarrow{\boldsymbol{b}} V$, $X \xrightarrow{\boldsymbol{aa}} X$.

(b) For G_5: States $\{S_0, S_1, S_2\}$, start S_0, final $\{S_0\}$ (since $S_0 \to \lambda$). Transitions: $S_0 \xrightarrow{\mathbf{0}} S_2$, $S_0 \xrightarrow{\mathbf{1}} S_1$, $S_1 \xrightarrow{\mathbf{0}} S_1$, $S_1 \xrightarrow{\mathbf{1}} S_2$, $S_2 \xrightarrow{\mathbf{0}} S_2$, $S_2 \xrightarrow{\mathbf{1}} S_0$.

(c) For G_6: States $\{Z, T\}$. Transitions: $Z \xrightarrow{\boldsymbol{a}} Z$, $Z \xrightarrow{\boldsymbol{b}} T$, $T \xrightarrow{\boldsymbol{a}} Z$. No final states (no terminal productions). $L(G_6) = \emptyset$.

(d) For G_7: States $\{S, B, C\}$, with implicit final state f. Transitions: $S \xrightarrow{\boldsymbol{a}} S$, $S \xrightarrow{\boldsymbol{ab}} B$, $S \xrightarrow{\boldsymbol{c}} C$, $B \xrightarrow{\boldsymbol{ab}} B$, $B \xrightarrow{\lambda} f$, $C \xrightarrow{\boldsymbol{c}} C$, $C \xrightarrow{\boldsymbol{ca}} f$.

8.6. **Right-linear grammars for the given languages.**

(a) All words in $\{\boldsymbol{a},\boldsymbol{b},\boldsymbol{c}\}^*$ *not* containing two consecutive $\boldsymbol{b}$s:

$$G = \langle \{S, A\}, \{\boldsymbol{a},\boldsymbol{b},\boldsymbol{c}\}, S, \{S \to \boldsymbol{a}S|\boldsymbol{b}A|\boldsymbol{c}S|\lambda, A \to \boldsymbol{a}S|\boldsymbol{c}S|\lambda\}\rangle$$

State S = "last symbol was not $\boldsymbol{b}$" (or start); state A = "last symbol was $\boldsymbol{b}$". From A, reading another $\boldsymbol{b}$ is disallowed.

(b) All words *containing* two consecutive $\boldsymbol{b}$s:

$$G = \langle \{S, A, B\}, \{\boldsymbol{a},\boldsymbol{b},\boldsymbol{c}\}, S, \{S \to \boldsymbol{a}S|\boldsymbol{c}S|\boldsymbol{b}A, A \to \boldsymbol{b}B|\boldsymbol{a}S|\boldsymbol{c}S, B \to \boldsymbol{a}B|\boldsymbol{b}B|\boldsymbol{c}B|\lambda\}\rangle$$

(c) All words with $|w|_{\boldsymbol{a}} = |w|_{\boldsymbol{b}}$: This language is not regular (it includes $\{\boldsymbol{a}^n\boldsymbol{b}^n\}$), so no right-linear grammar exists.

(d) All words with an even number of $\boldsymbol{a}$s:

$$G = \langle \{S, A\}, \{\boldsymbol{a},\boldsymbol{b},\boldsymbol{c}\}, S, \{S \to \boldsymbol{b}S|\boldsymbol{c}S|\boldsymbol{a}A|\lambda, A \to \boldsymbol{b}A|\boldsymbol{c}A|\boldsymbol{a}S\}\rangle$$

(e) All words *not* ending in $\boldsymbol{b}$:

$$G = \langle \{S, A\}, \{\boldsymbol{a},\boldsymbol{b},\boldsymbol{c}\}, S, \{S \to \boldsymbol{a}S|\boldsymbol{b}A|\boldsymbol{c}S|\boldsymbol{a}|\boldsymbol{c}|\lambda, A \to \boldsymbol{a}S|\boldsymbol{b}A|\boldsymbol{c}S|\boldsymbol{a}|\boldsymbol{c}\}\rangle$$

(Equivalently, $G = \langle \{S, A\}, \ldots, \{S \to (\boldsymbol{a}|\boldsymbol{b}|\boldsymbol{c})^*(\boldsymbol{a}|\boldsymbol{c})|\lambda\}\rangle$.)

(f) All words containing no $\boldsymbol{c}$s:

$$G = \langle \{S\}, \{\boldsymbol{a},\boldsymbol{b},\boldsymbol{c}\}, S, \{S \to \boldsymbol{a}S|\boldsymbol{b}S|\lambda\}\rangle$$

8.7. **Left-linear grammars for the languages in Exercise 8.6.**

A left-linear grammar has productions of the form $A \to w$ or $A \to Bw$.

(a) No two consecutive $\boldsymbol{b}$s:

$$G = \langle \{S, A\}, \{\boldsymbol{a},\boldsymbol{b},\boldsymbol{c}\}, S, \{S \to S\boldsymbol{a}|S\boldsymbol{c}|A\boldsymbol{b}|\lambda, A \to S\boldsymbol{a}|S\boldsymbol{c}|A\boldsymbol{b}\cdot(\text{blocked})\}\rangle$$

More carefully: S = words not ending in $\boldsymbol{b}$; A = words ending in exactly one $\boldsymbol{b}$ (last char is $\boldsymbol{b}$, but second-to-last is not $\boldsymbol{b}$). Productions: $S \to S\boldsymbol{a}|S\boldsymbol{c}|A\boldsymbol{a}|A\boldsymbol{c}|\lambda$, $A \to S\boldsymbol{b}$.

(b) Containing two consecutive $\boldsymbol{b}$s: $S \to S\boldsymbol{a}|S\boldsymbol{b}|S\boldsymbol{c}|A\boldsymbol{b}$, $A \to B\boldsymbol{b}$, $B \to B\boldsymbol{a}|B\boldsymbol{b}|B\boldsymbol{c}|\lambda$.

(c) Even number of $\boldsymbol{a}$s: $S \to S\boldsymbol{b}|S\boldsymbol{c}|A\boldsymbol{a}|\lambda$, $A \to S\boldsymbol{a}|A\boldsymbol{b}|A\boldsymbol{c}$.

(d) Not ending in $\boldsymbol{b}$: $S \to S\boldsymbol{a}|S\boldsymbol{c}|A\boldsymbol{a}|A\boldsymbol{c}|\lambda$, $A \to S\boldsymbol{b}|A\boldsymbol{b}$.

(e) No $\boldsymbol{c}$s: $S \to S\boldsymbol{a}|S\boldsymbol{b}|\lambda$.

8.8. **Complete the inductive portion of the proof of Theorem 8.8.**

Theorem 8.8 concerns the construction of the Kleene closure grammar $G_* = \langle \Omega \cup \{Z\}, \Sigma, Z, P \cup \{Z \to S, Z \to ZZ, Z \to \lambda\}\rangle$ for a right-linear grammar G. We prove by induction on derivation length that $L(G_*) = L(G)^*$.

Claim: $w \in L(G_*)$ iff $w \in L(G)^*$.

($\Rightarrow$) By induction on the derivation $Z \overset{*}{\Rightarrow} w$ in G_*: if the first production is $Z \to \lambda$ then $w = \lambda \in L(G)^*$. If $Z \to S \overset{*}{\Rightarrow} w$ then $w \in L(G) \subseteq L(G)^*$. If $Z \to ZZ \overset{*}{\Rightarrow} w$, then by IH both halves are in $L(G)^*$, so $w \in L(G)^*$.

($\Leftarrow$) If $w = w_1 w_2 \cdots w_k$ with each $w_i \in L(G)$, then $Z \Rightarrow ZZ \Rightarrow ZZZ \Rightarrow \cdots \overset{*}{\Rightarrow} w_1 w_2 \cdots w_k$ by induction on k. □

8.9. **Complete the inductive portion of the proof of Theorem 8.5.**

Theorem 8.5 concerns the Kleene closure. The key inductive claim is: for any $n \geq 0$ and $w \in L(G)^n$, $Z \stackrel{*}{\Rightarrow} w$ in G_*.

Base ($n = 0$): $w = \lambda$ and $Z \to \lambda$.

Step: If $w = uv$ with $u \in L(G)$ and $v \in L(G)^{n-1}$, then by IH $Z \stackrel{*}{\Rightarrow} v$ and (from grammar G) $S \stackrel{*}{\Rightarrow} u$. Thus $Z \Rightarrow ZZ \stackrel{*}{\Rightarrow} vZ \stackrel{*}{\Rightarrow} vu$... (carefully ordering): $Z \Rightarrow ZZ$ where the left $Z \stackrel{*}{\Rightarrow} u$ and the right $Z \stackrel{*}{\Rightarrow} v$ (by IH). □

8.10. **Use the efficient algorithm of Theorem 8.3 to find regular expressions for $L(G_5)$, $L(G_6)$, $L(G_7)$ in Exercise 8.3.**

The efficient algorithm eliminates one variable at a time using Arden's lemma.

- G_5: Equations: $S_0 = \mathbf{0}S_2 \cup \mathbf{1}S_1 \cup \lambda$, $S_1 = \mathbf{0}S_1 \cup \mathbf{1}S_2$, $S_2 = \mathbf{0}S_2 \cup \mathbf{1}S_0$. Solve for S_2 from third equation: $S_2 = \mathbf{0}^*\mathbf{1}S_0$. Substitute into S_1: $S_1 = \mathbf{0}S_1 \cup \mathbf{10}^*\mathbf{1}S_0$, so $S_1 = \mathbf{0}^*\mathbf{10}^*\mathbf{1}S_0$. Substitute S_1 and S_2 into S_0: $S_0 = \mathbf{0} \cdot \mathbf{0}^*\mathbf{1}S_0 \cup \mathbf{1} \cdot \mathbf{0}^*\mathbf{10}^*\mathbf{1}S_0 \cup \lambda = (\mathbf{0}^+\mathbf{1} \cup \mathbf{10}^*\mathbf{10}^*\mathbf{1})S_0 \cup \lambda$. By Arden's lemma: $L(G_5) = (\mathbf{0}^+\mathbf{1} \cup \mathbf{10}^*\mathbf{10}^*\mathbf{1})^*$. Simplified: $(\mathbf{0} \cup \mathbf{10}^*\mathbf{1})^*$.
- G_6: Equations: $Z = \boldsymbol{a}Z \cup \boldsymbol{b}T$, $T = \boldsymbol{a}Z$. Substitute: $Z = \boldsymbol{a}Z \cup \boldsymbol{ba}Z = (\boldsymbol{a} \cup \boldsymbol{ba})Z \cup \emptyset$. By Arden's lemma: $Z = (\boldsymbol{a} \cup \boldsymbol{ba})^* \cdot \emptyset = \emptyset$. So $L(G_6) = \emptyset$.
- G_7: Equations: $S = \boldsymbol{a}S \cup \boldsymbol{ab}B \cup \boldsymbol{c}C$, $B = \boldsymbol{ab}B \cup \lambda$, $C = \boldsymbol{c}C \cup \boldsymbol{ca}$. Solve $B = (\boldsymbol{ab})^*$. Solve $C = \boldsymbol{c}^*\boldsymbol{ca}$. Substitute: $S = \boldsymbol{a}S \cup \boldsymbol{ab}(\boldsymbol{ab})^* \cup \boldsymbol{c} \cdot \boldsymbol{c}^*\boldsymbol{ca} = \boldsymbol{a}S \cup \boldsymbol{a}(\boldsymbol{ab})^+ \cup \boldsymbol{c}^+\boldsymbol{a}$. By Arden's: $L(G_7) = \boldsymbol{a}^*(\boldsymbol{a}(\boldsymbol{ab})^+ \cup \boldsymbol{c}^+\boldsymbol{a}) = \boldsymbol{a}^+(\boldsymbol{ab})^+ \cup \boldsymbol{a}^*\boldsymbol{c}^+\boldsymbol{a}$.

8.11. **Left-linear version of Theorem 8.3 and Lemmas 8.4–8.5.**

(a) For left-linear grammars, the language equations take the form $A = BA \cup w$ (variables appear on the right of concatenations). The restatement: if a language equation system for a left-linear grammar is $\{X_i = \bigcup_j X_j\alpha_{ji} \cup \beta_i\}$, then solve by right-side substitution.

(b) Lemma 8.4 restated for left-linear: If $X = XA \cup B$ (where A, B do not contain X), then $X = BA^*$. Lemma 8.5 restated: If $X = XA \cup Y$ and X does not appear in A or Y, then $X = YA^*$.

(c) For G_2 (Example 8.12), apply left-linear equations and solve using right-iteration (A^* on the right): The result is the same language, expressed with right-star. The regular expression for $L(G_2)$ is obtained by the symmetric procedure.

8.12. **Grammar Q for binary numbers with decimal point.**

(a) The NDFA corresponding to Q has states $\{I, F, h\}$ where h is the halt (final) state. Start state I, final state F (since $F \to \lambda$). Transitions: $I \xrightarrow{\mathbf{0}} I$, $I \xrightarrow{\mathbf{1}} I$, $I \xrightarrow{\mathbf{0.}} F$ (i.e., $I \xrightarrow{\mathbf{0}} \cdot \stackrel{\cdot}{\to} F$, but for an NDFA treating "**0.**" as a two-symbol transition), $I \xrightarrow{\mathbf{1.}} F$, $F \xrightarrow{\mathbf{0}} F$, $F \xrightarrow{\mathbf{1}} F$, $F \to h$ (final).
Properly, introduce an intermediate state D for "seen the dot": States $\{I, D, F\}$, start I, final $\{F\}$. $\delta(I,\mathbf{0}) = I$, $\delta(I,\mathbf{1}) = I$, $\delta(I,\mathbf{.}) = D$, $\delta(D,\mathbf{0}) = F$, $\delta(D,\mathbf{1}) = F$, $\delta(F,\mathbf{0}) = F$, $\delta(F,\mathbf{1}) = F$, $\delta(F,\lambda) =$ accept. (And $I \xrightarrow{\mathbf{0}} D$ and $I \xrightarrow{\mathbf{1}} D$ would model **0.** and **1.** productions.)

(b) Right-linear equations:

$$I = \mathbf{0}I \cup \mathbf{1}I \cup \mathbf{0.}F \cup \mathbf{1.}F$$
$$F = \lambda \cup \mathbf{0}F \cup \mathbf{1}F$$

(c) Solving: $F = (\mathbf{0} \cup \mathbf{1})^*$ (by Arden's on $F = \mathbf{0}F \cup \mathbf{1}F \cup \lambda$). Substituting into I: $I = (\mathbf{0} \cup \mathbf{1})I \cup (\mathbf{0} \cup \mathbf{1}).(\mathbf{0} \cup \mathbf{1})^*$. By Arden's: $I = (\mathbf{0} \cup \mathbf{1})^+.(\mathbf{0} \cup \mathbf{1})^*$.

8.13. **Right-linear grammars for the given regular expressions.**

(a) $(\boldsymbol{a} \cup \boldsymbol{b})\boldsymbol{c}^*(\boldsymbol{d} \cup (\boldsymbol{ab})^*)$:

$$G = \langle \{S, A, B, C\}, \{\boldsymbol{a}, \boldsymbol{b}, \boldsymbol{c}, \boldsymbol{d}\}, S, P \rangle$$
$$P = \{S \to \boldsymbol{a}A | \boldsymbol{b}A,\ A \to \boldsymbol{c}A | \boldsymbol{d} | \boldsymbol{a}B | \lambda,\ B \to \boldsymbol{b}C,\ C \to \boldsymbol{a}B | \lambda\}$$

State A handles $\boldsymbol{c}^*$ and the choice of $\boldsymbol{d}$ or $(\boldsymbol{ab})^*$; state B after seeing $\boldsymbol{a}$ in the $(\boldsymbol{ab})^*$ part; state C after $\boldsymbol{ab}$.

(b) $(\boldsymbol{a} \cup \boldsymbol{b})^*\boldsymbol{a}(\boldsymbol{a} \cup \boldsymbol{b})^*$ (strings containing at least one $\boldsymbol{a}$):

$$G = \langle \{S, A\}, \{\boldsymbol{a}, \boldsymbol{b}\}, S, \{S \to \boldsymbol{a}S | \boldsymbol{b}S | \boldsymbol{a}A,\ A \to \boldsymbol{a}A | \boldsymbol{b}A | \lambda\} \rangle$$

8.14. **Left-linear grammars for the same expressions.**

(a) $(\boldsymbol{a} \cup \boldsymbol{b})\boldsymbol{c}^*(\boldsymbol{d} \cup (\boldsymbol{ab})^*)$: Reverse the order of derivation.

$$G = \langle \{S, A, B\}, \{\boldsymbol{a}, \boldsymbol{b}, \boldsymbol{c}, \boldsymbol{d}\}, S, \{S \to A\boldsymbol{d} | B | \boldsymbol{a} | \boldsymbol{b},\ A \to A\boldsymbol{c} | \boldsymbol{a} | \boldsymbol{b},\ B \to B\boldsymbol{ab} | \boldsymbol{a} | \boldsymbol{b}\} \rangle$$

Here A generates prefixes of $(\boldsymbol{a} \cup \boldsymbol{b})\boldsymbol{c}^*\boldsymbol{d}$ and B generates $(\boldsymbol{a} \cup \boldsymbol{b})\boldsymbol{c}^*(\boldsymbol{ab})^*$.

(b) $(\boldsymbol{a} \cup \boldsymbol{b})^*\boldsymbol{a}(\boldsymbol{a} \cup \boldsymbol{b})^*$:

$$G = \langle \{S, A\}, \{\boldsymbol{a}, \boldsymbol{b}\}, S, \{S \to S\boldsymbol{a} | S\boldsymbol{b} | A\boldsymbol{a},\ A \to A\boldsymbol{a} | A\boldsymbol{b} | \lambda\} \rangle$$

8.15. **Algorithm to convert right-linear to left-linear grammar.**

(a) **Algorithm**: 1. From the right-linear grammar G, construct the DFA A_G (Lemma 8.1). 2. Reverse all transitions in A_G to get A^r: swap initial and final states, reverse all arcs. 3. The reversed automaton A^r accepts $L(G)^r$. 4. Construct a left-linear grammar G^r from A^r. 5. The language $L(G^r) = L(G)^r$.

Alternatively, to get a grammar for $L(G)$ (not $L(G)^r$), apply the reversal twice: reverse G to get G' for $L(G)^r$, then reverse again.

(b) Applying to G_4: Construct the automaton, reverse it, derive the left-linear grammar. The result is a left-linear grammar with nonterminals corresponding to states of the reversed DFA and productions of the form $A \to Bw$ or $A \to w$.

8.16. **Algorithm to convert a regular grammar to one generating the complement.**

(a) Construct the DFA A from grammar G (via Lemma 8.1 or 8.2).

(b) Compute the complement DFA A^c by swapping final and nonfinal states (after making A complete/total).

(c) Construct a right-linear grammar G^c from A^c. Then $L(G^c) = \overline{L(G)}$.

8.17. **Algorithm to determine whether $L(G) = \emptyset$ for a regular grammar G.**

(a) Build the automaton A_G from G.

(b) Compute the set of states reachable from the start state (BFS/DFS from start state).

(c) If no final state is reachable, $L(G) = \emptyset$; otherwise $L(G) \neq \emptyset$.

This terminates since the automaton has finitely many states.

8.18. **Algorithm to determine whether $L(G)$ is infinite for a regular grammar G.**

(a) Build the automaton A_G from G.

(b) Find all states reachable from the start state.

(c) Find all states from which a final state is reachable.

(d) Check whether the intersection of these two sets (the "useful" states) contains any cycle (e.g., by DFS looking for back edges).

(e) $L(G)$ is infinite iff a cycle exists among the useful states.

8.19. **Algorithm to determine whether two right-linear grammars G_1 and G_2 generate the same language.**

(a) Construct DFAs A_1 and A_2 from G_1 and G_2 respectively.

(b) Compute the minimized DFA for each.

(c) Check isomorphism of the two minimized DFAs (compare state by state).

(d) $L(G_1) = L(G_2)$ iff the minimal DFAs are isomorphic.

8.20. **Determine $L(H)$ for grammar H.**

The grammar is $H = \langle \{A, B, S\}, \{\boldsymbol{a}, \boldsymbol{b}, \boldsymbol{c}\}, S, \{S \to \boldsymbol{a}SB\boldsymbol{c}, S \to \lambda, SB \to \boldsymbol{b}S, \boldsymbol{c}B \to B\boldsymbol{c}\}\rangle$.

This is a context-sensitive grammar. We trace derivations:

$$S \Rightarrow \lambda$$
$$S \Rightarrow \boldsymbol{a}SB\boldsymbol{c} \Rightarrow \boldsymbol{a}B\boldsymbol{c} \Rightarrow \boldsymbol{a}B\boldsymbol{c}$$

But $SB \to \boldsymbol{b}S$ requires S immediately before B: $\boldsymbol{a}SB\boldsymbol{c} \Rightarrow \boldsymbol{ab}S\boldsymbol{c} \Rightarrow \boldsymbol{abc}$.

$$S \Rightarrow \boldsymbol{a}SB\boldsymbol{c} \Rightarrow \boldsymbol{aa}SB\boldsymbol{c}B\boldsymbol{c} \Rightarrow \boldsymbol{aa}SB\boldsymbol{c}B\boldsymbol{c}$$

Using $SB \to \boldsymbol{b}S$: $\boldsymbol{aab}S\boldsymbol{c}B\boldsymbol{c}$. Using $\boldsymbol{c}B \to B\boldsymbol{c}$: $\boldsymbol{aab}SB\boldsymbol{cc}$. Using $SB \to \boldsymbol{b}S$: $\boldsymbol{aabb}S\boldsymbol{cc} \Rightarrow \boldsymbol{aabbcc}$. In general: $L(H) = \{\boldsymbol{a}^n\boldsymbol{b}^n\boldsymbol{c}^n \mid n \geq 0\}$.

8.21. **What is wrong with the concatenation construction $G^\bullet = \langle \Omega_1 \cup \Omega_2 \cup \{Z\}, \Sigma, Z, P_1 \cup P_2 \cup \{Z \to S_1 \cdot S_2\}\rangle$?**

The production $Z \to S_1 S_2$ has two nonterminals on the right-hand side. In a right-linear grammar, all productions must have the form $A \to w$ or $A \to wB$ (a string of terminals followed by at most one nonterminal). The production $S_1 S_2$ violates this since S_1 is a nonterminal followed by S_2 (also nonterminal), with no terminal prefix. This is a context-free production, not a right-linear one, so the resulting grammar is not right-linear and cannot directly represent a language in $\mathfrak{G}_\Sigma$.

8.22. **Prove $\mathfrak{G}_\Sigma$ is closed under concatenation.**

(a) Construction: Let $G_1 = \langle \Omega_1, \Sigma, S_1, P_1 \rangle$ and $G_2 = \langle \Omega_2, \Sigma, S_2, P_2 \rangle$ be right-linear grammars with $\Omega_1 \cap \Omega_2 = \emptyset$. Define:

$$G^\bullet = \langle \Omega_1 \cup \Omega_2, \Sigma, S_1, P^\bullet \rangle$$

where $P^\bullet$ is obtained from P_1 by replacing each terminal production $A \to w$ (where $A \in \Omega_1$) with $A \to wS_2$, plus all productions from P_2 unchanged.

(b) Proof that $L(G^\bullet) = L(G_1) \cdot L(G_2)$: If $u \in L(G_1)$ and $v \in L(G_2)$, then $S_1 \overset{*}{\Rightarrow} u$ in G_1 via some derivation ending with $A \to u'$ (terminal production). In $G^\bullet$, this becomes $S_1 \overset{*}{\Rightarrow} u'S_2 \overset{*}{\Rightarrow} u'v = uv$. Conversely, any derivation in $G^\bullet$ must use rules from P_1 until reaching a production $A \to wS_2$, producing some prefix, then switch to P_2. □

8.23. **Construct a λ-NDFA A' with one start and one final state from an NDFA A without λ-transitions.**

Let $A = \langle \Sigma, S, s_0, \delta, F \rangle$ be an NDFA without λ-transitions.

Construction of the right-linear grammar G_A from A: Each state is a nonterminal; start state s_0 is start symbol; for each transition $\delta(s, a) \ni t$, add production $s \to \boldsymbol{a}t$; for each $s \in F$, add production $s \to \lambda$.

Apply Kleene's theorem (Chapter 6 constructions) to G_A to build a λ-NDFA A' with exactly one start state and one final state such that $L(A') = L(G_A) = L(A)$.

8.24. **Complete the proof of Lemma 8.2.**

(a) Inductive statement: For all $n \geq 0$ and all $x \in \Sigma^n$ and all nonterminals A: $A \overset{*}{\Rightarrow} x$ in G iff $\overline{\delta}_A(A, x) \in F$.

(b) Proof by induction on n: **Base** ($n = 0$): $A \overset{*}{\Rightarrow} \lambda$ iff $A \to \lambda \in P$ iff $A \in F$ in the NDFA iff $\overline{\delta}(A, \lambda) = A \in F$. **Step**: Assume the result holds for length n. For $x = ay$ ($|y| = n$): $A \overset{*}{\Rightarrow} ay$ iff $(\exists B)$ $A \to \boldsymbol{a}B \in P$ and $B \overset{*}{\Rightarrow} y$ iff $(\exists B)$ $B \in \delta(A, \boldsymbol{a})$ and (by IH) $\overline{\delta}(B, y) \in F$ iff $\overline{\delta}(A, ay) \in F$. □

8.25. **Complete the proof of Lemma 8.3.**

(a) Inductive statement: For all $n \geq 0$ and $x \in \Sigma^n$ and nonterminal A: $A \overset{*}{\Rightarrow} x$ in the left-linear grammar G iff the reversed string x^r leads from the start state to A in the reversed automaton.

(b) Proof by induction on $|x|$, symmetric to Exercise 8.23. □

8.26. **Reasons for the assertions in the second half of the proof of Theorem 8.2.**

Theorem 8.2 establishes the equivalence between right-linear grammars and DFAs. The second half (DFA ⇒ grammar) uses the construction of G_A from A and verifies $L(A) = L(G_A)$ by applying Lemma 8.1, whose proof relies on Exercise 8.2. Each step in the proof follows from the definitions of transitions, derivations, and the inductive correspondence proved in Exercise 8.2. The final state condition in A corresponds exactly to a terminal production in G_A.

8.27. **Formal inductive proofs.**

(a) $L(G_1) = \{\boldsymbol{a}^n\boldsymbol{baa} \mid n \in \mathbb{N}\}$ (Example 8.7):

Claim: For all $n \geq 0$, $S \overset{*}{\Rightarrow} \boldsymbol{a}^n\boldsymbol{baa}$ in G_1. **Base** ($n = 0$): $S \Rightarrow \boldsymbol{baa}$ (production $S \to \boldsymbol{baa}$). **Step**: If $S \overset{*}{\Rightarrow} \boldsymbol{a}^n\boldsymbol{baa}$, then $S \Rightarrow \boldsymbol{a}S \overset{*}{\Rightarrow} \boldsymbol{a} \cdot \boldsymbol{a}^n\boldsymbol{baa} = \boldsymbol{a}^{n+1}\boldsymbol{baa}$. Conversely, any derivation from S either uses $S \to \boldsymbol{baa}$ (giving $\lambda \cdot \boldsymbol{baa}$) or $S \to \boldsymbol{a}S$ (giving a longer string by IH). So $L(G_1) = \{\boldsymbol{a}^n\boldsymbol{baa}\}$. □

(b) $L(G_8) = \{\boldsymbol{a}^n\boldsymbol{baa} \mid n \in \mathbb{N}\}$ (Example 8.9): The proof is analogous, using the inductive structure of the left-linear grammar G_8. □

8.28. **Mixed grammar C with both right-linear and left-linear productions.**

$C = \langle\{S, A, B\}, \{\mathbf{0}, \mathbf{1}\}, S, \{S \to \mathbf{0}A|\mathbf{1}B|\mathbf{0}|\mathbf{1}|\lambda, A \to S\mathbf{0}, B \to S\mathbf{1}\}\rangle$.

(a) $L(C) = \{w \in \{\mathbf{0}, \mathbf{1}\}^* \mid w = w^r\}$ — the set of all palindromes over $\{\mathbf{0}, \mathbf{1}\}$.
Derivation: $S \Rightarrow \mathbf{0}A \Rightarrow \mathbf{0}S\mathbf{0}$. Each step wraps the current sentential form between matching symbols. The grammar generates all even-length and odd-length palindromes.

(b) Yes, $L(C) = \{w \mid w = w^r\}$ is not regular (it requires unbounded memory to match the first and last symbols). So $L(C)$ is FAD only for the trivial case; in general, $L(C)$ is not FAD.

(c) The definition of regular grammars should *not* be expanded to include such mixed grammars, because the resulting class of languages would no longer coincide with the regular languages. Mixed grammars generate languages (like palindromes) that are context-free but not regular.

8.29. **Importance of $\Omega_1 \cap \Omega_2 = \emptyset$ in Theorem 8.4.**

(a) **Why important**: If $\Omega_1 \cap \Omega_2 \neq \emptyset$, then a shared nonterminal A would have its productions from both P_1 and P_2 merged. Derivations starting from S_1 could use P_2 productions for A and vice versa, producing strings not in $L(G_1) \cup L(G_2)$. **Example**: Let G_1 have $\{S_1 \to \boldsymbol{a}A, A \to \boldsymbol{b}\}$ and G_2 have $\{S_2 \to \boldsymbol{c}A, A \to \boldsymbol{d}\}$. If A is shared, the union grammar with $Z \to S_1|S_2$ would derive $\boldsymbol{a}A$ and then use $A \to \boldsymbol{d}$ from G_2, producing $\boldsymbol{ad} \notin L(G_1) \cup L(G_2)$.

(b) **Why possible**: We can always rename nonterminals in G_2 (choosing fresh names) to make $\Omega_1 \cap \Omega_2 = \emptyset$ without changing the language $L(G_2)$. This is a standard renaming/relabeling argument.

8.30. **If A_G is disconnected, what does this say about G?**

If A_G (the NDFA constructed from G) is disconnected, it means some states (nonterminals) are not reachable from the start state. This means the corresponding nonterminals in G are *useless*: they cannot appear in any derivation from the start symbol. The portions of G defining those nonterminals contribute nothing to $L(G)$ and can be removed without affecting the language generated.

8.31. **Apply Lemma 8.1 to the automata in Figure 8.4.**

For each DFA in Figure 8.4, Lemma 8.1 yields a right-linear grammar by treating each state q_i as a nonterminal Q_i, adding a production $Q_i \to \boldsymbol{a}Q_j$ for each transition $\delta(q_i, \boldsymbol{a}) = q_j$, and adding $Q_i \to \lambda$ for each final state q_i.

(Without the figures, the construction follows this template systematically for each automaton shown.)

8.32. **Restate Lemma 8.1 for NDFAs.**

(a) **Restated Lemma**: Given an NDFA $A = \langle\Sigma, S, s_0, \delta, F\rangle$, define a right-linear grammar $G_A = \langle S, \Sigma, s_0, P\rangle$ where for each $t \in \delta(s, a)$ add production $s \to \boldsymbol{a}t$, and for each $s \in F$ add $s \to \lambda$. Then $L(G_A) = L(A)$.

(b) **Proof**: By induction on string length, using the fact that NDFA transitions correspond to sets of choices, and each path through the NDFA corresponds to a derivation in G_A.

(c) Applied to Figure 8.5: For each NDFA, apply the construction and produce the corresponding grammar with multiple productions for states with nondeterministic transitions.

8.33. **Context-free grammars for languages over $\Sigma = \{\boldsymbol{a}, \boldsymbol{b}\}$.**

(a) L_1 = words where last letter matches first letter:

$$G_1 : S \to \boldsymbol{a}X\boldsymbol{a} \mid \boldsymbol{b}X\boldsymbol{b} \mid \boldsymbol{a} \mid \boldsymbol{b} \mid \lambda, \quad X \to \boldsymbol{a}X \mid \boldsymbol{b}X \mid \lambda$$

(b) L_2 = odd-length words where first letter matches center letter: Let $n = 2k+1$.

$$G_2 : S \to \boldsymbol{a}X\boldsymbol{a}Y \mid \boldsymbol{b}X\boldsymbol{b}Y, \quad X \to \boldsymbol{a}X\boldsymbol{a} \mid \boldsymbol{b}X\boldsymbol{b} \mid \lambda, \quad Y \to \boldsymbol{a}Y \mid \boldsymbol{b}Y \mid \lambda$$

(Pairs up from both ends; X handles the center matching, Y handles the suffix.)

(c) L_3 = words where last letter matches none of the other letters:

$$G_3 : S \to X\boldsymbol{a} \mid X\boldsymbol{b}, \quad \text{where } X \text{ generates strings not ending in } \boldsymbol{a} \text{ (resp. } \boldsymbol{b})$$

Since $\Sigma = \{\boldsymbol{a}, \boldsymbol{b}\}$, "last letter matches none of the others" means all other letters differ. This is regular: last letter $\boldsymbol{a}$, all others $\boldsymbol{b}$; or last letter $\boldsymbol{b}$, all others $\boldsymbol{a}$: $L_3 = \boldsymbol{b}^*\boldsymbol{a} \cup \boldsymbol{a}^*\boldsymbol{b}$. So $G_3 : S \to B\boldsymbol{a} \mid A\boldsymbol{b} \mid \boldsymbol{a} \mid \boldsymbol{b}$, $B \to B\boldsymbol{b} \mid \lambda$, $A \to A\boldsymbol{a} \mid \lambda$.

(d) L_4 = even-length words where two center letters match:

$$G_4 : S \to \boldsymbol{a}S\boldsymbol{a} \mid \boldsymbol{a}S\boldsymbol{b} \mid \boldsymbol{b}S\boldsymbol{a} \mid \boldsymbol{b}S\boldsymbol{b} \mid \boldsymbol{a}\boldsymbol{a} \mid \boldsymbol{b}\boldsymbol{b}$$

(e) L_5 = odd-length words where center letter matches none of the others: Similar to L_3; since $|\Sigma| = 2$, if center is $\boldsymbol{a}$ all others are $\boldsymbol{b}$, and vice versa. So $L_5 = \{\boldsymbol{b}^k\boldsymbol{a}\boldsymbol{b}^k\} \cup \{\boldsymbol{a}^k\boldsymbol{b}\boldsymbol{a}^k\}$, which is context-free but not regular:

$$G_5 : S \to \boldsymbol{b}S\boldsymbol{b} \mid \boldsymbol{a}T\boldsymbol{a} \mid \boldsymbol{a} \mid \boldsymbol{b}, \quad T \to \boldsymbol{a}T\boldsymbol{a} \mid \boldsymbol{b}$$

(f) Regular languages: L_1 is not regular (requires matching ends); L_2 is not regular; L_3 is regular ($\boldsymbol{b}^*\boldsymbol{a} \cup \boldsymbol{a}^*\boldsymbol{b}$); L_4 is not regular; L_5 is not regular.

8.34. **Context-free grammars for the given languages.**

(a) $L = \{x \in \{\boldsymbol{a}, \boldsymbol{b}\}^* \mid |x|_{\boldsymbol{a}} < |x|_{\boldsymbol{b}}\}$:

$$G : S \to \boldsymbol{b} \mid BS \mid SB \mid BSB, \quad B \to \boldsymbol{b} \mid \boldsymbol{a}B\boldsymbol{a} \mid \boldsymbol{b}B\boldsymbol{b}$$

More carefully: Use the grammar for balanced strings plus extra $\boldsymbol{b}$s. A cleaner version: $S \to SS \mid \boldsymbol{b}S\boldsymbol{a} \mid S\boldsymbol{b}\boldsymbol{a} \mid \boldsymbol{b}$.

(b) $G = \{x \in \{\boldsymbol{a}, \boldsymbol{b}\}^* \mid |x|_{\boldsymbol{a}} \geq |x|_{\boldsymbol{b}}\}$: $S \to SS \mid \boldsymbol{a} \mid \boldsymbol{a}S\boldsymbol{b} \mid \boldsymbol{a}S \mid \lambda$.

(c) $K = \{w \in \{\mathbf{0}, \mathbf{1}\}^* \mid w = w^r\}$ (palindromes): $S \to \mathbf{0}S\mathbf{0} \mid \mathbf{1}S\mathbf{1} \mid \mathbf{0} \mid \mathbf{1} \mid \lambda$.

(d) $\Phi = \{\boldsymbol{a}^j\boldsymbol{b}^k\boldsymbol{c}^m \mid j \geq 3, k = m\}$: $S \to \boldsymbol{a}\boldsymbol{a}\boldsymbol{a}AT$, $A \to \boldsymbol{a}A \mid \lambda$, $T \to \boldsymbol{b}T\boldsymbol{c} \mid \lambda$.

8.35. **More context-free grammars.**

(a) $L_1 = \{x \in \{\boldsymbol{a},\boldsymbol{b}\}^* \mid |x|_{\boldsymbol{a}} = 2|x|_{\boldsymbol{b}}\}$: $S \to \boldsymbol{aa}S\boldsymbol{b} \mid \boldsymbol{a}S\boldsymbol{ab} \mid S\boldsymbol{aab} \mid \lambda$ (or use $S \to SS \mid \boldsymbol{aab} \mid \boldsymbol{aba} \mid \boldsymbol{baa} \mid \lambda$).

(b) $L_2 = \{x \in \{\boldsymbol{a},\boldsymbol{b}\}^* \mid |x|_{\boldsymbol{a}} \neq |x|_{\boldsymbol{b}}\}$: $S \to A \mid B$, where A generates strings with more $\boldsymbol{a}$s than $\boldsymbol{b}$s ($A \to \boldsymbol{a} \mid \boldsymbol{a}A\boldsymbol{b} \mid \boldsymbol{a}A \mid A\boldsymbol{a} \mid AA$) and B generates strings with more $\boldsymbol{b}$s. Alternatively: $L_2 = \overline{L_{eq}}$ where $L_{eq} = \{|x|_{\boldsymbol{a}} = |x|_{\boldsymbol{b}}\}$.

(c) Postfix expressions over $\{\boldsymbol{A},\boldsymbol{B},\boldsymbol{+},\boldsymbol{-}\}$: $S \to \boldsymbol{A} \mid \boldsymbol{B} \mid S S\boldsymbol{+} \mid S S\boldsymbol{-}$.

(d) Parenthesized infix expressions over $\{\boldsymbol{A},\boldsymbol{B},\boldsymbol{+},\boldsymbol{-},\boldsymbol{(},\boldsymbol{)}\}$: $S \to \boldsymbol{A} \mid \boldsymbol{B} \mid \boldsymbol{(}S\boldsymbol{+}S\boldsymbol{)} \mid \boldsymbol{(}S\boldsymbol{-}S\boldsymbol{)}$.

8.36. **Context-sensitive grammars for the given languages.**

(a) $\Gamma = \{w\boldsymbol{2}w \mid w \in \{\boldsymbol{0},\boldsymbol{1}\}^*\}$: This is the language of strings of the form $w2w$ (copy language with separator). A context-sensitive grammar can be constructed by "moving" symbols past each other:

$$S \to \boldsymbol{0}S\boldsymbol{0} \mid \boldsymbol{1}S\boldsymbol{1} \mid \boldsymbol{2}$$

The grammar $S \to 0S0 \mid 1S1 \mid 2$ generates $\{w \cdot 2 \cdot w^r\}$, not $\Gamma = \{w \cdot 2 \cdot w\}$. Since Γ requires copying (not reversing), it is not context-free. A context-sensitive grammar synchronizes the two copies of w using marker symbols. Introduce nonterminals A_0, A_1 as "moving" copies:

$$\begin{aligned} S &\to \boldsymbol{2} \mid \boldsymbol{0}SC_0 \mid \boldsymbol{1}SC_1 \\ C_0\boldsymbol{2} &\to \boldsymbol{20} \quad \text{(move 0-copy past center)} \\ C_1\boldsymbol{2} &\to \boldsymbol{21} \\ C_0C_0 &\to C_0\boldsymbol{0}, \quad \text{etc.} \end{aligned}$$

The complete grammar is complex; the key idea is to generate w on the left, place a $\boldsymbol{2}$ marker, then use context-sensitive rules to copy and place w on the right.

(b) $\Phi = \{\boldsymbol{b}^{2^j} \mid j \in \mathbb{N}\} = \{\boldsymbol{b},\boldsymbol{bb},\boldsymbol{bbbb},\ldots\}$: A context-sensitive grammar doubles the string at each step: $S \to \boldsymbol{b} \mid \boldsymbol{b}Ab$, $A \to \boldsymbol{b}A\boldsymbol{b} \mid \boldsymbol{bb}$. The standard CSG uses "doubling" rules where a marker propagates through the string, doubling it:

$$\begin{aligned} S &\to \boldsymbol{b} \mid B \\ B &\to DB \\ DB &\to DD \\ D &\to \boldsymbol{bb} \end{aligned}$$

(This is a sketch; the full CSG uses left/right markers and $O(n)$ nonterminals.)

8.37. **Show G (context-sensitive) is not equivalent to G'' in Example 8.3.**

The grammar G has $SB \to \boldsymbol{b}S$ but G'' has $TB \to \boldsymbol{b}T$ with $S \to T$ and $T \to \lambda$. The difference is that G allows S to be rewritten to λ directly, while G'' uses T as an intermediary. In G, the production $S \to \lambda$ can fire while B's are still present (e.g., $\boldsymbol{a}SB\boldsymbol{c} \Rightarrow \boldsymbol{a}B\boldsymbol{c}$ by $S \to \lambda$, then $\boldsymbol{a}B\boldsymbol{c}$ is stuck since B cannot be eliminated without an S or c context). In G'', T can only go to λ when all B's have been converted.

Thus G may generate strings with leftover nonterminals (dead derivations), while G'' is designed to always complete. Formally: $\boldsymbol{aaSBBcc} \Rightarrow^* \boldsymbol{aaBBcc}$ in G (using $S \to \lambda$), which is stuck. In G'' this cannot happen since T only appears in the rule $TB \to \boldsymbol{b}T$. So G does not generate $\boldsymbol{aabbcc}$ in the same way, and the two grammars may differ on some strings.

8.38. **Context-free grammars for the given languages.**

(a) $L_1 = \boldsymbol{a}^*(\boldsymbol{b} \cup \boldsymbol{c})^* \cap \{x \mid |x|_{\boldsymbol{a}} = |x|_{\boldsymbol{b}} + |x|_{\boldsymbol{c}}\}$: Strings in L_1 are of the form $\boldsymbol{a}^n\boldsymbol{b}^j\boldsymbol{c}^k$ where $n = j + k$:

$$G_1 : S \to \boldsymbol{a}S\boldsymbol{b} \mid \boldsymbol{a}S\boldsymbol{c} \mid \lambda$$

Each step adds one $\boldsymbol{a}$ at the left and one $\boldsymbol{b}$ or $\boldsymbol{c}$ at the right.

(b) $L_2 = \{\boldsymbol{a}^i\boldsymbol{b}^j\boldsymbol{c}^k \mid i + j = k\}$:

$$G_2 : S \to \boldsymbol{a}S\boldsymbol{c} \mid \boldsymbol{b}T\boldsymbol{c} \mid \lambda, \quad T \to \boldsymbol{b}T\boldsymbol{c} \mid \lambda$$

(c) $L_3 = \{x \in \{\boldsymbol{a}, \boldsymbol{b}, \boldsymbol{c}\}^* \mid |x|_{\boldsymbol{a}} + |x|_{\boldsymbol{b}} = |x|_{\boldsymbol{c}}\}$:

$$G_3 : S \to \boldsymbol{a}S\boldsymbol{c} \mid \boldsymbol{b}S\boldsymbol{c} \mid SS \mid \lambda$$

8.39. **Prove $L(G') = L(G) \cup \{\lambda\}$ (Definition 8.4).**

Definition 8.4 constructs G' from G by adding a new start symbol Z with $Z \to S \mid \lambda$ (where S was the original start). Then: $x \in L(G')$ iff $Z \overset{*}{\Rightarrow} x$ in G'. $Z \to \lambda$ gives $\lambda \in L(G')$. $Z \to S \overset{*}{\Rightarrow} x$ gives $x \in L(G) \subseteq L(G')$. Conversely, any derivation from Z begins with either $Z \to \lambda$ or $Z \to S$, so $L(G') \subseteq L(G) \cup \{\lambda\}$. Hence $L(G') = L(G) \cup \{\lambda\}$. □

8.40. **Prove $L(G') = L(G) \cup \{\lambda\}$ (Definition 8.6).**

The proof is identical in structure to Exercise 8.40, using the specific construction given in Definition 8.6 (which adds a new start symbol with a lambda production). □

8.41. **Properties of the normal form-preserving construction.**

(a) If G is right-linear (Theorem 8.8), the naive star construction $Z \to S \mid ZZ \mid \lambda$ does *not* preserve right-linearity: $Z \to ZZ$ places a nonterminal in a non-final position, violating the form $A \to wB$ or $A \to w$. A right-linear Kleene-star construction avoids ZZ by folding the restart into the accepting productions of G: whenever $A \to w$ is a terminal production of G, add $A \to wS$ (to restart) and $A \to w$ (to halt). This keeps all productions right-linear.

(b) Example: $G_1 : S_1 \to \boldsymbol{a}$ and $G_2 : S_2 \to S_2\boldsymbol{b} \mid \boldsymbol{b}$. Both are in normal form (right-linear), but $G^{\cup}$: $Z \to S_1 \mid S_2$ combined may not be in the prescribed form if that form requires a unique start symbol structure.

(c) The $G^{\bullet}$ construction for concatenation in Example 8.18 is not normal form-preserving in general, since it requires replacing terminal productions $A \to w$ with $A \to wS_2$, which is a right-linear production but may violate other normal form constraints.

8.42. **Left-linear grammars for union, concatenation, and Kleene closure.**

Given left-linear grammars $G_1 = \langle \Omega_1, \Sigma, S_1, P_1 \rangle$ and $G_2 = \langle \Omega_2, \Sigma, S_2, P_2 \rangle$ with $\Omega_1 \cap \Omega_2 = \emptyset$:

(a) **Union**: $G^{\cup} = \langle \Omega_1 \cup \Omega_2 \cup \{Z\}, \Sigma, Z, P_1 \cup P_2 \cup \{Z \to S_1 \mid S_2\}\rangle$. The production $Z \to S_1$ is left-linear (since $Z \to S_1$ has a nonterminal on the right with no symbols; this is a unit production, which is allowed). $L(G^{\cup}) = L(G_1) \cup L(G_2)$.

(b) **Concatenation**: For left-linear grammars, to concatenate $L(G_1) \cdot L(G_2)$: replace each "initial" production $A \to w$ in G_1 (where A generates a terminal string) with $A \to S_2 w$ (prepend S_2 which generates $L(G_2)$ on the left). More precisely, for each production $S_1 \to w$ in G_1, replace with $S_1 \to S_2 w$ (left-linear). The resulting grammar generates $L(G_1) \cdot L(G_2)$.

(c) **Kleene closure**: Add new start Z with $Z \to Z S_1 \mid \lambda$. This is left-linear. $L(G_*) = L(G_1)^*$.

Chapter 9

Normal Forms and Pumping Lemmas — Solutions

9.1. **Characterize the parse trees of left-linear grammars.**

In a left-linear grammar, every production has the form $A \to w$ (terminal string) or $A \to Bw$ (nonterminal followed by a terminal string). Consequently, in any parse tree for a derivation in a left-linear grammar:

- The tree has a "left-spine": the leftmost child of any internal node is always a nonterminal, and the remaining children are terminal symbols.
- All nonterminals appear in the leftmost branch of the parse tree.
- The tree is maximally left-branching; at every level, the left subtree contains all nonterminal structure while the right siblings are terminals.

This is the mirror image of right-linear parse trees (which are maximally right-branching). The depth of the parse tree equals the number of nonterminals in the derivation.

9.2. **Context-free grammars for the following languages.**

(a) $\{\boldsymbol{a}^n\boldsymbol{b}^n\boldsymbol{c}^m\boldsymbol{d}^m \mid n, m \in \mathbb{N}\}$:

$$S \to AB, \quad A \to \boldsymbol{a}A\boldsymbol{b} \mid \lambda, \quad B \to \boldsymbol{c}B\boldsymbol{d} \mid \lambda.$$

(b) $\{\boldsymbol{a}^i\boldsymbol{b}^j\boldsymbol{c}^j\boldsymbol{d}^i \mid i, j \in \mathbb{N}\}$:

$$S \to \boldsymbol{a}S\boldsymbol{d} \mid T \mid \lambda, \quad T \to \boldsymbol{b}T\boldsymbol{c} \mid \lambda.$$

(c) $\{\boldsymbol{a}^n\boldsymbol{b}^n\boldsymbol{c}^m\boldsymbol{d}^m\} \cup \{\boldsymbol{a}^i\boldsymbol{b}^j\boldsymbol{c}^j\boldsymbol{d}^i\}$:

$$S \to S_1 \mid S_2$$

where S_1 generates the first language (as in part (a)) and S_2 generates the second (as in part (b)).

9.3. **Unambiguous context-free grammars.**

(a) i. For $\{\boldsymbol{a}^n\boldsymbol{b}^n\boldsymbol{c}^m\boldsymbol{d}^m\}$: The grammar $S \to AB$, $A \to \boldsymbol{a}A\boldsymbol{b} \mid \lambda$, $B \to \boldsymbol{c}B\boldsymbol{d} \mid \lambda$ is already unambiguous (every string has a unique parse).

ii. For $\{\boldsymbol{a}^i\boldsymbol{b}^j\boldsymbol{c}^j\boldsymbol{d}^i\}$: The grammar $S \to \boldsymbol{a}S\boldsymbol{d} \mid T \mid \lambda$, $T \to \boldsymbol{b}T\boldsymbol{c} \mid \lambda$ is unambiguous.

iii. For the union $L_1 \cup L_2$: Let $S \to S_1 \mid S_2$. The string $\boldsymbol{a}^n\boldsymbol{b}^n\boldsymbol{c}^n\boldsymbol{d}^n$ is in both L_1 (with $m = n$) and L_2 (with $i = j = n$), so any grammar must choose between two derivations for such strings. The language is inherently ambiguous at these strings if we require both sets in the union.

(b) $\mathfrak{U}_\Sigma$ is not closed under union: The union $L_1 \cup L_2$ may be inherently ambiguous even when L_1 and L_2 are individually unambiguous. The classic example is $L_1 = \{\boldsymbol{a}^i\boldsymbol{b}^j\boldsymbol{c}^k \mid i = j\}$ and $L_2 = \{\boldsymbol{a}^i\boldsymbol{b}^j\boldsymbol{c}^k \mid j = k\}$: both are unambiguous, but $L_1 \cup L_2$ is inherently ambiguous (the string $\boldsymbol{a}^n\boldsymbol{b}^n\boldsymbol{c}^n$ has two distinct parse trees).

(c) $\mathfrak{U}_\Sigma$ is *not* closed under union (as argued above).

9.4. **State and prove the inductive result needed in Theorem 9.2.**

Claim: If $G = \langle \Omega, \Sigma, S, P\rangle$ is a context-free grammar in which each nonterminal A has exactly one rule (the grammar is in single-rule form after Theorem 9.2's construction), then G is unambiguous.

The key inductive result for Theorem 9.2 (elimination of epsilon and unit productions for ambiguity resolution) is:

Lemma: For each nonterminal A and string x, the number of parse trees for x derivable from A is the same in G as in the constructed unambiguous grammar G'.

Proof: By induction on $|x|$. Base case $|x| = 0$: $A \overset{*}{\Rightarrow} \lambda$ iff $A \in N_\lambda$ (nullable nonterminals), and the construction handles this uniquely. Inductive step: use the structure of the production applied at the root of the parse tree. □

9.5. **Algorithm to compute $B^u = \{C \mid B \overset{*}{\Rightarrow} C\}$ for each nonterminal B in P^u.**

Consider the unit-production graph: nodes are nonterminals, directed edge $B \to C$ for each unit production $B \to C$ in P^u. Then B^u is the set of nonterminals reachable from B in this graph (including B itself).

Algorithm: For each nonterminal B, compute B^u by BFS or DFS from B in the unit-production graph:

(a) Initialize $B^u = \{B\}$.

(b) While there exists $C \in B^u$ with a unit production $C \to D \in P^u$ and $D \notin B^u$: add D to B^u.

(c) Repeat until no new nonterminals are added.

This is a standard graph reachability computation and terminates since $|\Omega|$ is finite.

9.6. **Closure of $\mathfrak{C}_\Sigma$ under concatenation.**

(a) **Problem with naive construction**: The production $Z \to S_1 \cdot S_2$ is fine as a context-free production, but the issue is with lambda productions. If $\lambda \in L(G_1)$ (i.e., $S_1 \overset{*}{\Rightarrow} \lambda$), then $Z \to S_1S_2 \overset{*}{\Rightarrow} S_2$, so $L(G^*) \supseteq L(G_2)$, not just $L(G_1)\cdot L(G_2)$ minus λ-contributions. Similarly, if $\lambda \in L(G_2)$, then $Z \overset{*}{\Rightarrow} S_1$, giving $L(G^*) \supseteq L(G_1)$. The issue: when both G_1 and G_2 derive λ, the simple construction may include unintended strings or exclude others due to improper lambda-production treatment.

(b) **Fix**: First transform G_1 and G_2 so that λ is not derivable from their respective start symbols (or handle it explicitly). If $\lambda \in L(G_i)$, add explicit productions:

$$Z \to S_1 S_2 \mid S_1 \mid S_2 \mid \lambda \quad \text{(appropriate subset based on whether } \lambda \in L(G_i)\text{)}$$

Specifically: add $Z \to S_2$ if $\lambda \in L(G_1)$; add $Z \to S_1$ if $\lambda \in L(G_2)$; add $Z \to \lambda$ if $\lambda \in L(G_1) \cap L(G_2)$.

(c) **Proof**: $w \in L(G^*_{\text{corrected}})$ iff $Z \overset{*}{\Rightarrow} w$ iff $w = uv$ with $u \in L(G_1)$ and $v \in L(G_2)$ (considering the λ cases via the added productions) iff $w \in L(G_1) \cdot L(G_2)$. □

9.7. $\{\boldsymbol{a}^i \boldsymbol{b}^j \boldsymbol{c}^k \mid i, j, k \in \mathbb{N},\ i + j = k\}$ **is context free.**

Grammar: $S \to \boldsymbol{a} S \boldsymbol{c} \mid T$, $T \to \boldsymbol{b} T \boldsymbol{c} \mid \lambda$.

Proof: Any derivation produces $\boldsymbol{a}^i \cdot \boldsymbol{b}^j \boldsymbol{c}^j \cdot \boldsymbol{c}^i = \boldsymbol{a}^i \boldsymbol{b}^j \boldsymbol{c}^{i+j}$. Taking $k = i + j$, we get exactly the language. □

9.8. **Ambiguous right-linear grammar and its disambiguation.**

(a) $G = \langle \{S, A, B\}, \{\boldsymbol{a}\}, S, \{S \to A, S \to B, A \to \boldsymbol{aa}A, A \to \lambda, B \to \boldsymbol{aaa}B, B \to \lambda\} \rangle$. The string $\boldsymbol{aaaaaa} = \boldsymbol{a}^6$ can be derived as $S \Rightarrow A \overset{*}{\Rightarrow} \boldsymbol{aa}A \overset{*}{\Rightarrow} \boldsymbol{aaaa}A \overset{*}{\Rightarrow} \boldsymbol{aaaaaa}$ (via A: three $\boldsymbol{aa}$ steps) or as $S \Rightarrow B \overset{*}{\Rightarrow} \boldsymbol{aaa}B \overset{*}{\Rightarrow} \boldsymbol{aaaaaa}$ (via B: two $\boldsymbol{aaa}$ steps). So G is ambiguous.

(b) $L(G) = \{w \mid |w| \equiv 0 \pmod 2\} \cup \{w \mid |w| \equiv 0 \pmod 3\} = \{w \mid 2 \mid |w| \text{ or } 3 \mid |w|\}$. To remove ambiguity, find the minimal DFA for $L(G)$ and construct the corresponding unambiguous right-linear grammar. The minimal DFA has states tracking $|w| \pmod 6$, with $n \in \{0, 2, 3, 4, 6 \equiv 0\}$ being accepting (length divisible by 2 or 3 = not $1, 5 \pmod 6$). This gives an unambiguous grammar.

9.9. **Grammar for regular expressions, modified.**

(a) Grammar that strips outermost parentheses:

$$R \to \boldsymbol{a} \mid \boldsymbol{b} \mid \boldsymbol{c} \mid \boldsymbol{\epsilon} \mid \boldsymbol{\emptyset} \mid E \cdot E \mid E \cup E \mid R^*$$

where $E \to (R)$ or E is any atomic expression.

(b) Grammar allowing extraneous parentheses:

$$R \to (R) \mid \boldsymbol{a} \mid \boldsymbol{b} \mid \boldsymbol{c} \mid \boldsymbol{\epsilon} \mid \boldsymbol{\emptyset} \mid (R \cdot R) \mid (R \cup R) \mid R^*$$

9.10. **Leftmost and rightmost derivations for $((\boldsymbol{a}^* \cdot \boldsymbol{b}) \cup (\boldsymbol{c} \cdot \boldsymbol{d})^*)$.**

(a) **Leftmost derivation** (always expand leftmost nonterminal):

$$\begin{aligned}
R &\Rightarrow (R \cup R) \\
&\Rightarrow ((R \cdot R) \cup R) \\
&\Rightarrow ((R^* \cdot R) \cup R) \\
&\Rightarrow ((\boldsymbol{a}^* \cdot R) \cup R) \\
&\Rightarrow ((\boldsymbol{a}^* \cdot \boldsymbol{b}) \cup R) \\
&\Rightarrow ((\boldsymbol{a}^* \cdot \boldsymbol{b}) \cup (R \cdot R)^*) \\
&\Rightarrow ((\boldsymbol{a}^* \cdot \boldsymbol{b}) \cup (\boldsymbol{c} \cdot R)^*) \\
&\Rightarrow ((\boldsymbol{a}^* \cdot \boldsymbol{b}) \cup (\boldsymbol{c} \cdot \boldsymbol{d})^*)
\end{aligned}$$

(b) **Rightmost derivation** (always expand rightmost nonterminal):

$$\begin{aligned} R &\Rightarrow (R \cup R) \\ &\Rightarrow (R \cup (R \cdot R)^*) \\ &\Rightarrow (R \cup (R \cdot \boldsymbol{d})^*) \\ &\Rightarrow (R \cup (\boldsymbol{c} \cdot \boldsymbol{d})^*) \\ &\Rightarrow ((R \cdot R) \cup (\boldsymbol{c} \cdot \boldsymbol{d})^*) \\ &\Rightarrow ((R \cdot \boldsymbol{b}) \cup (\boldsymbol{c} \cdot \boldsymbol{d})^*) \\ &\Rightarrow ((R^* \cdot \boldsymbol{b}) \cup (\boldsymbol{c} \cdot \boldsymbol{d})^*) \\ &\Rightarrow ((\boldsymbol{a}^* \cdot \boldsymbol{b}) \cup (\boldsymbol{c} \cdot \boldsymbol{d})^*) \end{aligned}$$

9.11. **Prove $L(G) = L(G')$ (Theorem 9.5) by induction on derivation steps.**

Theorem 9.5 concerns the elimination of lambda productions. Let G' be obtained from G by systematically handling lambda productions.

Claim: For all $n \geq 0$, if $S \overset{*}{\Rightarrow}_G x$ in n steps with $x \in \Sigma^*$, then $S \overset{*}{\Rightarrow}_{G'} x$, and vice versa.

Proof (by induction on number of steps): **Base**: 0 steps: $x = \lambda$ (if S is nullable); handled by construction of G' which adds $Z \to \lambda$ if S is nullable. **Step**: Induct on derivation length. Each step $\alpha \Rightarrow_G \beta$ uses a production $A \to \gamma$. If γ contains no lambda productions, the same step works in G'. If γ contains a lambda-derivable nonterminal B (i.e., $B \overset{*}{\Rightarrow}^* \lambda$), then G' contains the production $A \to \gamma'$ where γ' is γ with B deleted; this simulates the combined derivation in one step. □

9.12. **For CNF grammar G and $x \in L(G)$, $x \neq \lambda$: number of productions in any derivation of x is $2|x|$.**

Correct statement: In a CNF grammar, any derivation of a string x with $|x| \geq 1$ uses exactly $2|x| - 1$ productions.

Proof: Each production is either of the form $A \to BC$ (increases the count of nonterminals by 1) or $A \to a$ (converts one nonterminal to a terminal, does not increase count). Start with 1 nonterminal. After k productions of the form $A \to BC$, we have $k+1$ nonterminals and 0 terminals. Each terminal production reduces nonterminals by 1 and adds 1 terminal. To produce $|x|$ terminals, we need $|x|$ terminal productions. The total nonterminals created = $k+1$ and consumed = $|x|$, so $k+1 = |x|$, giving $k = |x| - 1$ binary productions. Total productions = $(|x| - 1) + |x| = 2|x| - 1$. □

9.13. **Bounds on parse tree depth for CNF grammars.**

(a) **Lower bound**: For $x \in L(G)$ with $|x| = n > 0$, the parse tree has n leaf nodes (one per terminal). Since each internal node has exactly 2 children (CNF), a complete binary tree with n leaves has depth at least $\lceil \log_2 n \rceil$. Thus the depth of the parse tree is at least $\lceil \log_2 n \rceil$.

(b) **Upper bound**: The parse tree can be maximally skewed (a path): at each level, one child is a leaf (terminal) and the other is the next internal node. This gives depth $n - 1$ from the root to the last nonterminal, plus 1 for the final terminal, giving depth n. (Or $2n - 1$ if we count from a different convention.) Thus the depth is at most $n = |x|$.

Summary: $\lceil \log_2 |x| \rceil \leq \text{depth} \leq |x|$.

9.14. **Convert grammars to Chomsky Normal Form.**

(a) $\langle\{S,B,C\},\{\boldsymbol{a},\boldsymbol{b},\boldsymbol{c}\},S,\{S\to\boldsymbol{a}B, S\to\boldsymbol{ab}C, B\to\boldsymbol{bc}, C\to\boldsymbol{c}\}\rangle$:

Introduce terminal nonterminals: $T_a\to\boldsymbol{a}$, $T_b\to\boldsymbol{b}$, $T_c\to\boldsymbol{c}$. - $S\to\boldsymbol{a}B$ becomes $S\to T_aB$ (already binary CNF). - $S\to\boldsymbol{ab}C$ becomes $S\to T_aD_1$ where $D_1\to T_bC$. - $B\to\boldsymbol{bc}$ becomes $B\to T_bT_c$. - $C\to\boldsymbol{c}$ becomes $C\to T_c$ — but $C\to T_c$ is a unit production! Replace: wherever C appears as a target, substitute. Since $C\to\boldsymbol{c}$ and C does not appear in the right side of B or S (only in $S\to\boldsymbol{ab}C$), we can just keep $C\to T_c$ and note it will be eliminated: replace $S\to T_aD_1$ (where $D_1\to T_bC$) by substituting C: $D_1\to T_bT_c$.

Final CNF:

$$S\to T_aB\mid T_aD_1,\quad D_1\to T_bT_c,\quad B\to T_bT_c,\quad T_a\to\boldsymbol{a},\quad T_b\to\boldsymbol{b},\quad T_c\to\boldsymbol{c}.$$

(b) $\langle\{S,A,B\},\{\boldsymbol{a},\boldsymbol{b},\boldsymbol{c}\},S,\{S\to\boldsymbol{c}BA, S\to B, A\to\boldsymbol{c}B, A\to A\boldsymbol{bb}S, B\to\boldsymbol{aaa}\}\rangle$:

Step 1: Remove unit production $S\to B$: add all B-rules to S: $S\to\boldsymbol{aaa}$. Step 2: Introduce terminal symbols: $T_a\to\boldsymbol{a}$, $T_b\to\boldsymbol{b}$, $T_c\to\boldsymbol{c}$. Step 3: Break long right-hand sides: - $S\to\boldsymbol{c}BA$: $S\to T_cE_1$, $E_1\to BA$. - $S\to\boldsymbol{aaa}$: $S\to T_aE_2$, $E_2\to T_aT_a$. - $A\to\boldsymbol{c}B$: $A\to T_cB$. - $A\to A\boldsymbol{bb}S$: $A\to AE_3$, $E_3\to T_bE_4$, $E_4\to T_bS$. - $B\to\boldsymbol{aaa}$: $B\to T_aE_2$ (reuse E_2).

Final CNF:

$$\begin{aligned}
S&\to T_cE_1\mid T_aE_2\\
E_1&\to BA,\quad E_2\to T_aT_a\\
A&\to T_cB\mid AE_3\\
E_3&\to T_bE_4,\quad E_4\to T_bS\\
B&\to T_aE_2\\
T_a&\to\boldsymbol{a},\quad T_b\to\boldsymbol{b},\quad T_c\to\boldsymbol{c}
\end{aligned}$$

(c) $\langle\{R\},\{\boldsymbol{a},\boldsymbol{b},\boldsymbol{c},\boldsymbol{(},\boldsymbol{)},\boldsymbol{\epsilon},\boldsymbol{\emptyset},\cup,\cdot,\boldsymbol{*}\},R,\{R\to\boldsymbol{a}|\boldsymbol{b}|\boldsymbol{c}|\boldsymbol{\epsilon}|\boldsymbol{\emptyset}|(R\cdot R)|(R\cup R)|R^{\boldsymbol{*}}\}\rangle$:

Introduce terminal nonterminals for each terminal symbol: $T_a, T_b, T_c, T_\epsilon, T_\emptyset, T_(, T_), T_\cup, T_\cdot, T_*$.

$R\to\boldsymbol{a}|\boldsymbol{b}|\boldsymbol{c}|\boldsymbol{\epsilon}|\boldsymbol{\emptyset}$: Each of these is a single-terminal production $R\to a$ (with $|a|=1$), which is already in CNF form.

$R\to(R\cdot R)$: Break as $R\to T_(D_1$, $D_1\to RD_2$, $D_2\to T_\cdot D_3$, $D_3\to RT_)$. $R\to(R\cup R)$: Similarly $R\to T_(D_4$, $D_4\to RD_5$, $D_5\to T_\cup D_6$, $D_6\to RT_)$. $R\to R^{\boldsymbol{*}}$: $R\to RD_7$, $D_7\to T_*$—but $D_7\to T_*$ is a unit production; substitute: $R\to RT_*$.

Final CNF:

$$\begin{aligned}
R&\to\boldsymbol{a}|\boldsymbol{b}|\boldsymbol{c}|\boldsymbol{\epsilon}|\boldsymbol{\emptyset}\mid T_(D_1\mid T_(D_4\mid RT_*\\
D_1&\to RD_2,\quad D_2\to T_\cdot D_3,\quad D_3\to RT_)\\
D_4&\to RD_5,\quad D_5\to T_\cup D_6,\quad D_6\to RT_)\\
T_(&\to\boldsymbol{(},\quad T_)\to\boldsymbol{)},\quad T_\cdot\to\cdot,\quad T_\cup\to\cup,\quad T_*\to\boldsymbol{*}
\end{aligned}$$

(d) $\langle\{T\},\{\boldsymbol{a},\boldsymbol{b},\boldsymbol{c},\boldsymbol{d},\boldsymbol{-},\boldsymbol{+}\},T,\{T\to\boldsymbol{a}|\boldsymbol{b}|\boldsymbol{c}|\boldsymbol{d}|T\boldsymbol{-}T|T\boldsymbol{+}T\}\rangle$:

$T\to\boldsymbol{a}|\boldsymbol{b}|\boldsymbol{c}|\boldsymbol{d}$: Already in CNF (terminal productions). $T\to T\boldsymbol{-}T$: $T\to TD_-$ where $D_-\to T_-T$ and $T_-\to\boldsymbol{-}$. $T\to T\boldsymbol{+}T$: $T\to TD_+$ where $D_+\to T_+T$ and $T_+\to\boldsymbol{+}$.

Final CNF:

$$T\to\boldsymbol{a}|\boldsymbol{b}|\boldsymbol{c}|\boldsymbol{d}\mid TD_-\mid TD_+,\quad D_-\to T_-T,\quad D_+\to T_+T,\quad T_-\to\boldsymbol{-},\quad T_+\to\boldsymbol{+}.$$

9.15. **Convert grammars to Greibach Normal Form.**

(a) $\langle\{S_1,S_2\},\{\boldsymbol{a},\boldsymbol{b},\boldsymbol{c},\boldsymbol{d},\boldsymbol{e}\},S_1,\{S_1\to S_2S_1\boldsymbol{e},S_1\to S_2\boldsymbol{b},S_2\to S_1S_2,S_2\to\boldsymbol{c}\}\rangle$:

Step 1: Eliminate left-recursion and substitute. $S_2\to S_1S_2\mid\boldsymbol{c}$. Substitute S_1 into $S_2\to S_1S_2$: $S_2\to S_2S_1\boldsymbol{e}S_2\mid S_2\boldsymbol{b}S_2\mid\boldsymbol{c}S_2\mid\boldsymbol{c}$. Eliminate left recursion in S_2: Let $S_2=\boldsymbol{c}(S_1\boldsymbol{e}S_2\mid\boldsymbol{b}S_2\mid\lambda)^*$. Introduce $S_2'\to S_1\boldsymbol{e}S_2S_2'\mid\boldsymbol{b}S_2S_2'\mid S_1\boldsymbol{e}S_2\mid\boldsymbol{b}S_2\mid\lambda$. So $S_2\to\boldsymbol{c}S_2'$, $S_2\to\boldsymbol{c}$. Then: $S_1\to S_2S_1\boldsymbol{e}\mid S_2\boldsymbol{b}$. Substitute $S_2\to\boldsymbol{c}S_2'$ and $S_2\to\boldsymbol{c}$: $S_1\to\boldsymbol{c}S_2'S_1\boldsymbol{e}\mid\boldsymbol{c}S_1\boldsymbol{e}\mid\boldsymbol{c}S_2'\boldsymbol{b}\mid\boldsymbol{c}\boldsymbol{b}$. This is now in GNF (each rule starts with a terminal).

(b) $\langle\{S_1,S_2,S_3\},\ldots,\{S_1\to S_3S_1,S_1\to S_2\boldsymbol{a},S_2\to\boldsymbol{b}\boldsymbol{e},S_3\to S_2\boldsymbol{c}\}\rangle$:

$S_2\to\boldsymbol{b}\boldsymbol{e}$ (already in GNF). $S_3\to S_2\boldsymbol{c}\to\boldsymbol{b}\boldsymbol{e}\boldsymbol{c}$ (substitute): $S_3\to\boldsymbol{b}\boldsymbol{e}\boldsymbol{c}$. $S_1\to S_3S_1\to\boldsymbol{b}\boldsymbol{e}\boldsymbol{c}S_1$ and $S_1\to S_2\boldsymbol{a}\to\boldsymbol{b}\boldsymbol{e}\boldsymbol{a}=\boldsymbol{b}\boldsymbol{e}\boldsymbol{a}$. GNF: $S_1\to\boldsymbol{b}\boldsymbol{e}\boldsymbol{c}S_1\mid\boldsymbol{b}\boldsymbol{e}\boldsymbol{a}$, $S_2\to\boldsymbol{b}\boldsymbol{e}$, $S_3\to\boldsymbol{b}\boldsymbol{e}\boldsymbol{c}$.

(c) $\langle\{S_1,S_2,S_3\},\ldots,\{S_1\to S_1S_2\boldsymbol{c},S_1\to\boldsymbol{d}S_3,S_2\to S_1S_1,S_2\to\boldsymbol{a},S_3\to S_3\boldsymbol{e}\}\rangle$:

$S_3\to S_3\boldsymbol{e}$ |? — there's no base case for S_3 given. Assuming S_3 should have a base; looking at $S_1\to\boldsymbol{d}S_3$, S_3 appears only through $S_3\to S_3\boldsymbol{e}$, which is left-recursive with no base. This may indicate $L(S_3)=\emptyset$ unless there is a missing production. Assuming the grammar is as given and we treat $S_3\to S_3\boldsymbol{e}$ as the only rule for S_3: S_3 generates no terminal strings (left-recursive with no base). Then $S_1\to\boldsymbol{d}S_3$ generates nothing. So we focus on $S_1\to S_1S_2\boldsymbol{c}\mid\boldsymbol{d}S_3$ (empty) and $S_2\to S_1S_1\mid\boldsymbol{a}$.

For the left-recursive part: $S_1\to S_1S_2\boldsymbol{c}\mid\boldsymbol{d}S_3$ (taking S_3 as non-empty via some additional rule like $S_3\to\boldsymbol{d}$): If $S_3\to\boldsymbol{d}$ (assumed), then $S_1\to\boldsymbol{d}S_3\to\boldsymbol{d}\boldsymbol{d}$ and $S_1\to S_1S_2\boldsymbol{c}$. Eliminate left recursion: $S_1\to\boldsymbol{d}\boldsymbol{d}S_1'$, $S_1'\to S_2\boldsymbol{c}S_1'\mid\lambda$. Substitute $S_2\to S_1S_1\mid\boldsymbol{a}$: $S_1'\to S_1S_1\boldsymbol{c}S_1'\mid\boldsymbol{a}\boldsymbol{c}S_1'\mid\boldsymbol{a}\boldsymbol{c}\mid\ldots$ Continue substituting to get GNF; each step replaces leading nonterminals with their GNF expansions.

9.16. **Prove there exists a CFG G'' equivalent to G' (with added $A\to\lambda$ rules).**

G' is obtained from G by adding rules $A\to\lambda$ for various nonterminals A. We want to show G'' equivalent to G' where G'' handles all lambda productions via the standard construction (Definition 9.4 / Theorem 9.5).

Proof: The standard elimination of lambda productions (Theorem 9.5) shows that for any CFG, there is an equivalent CFG with no lambda productions except possibly $Z\to\lambda$ (if $\lambda\in L(G)$). This is done by: (1) finding all nullable nonterminals (A such that $A\overset{*}{\Rightarrow}\lambda$); (2) for each production $B\to\alpha A\beta$ where A is nullable, adding $B\to\alpha\beta$; (3) removing all $A\to\lambda$ except $Z\to\lambda$ if needed. The resulting G'' satisfies $L(G'')=L(G')=L(G)\cup\{\lambda\mid\lambda\in L(G')\}$. □

9.17. **Generalization of Lemma 9.1.**

Lemma: If $G=\langle\Omega,\Sigma,S,P\rangle$, $X\to\gamma B\alpha\in P$, all B-rules are $\{B\to\beta_i\}$, and G' replaces $X\to\gamma B\alpha$ with $\{X_B\to\gamma\beta_i\alpha\}$, then $L(G)=L(G')$.

Proof: By induction on derivation length.

($L(G)\subseteq L(G')$): Any derivation in G that uses $X\to\gamma B\alpha$ followed later by $B\to\beta_i$ can be replaced in G' by the single production $X_B\to\gamma\beta_i\alpha$. All other derivations are unchanged.

($L(G')\subseteq L(G)$): Any production $X_B\to\gamma\beta_i\alpha$ in G' can be simulated in G by $X\to\gamma B\alpha$ followed by $B\to\beta_i$. □

9.18. $P=\{d^p\mid p$ **prime**$\}$ **is not context free.**

(a) **Pumping theorem proof**: Assume P is context free with pumping constant n. Take $s = \boldsymbol{d}^p$ where $p > n$ is prime. By the pumping lemma, $s = uvwxy$ with $|vwx| \le n$, $|vx| \ge 1$, and $uv^i wx^i y \in P$ for all $i \ge 0$.

Let $|vx| = m$ where $1 \le m \le n$. Then $uv^i wx^i y = \boldsymbol{d}^{p+(i-1)m}$. For $i = 0$: $\boldsymbol{d}^{p-m}$. For $i = 2$: $\boldsymbol{d}^{p+m}$.

Choose $i = p+1$: the resulting string has length $p + pm = p(1+m)$. Since $p \ge 2$ and $m \ge 1$, $p(1+m)$ is composite (product of two integers each ≥ 2). So $\boldsymbol{d}^{p(1+m)} \notin P$, contradiction. □

(b) **Using nonregularity**: Since $P \subseteq \{\boldsymbol{d}\}^*$ and P is not regular (the set of lengths is the set of primes, which is not eventually periodic), if P were context free over a unary alphabet, then by Parikh's theorem P would have a semilinear set of lengths. But the set of primes is not semilinear (semilinear sets over $\mathbb{N}$ are eventually periodic/union of arithmetic progressions). Therefore P is not context free. □

9.19. $\Gamma = \{w\mathbf{2}w \mid w \in \{\mathbf{0},\mathbf{1}\}^*\}$ **is not context free.**

Assume Γ is context free with pumping constant n. Choose $s = \mathbf{0}^n\mathbf{1}^n\mathbf{2}\mathbf{0}^n\mathbf{1}^n \in \Gamma$ (take $w = \mathbf{0}^n\mathbf{1}^n$, $|s| = 4n+1$).

By the pumping lemma: $s = uvwxy$, $|vwx| \le n$, $|vx| \ge 1$, all pumped strings in Γ.

Since $|vwx| \le n$, the substring vwx lies entirely within one of the following regions: the first $\mathbf{0}^n$-block, the $\mathbf{1}^n$-block, the $\mathbf{2}$ separator, the second $\mathbf{0}^n$-block, or the second $\mathbf{1}^n$-block (it cannot span the $\mathbf{2}$ and both sides).

In any case, pumping changes the count of some symbol on one side of $\mathbf{2}$ without affecting the other side, or changes only one block. The result is a string that cannot be of the form $w'\mathbf{2}w'$ for any w'. Contradiction. □

9.20. $\Psi = \{ww \mid w \in \{\mathbf{0},\mathbf{1}\}^*\}$ **is not context free.**

Assume Ψ is context free with pumping constant n. Choose $s = \mathbf{0}^n\mathbf{1}^n\mathbf{0}^n\mathbf{1}^n \in \Psi$ (take $w = \mathbf{0}^n\mathbf{1}^n$). $|s| = 4n$.

By the pumping lemma: $s = uvwxy$, $|vwx| \le n$, $|vx| \ge 1$, all pumped strings in Ψ.

Since $|vwx| \le n$, vwx is contained within a span of n consecutive characters. The string s has the pattern $0^n1^n0^n1^n$. The middle two blocks meet at position $2n$; the window vwx cannot span from the first 0^n-block to the second 0^n-block.

In all cases, pumping (or de-pumping with $i = 0$) changes the length or composition of one half of s without matching change in the other half, so the result cannot have the form $w'w'$. Contradiction. □

9.21. $\Xi = \{\boldsymbol{b}^{2^j} \mid j \in \mathbb{N}\}$ **is not context free.**

Assume Ξ is context free with pumping constant n. Choose j such that $2^j > n$, and let $s = \boldsymbol{b}^{2^j}$.

By the pumping lemma: $s = uvwxy$, $|vwx| \le n$, $|vx| = m \ge 1$. Then $|uv^i wx^i y| = 2^j + (i-1)m$ for each $i \ge 0$.

For $i = 2$: length $= 2^j + m$. We need $2^j + m$ to be a power of 2. But $2^j < 2^j + m \le 2^j + n < 2^{j+1}$ (since $m \le n < 2^j$). So $2^j + m$ is strictly between 2^j and 2^{j+1}, which means it is not a power of 2. Contradiction. □

9.22. $\Phi = \{\boldsymbol{a}^{j^2} \mid j \in \mathbb{N}\}$ **is not context free; application to** Φ'**.**

(a) Φ **is not context free**: Assume Φ is context free with pumping constant n. Let $j > n$; take $s = \boldsymbol{a}^{j^2}$.

By the pumping lemma: $s = uvwxy$, $|vwx| \le n$, $|vx| = m \ge 1$. Then pumped strings have length $j^2 + (i-1)m$. For $i = 2$: length $j^2 + m$. We need $j^2 + m$ to be a perfect square. Choose $j > n/2$ so that $n < 2j$; then $j^2 < j^2 + m \le j^2 + n < j^2 + 2j + 1 = (j+1)^2$. So $j^2 + m$ is strictly between two consecutive perfect squares, hence not a perfect square. Contradiction. □

(b) Φ' **not context free via homomorphism**: Define $h : \{\boldsymbol{b}, \boldsymbol{c}, \boldsymbol{d}\}^* \to \{\boldsymbol{a}\}^*$ by $h(\boldsymbol{c}) = \boldsymbol{a}$ and $h(\boldsymbol{b}) = h(\boldsymbol{d}) = \lambda$. If Φ' were a CFL, then $h(\Phi') = \Phi$ would also be a CFL (CFLs are closed under homomorphism), contradicting part (a). Hence $\Phi' \notin \mathfrak{C}_\Sigma$.

(c) **Direct Ogden's lemma proof**: Mark all $\boldsymbol{c}$-symbols in a long string in Φ'. By Ogden's lemma, the pumped substring must touch at least one marked position. Since $|x|_{\boldsymbol{c}} = (|x|_{\boldsymbol{d}})^2$, pumping $\boldsymbol{c}$s or $\boldsymbol{d}$s independently destroys the quadratic relation.

(d) **Pumping theorem**: The standard pumping theorem (without Ogden) may be insufficient here because it does not distinguish which positions are pumped. It may be possible to directly apply the pumping lemma, but choosing the right string is tricky. Ogden's lemma is more convenient.

9.23. $L = \{y \in \{\mathbf{0}, \mathbf{1}\}^* \mid |y|_{\mathbf{0}} = |y|_{\mathbf{1}}\}$ **is context free.**

Grammar: $S \to \mathbf{0}S\mathbf{1} \mid \mathbf{1}S\mathbf{0} \mid SS \mid \lambda$.

Proof: By induction, any string with equal numbers of **0**s and **1**s is derivable, and all derivations yield strings with equal counts. So $L(G) = L$. □

9.24. **Formal recursive definition of the path in Theorem 9.9 (pumping lemma).**

(a) Let the parse tree have root r corresponding to start symbol S. Define a *leftmost longest path* recursively: - **Boundary**: If r is a leaf, the path is $\{r\}$. - **Rule**: From a node v with children $c_1, c_2, \ldots, c_k$, choose c_i to be the child whose subtree has the greatest number of leaves. (Break ties by choosing the leftmost.) Append c_i to the path, and recurse from c_i.

(b) The depth of this path is at least $\log_2$ (number of leaves) $= \log_2 |x|$ (by the binary branching property of CNF). If $|x| \ge b^{h+1}$ where b is the maximum branching factor and $h = |\Omega|$, then the path has length $> h$, so some nonterminal repeats on the path. Let A be the repeated nonterminal at positions $i < j$ on the path. Then the subtree at position j generates w, and the subtree at position i generates vwx (with v, x possibly empty but $|vx| \ge 1$ by the choice that $vwx \ne w$). Pumping follows from substituting the subtree at j for the subtree at i. □

9.25. $\mathfrak{C}_\Sigma$ **is closed under** $\cup$ **via direct grammar construction.**

Let $G_1 = \langle \Omega_1, \Sigma, S_1, P_1 \rangle$ and $G_2 = \langle \Omega_2, \Sigma, S_2, P_2 \rangle$ with $\Omega_1 \cap \Omega_2 = \emptyset$. Choose $Z \notin \Omega_1 \cup \Omega_2$ and define:

$$G^{\cup} = \langle \Omega_1 \cup \Omega_2 \cup \{Z\}, \Sigma, Z, P_1 \cup P_2 \cup \{Z \to S_1, Z \to S_2\} \rangle.$$

Then $w \in L(G^{\cup})$ iff $Z \overset{*}{\Rightarrow} w$ iff $Z \Rightarrow S_i \overset{*}{\Rightarrow} w$ for $i = 1$ or $i = 2$ iff $w \in L(G_1) \cup L(G_2)$. □

9.26. **Closure properties of** $\mathfrak{X}_\Sigma$ **(languages that are NOT context free).**

(a) **Union**: Not closed. Let $L = \{\boldsymbol{a}^n\boldsymbol{b}^n\boldsymbol{c}^n \mid n \geq 0\}$. Then $L \notin \mathfrak{C}_\Sigma$ (standard pumping argument) and $\overline{L} \notin \mathfrak{C}_\Sigma$ (if $\overline{L}$ were a CFL, then $L = \overline{\overline{L}}$ would be a CFL by closure under complement restricted to this context — more directly: the complement of a non-CFL need not be a CFL, but here $\overline{L} \cap \boldsymbol{a}^*\boldsymbol{b}^*\boldsymbol{c}^* = \boldsymbol{a}^*\boldsymbol{b}^*\boldsymbol{c}^* \setminus L$, and if $\overline{L}$ were a CFL its intersection with the regular language $\boldsymbol{a}^*\boldsymbol{b}^*\boldsymbol{c}^*$ would be a CFL, but that intersection is not a CFL). So $L, \overline{L} \in \mathfrak{X}_\Sigma$, yet $L \cup \overline{L} = \Sigma^* \in \mathfrak{C}_\Sigma \setminus \mathfrak{X}_\Sigma$. Hence $\mathfrak{X}_\Sigma$ is *not* closed under union.

(b) **Complement**: Not closed. If $L \in \mathfrak{X}_\Sigma$ (L is not CFL), $\overline{L}$ may or may not be in $\mathfrak{X}_\Sigma$. Since CFLs are not closed under complement, there exist non-CFLs whose complements are also non-CFLs, and non-CFLs whose complements are CFLs.

(c) **Intersection**: Not closed. Similar argument.

(d) **Kleene closure**: Not closed. $L = \{\boldsymbol{a}^p \mid p \text{ prime}\} \in \mathfrak{X}_\Sigma$ (not regular, hence if unary: not CFL by Parikh), but $L^* = \{\boldsymbol{a}^n \mid n \geq 0 \text{ or sum of primes}\} = \{\boldsymbol{a}^n \mid n \neq 1\} \cup \{\lambda\}$ which is regular (hence CFL). So $L^* \notin \mathfrak{X}_\Sigma$.

(e) **Concatenation**: Not closed. $L \cdot \Sigma^*$ for a non-CFL L may be a CFL or not; specific examples show $\mathfrak{X}_\Sigma$ is not closed.

9.27. **$\mathfrak{C}_\Sigma$ is closed under reversal.**

Let $G = \langle \Omega, \Sigma, S, P \rangle$ generate L. Define $G^r = \langle \Omega, \Sigma, S, P^r \rangle$ where $P^r = \{A \to \alpha^r \mid A \to \alpha \in P\}$ (reverse the right-hand side of each production).

Claim: $L(G^r) = L^r$.

Proof: By induction on derivation length. If $S \overset{*}{\Rightarrow}_G w$, then $S \overset{*}{\Rightarrow}_{G^r} w^r$ (by reversing each production step). Formally: if $\alpha \Rightarrow_G \beta$ (applying $A \to \gamma$ in context $\alpha = \phi A \psi$), then $\beta^r = \psi^r \gamma^r \phi^r \Rightarrow_{G^r} \psi^r A \phi^r = \alpha^r$ in G^r (using $A \to \gamma^r$). By induction, full derivations reverse. □

9.28. **$\mathfrak{C}_\Sigma$ is closed under operator T ($T(L) = \{w^r w \mid w \in L\}$).**

Claim: T does not preserve $\mathfrak{C}_\Sigma$ in general; i.e., there exists $L \in \mathfrak{C}_\Sigma$ such that $T(L) \notin \mathfrak{C}_\Sigma$.

Counterexample: Let $L = \{\boldsymbol{a}^n\boldsymbol{b}^n \mid n \geq 0\} \in \mathfrak{C}_\Sigma$. Then

$$T(L) = \{(\boldsymbol{a}^n\boldsymbol{b}^n)^r(\boldsymbol{a}^n\boldsymbol{b}^n) \mid n \geq 0\} = \{\boldsymbol{b}^n\boldsymbol{a}^n\boldsymbol{a}^n\boldsymbol{b}^n \mid n \geq 0\} = \{\boldsymbol{b}^n\boldsymbol{a}^{2n}\boldsymbol{b}^n \mid n \geq 0\}.$$

We claim $T(L) \notin \mathfrak{C}_\Sigma$. Suppose for contradiction that $T(L)$ is a CFL with pumping constant N. Take $s = \boldsymbol{b}^N\boldsymbol{a}^{2N}\boldsymbol{b}^N \in T(L)$ and write $s = uvwxy$ with $|vwx| \leq N$, $|vx| \geq 1$. Since $|vwx| \leq N < N+1 \leq N+2N+N = |s|$, the window vwx spans at most two of the three contiguous blocks $[\boldsymbol{b}^N][\boldsymbol{a}^{2N}][\boldsymbol{b}^N]$. We analyse each case when pumping $i = 2$ (replacing v, x by v^2, x^2):

- *vwx within the first $\boldsymbol{b}$-block*: $v = \boldsymbol{b}^p$, $x = \boldsymbol{b}^q$ with $p + q \geq 1$. Pumped string is $\boldsymbol{b}^{N+p+q}\boldsymbol{a}^{2N}\boldsymbol{b}^N$. For membership in $T(L)$, the outer $\boldsymbol{b}$-blocks must match, but $N + p + q > N$. **Not in $T(L)$.**
- *vwx spans first $\boldsymbol{b}$-block and $\boldsymbol{a}$-block*: v adds $p \geq 0$ $\boldsymbol{b}$'s, x adds $q \geq 0$ $\boldsymbol{a}$'s (or vice versa), $p + q \geq 1$. Pumped string is $\boldsymbol{b}^{N+p}\boldsymbol{a}^{2N+q}\boldsymbol{b}^N$. For $T(L)$ membership need $N + p = n$ and $2N + q = 2n$ and $N = n$; from $N = n$: $p = 0$ and $q = 0$, contradicting $p + q \geq 1$. **Not in $T(L)$.**
- *vwx within the $\boldsymbol{a}$-block*: $v = \boldsymbol{a}^p$, $x = \boldsymbol{a}^q$, $p + q \geq 1$. Pumped string is $\boldsymbol{b}^N\boldsymbol{a}^{2N+p+q}\boldsymbol{b}^N$. Need $2N + p + q = 2N$, impossible. **Not in $T(L)$.**
- *vwx spans $\boldsymbol{a}$-block and last $\boldsymbol{b}$-block*: symmetric to the second case; the outer $\boldsymbol{b}$-blocks become unequal. **Not in $T(L)$.**

- *vwx within the last $\boldsymbol{b}$-block*: symmetric to the first case. **Not in** $T(L)$**.**

In every case, pumping $i = 2$ produces a string outside $T(L)$, contradicting the pumping lemma. Hence $T(L) \notin \mathfrak{C}_\Sigma$.

Note: $T(\Sigma^*) = \{w^r\, w \mid w \in \Sigma^*\}$ is in fact the language of *even-length palindromes*, which *is* context free (grammar $S \to \boldsymbol{a}S\boldsymbol{a} \mid \boldsymbol{b}S\boldsymbol{b} \mid \lambda$ for $\Sigma = \{\boldsymbol{a}, \boldsymbol{b}\}$), so it cannot serve as a witness for non-closure. The counterexample above using $L = \{\boldsymbol{a}^n\boldsymbol{b}^n\}$ is correct.

Hence $\mathfrak{C}_\Sigma$ is *not* closed under T. □

9.29. **$\mathfrak{C}_{\{\boldsymbol{a},\boldsymbol{b}\}}$ is not closed under relative complement.**

Disproof: Let $L_1 = \{x \in \{\boldsymbol{a}, \boldsymbol{b}\}^* \mid |x|_{\boldsymbol{a}} = |x|_{\boldsymbol{b}}\}$ (CFL) and $L_2 = \{\boldsymbol{a}\}^*\{\boldsymbol{b}\}^*$ (regular, hence CFL). Then:

$$L_1 - L_2 = \{x \mid |x|_{\boldsymbol{a}} = |x|_{\boldsymbol{b}}\} - \{\boldsymbol{a}^n\boldsymbol{b}^n\} = \{x \mid |x|_{\boldsymbol{a}} = |x|_{\boldsymbol{b}}, x \notin \boldsymbol{a}^*\boldsymbol{b}^*\}.$$

This language (balanced strings that are not in $\boldsymbol{a}^*\boldsymbol{b}^*$) is context free (construct a grammar). So this does not give a counterexample.

A better approach: use the fact that if $\mathfrak{C}_\Sigma$ were closed under relative complement and we know $\mathfrak{C}_\Sigma$ is closed under union, then $\mathfrak{C}_\Sigma$ would be closed under complement (since $\overline{L_1} = \Sigma^* - L_1$, and Σ^* is CFL). But $\mathfrak{C}_\Sigma$ is not closed under complement (see Exercise 9.30). Hence $\mathfrak{C}_\Sigma$ is *not* closed under relative complement. □

9.30. **$\mathfrak{C}_\Sigma$ is not closed under intersection or complement.**

(a) Let $L_1 = \{\boldsymbol{a}^i\boldsymbol{b}^i\boldsymbol{c}^j \mid i, j \in \mathbb{N}\}$ and $L_2 = \{\boldsymbol{a}^i\boldsymbol{b}^j\boldsymbol{c}^j \mid i, j \in \mathbb{N}\}$. Both are context free. Their intersection:

$$L_1 \cap L_2 = \{\boldsymbol{a}^n\boldsymbol{b}^n\boldsymbol{c}^n \mid n \in \mathbb{N}\},$$

which is not context free (by the pumping lemma). So $\mathfrak{C}_\Sigma$ is not closed under intersection.

If $\mathfrak{C}_\Sigma$ were closed under complement, then $\overline{L_1}$ and $\overline{L_2}$ would be context free, and $\overline{\overline{L_1} \cup \overline{L_2}} = L_1 \cap L_2$ would be context free (by De Morgan and closure under union and complement). But $L_1 \cap L_2$ is not CFL. Contradiction. So $\mathfrak{C}_\Sigma$ is not closed under complement.

(b) For $|\Sigma| > 1$: given the result for $\Sigma = \{\boldsymbol{a}, \boldsymbol{b}, \boldsymbol{c}\}$ from part (a), define a homomorphism $h : \{\boldsymbol{a}, \boldsymbol{b}, \boldsymbol{c}\}^* \to \Sigma^*$ (injecting the three-letter alphabet into any alphabet with more than one symbol). By closure of CFLs under inverse homomorphism, the preimage of a CFL under h is a CFL. If $\mathfrak{C}_\Sigma$ were closed under complement for Σ, pulling back via h would show closure for three-letter alphabets, contradicting part (a). □

9.31. **Automaton-based method for Theorem 9.3 (epsilon elimination).**

Consider a pushdown automaton (PDA) with states labeled by subsets $T \subseteq \Omega$ (the power set of nonterminals). Start state: $\{S\}$. Transitions: from state T, for each nonterminal $A \in T$ and production $A \to \gamma$, compute the set of nonterminals derivable in one step from γ, and union these into the new state.

This is similar to the subset construction for NFAs, adapted for CFGs. The set T represents all nonterminals reachable in derivations. The nullable nonterminals (N_λ) are those states from which λ is reachable.

9.32. **Grammars in GNF with at most two symbols on the right.**

(a) This is not a normal form for all CFLs: The language $\{\boldsymbol{a}^n \mid n \geq 1\}$ can be generated (e.g., $S \to \boldsymbol{a}S \mid \boldsymbol{a}$), but more complex CFLs like $\{ww^r\}$ require productions of the form $S \to \boldsymbol{a}S\boldsymbol{a}$ in GNF which has 3 symbols on the right (more than 2). Converting to this restricted GNF may require more nonterminals and fail to represent the language if we strictly limit to 2 symbols. In fact $\boldsymbol{a}Xa$ (where a is terminal and X is nonterminal) has 3 symbols; with at most 2 on the right in GNF (first terminal, at most one nonterminal), we can only write $\boldsymbol{a}X$ (dropping the trailing $\boldsymbol{a}$). So $\{ww^r \mid w \in \{\boldsymbol{a}, \boldsymbol{b}\}^*\}$ cannot be generated.

(b) Languages generated by this restricted form: GNF with at most two RHS symbols means productions are $A \to \boldsymbol{a}$ or $A \to \boldsymbol{a}B$. These are exactly the *right-linear grammars*, which generate precisely the *regular languages* ($\mathfrak{D}_\Sigma = \mathfrak{G}_\Sigma = \mathfrak{R}_\Sigma$). □

9.33. **Prove L^* must be regular for *any* collection L over Σ.**

Over a unary alphabet $\Sigma = \{\boldsymbol{a}\}$, L^* is always regular for any $L \subseteq \Sigma^*$.

Proof: Identify L with the set of lengths $A = \{|w| \mid w \in L\} \subseteq \mathbb{N}$. Then $L^* = \{\boldsymbol{a}^n \mid n \in A^+\}$, where $A^+ = \{a_1 + \cdots + a_k \mid k \geq 0,\ a_i \in A\}$. By the Sylvester–Frobenius theorem, if A is non-empty with $\gcd(A) = d$, then every sufficiently large multiple of d belongs to A^+. Hence A^+ is eventually periodic, and the corresponding language L^* is regular. □

(The claim is false over alphabets of size > 1: $L = \{\boldsymbol{a}^n\boldsymbol{b}^n \mid n \geq 1\}$ is a CFL whose Kleene closure is not regular.)

9.34. **If $|\Sigma| = 1$, is $\mathfrak{C}_\Sigma$ closed under complementation?**

Yes. Over a unary alphabet $\Sigma = \{\boldsymbol{a}\}$, every context-free language L corresponds to a semilinear set of lengths (by Parikh's theorem). The complement of a semilinear set is also semilinear (since semilinear sets are closed under Boolean operations). A semilinear subset of $\mathbb{N}$ corresponds to a regular language over $\{\boldsymbol{a}\}$. Hence $\overline{L}$ is regular, and in particular, context-free. So $\mathfrak{C}_\Sigma$ is closed under complementation when $|\Sigma| = 1$. □

9.35. $\{\boldsymbol{a}^n\boldsymbol{b}^n\boldsymbol{c}^m \mid n, m \in \mathbb{N}\}$ **is context free.**

Grammar: $S \to AB$, $A \to \boldsymbol{a}A\boldsymbol{b} \mid \lambda$, $B \to \boldsymbol{c}B \mid \lambda$.

A generates $\{\boldsymbol{a}^n\boldsymbol{b}^n\}$ and B generates $\boldsymbol{c}^*$. So $L(G) = \{\boldsymbol{a}^n\boldsymbol{b}^n\boldsymbol{c}^m\}$. □

9.36. **Use Ogden's lemma to prove $\{\boldsymbol{a}^k\boldsymbol{b}^n\boldsymbol{c}^m \mid k \neq n \wedge n \neq m\}$ is not context free.**

Assume $L = \{\boldsymbol{a}^k\boldsymbol{b}^n\boldsymbol{c}^m \mid k \neq n, n \neq m\}$ is context free with Ogden constant N.

Choose $p > N$ and let $s = \boldsymbol{a}^{p!+p}\boldsymbol{b}^p\boldsymbol{c}^{p!+p} \in L$ (since $p! + p \neq p$ and $p \neq p! + p$). Mark all p occurrences of $\boldsymbol{b}$ in s.

By Ogden's lemma: $s = uvwxy$ where vwx contains at most $N \leq p$ marked positions, vx contains at least 1 marked position, and all pumped strings $uv^iwx^iy \in L$.

Let $m = |vx|_{\boldsymbol{b}} \geq 1$ (the number of $\boldsymbol{b}$s in vx; at least 1 since vx contains a marked symbol). Choose $i - 1 = p!/m$, which is a positive integer since $m \leq N < p$ implies $m \leq p - 1$ and hence $m \mid p!$. The pumped string has $\boldsymbol{b}$-count equal to $p + (i-1)m = p + p! = p! + p$.

Since $|vwx| \le N < p$ and the $\boldsymbol{b}$-block has exactly p symbols, the window vwx cannot reach both the $\boldsymbol{a}$-block on the left and the $\boldsymbol{c}$-block on the right simultaneously. Let $s_a = |vx|_{\boldsymbol{a}}$ and $s_c = |vx|_{\boldsymbol{c}}$. Then either $s_a = 0$ (window does not touch the $\boldsymbol{a}$-block) or $s_c = 0$ (window does not touch the $\boldsymbol{c}$-block).

- If $s_a = 0$: the $\boldsymbol{a}$-count remains $p! + p = \boldsymbol{b}$-count, so $k = n = p! + p$, violating $k \neq n$. Contradiction.
- If $s_c = 0$: the $\boldsymbol{c}$-count remains $p! + p = \boldsymbol{b}$-count, so $n = m = p! + p$, violating $n \neq m$. Contradiction.

In every case the pumped string lies outside L, contradicting Ogden's lemma. □

Chapter 10

Pushdown Automata — Solutions

10.1. **Show by induction:** $\overline{\delta}(s_0, x) = t \Leftrightarrow \langle s_0, x, \text{¢}\rangle \vdash^* \langle t, \lambda, \text{¢}\rangle$**.**

Claim ($P(n)$): For all $x \in \Sigma^n$, $\overline{\delta}(s_0, x) = t$ iff $\langle s_0, x, \text{¢}\rangle \vdash^* \langle t, \lambda, \text{¢}\rangle$.

Base ($n = 0$): $\overline{\delta}(s_0, \lambda) = s_0$ and $\langle s_0, \lambda, \text{¢}\rangle \vdash^* \langle s_0, \lambda, \text{¢}\rangle$ (zero moves). Both sides give $t = s_0$. ✓

Step: Assume $P(n)$. For $x = ya$ ($|y| = n$, $a \in \Sigma$):

($\Rightarrow$): $\overline{\delta}(s_0, ya) = \delta(\overline{\delta}(s_0, y), a) = t$. Let $q = \overline{\delta}(s_0, y)$. By IH, $\langle s_0, y, \text{¢}\rangle \vdash^* \langle q, \lambda, \text{¢}\rangle$. By Theorem 10.1, $\langle q, a, \text{¢}\rangle \vdash \langle t, \lambda, \text{¢}\rangle$ (single step reading a). So $\langle s_0, ya, \text{¢}\rangle \vdash^* \langle t, \lambda, \text{¢}\rangle$.

($\Leftarrow$): If $\langle s_0, ya, \text{¢}\rangle \vdash^* \langle t, \lambda, \text{¢}\rangle$, then the computation passes through some state q after reading y: $\langle s_0, y, \text{¢}\rangle \vdash^* \langle q, \lambda, \text{¢}\rangle$. By IH, $\overline{\delta}(s_0, y) = q$. The last step reads a from state q to reach t, so $\delta(q, a) = t$, hence $\overline{\delta}(s_0, ya) = t$. □

10.2. **Define a one-state DPDA P_1' accepting $\{\boldsymbol{a}^n\boldsymbol{b}^n \mid n \geq 0\}$ via final state.**

A one-state DPDA accepting via final state always accepts λ: the unique state must be both the initial state and the only final state, so the machine is in a final state at the very start with empty input. Hence the best a one-state final-state DPDA can achieve is $\{\boldsymbol{a}^n\boldsymbol{b}^n \mid n \geq 0\}$ (which includes λ).

Let $P_1' = \langle\{\boldsymbol{a}, \boldsymbol{b}\}, \{Z, A\}, \{t\}, t, \delta, Z, \{t\}\rangle$ where:

$$\delta(t, \boldsymbol{a}, Z) = \langle t, AZ\rangle \quad \text{(push } A \text{ on reading first } \mathbf{a}\text{)}$$
$$\delta(t, \boldsymbol{a}, A) = \langle t, AA\rangle \quad \text{(push another } A \text{ for each subsequent } \mathbf{a}\text{)}$$
$$\delta(t, \boldsymbol{b}, A) = \langle t, \lambda\rangle \quad \text{(pop } A \text{ on reading } \mathbf{b}\text{)}$$

The machine pushes one A for each $\boldsymbol{a}$ read, then pops one A for each $\boldsymbol{b}$ read. It accepts when the input is exhausted in state t: this occurs precisely when the $\boldsymbol{a}$s and $\boldsymbol{b}$s are balanced (stack returned to Z) or when the input is empty. Note: $\delta(t, \boldsymbol{b}, Z)$ is undefined, so a string with more $\boldsymbol{b}$s than $\boldsymbol{a}$s causes the machine to get stuck (reject). Hence $\Lambda(P_1') = \{\boldsymbol{a}^n\boldsymbol{b}^n \mid n \geq 0\}$.

10.3. **PDA P_2': proofs and language.**

$P_2' = \langle\{\boldsymbol{a}, \boldsymbol{b}\}, \{S, C\}, \{t\}, t, \delta, S, \{t\}\rangle$ with $\delta(t, \boldsymbol{a}, S) = \{\langle t, SC\rangle, \langle t, C\rangle\}$, $\delta(t, \boldsymbol{b}, C) = \{\langle t, \lambda\rangle\}$, others empty.

(a) **Inductive proof**: ($\forall i \in \mathbb{N}$): if $\langle t, \boldsymbol{a}^i, S\rangle \vdash^* \langle t, \lambda, \alpha\rangle$ then $\alpha = SC^i$ or $\alpha = C^i$.
Base ($i = 0$): $\langle t, \lambda, S\rangle \vdash^* \langle t, \lambda, S\rangle$; here $\alpha = S = SC^0$. ✓

Step: Assume the result for i. Reading $\boldsymbol{a}^{i+1}$ from S: first move on $(\boldsymbol{a}, S)$ either pushes SC or C.

Case 1: Push SC. Then $\langle t, \boldsymbol{a}^{i+1}, S\rangle \vdash \langle t, \boldsymbol{a}^{i}, SC\rangle$. Now process $\boldsymbol{a}^{i}$ from S: by IH, $\langle t, \boldsymbol{a}^{i}, S\rangle \vdash^{*} \langle t, \lambda, \beta\rangle$ where $\beta \in \{SC^{i}, C^{i}\}$. So $\langle t, \boldsymbol{a}^{i}, SC\rangle \vdash^{*} \langle t, \lambda, \beta C\rangle \in \{SC^{i+1}, C^{i+1}\}$.

Case 2: Push C. Then $\langle t, \boldsymbol{a}^{i+1}, S\rangle \vdash \langle t, \boldsymbol{a}^{i}, C\rangle$. Processing $\boldsymbol{a}^{i}$ from C: there are no moves (neither $\delta(t, \boldsymbol{a}, C)$ nor $\delta(t, \lambda, C)$ are defined for reading $\boldsymbol{a}$), so this branch is a dead end unless $i = 0$.

So $\alpha \in \{SC^{i+1}, C^{i+1}\}$. □

(b) **Inductive proof**: $(\forall i \in \mathbb{N})$: if $\langle t, x, C^{i}\rangle \vdash^{*} \langle t, \lambda, \beta\rangle$ then $x = \boldsymbol{b}^{i}$.

Base $(i = 0)$: $\langle t, x, \lambda\rangle$ can only have $x = \lambda$ (stack empty, no moves possible). So $x = \lambda = \boldsymbol{b}^{0}$. ✓

Step: Assume for i. From $\langle t, x, C^{i+1}\rangle$: the top of the stack is C. The only move consuming C reads $\boldsymbol{b}$: $\delta(t, \boldsymbol{b}, C) = \langle t, \lambda\rangle$. So x must start with $\boldsymbol{b}$: $x = \boldsymbol{b}x'$ and $\langle t, \boldsymbol{b}x', C^{i+1}\rangle \vdash \langle t, x', C^{i}\rangle$. By IH, $x' = \boldsymbol{b}^{i}$. Hence $x = \boldsymbol{b}^{i+1}$. □

(c) P_2' accepts by empty stack (the final-state component $\{t\}$ is not used for acceptance here; a string is accepted iff the machine empties its stack while in state t with empty input). From (a), processing $\boldsymbol{a}^{n}$ from start symbol S can yield stack C^{n} (by choosing $\delta(t, \boldsymbol{a}, S) = \langle t, SC\rangle$ at each of the first $n-1$ steps and then $\langle t, C\rangle$ at the last, so the final stack is C^{n}). From (b), the stack C^{n} is emptied by reading exactly $\boldsymbol{b}^{n}$. Thus:

$$L(P_2') = \{\boldsymbol{a}^{n}\boldsymbol{b}^{n} \mid n \geq 1\}.$$

For $n = 0$: the initial configuration $\langle t, \lambda, S\rangle$ has S still on the stack with no moves available, so the stack is never emptied and $\lambda \notin L(P_2')$. Hence $L(P_2') = \{\boldsymbol{a}^{n}\boldsymbol{b}^{n} \mid n \geq 1\}$.

10.4. $L = \{\boldsymbol{a}^{i}\boldsymbol{b}^{j}\boldsymbol{c}^{k} \mid i, j, k \in \mathbb{N},\ i + j = k\}$.

(a) **PDA via final state**: Push A for each $\boldsymbol{a}$, push A for each $\boldsymbol{b}$, pop A for each $\boldsymbol{c}$; accept when stack (minus bottom marker) is empty. $P = \langle\{\boldsymbol{a}, \boldsymbol{b}, \boldsymbol{c}\}, \{Z, A\}, \{q_0, q_1, q_f\}, q_0, \delta, Z, \{q_f\}\rangle$: $\delta(q_0, \boldsymbol{a}, Z) = \langle q_0, AZ\rangle$, $\delta(q_0, \boldsymbol{a}, A) = \langle q_0, AA\rangle$, $\delta(q_0, \boldsymbol{b}, Z) = \langle q_1, AZ\rangle$, $\delta(q_0, \boldsymbol{b}, A) = \langle q_1, AA\rangle$, $\delta(q_1, \boldsymbol{b}, A) = \langle q_1, AA\rangle$, $\delta(q_1, \boldsymbol{c}, A) = \langle q_1, \lambda\rangle$, $\delta(q_1, \lambda, Z) = \langle q_f, Z\rangle$.

(b) **PDA via empty stack**: Same idea, but pop Z at the end instead of moving to a final state.

(c) **Counting automaton**: No; a counting automaton has fixed counting capacity and cannot track the unbounded sum $i + j$.

(d) **DPDA**: Yes. The DPDA reads $\boldsymbol{a}$s (pushing), then $\boldsymbol{b}$s (pushing), then $\boldsymbol{c}$s (popping), accepting when the stack is empty at end of input. Since the transitions are deterministic at each step, a DPDA exists.

(e) **Grammar from PDA** (using Definition 10.7): The grammar derived from the PDA in part (a) produces variables $\langle q_i, A, q_j\rangle$ meaning "starting in state q_i with A on top of the stack, the PDA moves to state q_j and empties the stack." The specific grammar productions are derived systematically from the transition function.

10.5. $L = \{x \in \{\boldsymbol{a}, \boldsymbol{b}, \boldsymbol{c}\}^{*} \mid |x|_{\boldsymbol{a}} + |x|_{\boldsymbol{b}} = |x|_{\boldsymbol{c}}\}$.

(a) **PDA via final state**: Use two stack symbols A (surplus of $\boldsymbol{a}/\boldsymbol{b}$) and B (surplus of $\boldsymbol{c}$) to handle symbols in any order.

P: single reading state q, final state f, bottom marker Z:

$$\begin{aligned}
&\delta(q,\boldsymbol{a},Z)=\langle q,AZ\rangle, && \delta(q,\boldsymbol{a},A)=\langle q,AA\rangle, && \delta(q,\boldsymbol{a},B)=\langle q,\lambda\rangle,\\
&\delta(q,\boldsymbol{b},Z)=\langle q,AZ\rangle, && \delta(q,\boldsymbol{b},A)=\langle q,AA\rangle, && \delta(q,\boldsymbol{b},B)=\langle q,\lambda\rangle,\\
&\delta(q,\boldsymbol{c},Z)=\langle q,BZ\rangle, && \delta(q,\boldsymbol{c},B)=\langle q,BB\rangle, && \delta(q,\boldsymbol{c},A)=\langle q,\lambda\rangle.
\end{aligned}$$

When $\boldsymbol{a}$ or $\boldsymbol{b}$ is read with B on top, the pending $\boldsymbol{c}$-surplus is cancelled, and vice versa. Accept by final state: $\delta(q,\lambda,Z)=\langle f,Z\rangle$ (fires when input exhausted and stack balanced).

(b) **PDA via empty stack**: Pop Z as well. Add $\delta(q,\lambda,Z)=\langle q,\lambda\rangle$ (the stack becomes empty when balanced).

(c) **Counting automaton**: No. The counts $|x|_{\boldsymbol{a}}+|x|_{\boldsymbol{b}}$ and $|x|_{\boldsymbol{c}}$ can be unbounded.

(d) **DPDA**: Yes. The two-symbol construction in part (a) is deterministic: for each (q,σ,X) triple there is at most one transition, since every input symbol and every stack top symbol determine a unique action. The only potential conflict is the λ-transition $\delta(q,\lambda,Z)=\langle f,Z\rangle$, which must not coexist with other q-transitions on Z. This is resolved by moving the λ-acceptance to a separate final state reached only when input is exhausted; in practice, the construction is deterministic for all reachable configurations. Hence a DPDA for L exists.

(e) **Grammar from PDA**: Apply Definition 10.7 to the PDA in part (a) to derive the context-free grammar.

10.6. **Closure of $\mathfrak{P}_\Sigma$ and $\mathscr{A}_\Sigma$ under inverse homomorphism.**

(a) **$\mathfrak{P}_\Sigma$ is closed under inverse homomorphism**: Given a context-free language L over Σ and a homomorphism $h:\Delta^*\to\Sigma^*$, we want $h^{-1}(L)=\{w\in\Delta^*\mid h(w)\in L\}$ to be a CFL.

Proof: Construct a PDA for $h^{-1}(L)$ from the PDA P for L. The new PDA P' uses a buffer: for each input symbol $a\in\Delta$, it replaces a with $h(a)$ in a buffer and then simulates P on $h(a)$ before reading the next input symbol. The new PDA processes $h(a)$ letter-by-letter from the buffer. Since h maps to a finite string, this is a finite process per input symbol. Thus P' accepts exactly $h^{-1}(L)$, which is a CFL.

(b) **$\mathscr{A}_\Sigma$ (DCFL) is closed under inverse homomorphism**: Given a DCFL L and a homomorphism h, $h^{-1}(L)$ is also a DCFL.

Proof: The same buffering construction applied to a DPDA for L yields a DPDA for $h^{-1}(L)$, since the DPDA is deterministic and the buffering of $h(a)$ (a fixed string per input symbol a) preserves determinism. □

10.7. **A finite language not recognized by any one-state PDA via final state.**

Example: $L=\{\boldsymbol{ab}\}$.

Proof: Every one-state PDA P with state set $\{t\}$, start state t, and final states $\{t\}$ trivially accepts λ: the initial configuration has P already in the (unique) final state with empty input, so λ is accepted regardless of the stack content. Since $\lambda\notin L=\{\boldsymbol{ab}\}$, no one-state PDA via final state can recognize L.

More generally, any finite language that does not contain λ is not recognizable by any one-state PDA via final state, since such a machine always accepts λ. □

10.8. $L = \{\boldsymbol{a}^n\boldsymbol{b}^n\boldsymbol{c}^m\boldsymbol{d}^m \mid n, m \in \mathbb{N}\}$.

(a) **PDA via final state**: Push A on $\boldsymbol{a}$; pop A on $\boldsymbol{b}$; push B on $\boldsymbol{c}$; pop B on $\boldsymbol{d}$; accept when balanced. States: $\{q_0, q_1, q_2, q_f\}$. Initial state q_0, final state q_f. Stack alphabet $\{Z, A, B\}$. $\delta(q_0, \boldsymbol{a}, Z) = \langle q_0, AZ\rangle$, $\delta(q_0, \boldsymbol{a}, A) = \langle q_0, AA\rangle$, $\delta(q_0, \boldsymbol{b}, A) = \langle q_1, \lambda\rangle$, $\delta(q_1, \boldsymbol{b}, A) = \langle q_1, \lambda\rangle$, After reading $\boldsymbol{b}^n$, the machine is in state q_1 with Z on top of the stack. Then: $\delta(q_1, \boldsymbol{c}, Z) = \langle q_2, BZ\rangle$, $\delta(q_2, \boldsymbol{c}, B) = \langle q_2, BB\rangle$, $\delta(q_2, \boldsymbol{d}, B) = \langle q_2, \lambda\rangle$, $\delta(q_2, \lambda, Z) = \langle q_f, Z\rangle$. Also $\delta(q_0, \lambda, Z) = \langle q_1, Z\rangle$ to handle $n = 0$, and $\delta(q_1, \lambda, Z) = \langle q_2, Z\rangle$ for $m = 0$ (with appropriate final states).

(b) **PDA via empty stack**: Similar; pop Z at the end.

(c) **DPDA**: Yes. The DPDA deterministically pushes As, then pops As, then pushes Bs, then pops Bs. Each step is deterministic.

(d) **Counting automaton**: No. Both n and m are unbounded and independently variable.

(e) **Grammar from PDA** (Definition 10.7): Apply the systematic construction from the PDA in part (b) to obtain a context-free grammar. Variables are of the form $\langle q_i, X, q_j\rangle$ for states q_i, q_j and stack symbol X.

10.9. **Prove by induction (Theorem 10.2):** $\langle s, x, S\rangle \overset{*}{\vdash} \langle s, \lambda, \beta\rangle \Leftrightarrow S \overset{*}{\Rightarrow} x\beta$ **as a leftmost derivation.**

Claim ($P(n)$): For all $n \geq 0$ and strings x, β: $\langle s, x, S\rangle \overset{*}{\vdash} \langle s, \lambda, \beta\rangle$ in n moves iff $S \overset{*}{\Rightarrow} x\beta$ in n leftmost-derivation steps.

Base ($n = 0$): $\langle s, \lambda, S\rangle \overset{*}{\vdash} \langle s, \lambda, S\rangle$ (zero moves) iff $x = \lambda, \beta = S$, and $S \overset{*}{\Rightarrow}{}^{0}\, \lambda \cdot S$ (zero steps). ✓

Step: Assume $P(n)$ for all shorter computations. A computation of $n+1$ steps: $\langle s, ay, S\rangle \vdash \langle s, y, \gamma\rangle \overset{*}{\vdash} \langle s, \lambda, \beta\rangle$ (first move reads a, pushes γ for top of S). By the construction of the PDA (Theorem 10.2), this first step corresponds to applying a grammar production $S \to a\gamma$ (GNF production, starting with terminal a). By $P(n)$, $\langle s, y, \gamma\rangle \overset{*}{\vdash} \langle s, \lambda, \beta\rangle$ iff $\gamma \overset{*}{\Rightarrow} y\beta$ as leftmost derivation. So the full computation corresponds to $S \Rightarrow a\gamma \overset{*}{\Rightarrow} ay\beta = x\beta$. □

10.10. **Grammar for regular expressions: convert to GNF, find corresponding PDA.**

Grammar: $R \to \boldsymbol{a}|\boldsymbol{b}|\boldsymbol{c}|\boldsymbol{\epsilon}|\boldsymbol{\emptyset}|(R \cdot R)|(R \cup R)|R^{\boldsymbol{*}}$.

(a) **GNF conversion**: Introduce nonterminal Y to handle the middle of productions. The right-hand sides starting with **(** are already in GNF form. Productions starting with R:

$R^{\boldsymbol{*}} = R^{\boldsymbol{*}}$ needs to start with a terminal. But $R^{\boldsymbol{*}}$ starts with nonterminal R. Expand R: $R \to \boldsymbol{a}^{\boldsymbol{*}}|\boldsymbol{b}^{\boldsymbol{*}}|\boldsymbol{c}^{\boldsymbol{*}}|\boldsymbol{\epsilon}^{\boldsymbol{*}}|\boldsymbol{\emptyset}^{\boldsymbol{*}}|(R \cdot R)^{\boldsymbol{*}}|(R \cup R)^{\boldsymbol{*}}|R^{\boldsymbol{**}}$

This introduces left recursion ($R^{\boldsymbol{**}}$). Use the standard GNF transformation:

In GNF, $R^{\boldsymbol{*}}$ needs the first symbol to be a terminal. Since $R^{\boldsymbol{*}}$ starts with R, which generates strings starting with $\boldsymbol{a}, \boldsymbol{b}, \boldsymbol{c}, \boldsymbol{\epsilon}, \boldsymbol{\emptyset}, \boldsymbol{(}$, we get: $R \to \boldsymbol{a}^{\boldsymbol{*}} \mid \boldsymbol{b}^{\boldsymbol{*}} \mid (R \cdot R)^{\boldsymbol{*}} \mid \ldots$

Let Y handle the tail after the first terminal in longer productions: $R \to \boldsymbol{(}YR\boldsymbol{)}$ where $Y \to R\cdot$ or $Y \to R\cup$.

More precisely in GNF:

$$R \to \boldsymbol{a} \mid \boldsymbol{b} \mid \boldsymbol{c} \mid \boldsymbol{\epsilon} \mid \boldsymbol{\emptyset}$$
$$R \to \boldsymbol{(}RYR\boldsymbol{)}$$
$$R \to \boldsymbol{(}RZR\boldsymbol{)}$$
$$R \to R^{\boldsymbol{*}} \to \text{(substitute and eliminate left recursion)}$$

For $R^{\boldsymbol{*}}$: introduce $R \to \boldsymbol{a}Y_a \mid \boldsymbol{b}Y_b \mid \ldots$ where $Y_a \to \boldsymbol{*}$.

(b) **PDA from GNF grammar** (Definition 10.6): The PDA has a single state q, initial stack symbol R. For each GNF production $R \to a\alpha$, add transition $\delta(q, a, R) = \langle q, \alpha\rangle$. The PDA accepts via empty stack.

(c) **PDA accepting via final state** (Theorem 10.4): From the empty-stack PDA P, construct P_f by adding a new initial state that pushes a marker, and a new final state that pops the marker. $L(P_f) = L(P)$.

10.11. $L = \{\boldsymbol{a}^i\boldsymbol{b}^j\boldsymbol{c}^j\boldsymbol{d}^i \mid i, j \in \mathbb{N}\}$**.**

(a) **PDA via final state**: Push A on $\boldsymbol{a}$; push B on $\boldsymbol{b}$; pop B on $\boldsymbol{c}$; pop A on $\boldsymbol{d}$; accept when stack returns to Z. States $\{q_0, q_1, q_2, q_3, q_f\}$, alphabet $\{Z, A, B\}$: $\delta(q_0, \boldsymbol{a}, Z) = \langle q_0, AZ\rangle$, $\delta(q_0, \boldsymbol{a}, A) = \langle q_0, AA\rangle$, $\delta(q_0, \boldsymbol{b}, A) = \langle q_1, BA\rangle$, $\delta(q_0, \boldsymbol{b}, Z) = \langle q_1, BZ\rangle$ (for $i = 0$), $\delta(q_1, \boldsymbol{b}, B) = \langle q_1, BB\rangle$, $\delta(q_1, \boldsymbol{c}, B) = \langle q_2, \lambda\rangle$, $\delta(q_2, \boldsymbol{c}, B) = \langle q_2, \lambda\rangle$, $\delta(q_2, \boldsymbol{d}, A) = \langle q_3, \lambda\rangle$, $\delta(q_3, \boldsymbol{d}, A) = \langle q_3, \lambda\rangle$, $\delta(q_2, \lambda, Z) = \langle q_f, Z\rangle$, $\delta(q_3, \lambda, Z) = \langle q_f, Z\rangle$.

(b) **PDA via empty stack**: Pop Z as well; add $\delta(\cdot, \lambda, Z) = \langle q_f, \lambda\rangle$ with appropriate state.

(c) **DPDA**: Yes. The language has a fixed block structure: $\boldsymbol{a}^i, \boldsymbol{b}^j, \boldsymbol{c}^j, \boldsymbol{d}^i$, so the DPDA deterministically processes each block. The stack remembers i (via As) and j (via Bs overlaid on top). This is deterministic.

(d) **Counting automaton**: No; both i and j are unbounded.

(e) **Grammar from PDA** (Definition 10.7): Apply the construction to the PDA in part (b). Variables $\langle q_i, X, q_j\rangle$ with appropriate productions derived from the transitions.

10.12. **Find G_{P_3} from Example 10.5 using Definition 10.7.**

PDA P_3 is given in the text (Example 10.5). Definition 10.7 (PDA to grammar) constructs variables $[q_i A q_j]$ for each state pair (q_i, q_j) and stack symbol A.

Productions: - For the initial variable: $S \to [q_0 Z q_j]$ for each state q_j. - For each transition $\delta(q, a, A) \ni \langle r, B_1 B_2 \cdots B_k\rangle$: $[qAq'] \to a[rB_1 s_1][s_1 B_2 s_2]\cdots[s_{k-1}B_k q']$ for all states $s_1, \ldots, s_{k-1}$. - For $\delta(q, a, A) \ni \langle r, \lambda\rangle$: $[qAr] \to a$.

The specific grammar for P_3 follows this schema with the particular transitions of P_3 filled in from Example 10.5.

10.13. **Prove by induction (Theorem 10.3):** $A^{st} \overset{*}{\Rightarrow} x \Leftrightarrow \langle s, x, A\rangle \overset{*}{\vdash} \langle t, \lambda, \lambda\rangle$**.**

Claim ($P(n)$): For all n, for all states s, t, stack symbol A, string x: $A^{st} \overset{*}{\Rightarrow} x$ in n steps iff $\langle s, x, A\rangle \overset{*}{\vdash} \langle t, \lambda, \lambda\rangle$ in n moves.

Base ($n = 1$): $A^{st} \to a$ (terminal production from grammar of Definition 10.7) corresponds to $\delta(s, a, A) \ni \langle t, \lambda\rangle$, giving $\langle s, a, A\rangle \vdash \langle t, \lambda, \lambda\rangle$. ✓

Step: Assume $P(k)$ for all $k < n$. A derivation $A^{st} \overset{*}{\Rightarrow} x$ of length n begins with $A^{st} \Rightarrow aB_1^{s_0 s_1} B_2^{s_1 s_2} \cdots B_m^{s_{m-1} t}$ (using a production from $\delta(s, a, A) \ni \langle s_0, B_1 \cdots B_m\rangle$). Then each $B_i^{s_{i-1} s_i}$ independently derives x_i in fewer than n steps. By IH, $\langle s_{i-1}, x_i, B_i\rangle \overset{*}{\vdash} \langle s_i, \lambda, \lambda\rangle$. Chaining: $\langle s, ax_1 \cdots x_m, A\rangle \vdash \langle s_0, x_1 \cdots x_m, B_1 \cdots B_m\rangle \overset{*}{\vdash} \langle t, \lambda, \lambda\rangle$. So $x = ax_1 \cdots x_m$ and $\langle s, x, A\rangle \overset{*}{\vdash} \langle t, \lambda, \lambda\rangle$. □

10.14. $L = \{\boldsymbol{a}^n\boldsymbol{b}^n\boldsymbol{c}^m\boldsymbol{d}^m\} \cup \{\boldsymbol{a}^i\boldsymbol{b}^j\boldsymbol{c}^j\boldsymbol{d}^i\}$**.**

(a) **PDA via final state**: Use nondeterminism to choose which language the input belongs to. P: from initial state, nondeterministically branch to handle either $\boldsymbol{a}^n\boldsymbol{b}^n\boldsymbol{c}^m\boldsymbol{d}^m$ (Exercise 10.8) or $\boldsymbol{a}^i\boldsymbol{b}^j\boldsymbol{c}^j\boldsymbol{d}^i$ (Exercise 10.11).

(b) **PDA via empty stack**: Combine with the union grammar $S \to S_1 | S_2$.

(c) **DPDA**: No. The string $\boldsymbol{a}^n\boldsymbol{b}^n\boldsymbol{c}^n\boldsymbol{d}^n$ is in both components of the union, and the DPDA would need to nondeterministically decide which structure applies. It can be shown this language is not deterministically context-free.

(d) **Counting automaton**: No; the string lengths are unbounded.

(e) **Grammar from PDA**: Apply Definition 10.7 to part (b).

10.15. **Find G_{P_G} from Example 10.6 using Definition 10.7.**

Example 10.6 gives a PDA P_G constructed from a context-free grammar G via the GNF construction. Applying Definition 10.7 to P_G recovers a grammar equivalent to G. The variables are $[qAq']$ and the productions follow the template in Definition 10.7. The resulting grammar may differ from G but generates the same language.

10.16. **Prove (Theorem 10.4): $\langle s, xy, \alpha\rangle \overset{*}{\vdash} \langle s, y, \beta\rangle$ in P iff $\langle s, xy, \alpha Y\rangle \overset{*}{\vdash} \langle s, y, \beta Y\rangle$ in P_f.**

Proof by induction on the number of moves:

Base (0 moves): Both sides are trivially equivalent (same configuration).

Step: If $\langle s, xy, \alpha\rangle \vdash \langle s', x'y, \alpha'\rangle \overset{*}{\vdash} \langle s, y, \beta\rangle$ in P: The first move in P corresponds to $\delta_P(s, a, A) \ni \langle s', \gamma\rangle$ (reading a or λ, popping A, pushing γ). In P_f, the same move is available: δ_{P_f} includes all moves of δ_P plus new moves for the marker Y. The marker Y sits below α and is not affected by moves on α. By IH on the remaining $k-1$ moves, the result follows for P_f with Y appended. □

10.17. **Prove (Theorem 10.5): same correspondence for P_λ.**

Proof by induction: Symmetric to Exercise 10.16; the marker Y at the bottom of the stack (representing "no more stack to pop") is unaffected by all moves above it, so the computation in P with stack α corresponds exactly to the computation in P_λ with stack αY. □

10.18. $\{x \in \{\boldsymbol{a}, \boldsymbol{b}, \boldsymbol{c}\}^* \mid |x|_{\boldsymbol{a}} = |x|_{\boldsymbol{b}} \wedge |x|_{\boldsymbol{b}} > |x|_{\boldsymbol{c}}\}$ **is not context free.**

Proof using closure properties:

Let $K = \{x \mid |x|_{\boldsymbol{a}} = |x|_{\boldsymbol{b}} \wedge |x|_{\boldsymbol{b}} > |x|_{\boldsymbol{c}}\}$.

Suppose $K \in \mathfrak{P}_\Sigma$. Consider the regular language $R = \boldsymbol{a}^*\boldsymbol{b}^*\boldsymbol{c}^*$ (strings of the form $\boldsymbol{a}^i\boldsymbol{b}^j\boldsymbol{c}^k$).

$K \cap R = \{\boldsymbol{a}^n\boldsymbol{b}^n\boldsymbol{c}^k \mid n > k\}$.

If K were a CFL, then $K \cap R$ would be a CFL (by Theorem 10.6: CFL ∩ regular = CFL). But $\{\boldsymbol{a}^n\boldsymbol{b}^n\boldsymbol{c}^k \mid n > k\}$ is not context free (one can apply the pumping lemma to strings $\boldsymbol{a}^p\boldsymbol{b}^p\boldsymbol{c}^{p-1}$; any pumping either changes the $\boldsymbol{a}$-count differently from the $\boldsymbol{b}$-count, or changes the $\boldsymbol{c}$-count, in each case violating the constraints).

So K is not a CFL. □

10.19. **Two-tape automaton.**

(a) **Transition function**: For a two-tape automaton with tapes over Σ, the state transition function δ has domain $S \times (\Sigma \cup \{\boldsymbol{B}\}) \times (\Sigma \cup \{\boldsymbol{B}\})$ (current state, symbol read from tape 1, symbol read from tape 2) and range $S \times (\Sigma \cup \{\boldsymbol{B}\}) \times \{L, R, S\} \times (\Sigma \cup \{\boldsymbol{B}\}) \times \{L, R, S\}$ (new state, write to tape 1, move head 1, write to tape 2, move head 2).

(b) **Two-tape automaton for $\{\boldsymbol{a}^n\boldsymbol{b}^n\boldsymbol{c}^n \mid n \geq 1\}$**: - Tape 1 contains the input $\boldsymbol{a}^n\boldsymbol{b}^n\boldsymbol{c}^n$. - Phase 1: Use head 1 to read $\boldsymbol{a}$s; use head 2 to record a mark for each $\boldsymbol{a}$. - Phase 2: Read $\boldsymbol{b}$s on tape 1 while erasing marks on tape 2; verify counts match. - Phase 3: Read $\boldsymbol{c}$s on tape 1; use head 2 (now at start) to verify count matches. - Accept if all three counts match and tape 1 is exhausted.

10.20. **$\{\boldsymbol{a}^n\boldsymbol{b}^n\boldsymbol{c}^n\}$ is not context free; $\{x \mid |x|_{\boldsymbol{a}} = |x|_{\boldsymbol{b}}\}$ is not context free.**

(a) **$\{\boldsymbol{a}^n\boldsymbol{b}^n\boldsymbol{c}^n\}$ is not CFL**: Assume it is, with pumping constant N. Take $s = \boldsymbol{a}^N\boldsymbol{b}^N\boldsymbol{c}^N$. By the pumping lemma, $s = uvwxy$ with $|vwx| \leq N$ and $|vx| \geq 1$.

Since $|vwx| \leq N$, the substring vwx cannot span all three blocks. So v and x together involve at most two of $\{\boldsymbol{a}, \boldsymbol{b}, \boldsymbol{c}\}$. Pumping changes the count of at most two symbols while leaving the third unchanged. But all three must be equal, so the pumped string fails $|u|_{\boldsymbol{a}} = |u|_{\boldsymbol{b}} = |u|_{\boldsymbol{c}}$ for some $i \neq 1$. Contradiction. □

(b) $L = \{x \mid |x|_{\boldsymbol{a}} = |x|_{\boldsymbol{b}}\}$ **over $\{\boldsymbol{a}, \boldsymbol{b}, \boldsymbol{c}\}$ is a CFL**:

The grammar G:

$$S \to \boldsymbol{c}S \mid SS \mid \boldsymbol{a}S\boldsymbol{b} \mid \boldsymbol{b}S\boldsymbol{a} \mid \lambda$$

generates exactly $\{x \in \{\boldsymbol{a}, \boldsymbol{b}, \boldsymbol{c}\}^* \mid |x|_{\boldsymbol{a}} = |x|_{\boldsymbol{b}}\}$: every production that introduces an $\boldsymbol{a}$ also introduces a $\boldsymbol{b}$ (and vice versa), while $\boldsymbol{c}$-steps and concatenation preserve balance.

Note: The exercise may have intended the language $\{x \mid |x|_{\boldsymbol{a}} = |x|_{\boldsymbol{b}} = |x|_{\boldsymbol{c}}\}$, which *is* not CFL. The proof for that language follows the hint: if $\{x \mid |x|_{\boldsymbol{a}} = |x|_{\boldsymbol{b}} = |x|_{\boldsymbol{c}}\}$ were CFL, intersect with the regular language $\boldsymbol{a}^*\boldsymbol{b}^*\boldsymbol{c}^*$ (closure of CFL under intersection with regular languages) to obtain $\{\boldsymbol{a}^n\boldsymbol{b}^n\boldsymbol{c}^n \mid n \geq 0\}$, which would then be CFL — contradicting part (a). □

10.21. **DPDA constructions and non-DCFL example.**

(a) DPDA for $\{\boldsymbol{c}^n\boldsymbol{b}^m \mid n \geq 1, n = m\} \cup \{\boldsymbol{a}^n\boldsymbol{b}^m \mid n \geq 1, n = 2m\}$:

States $\{q_0, q_c, q_a, q_f\}$. Stack alphabet $\{Z, A\}$.

From q_0: if first symbol is $\boldsymbol{c}$, go to q_c (push A); if first is $\boldsymbol{a}$, go to q_a (push A).

q_c: push A for each $\boldsymbol{c}$; pop A for each $\boldsymbol{b}$; accept when stack has Z and input done.

q_a: push A for each $\boldsymbol{a}$; when $\boldsymbol{b}$ comes, pop two As per $\boldsymbol{b}$; accept when stack has Z and input done.

(b) Define $h : \{\boldsymbol{a}, \boldsymbol{b}, \boldsymbol{c}\}^* \to \{\boldsymbol{a}, \boldsymbol{b}\}^*$ by $h(\boldsymbol{a}) = \boldsymbol{a}$, $h(\boldsymbol{b}) = \boldsymbol{b}$, $h(\boldsymbol{c}) = \boldsymbol{a}$. Then: $h(\{\boldsymbol{c}^n\boldsymbol{b}^m \mid n = m\}) = \{\boldsymbol{a}^n\boldsymbol{b}^n \mid n \geq 1\}$ and $h(\{\boldsymbol{a}^n\boldsymbol{b}^m \mid n = 2m\}) = \{\boldsymbol{a}^n\boldsymbol{b}^m \mid n = 2m\}$. $h(L) = \{\boldsymbol{a}^n\boldsymbol{b}^n\} \cup \{\boldsymbol{a}^n\boldsymbol{b}^m \mid n = 2m\}$. This language is not a DCFL (it can be shown that any DPDA accepting it would have conflicting behavior on some strings).

10.22. **Ogden's lemma: $\{\boldsymbol{a}^k\boldsymbol{b}^n\boldsymbol{c}^m \mid k \neq n \wedge n \neq m\}$ is not CFL.**

Assume $L = \{\boldsymbol{a}^k\boldsymbol{b}^n\boldsymbol{c}^m \mid k \neq n, n \neq m\}$ is CFL with Ogden constant N.

Choose $p > N$. Let $s = \boldsymbol{a}^{p!+p}\boldsymbol{b}^p\boldsymbol{c}^{p!+p} \in L$ (since $p! + p \neq p$). Mark all $\boldsymbol{b}$-symbols.

By Ogden's lemma: $s = uvwxy$ with vwx containing at most N marked positions, vx has at least one marked position, all pumped strings in L.

Since all marks are on $\boldsymbol{b}$s, and $|vwx| \le N \le p$, vwx lies within the $\boldsymbol{b}$-block (possibly with some $\boldsymbol{a}$s or $\boldsymbol{c}$s on the boundary). Since vx has at least one $\boldsymbol{b}$, $|vx|_{\boldsymbol{b}} \ge 1$; let $|vx|_{\boldsymbol{b}} = j$ $(1 \le j \le N)$.

For the pumped string $uv^i wx^i y$: the count of $\boldsymbol{b}$s is $p + (i-1)j$. The counts of $\boldsymbol{a}$s and $\boldsymbol{c}$s may also change if v or x extend into the $\boldsymbol{a}$ or $\boldsymbol{c}$ blocks.

Case 1: vx contains only $\boldsymbol{b}$s. Then $k = p! + p$ and $m = p! + p$ for all i. Choose i such that $p + (i-1)j = p! + p$: then $(i-1)j = p!$, so $i - 1 = p!/j$. Since $j \le N \le p - 1$, $j \mid p!$, so i is an integer. The pumped string has $|\boldsymbol{b}| = p! + p = k = m$, violating $k \ne n$ or $n \ne m$. Contradiction.

Case 2: vx spans the $\boldsymbol{a}/\boldsymbol{b}$ boundary. Let $|vx|_{\boldsymbol{a}} = \ell \ge 0$ and $|vx|_{\boldsymbol{b}} = j \ge 1$ (at least one marked $\boldsymbol{b}$). Choose $i - 1 = p!/j$ (an integer since $j \le N < p$). Then the pumped string has $n = p + p! = p! + p$ and $m = p! + p$ (the $\boldsymbol{c}$-block is untouched), so $n = m$, violating $n \ne m$. Contradiction.

Case 3: vx spans the $\boldsymbol{b}/\boldsymbol{c}$ boundary. Let $|vx|_{\boldsymbol{b}} = j \ge 1$ and $|vx|_{\boldsymbol{c}} = r \ge 0$. Choose $i - 1 = p!/j$. The pumped string has $n = p! + p$ and $k = p! + p$ (the $\boldsymbol{a}$-block is untouched), so $k = n$, violating $k \ne n$. Contradiction. □

10.23. **Prove (Theorem 10.6) by induction.**

Theorem 10.6 states that if P_1 is a PDA and A_2 is a DFA (or regular language), then their "intersection" machine $P^{\cap}$ (Product machine) satisfies the correspondence stated.

Proof by induction on the number of moves:

Base (0 moves): $\langle\langle s_1, s_2\rangle, y, \alpha\rangle \vdash^* \langle\langle s_1, s_2\rangle, y, \alpha\rangle$ (trivially). Here $t_1 = s_1$, $t_2 = s_2$, $x = \lambda$, so $\overline{\delta}_2(s_2, \lambda) = s_2 = t_2$. ✓

Step: If the product machine takes $k+1$ moves from $\langle\langle s_1, s_2\rangle, xy, \alpha\rangle$: the first move reads some a (or λ) and moves to $\langle\langle s_1', s_2'\rangle, \cdot, \cdot\rangle$. In the product machine, this corresponds to a joint transition: P_1 moves from s_1 and A_2 reads a to move from s_2. By IH on k moves, the remaining computation in the product machine corresponds to $P^{\cap}$ and $\overline{\delta}_2$. The A_2 component is precisely $\overline{\delta}_2(s_2, x)$ after reading x. □

10.24. **Prove: DPDA P has equivalent DPDA P' with total transitions, no empty stack, scans full input.**

(a) **Total transitions**: Add a dead state q_d and for every undefined $\delta(q, a, X)$, add $\delta(q, a, X) = \langle q_d, X\rangle$ (stay in dead state, preserve stack). q_d is nonfinal, so no new strings are accepted.

(b) **Never empty stack**: Add a bottom-of-stack marker **\$** pushed at the start. Rules that would pop the last symbol instead pop **\$** and push it back (loop). Alternatively, ensure all stack-popping moves check that the stack has more than **\$**.

(c) **Always scans full input**: After reaching a final state or dead state, keep reading input until **¢** (end marker) is reached without changing acceptance. Add transitions from all states (final or dead) that read any input symbol and stay in the same state.

All three modifications preserve determinism and do not change $L(P)$. □

10.25. **$\mathscr{A}_\Sigma$ (DCFL) is closed under complementation.**

Using Exercise 10.24: For a DPDA P (accepting via final state), P can be made total (all transitions defined), never emptying stack, and always scanning full input. Under these conditions:

Every computation on every input string either ends in a final state (accept) or a nonfinal state (reject). There are no infinite loops or premature halts.

Construct P' by swapping final and nonfinal states of P: a string is accepted by P' iff it is rejected by P. The main issue (mentioned in the hint) is λ-moves cycling through both final and nonfinal states: since Exercise 10.24 ensures the DPDA is well-behaved (complete, non-emptying, full-input scanning), after processing any input string, the computation ends deterministically in exactly one state. Swapping final/nonfinal gives the complement. □

10.26. **Example: $\mathscr{A}_\Sigma$ not closed under concatenation.**

Caution: the naive example $\{\boldsymbol{a}^n\boldsymbol{b}^n\}^2 = \{\boldsymbol{a}^n\boldsymbol{b}^n\boldsymbol{a}^m\boldsymbol{b}^m \mid n,m \geq 0\}$ IS in fact a DCFL — the DPDA detects the split between the two factors by stack-emptiness after the first **b**-block, then restarts. So that pair does not witness non-closure.

A correct witness uses the fact that $\mathscr{A}_\Sigma$ is not closed under union (standard result: $L_1 = \{\boldsymbol{a}^n\boldsymbol{b}^n\boldsymbol{c}^m\}$ and $L_2 = \{\boldsymbol{a}^m\boldsymbol{b}^n\boldsymbol{c}^n\}$ are DCFL but $L_1 \cup L_2$ is not), combined with the fact that if $\mathscr{A}_\Sigma$ were closed under concatenation and complement (it is closed under complement by Exercise 10.25), one could derive closure under union via a construction of Ginsburg and Greibach (1966), contradicting that result. Hence $\mathscr{A}_\Sigma$ is not closed under concatenation. □

10.27. **Example: $\mathscr{A}_\Sigma$ not closed under Kleene closure.**

$L = \{\boldsymbol{a}^n\boldsymbol{b}^n \mid n \geq 1\}$ is a DCFL. Its Kleene closure L^* is also a DCFL: a DPDA pushes A for each **a**, pops A for each **b**, and when the stack empties it enters an accepting state and is ready to begin another **ab**-block or accept end-of-input. Reading **b** with an empty stack, or ending while the stack is non-empty, causes rejection. This scan is deterministic.

For a genuine witness that $\mathscr{A}_\Sigma$ is not closed under Kleene closure, consider a DCFL whose star is not deterministic. For example, $L_3 = \{w\,w^r\,\boldsymbol{c} \mid w \in \{\boldsymbol{a},\boldsymbol{b}\}^*\}$ is a DCFL (the **c** marks the center). $(L_3)^*$ requires the DPDA to guess how many copies of L_3 are concatenated and where each **c**-terminated palindrome ends, which cannot be done deterministically.

10.28. $\{\boldsymbol{c}\boldsymbol{a}^n\boldsymbol{b}^m \mid n = m\} \cup \{\boldsymbol{a}^n\boldsymbol{b}^m \mid n = 2m\}$ **is a DCFL.**

A DPDA can distinguish the two cases by the first symbol: if it reads $\boldsymbol{c}$, it proceeds to recognize $\boldsymbol{a}^n\boldsymbol{b}^n$ (push one A per $\boldsymbol{a}$, pop one per $\boldsymbol{b}$); if it reads $\boldsymbol{a}$, it proceeds to recognize $\boldsymbol{a}^n\boldsymbol{b}^{n/2}$ (push two As per $\boldsymbol{a}$, pop two per $\boldsymbol{b}$, or equivalently push one per $\boldsymbol{a}$ and pop one per two $\boldsymbol{b}$s). Since the first symbol deterministically determines which component applies, the DPDA is well-defined. □

10.29. $L_1 - R_2$ **is context free if** L_1 **is CFL and** R_2 **is regular.**

(a) **Via product machine**: Extend the intersection proof (Theorem 10.6). Construct a product machine where the PDA simulates L_1 and the DFA simulates R_2; modify final states to accept when the PDA is in a final state and the DFA is in a *nonfinal* state. This gives $L_1 \cap \overline{R_2} = L_1 - R_2$.

(b) **Via closure**: $L_1 - R_2 = L_1 \cap \overline{R_2}$. Since regular languages are closed under complement, $\overline{R_2}$ is regular. Since CFLs are closed under intersection with regular languages (Theorem 10.6), $L_1 \cap \overline{R_2}$ is a CFL. □

10.30. $L_1 \cup R_2$ **is context free if** L_1 **is CFL and** R_2 **is regular.**

(a) **Via modified product machine**: Accept when the PDA is in a final state (for L_1) OR the DFA is in a final state (for R_2). This is a product machine with final states $F_1 \times S_2 \cup S_1 \times F_2$.

(b) **Via closure**: $L_1 \cup R_2$: since R_2 is regular, it is also a CFL. CFLs are closed under union: $L_1 \cup R_2 \in \mathfrak{P}_\Sigma$. □

10.31. **If L_1 is DCFL and R_2 is regular, $R_2 - L_1$ is DCFL.**

$R_2 - L_1 = R_2 \cap \overline{L_1}$. Since L_1 is DCFL, $\overline{L_1}$ is DCFL (Exercise 10.25). Since R_2 is regular (hence DCFL), and DCFLs are closed under intersection with regular languages (Theorem 10.6 analog for DPDAs), $R_2 \cap \overline{L_1}$ is DCFL. □

10.32. $\{w\mathbf{2}w^r\}$ **is DCFL;** $\{ww^r\}$ **is not DCFL.**

(a) $\{w\mathbf{2}w^r\}$ **is DCFL**: The DPDA pushes all symbols before **2** onto the stack, then pops matching symbols for each character after **2** (reversed). Since **2** marks the center deterministically, the DPDA knows exactly when to switch from push to pop mode. □

(b) $\{ww^r\}$ **is not DCFL**: The palindrome-by-even-length language $\{ww^r\}$ is a CFL (grammar: $S \to \boldsymbol{a}S\boldsymbol{a} \mid \boldsymbol{b}S\boldsymbol{b} \mid \lambda$). However, the center of the string is not marked; the DPDA must guess when the first half ends. It can be shown using the Closure properties of DCFLs that $\{ww^r\}$ is not a DCFL. □

10.33. **Non-closure of $\mathscr{A}_\Sigma$ under various operations.**

(a) $L_1 \cdot L_2$ need not be DCFL: Example from Exercise 10.26.

(b) $L_1 - L_2$ need not be DCFL: Let L_1 be any DCFL and L_2 be any CFL that is not a DCFL; then $L_1 - L_2 = L_1 \cap \overline{L_2}$. Since DCFLs are not closed under complement, $\overline{L_2}$ need not be a CFL at all. A concrete pair: $L_1 = \{\boldsymbol{a}^n\boldsymbol{b}^n\boldsymbol{c}^m \mid n, m \geq 0\}$ (DCFL) and $L_2 = \{\boldsymbol{a}^n\boldsymbol{b}^m\boldsymbol{c}^m \mid n, m \geq 0\}$ (DCFL). Then $L_1 - L_2 = \{w \in L_1 \mid |w|_{\boldsymbol{b}} \neq |w|_{\boldsymbol{c}}\}$; verifying this is not DCFL follows from the non-closure of DCFLs under difference.

(c) L_1^* need not be DCFL: Exercise 10.27.

(d) L_1^r need not be DCFL: $L = \{ww^r\boldsymbol{c}\}$ (DCFL); $L^r = \{\boldsymbol{c}ww^r\}$ which is also DCFL. A better example: L is a DCFL such that L^r is CFL but not DCFL.

10.34. **Closure of $\mathfrak{P}_\Sigma$ and $\mathscr{A}_\Sigma$ under quotient.**

(a) $\mathfrak{P}_\Sigma$ **closed under quotient**: $L/R = \{x \mid \exists y \in R : xy \in L\}$ (right quotient by regular R). Since $\mathfrak{P}_\Sigma$ is closed under intersection with regular languages and homomorphisms, and the quotient by a regular language preserves context-freeness. **True**: $\mathfrak{P}_\Sigma$ is closed under right quotient by regular languages.

(b) $\mathscr{A}_\Sigma$ **closed under quotient**: Similarly, DCFLs are closed under right quotient by regular languages. **True**.

10.35. **Closure under operator b (Theorem 5.11).**

(a) $\mathfrak{P}_\Sigma$ is closed under b (the "bridge" or "between" operator): by constructing an appropriate PDA that handles the b operation using intersection with a regular language.

(b) $\mathscr{A}_\Sigma$ is closed under b: similarly, using the DPDA construction.

10.36. **Closure under operator Y (Theorem 5.7).**

(a) $\mathfrak{P}_\Sigma$ is closed under Y: $Y(L) = \{x \mid \exists y \in L : x \text{ is a subsequence of } y\}$ or similar. The specific construction depends on Definition 5.7. Typically: Yes, $\mathfrak{P}_\Sigma$ is closed if Y is a morphism-based or homomorphism-based operation.

(b) $\mathscr{A}_\Sigma$ is closed under Y: depends on the specific definition.

10.37. **Closure under prefix operator P (Exercise 5.16).**

(a) $\mathfrak{P}_\Sigma$ **closed under** P: $P(L) = \{x \mid \exists y : xy \in L\}$ (prefix language). If L is CFL with PDA M (accepting via empty stack), then $P(L)$ is recognized by modifying M to accept at any configuration reached on input x that could be extended to an accepting computation. Since the PDA is nondeterministic, $P(L)$ is CFL. **True**.

(b) $\mathscr{A}_\Sigma$ **closed under** P: For DPDAs, the prefix language is also DCFL. **True**.

10.38. **Closure under operator F (Exercise 5.19).**

(a) $\mathfrak{P}_\Sigma$ **closed under** F: $F(L)$ typically denotes the "suffix" or "final segment" language, or another specific operator from Exercise 5.19. If $F(L) = \{y \mid \exists x : xy \in L\}$ (suffix), then: CFLs are not closed under suffix in general. **False** in general.

(b) $\mathscr{A}_\Sigma$ **closed under** F: DCFLs are also not closed under suffix in general. **False**.

Chapter 11

Turing Machines — Solutions

11.1. **How is a Turing machine like an elevator?**

Analogy:

- **States** correspond to *floors* (the elevator's current position/state).
- **Input** corresponds to *button presses* (the sequence of requested floors).
- The elevator transitions from floor to floor based on button requests, just as the Turing machine transitions from state to state based on input symbols.

Essential missing component: An elevator has only *finite, bounded memory* — it can remember which buttons have been pressed (a finite set), but it cannot write new symbols to an unbounded tape. The critical component preventing an elevator from modeling a general computing device is the *unbounded read/write tape*. Without it, the elevator can only implement a finite state machine (regular language computation), not a Turing machine. The ability to store and retrieve arbitrarily large amounts of information is essential for general computation.

11.2. **Turing machine for palindromes $L = \{w \mid w = w^r\}$ over $\Sigma = \{\boldsymbol{a}, \boldsymbol{b}, \boldsymbol{c}\}$.**

(a) **One-tape, one-head TM**:
Strategy: Repeatedly match the leftmost and rightmost unprocessed symbols.

1. Mark the leftmost symbol a (replace with $\overline{a}$); move right to the rightmost unmarked symbol.
2. If the rightmost symbol matches a, mark it and move left; continue.
3. If the rightmost symbol does not match, reject.
4. When all symbols are marked (or only the center symbol for odd-length), accept.

States: $\{q_{\text{start}}, q_a, q_b, q_c, q_{\text{right}}, q_{\text{check},a}, q_{\text{check},b}, q_{\text{check},c}, q_{\text{return}}, q_{\text{accept}}, q_{\text{reject}}\}$.
Transitions: $\delta(q_{\text{start}}, \boldsymbol{a}) = (q_a, \overline{\boldsymbol{a}}, R)$ [mark $\boldsymbol{a}$, go right]; etc.

(b) **Linear Bounded Automaton (LBA)**: Since the tape head never needs to go beyond the input boundaries (matching leftmost/rightmost within the input), the TM above is already an LBA. The computation uses only the n tape cells containing the input.

(c) **Nondeterminism/multiple tapes**: A two-tape TM copies the input to tape 2, reverses tape 2, and compares tape 1 with tape 2 symbol by symbol — $O(n)$ time. A nondeterministic TM can guess the center and verify matching from both ends simultaneously.

11.3. **TM recognizing $\{\boldsymbol{a}^n \mid n$ is a perfect square$\}$ over $\Sigma = \{\boldsymbol{a}\}$.**

Strategy: The k-th perfect square is k^2. Note $(k+1)^2 = k^2 + (2k+1)$: successive perfect squares differ by successive odd numbers $1, 3, 5, 7, \ldots$

Algorithm:

1. If input is λ: accept (since $0 = 0^2$).
2. Mark the first $\boldsymbol{a}$ and subtract 1; this corresponds to $1^2 = 1$.
3. Then subtract 3; if exhausted, accept (that was $2^2 = 4$). If not enough, reject.
4. Then subtract 5; if exhausted accept ($3^2 = 9$). Etc.
5. Subtract $2k+1$ at each step for $k = 0, 1, 2, \ldots$; accept if the tape is exactly emptied, reject if running short.

The TM maintains the current step k (in unary, using a separate tape region or using a scratch area) and repeats the subtraction of $2k+1$. It accepts when exactly k^2 symbols have been processed.

11.4. **TM for languages over $\Sigma = \{\boldsymbol{a}, \boldsymbol{b}, \boldsymbol{c}\}$.**

(a) $\{\boldsymbol{a}^k\boldsymbol{b}^n\boldsymbol{c}^m \mid k \neq n \wedge n \neq m\}$:
Strategy: Count $|x|_a$, $|x|_b$, $|x|_c$ separately and verify $|x|_a \neq |x|_b$ and $|x|_b \neq |x|_c$.
A practical approach:

1. Verify the input has the form $\boldsymbol{a}^k\boldsymbol{b}^n\boldsymbol{c}^m$ (check symbol ordering); reject if not.
2. Pair off $\boldsymbol{a}$s and $\boldsymbol{b}$s: erase one $\boldsymbol{a}$ and one $\boldsymbol{b}$ in each pass. If they run out simultaneously, $k = n$; reject.
3. Pair off $\boldsymbol{b}$s and $\boldsymbol{c}$s similarly. If they run out simultaneously, $n = m$; reject.
4. If both checks show inequality, accept.

(b) $\{x \mid |x|_a \neq |x|_b \wedge |x|_b \neq |x|_c\}$ (arbitrary mixing):
Strategy: Count occurrences by marking symbols.

1. In pass 1: pair $\boldsymbol{a}$s with $\boldsymbol{b}$s by crossing out one of each per scan. If the pairing runs out on one side before the other, $|x|_a \neq |x|_b$ confirmed. If they balance, $|x|_a = |x|_b$; reject.
2. In pass 2: pair $\boldsymbol{b}$s with $\boldsymbol{c}$s. Similarly check $|x|_b \neq |x|_c$.
3. Accept iff both checks confirm inequality.

11.5. **Machine M with $L(M) = L_1(M) = L_2(M) = L_3(M) = \{x \in \{\boldsymbol{a}, \boldsymbol{b}, \boldsymbol{c}\}^* \mid |x|_a = |x|_b = |x|_c\}$.**

(a) **Construction**: Design M to:

1. Verify $|x|_a = |x|_b = |x|_c$ by crossing-off pairing.
2. If yes: **halt** with the tape containing $\boldsymbol{Y}$ in cell 1 (accept state is also the halt state for L_1, L_2, L_3).
3. If no: **halt** with $\boldsymbol{N}$ on the tape (reject for all acceptance modes).

Specifically: - $L(M)$: accept = halting in designated final state. - $L_1(M)$: accept = halting. - $L_2(M)$: accept = halting with $\boldsymbol{Y}$ on tape. - $L_3(M)$: accept = halting on a blank tape (we can arrange M to clear tape for yes-instances).
By designing M to halt on all inputs and write $\boldsymbol{Y}$/$\boldsymbol{N}$ appropriately, all four acceptance modes agree.

(b) **Is this always possible?**: Yes, for any Turing-acceptable language L. Proof: Given any TM A accepting L, by the $L_1/L_2/L_3$ equivalence theorems (Exercise 11.19), there is a halting TM A' accepting L in any of the four modes. If we require all four modes to agree, we need A to halt on *all* inputs (not just those in L). This is possible iff L is *recursive* (decidable). For Turing-acceptable but not recursive languages, $L(M) = L$ but M may not halt on all inputs, preventing $L_1(M) = L$. So the answer is: **yes if and only if L is recursive**. For every recursive language, we can build M with all four modes agreeing.

11.6. $L = \{ww \mid w \in \{\boldsymbol{a}, \boldsymbol{b}, \boldsymbol{c}\}^*\}$.

(a) **One-tape TM**: Strategy:
1. If input has odd length, reject (since $|ww| = 2|w|$ is even).
2. Mark the midpoint by counting: move right counting n symbols, then left counting n symbols to find the center (binary search or two-pass count).
3. From the start, match symbol i with symbol $n+i$ (for each i from 1 to n), crossing off matched pairs.
4. Accept if all pairs match; reject on any mismatch.

(b) **LBA**: Since the computation uses only the $2n$ cells of the input, the machine is already an LBA.

(c) **Multiple tapes**: A two-tape TM copies the first half of the input to tape 2, then compares tape 2 with the second half of tape 1 symbol by symbol. A nondeterministic TM can guess the midpoint and verify.

11.7. **Lexicographic ordering Turing machines.**

Given $\Sigma = \{\boldsymbol{a}_1, \ldots, \boldsymbol{a}_n\}$ with lexicographic order based on base-n numbering.

(a) **Successor TM**: Machine M_{succ} maps x to its lexicographic successor.
Algorithm: Process x from right to left. Find the rightmost symbol $\boldsymbol{a}_i$ with $i < n$; replace it with $\boldsymbol{a}_{i+1}$ and replace all symbols to its right with $\boldsymbol{a}_1$. If all symbols are $\boldsymbol{a}_n$ (e.g., $x = \boldsymbol{a}_n^k$), the successor is $\boldsymbol{a}_1^{k+1}$: write $k+1$ copies of $\boldsymbol{a}_1$ (extending the tape by one cell).

(b) **Generator TM**: Start with blank tape. Write $\boldsymbol{a}_1$ (the first word). Apply M_{succ} to get the next word; erase the current word, write the successor. Repeat indefinitely. This generates words of Σ^* in lexicographic order, one at a time.

(c) **Enumerator TM**: Start with blank tape. Apply M_{succ} repeatedly, but instead of erasing the previous word, write the next word to the right (separated by a blank). This enumerates Σ^* by writing all words in sequence on the tape.

(d) **Deterministic simulation of NDTM**: Given a nondeterministic TM N, simulate it deterministically using the lexicographic ordering of computation paths. At step k, enumerate all possible k-step computation paths (sequences of transitions) using the lexicographic order, and simulate each one. If any path reaches an accepting state, accept. This gives a deterministic simulation (though potentially much slower). M_{succ} generates the next computation path to try.

11.8. **Semi-infinite tape Turing machine.**

A *semi-infinite tape* has a left boundary (leftmost cell = cell 1) and extends indefinitely to the right.

(a) **Two-track simulation**: Given a TM M with a bi-infinite tape, simulate it on a two-track semi-infinite tape: - Track 1 (top): cells $0, 1, 2, 3, \ldots$ of the original tape. - Track 2 (bottom): cells $-1, -2, -3, \ldots$ of the original tape (stored in reverse). - Cell k of the two-track tape contains (c_k, c_{-k}) for $k \geq 0$ (where c_i is the content of cell i on the original tape), with a special marker at cell 0 to indicate the boundary.

The head of the two-track TM simulates both directions: moving right on Track 1 corresponds to moving right; moving left (if at cell 0) switches to Track 2 and moves right.

(b) **Equivalence proof**: By induction on the number of TM steps: - Each step of M corresponds to a step (or at most two steps) of the two-track machine. - The content of the two tracks correctly represents the bi-infinite tape of M. - Acceptance/rejection conditions are preserved. □

11.9. **TM recognizing $\{\boldsymbol{a}^n \mid n$ is a power of 2$\}$.**

Strategy: Repeatedly check if n is divisible by 2 and halve it until reaching 1.

Algorithm:

1. If input is $\boldsymbol{a}$ ($n = 1 = 2^0$), accept.
2. If input has odd length > 1, reject.
3. Sweep left to right, marking every other $\boldsymbol{a}$ with $\overline{\boldsymbol{a}}$. If n is odd (and > 1) an unmarked $\boldsymbol{a}$ remains at the end — reject. Otherwise, replace all unmarked $\boldsymbol{a}$s with blanks; the $\overline{\boldsymbol{a}}$s form the halved string. Repeat.
4. Repeat until one symbol remains; accept.

11.10. **Three-head TM for $\{x \mid |x|_a = |x|_b = |x|_c\}$.**

Design: All three heads start at the leftmost character.

Strategy: - Head 1 scans for $\boldsymbol{a}$-symbols (moving right, skipping $\boldsymbol{b}$ and $\boldsymbol{c}$). - Head 2 scans for $\boldsymbol{b}$-symbols. - Head 3 scans for $\boldsymbol{c}$-symbols.

In each synchronization step: advance all three heads to their next unmatched symbol of the respective types. If at any point one head reaches the end while others haven't (or vice versa), reject. If all three heads reach the end simultaneously, accept.

No head ever needs to move left: Each head only moves right (it scans past symbols of other types, looking for its own). Since the heads never backtrack, none needs to move left. □

11.11. **Every CFL is Turing-acceptable.**

Proof (Lemma 11.1): Let L be a CFL with PDA $P = \langle \Sigma, \Gamma, S, s_0, \delta, Z_0, F \rangle$.

Construct a TM M that simulates P: - Tape 1: input string x. - Tape 2: stack of P (top of stack at left). - M simulates each PDA move: reads next input symbol (or λ), looks up δ, pops the stack top, pushes the new stack string (nondeterministically if $|\delta| > 1$). - If the PDA reaches a final state with empty input, M accepts.

Since the PDA is nondeterministic, M is a nondeterministic TM. Every NTM can be simulated by a deterministic TM (by enumerating computation paths as in Exercise 11.7(d)).

Thus $L = L(P) = L(M) \in \mathfrak{T}_\Sigma$. □

11.12. **Every type 1 language is a LBL.**

Proof (Theorem 11.3): Let $G = \langle \Sigma, \Omega, Z, P \rangle$ be a context-sensitive grammar (type 1: every production $\alpha \to \beta$ has $|\beta| \geq |\alpha|$, except possibly $Z \to \lambda$).

Construct a nondeterministic LBA M that accepts $L(G)$: - Tape: input x of length n. - M nondeterministically simulates derivations of G in *reverse*: it starts with the candidate string x and applies productions in reverse ($\beta \to \alpha$, which decreases string length since $|\alpha| \leq |\beta|$), trying to reduce back to Z. - Since productions only decrease (or maintain) length in reverse, the tape never needs to grow beyond the original input length n. - If M successfully reduces to Z, accept; otherwise reject.

Since all computations stay within the original n cells, M is a linear bounded automaton. □

11.13. **Convert LBA M to three-track strict LBA.**

(a) **New alphabets**: - The three-track tape has symbols $(a, b, c) \in (\Sigma \cup \{\boldsymbol{B}\}) \times \{L, R, \boldsymbol{B}\} \times \{L, R, \boldsymbol{B}\}$, where: - Track 1: original tape content. - Track 2: left-boundary marker (L in the leftmost cell, $\boldsymbol{B}$ elsewhere). - Track 3: right-boundary marker (R in the rightmost cell of input, $\boldsymbol{B}$ elsewhere).

(b) **New transitions**: For any old transition $\delta(s, a) = (s', a', d)$ in M: - The new TM reads the three-track symbol at the current head position. - It writes the new three-track symbol (updating track 1, preserving tracks 2 and 3). - If the head would move left from the leftmost cell (track 2 = L) or right from the rightmost (track 3 = R), the new TM instead enters a unique "boundary error" state and rejects. - This ensures the head never leaves the input region.

(c) **Proof of equivalence** (for $|x| \geq 2$): By induction on the number of steps: - The strict LBA M' exactly simulates M as long as the head stays within boundaries. - Since M is a linear bounded automaton, it never leaves the input region (this is the definition of LBA). - Therefore the boundary checks never trigger, and M' accepts exactly when M does. □

11.14. **Handle strings of length 1; strict LBA for type 1 languages.**

(a) **Strings of length 1**: Add a special symbol $\underline{\overline{\boldsymbol{b}}}$ that represents "left boundary AND right boundary" (single-cell input). The modified machine enters a special mode when it sees $\underline{\overline{\boldsymbol{b}}}$ and handles single-character inputs by appropriate transitions that do not require movement.

(b) **Strict LBA accepting λ**: The empty string requires the initial scan of a blank cell (since the tape head must look at something). Define: if the initial tape cell is blank, the strict LBA interprets this as λ and immediately decides acceptance based on whether $\lambda \in L$. For type 1 grammars that allow $Z \to \lambda$ (the only way $\lambda \in L$), the strict LBA accepts the blank tape. All other computations proceed as before. □

11.15. **Prove (Theorem 11.4) by induction: TM computation iff LBA grammar derivation.**

Statement: $\alpha s \beta \vdash^* \gamma t \omega$ in M (Turing machine) iff $[\#^i \alpha s \beta \#^j] \Rightarrow^* [\#^m \gamma t \omega \#^n]$ (in the grammar constructed for Theorem 11.4), for some $i, j, m, n \in \mathbb{N}$.

Base (0 transitions): $\alpha s \beta \vdash^* \alpha s \beta$, and $[\#^i \alpha s \beta \#^j] \Rightarrow^* [\#^i \alpha s \beta \#^j]$ (no derivation steps). ✓

Step: If $\alpha s \beta \vdash \gamma t \omega$ by one TM transition, the grammar applies the corresponding production (which is a context-sensitive rule encoding the TM transition). The # boundary markers are adjusted as needed (if the head moves to a new blank cell, a # is converted). By induction, chaining k steps: k TM transitions correspond to k grammar derivation steps. □

11.16. **General induction statement for Theorem 11.5.**

Theorem 11.5 establishes that every LBL is a type 1 (context-sensitive) language.

Inductive statement: For all $k \geq 0$, for all LBA configurations $C = (s, \alpha, i)$ (state s, tape α, head at position i) reachable from the initial configuration on input x in at most k steps:

C is reachable in k LBA steps from the initial configuration iff the sentential form ϕ_C (encoding C as a string of nonterminals in the grammar for $L(M)$) is derivable in k grammar steps from the start symbol.

Proof by induction on k: - Base ($k = 0$): The initial configuration corresponds to the start of the grammar derivation. ✓ - Step: Each LBA transition corresponds to a grammar production. By the IH, all k-step reachable configurations correspond to k-step derivations; adding one more LBA step extends to $k+1$ on both sides. □

11.17. **Type 0 language Kleene closure.**

(a) **Flaw in** $G_* = \langle \Sigma, \Omega \cup \{W\}, W, P \cup \{W \to \lambda, W \to WW, W \to Z\} \rangle$:

The production $W \to WW$ in combination with $W \to Z \to \ldots$ may generate strings with nonterminals from different "copies" of the grammar interfering with each other. Specifically, type 0 grammars can have productions $\alpha\beta \to \gamma$ (context-sensitive or unrestricted) where α and β come from different parts of the derivation. With two independent copies of G (from $W \to WW$) sharing nonterminals, the productions of one copy may interact with nonterminals of the other copy in unintended ways.

(b) **Example**: Let $G = \langle \{A, B, C\}, \{a, b\}, Z, \{Z \to AB, AB \to BA, A \to a, B \to b\} \rangle$. Then $L(G) = \{ab, ba\}$. With G_*: $W \to WW \to ZZ \to ABAB$. The productions of G might apply to the combined string $ABAB$ in unintended ways: $AB \to BA$ could apply to either the first AB or the second AB or the internal pair BA (B from copy 1 and A from copy 2), producing incorrect strings.

(c) **Correct** G_*: To avoid cross-copy interference, rename nonterminals: for each copy of G used in the derivation, use a fresh copy of all nonterminals (with indices). Alternatively, use the Turing machine characterization: since $\mathfrak{T}_\Sigma$ is closed under concatenation and Kleene closure can be simulated by nondeterministically splitting the input and recognizing each piece.

Formally: Let $G_* = \langle \Sigma, \Omega \cup \{W, W'\}, W, P' \rangle$ where P' contains a *fresh* copy P_i of P for each "level" of recursion, with nonterminals marked to prevent interaction. The construction ensures each derivation of a word from $L(G)$ uses an isolated copy of G.

(d) **Proof**: $w \in L(G_*)$ iff w can be written as $w = w_1 w_2 \cdots w_k$ with each $w_i \in L(G)$ iff $W \overset{*}{\Rightarrow} w$ in G_* (by the isolated copy construction). □

11.18. $\mathfrak{T}_\Sigma$ **is closed under various operations.**

(a) **Homomorphism**: Given $L \in \mathfrak{T}_\Sigma$ (TM M accepting L) and homomorphism h, construct TM M' that accepts $h(L)$: M' nondeterministically guesses a string w and verifies $h(w) = x$ (the input) and $w \in L(M)$. Alternatively, directly: for $h(L)$, the TM M' on input y nondeterministically guesses an x with $h(x) = y$ and simulates M on x.

(b) **Inverse homomorphism**: M' on input x computes $h(x)$ and simulates M on $h(x)$. Since h is a computable function (homomorphisms are computable), this works. So $h^{-1}(L) \in \mathfrak{T}_\Sigma$.

(c) **Concatenation**: Given TMs M_1 and M_2 for L_1 and L_2, a nondeterministic TM M for $L_1 \cdot L_2$ guesses a split point $x = uv$, simulates M_1 on u and M_2 on v, accepts if both accept.

(d) **Substitution**: For each symbol $a \in \Sigma$, let $L_a \in \mathfrak{T}_\Sigma$. The substitution $L[a \mapsto L_a]$ replaces each a in words of L with words from L_a. A nondeterministic TM handles this by guessing the substitution. □

11.19. **Equivalence of acceptance modes L_1 and L_2.**

(a) $L(M) = L_1(M) \Rightarrow L(M) = L_2(M)$: Given TM A_1 with $L = L_1(A_1)$ (accept = halt), construct A_2 with $L = L_2(A_2)$ (accept = halt with $\boldsymbol{Y}$ on tape).

Modification: When A_1 enters the halt state: - If A_1 was in a final/accepting configuration: A_2 erases the tape and writes $\boldsymbol{Y}$. - If A_1 was in a rejecting halt: A_2 writes $\boldsymbol{N}$.

For non-halting A_1 computations: same behavior. Since A_1 halts on inputs in L (by L_1), A_2 also halts on those inputs and writes $\boldsymbol{Y}$. For inputs not in L: A_1 may not halt; A_2 has the same non-halting behavior, so $L_2(A_2) = L_1(A_1) = L$. □

(b) $L(M) = L_2(M) \Rightarrow L(M) = L_3(M)$: Given TM A_2 with $L = L_2(A_2)$ (accept = halt with $\boldsymbol{Y}$), construct A_3 with $L = L_3(A_3)$ (accept = halt with blank tape).

Modification: When A_2 halts with $\boldsymbol{Y}$: A_3 erases the entire tape. When A_2 halts with $\boldsymbol{N}$: A_3 writes some non-blank symbol. Then $L_3(A_3)$ = strings for which A_3 halts on blank tape = strings for which A_2 wrote $\boldsymbol{Y}$ = $L_2(A_2) = L$. □

11.20. **$\mathfrak{O}_\Sigma$ (halting TM languages) is closed under the same operations.**

(a) **Homomorphism**: Given a halting TM M for $L \in \mathfrak{O}_\Sigma$, construct a halting TM for $h(L)$: on input y, nondeterministically enumerate all x with $h(x) = y$ in lexicographic order. For each x, simulate M on x (which halts since M is a halting TM). If M accepts some x, accept y; if all x are rejected, reject y. Since h is a homomorphism, the set of x with $h(x) = y$ is finite and computable. The TM halts on all inputs.

(b) **Inverse homomorphism**: On input x, compute $h(x)$ (finite computation), simulate halting TM M on $h(x)$; since M halts on all inputs, the whole machine halts.

(c) **Concatenation**: Nondeterministically split input $x = uv$ (finite choices), simulate halting TMs M_1 on u and M_2 on v (both halt). Accept if both accept. The constructed TM halts on all inputs.

(d) **Substitution**: For each a, the substituted language $L_a \in \mathfrak{O}_\Sigma$ has a halting TM. The substitution machine nondeterministically applies substitutions and simulates all relevant halting TMs, which all halt. Hence the combined machine halts. □

Chapter 12

Decidability — Solutions

12.1. **Verify assertions in proof of Theorem 12.1 concerning Theorem 2.7.**

Theorem 2.7 (Nerode's theorem) states that a language is FAD (finite automaton definable) iff its Nerode right congruence R_L has finite index.

Theorem 12.1 (decidability of $L(A) = \emptyset$) uses the following from Theorem 2.7: - The minimal DFA A_{min} has states in bijection with equivalence classes of R_L. - $L(A) = \emptyset$ iff no final state is reachable from s_0 iff no equivalence class of R_L containing an accepted string is reachable. - Reachability is decidable by BFS/DFS on the finite graph of A.

The assertion to verify: the equivalence classes of R_L for a finite-index R_L correspond exactly to the states of the minimal DFA. This follows directly from Theorem 2.7: each class $[x]_{R_L}$ maps to the state $\overline{\delta}(s_0, x)$, and this mapping is a bijection between classes and states. □

12.2. **Prove Lemma 12.1.**

Lemma 12.1 (typically states): For a DFA $A = \langle \Sigma, S, s_0, \delta, F \rangle$ with $|S| = n$, $L(A) \neq \emptyset$ iff $L(A)$ contains a string of length less than n.

Proof: ($\Leftarrow$) Obvious.

($\Rightarrow$): If $L(A) \neq \emptyset$, let $x \in L(A)$. If $|x| < n$, done. If $|x| \geq n$, then the sequence of states $s_0, \overline{\delta}(s_0, x_1), \overline{\delta}(s_0, x_1 x_2), \ldots, \overline{\delta}(s_0, x$ has $|x|+1 > n$ states, so by the pigeonhole principle, some state repeats. Say $\overline{\delta}(s_0, u) = \overline{\delta}(s_0, uv) = q$ where $|v| \geq 1$. Then $\overline{\delta}(s_0, u) = q = \overline{\delta}(s_0, uv)$, and since $x = uvw \in L(A)$, $\overline{\delta}(s_0, uw) = \overline{\delta}(s_0, uvw) \in F$. So $uw \in L(A)$ and $|uw| < |uvw| = |x|$. Repeat until length $< n$. □

12.3. **Number of nonfinal states in minimal DFA ≤ nonfinal states in any machine B with $L(B) = L$.**

Proof: Let A_{min} be the minimal DFA for L with n_{min} nonfinal states and B any machine with $L(B) = L$ having n_B nonfinal states.

Each nonfinal state q of A_{min} corresponds to a unique right-congruence class $[x]_{R_L}$ (by Nerode's theorem) with $\overline{\delta}_{min}(s_0, x) = q$ and $x \notin L$ (since q is nonfinal). In any machine B with $L(B) = L$, two strings in different R_L-classes must lead to different states (otherwise B would accept/reject some string incorrectly). Therefore B must have at least as many nonfinal states as A_{min}. More precisely, the number of nonfinal classes of R_L equals the number of nonfinal states in A_{min}, and B must have at least this many nonfinal states. □

12.4. **Decidability of $L(A_1) \subseteq L(A_2)$.**

Algorithm:

(a) Construct DFA A_2^c for $\overline{L(A_2)}$ (complement: swap final/nonfinal states of A_2's complete DFA).

(b) Construct DFA $A^\cap$ for $L(A_1) \cap \overline{L(A_2)}$ (product machine).

(c) $L(A_1) \subseteq L(A_2)$ iff $L(A_1) \cap \overline{L(A_2)} = \emptyset$ iff $L(A^\cap) = \emptyset$ (decidable by Theorem 12.1). □

12.5. **Decidability of $L(A)$ is cofinite.**

$L(A)$ is cofinite iff $\overline{L(A)}$ is finite.

Algorithm:

(a) Construct DFA A^c for $\overline{L(A)}$ (complement).

(b) $\overline{L(A)}$ is finite iff $L(A^c)$ is finite iff the DFA A^c (after removing unreachable states) has no cycle reachable from the start state that also leads to a final state (Theorem 12.2 analog).

(c) Check for cycles in A^c among useful states (BFS/DFS). Decidable. □

12.6. **Decidability: does $L(A)$ contain any string of length > 1228?**

Algorithm (using the pumping lemma and DFA structure): - By Theorem 12.2 (pumping lemma), $L(A)$ is infinite iff $L(A)$ contains a string of length $n \leq k < 2n$ (where $n = |S|$). - For strings of length > 1228: if $|S| \leq 1228$, enumerate all strings of length n to $2n-1$ and check if any is in $L(A)$ (finitely many checks). If $L(A)$ contains such a string, it also contains strings of length > 1228. - More directly: compute $\{\overline{\delta}(s_0, y) \mid |y| = 1229\}$ (reachable states at depth 1229) by BFS. If any final state is reachable at depth > 1228 (specifically, any path of length > 1228 reaches a final state), the answer is yes. BFS on the DFA graph answers this. □

12.7. **Decidability: does A accept any even-length strings?**

Algorithm:

(a) Construct DFA A_{even} that recognizes $\{x \mid |x| \text{ is even}\}$ (a simple 2-state DFA alternating between even/odd states).

(b) Construct product DFA $A^\cap$ for $L(A) \cap L(A_{even})$.

(c) A accepts an even-length string iff $L(A^\cap) \neq \emptyset$. Decidable by Theorem 12.1. □

12.8. **Decidability: R_1 and R_2 represent complementary languages.**

Algorithm:

(a) Convert regular expressions R_1 and R_2 to DFAs A_1 and A_2.

(b) Check $L(A_1) = \overline{L(A_2)}$: equivalently, check $L(A_1) \subseteq \overline{L(A_2)}$ and $\overline{L(A_2)} \subseteq L(A_1)$.

(c) By Exercise 12.4, each subset relation is decidable. Equivalently, check $L(A_1) \cup L(A_2) = \Sigma^*$ and $L(A_1) \cap L(A_2) = \emptyset$.

(d) Both conditions are decidable (using complement, intersection, and emptiness). □

12.9. **Decidability: R_1 and R_2 describe common strings.**

Algorithm: $L(R_1) \cap L(R_2) \neq \emptyset$?

(a) Convert R_1, R_2 to DFAs A_1, A_2.

(b) Compute the product DFA $A^{\cap}$ for $L(A_1) \cap L(A_2)$.

(c) Check $L(A^{\cap}) \neq \emptyset$ (decidable by Theorem 12.1). □

12.10. **Decidability: is there a DFA with < 31 states accepting $L(R_1)$?**

Algorithm:

(a) Convert R_1 to a DFA A (via NFA construction).

(b) Minimize A to get A_{min}.

(c) There is a DFA with < 31 states accepting $L(R_1)$ iff $|A_{min}| < 31$.

(d) Count the states of A_{min}: decidable. □

12.11. **Decidability: is there a DFA with > 31 states accepting $L(R_1)$?**

One-step algorithm: Yes, always. If A_{min} has k states, then we can construct a DFA with $k + \ell$ states for any $\ell > 0$ by adding dead (sink) states. If $k \leq 31$, add $32 - k$ unreachable dead states. If $k > 31$, A_{min} itself already has > 31 states.

Therefore, for *any* regular language, there always exists a DFA with > 31 states accepting it. The answer is **always yes**. □

12.12. **Decidability: $\exists$ λ-NDFA with ≤ 1 final state accepting R.**

Algorithm:

(a) Convert R to a λ-NDFA N using the Kleene/Thompson construction. This construction produces NDFAs with exactly one final state. So the answer is always yes for any regular expression R. □

12.13. **Decidability: $\exists$ NDFA (without λ-moves) with ≤ 1 final state accepting $L(A)$.**

Algorithm:

(a) Minimize the DFA A to get A_{min}.

(b) A DFA (hence NDFA) with one final state accepting L exists iff no two distinct final states of A_{min} are distinguishable by the language continuation. More precisely: the language L can be accepted by a one-final-state NDFA iff the set F of final-state equivalence classes (in the Myhill-Nerode quotient) can be "collapsed" into one: this is possible iff all strings in L lead to the same right-congruence class (i.e., L has a single final Nerode class), or more generally, the structure allows merging.

(c) Check: if A_{min} has exactly 1 final state, yes. Otherwise, check if the language structure allows a 1-final-state NDFA: convert to minimal DFA, check how many final Nerode classes exist. Decidable. □

12.14. **Decidability of $R_1 = R_2$ (same language).**

Algorithm:

(a) Convert R_1, R_2 to DFAs A_1, A_2.

(b) Minimize both: A_1^{min}, A_2^{min}.

(c) $L(R_1) = L(R_2)$ iff $A_1^{min} \cong A_2^{min}$ (isomorphic as DFAs). Check isomorphism: decidable in polynomial time. □

12.15. **Decidability: R_1 and R_2 generate the same right congruence ($R_{L_1} = R_{L_2}$).**

Algorithm: $R_{L_1} = R_{L_2}$ iff L_1 and L_2 have the same Nerode congruence iff the minimal DFAs for L_1 and L_2 are isomorphic (same state structure, same transitions, same final states). By Exercise 12.14, this reduces to checking $L(R_1) = L(R_2)$, which is decidable. □

12.16. **Prove Theorem 12.5.**

Theorem 12.5: The equivalence problem for DFAs is decidable.

Proof: Given DFAs A_1 and A_2, $L(A_1) = L(A_2)$ iff $L(A_1) \triangle L(A_2) = \emptyset$ (symmetric difference). $L(A_1) \triangle L(A_2) = (L(A_1) - L(A_2)) \cup (L(A_2) - L(A_1)) = (L(A_1) \cap \overline{L(A_2)}) \cup (L(A_2) \cap \overline{L(A_1)})$.

Construct the corresponding DFA (product machine) for this symmetric difference. Check if it is empty (decidable by Theorem 12.1). □

12.17. **Efficient algorithm for $\{\overline{\delta}(s_0, y) \mid y \in \Sigma^*,\ n \le |y| < 2n\}$.**

Algorithm: BFS on the DFA state graph, tracking string length.

(a) Initialize: at length 0, state is $\{s_0\}$.

(b) BFS layer by layer (one transition per layer, tracking the current layer as the string length).

(c) At layer n: record all states reached at length n (i.e., states reachable by strings of length n).

(d) At layers $n, n+1, \ldots, 2n-1$: continue BFS, recording states reached at each length.

(e) Return the union of all states reached at lengths n through $2n-1$.

Since the DFA has $|S|$ states, the BFS terminates in at most $2n$ steps, each taking $O(|S| \cdot |\Sigma|)$ work. Total: $O(n \cdot |S| \cdot |\Sigma|)$.

Justification for halting: The DFA has a finite number of states. BFS on the finite DFA graph terminates in at most $|S|$ layers. We run for $2n$ layers, which is finite. □

12.18. **Would intersecting $\{\overline{\delta}(s_0, y) \mid 5n \le |y| < 6n\}$ with F always work?**

Answer: Yes, it would always produce the correct answer.

Justification: By the pumping lemma (Theorem 12.2), $L(A)$ is infinite iff $L(A)$ contains a string of length $n \le k < 2n$ (where $n = |S|$).

Claim: $L(A)$ is infinite iff $\{\overline{\delta}(s_0, y) \mid 5n \le |y| < 6n\} \cap F \neq \emptyset$.

($\Rightarrow$): If $L(A)$ is infinite, it contains strings of arbitrary length, so in particular strings of length between $5n$ and $6n-1$. Hence the intersection is nonempty.

($\Leftarrow$): If the intersection is nonempty, there exists a string of length $5n \le k < 6n$ in $L(A)$. Since $k \ge 5n > n = |S|$, by the pigeonhole principle some state repeats in the path of length k; thus a shorter string is also accepted, and by applying the pumping argument, $L(A)$ is infinite.

So yes, this alternative range would work correctly. □

12.19. **Decidability of equivalence of two Mealy machines.**

Algorithm:

(a) Minimize both Mealy machines M_1 and M_2.

(b) Two Mealy machines are equivalent iff they produce identical outputs on all inputs.

(c) Construct a product machine $M^{\times}$ that tracks the states of M_1 and M_2 simultaneously and checks if outputs agree on every input.

(d) Use a reachability analysis: find all states (q_1, q_2) of the product machine reachable from the initial state where the outputs differ. If no such state is reachable, the machines are equivalent.

(e) Reachability is decidable (BFS/DFS on the finite product machine graph). □

12.20. **Decidability of equivalence of two Moore machines.**

Algorithm: Same as Exercise 12.19. Moore machines output based on state (not transition), so:

(a) Build the product machine tracking states of both Moore machines.

(b) Check if any reachable state pair (q_1, q_2) has $\omega_1(q_1) \neq \omega_2(q_2)$ (different outputs).

(c) If not, the machines are equivalent. Decidable by BFS. □

12.21. **Decide whether R represents strings of length > 28, without finite automata.**

Algorithm (direct on regular expressions):

By structural induction on the regular expression R: - $R = \emptyset$: max length $= -\infty$; answer: no. - $R = \lambda$ or $R = a$ ($a \in \Sigma$): max length ≤ 1; answer: no. - $R = R_1 \cup R_2$: yes if either R_1 or R_2 represents a string of length > 28. - $R = R_1 \cdot R_2$: yes if the sum of max lengths exceeds 28, or if either has unbounded length (due to *). - $R = R_1^*$: yes if R_1 represents any nonempty string (then R_1^{29} has length > 28); formally, R_1^* represents strings of length > 28 iff R_1 is not $\emptyset$ and R_1 is not $\{\lambda\}$ alone.

This induction on the structure of R terminates in finitely many steps. □

12.22. **Decide whether $L(G)$ contains a string of length > 28 for right-linear grammar G.**

Algorithm (without finite automata):

Solve the language equations for G: compute the set of nonterminals reachable from the start symbol, and check for cycles in the nonterminal dependency graph. If any cycle involves a nonterminal that generates nonempty strings, the language is infinite; in particular, it contains strings of length > 28. Otherwise, solve the equations to find the maximum derivable string length.

Alternatively: perform BFS in the derivation tree up to depth 29 (each step adds at most one terminal). If any terminal string of length > 28 is derivable, answer yes. This terminates since the grammar is finite. □

12.23. **Prove $Z_0 \subseteq Z_1 \subseteq \cdots \subseteq Z_n \subseteq \cdots \subseteq \Omega$.**

In the proof of Theorem 12.6 (decidability of $L(G) = \emptyset$ for CFG G), Z_i is the set of nonterminals reachable from the start symbol S in i or fewer derivation steps.

$Z_0 = \{S\}$ (only the start symbol). $Z_{i+1} = Z_i \cup \{B \mid \exists A \in Z_i, \exists \alpha : A \to \alpha \in P, B \in \alpha\}$ (all nonterminals appearing in right-hand sides of productions of nonterminals in Z_i).

Proof: $Z_0 \subseteq Z_1$ since $Z_1 = Z_0 \cup (\cdots) \supseteq Z_0$. By induction: if $Z_k \subseteq Z_{k+1}$, then $Z_{k+1} \subseteq Z_{k+2}$ since any nonterminal in Z_{k+1} is reachable in $k+1$ steps, hence reachable in $k+2$ steps as well. The chain is bounded above by Ω (which is finite). □

12.24. **If $Z_m = Z_{m+1}$, then Z_m is the set of all nonterminals reachable from S.**

Proof: If $Z_m = Z_{m+1}$, then no new nonterminals are added in step $m+1$. This means: every nonterminal appearing in the right-hand side of any production of any nonterminal in Z_m is already in Z_m.

Claim: Z_m is closed under reachability, i.e., every nonterminal reachable from S is in Z_m.

Suppose B is reachable from S (i.e., $S \overset{*}{\Rightarrow} \alpha B \beta$ for some α, β). The shortest derivation has some length k. By monotonicity ($Z_0 \subseteq Z_1 \subseteq \cdots$), $B \in Z_k$. Since $Z_m = Z_{m+1} = Z_{m+2} = \cdots$ (the sequence stabilizes), $B \in Z_m$ for all $k \leq m$ already. If $k > m$, then by the stabilization, every nonterminal reachable in k steps is already in Z_m. So Z_m = all reachable nonterminals. □

12.25. **Prove $\exists m \in \mathbb{N} : Z_m = Z_{m+1}$.**

Proof: The sequence $Z_0 \subseteq Z_1 \subseteq \cdots \subseteq \Omega$ is a non-decreasing chain of subsets of Ω. Since $|\Omega|$ is finite, the chain can increase at most $|\Omega|$ times. Therefore there exists $m \leq |\Omega|$ such that $Z_m = Z_{m+1}$ (the chain stabilizes). □

12.26. **Definition of W_i and monotonicity.**

(a) In the proof of Theorem 12.6, W_i is the set of nonterminals that can derive a terminal string in i or fewer steps: $W_0 = \emptyset$. $W_{i+1} = W_i \cup \{A \mid \exists (A \to \alpha) \in P$: every symbol in $\alpha \in \Sigma \cup W_i\}$. (A nonterminal A can derive a terminal string in $\leq i+1$ steps if some production of A has a right-hand side consisting only of terminals and nonterminals already known to derive terminal strings.)

(b) **Proof of monotonicity**: $W_0 = \emptyset \subseteq W_1$ (trivially). By induction: if $W_k \subseteq W_{k+1}$, then for any $A \in W_{k+1}$: A derives a terminal string in $\leq k+1$ steps, hence in $\leq k+2$ steps as well, so $A \in W_{k+2}$. Thus $W_{k+1} \subseteq W_{k+2}$. The chain is bounded by Ω. □

12.27. **If $W_m = W_{m+1}$, then W_m is the set of all nonterminals producing terminal strings.**

Proof: If $A \overset{*}{\Rightarrow} w \in \Sigma^*$ in k steps, then $A \in W_k$. Since $W_m = W_{m+k}$ for all $k \geq 0$ (by stabilization), $A \in W_m$. Conversely, every $A \in W_m$ derives a terminal string by definition. So W_m is precisely the set of productive nonterminals. □

12.28. **Prove $\exists m \in \mathbb{N} : W_m = W_{m+1}$.**

Proof: Same as Exercise 12.25: $W_0 \subseteq W_1 \subseteq \cdots \subseteq \Omega$ is a non-decreasing chain in the finite set Ω, so it stabilizes. □

12.29. **Decidability questions about NDFA A with λ-moves.**

(a) **Exists string with no complete path**: A "complete path" is one reading all input symbols. Check if there is any state (reachable from s_0 along some path) from which some transition is undefined. Convert A to DFA A^d (subset construction); check if the dead state (empty subset) is reachable in A^d. Decidable.

(b) **Exists string with a path to a nonfinal state**: BFS/DFS on A to find if any complete computation path ends in a nonfinal state. Decidable (finite automaton has finitely many configurations per string-length).

(c) **Exists accepted string with all paths leading to final states**: Find strings where every path from s_0 to an accepting state passes only through transitions that can also lead to final states. Check using the product of A with a tracking automaton. Decidable.

(d) **All accepted strings have all complete paths to final states**: More complex, but still decidable: check if the complement of this property is empty. Using DFA A^d (subset construction), acceptance corresponds to subsets containing at least one final state. Check if any reachable subset in A^d contains both a final and a nonfinal state of A. Decidable.

(e) **All strings have unique paths**: A string has a unique path iff the NDFA is deterministic on that string (no two applicable transitions from any state on any symbol along the path). Check: for each state q and symbol a, if $|\delta(q,a)| > 1$ or there are λ-move alternatives, the NDFA is nondeterministic on some strings. More carefully: check if any string leads to a state with multiple transition choices. Decidable (check transitions structure). □

12.30. **Decidability for two DFAs A_1 and A_2.**

(a) **Decidability of homomorphism existence**: A homomorphism $h : A_1 \to A_2$ (of DFAs) is a state map preserving start state, transitions, and taking final states to final states. Check all functions $h : S_1 \to S_2$: finitely many ($|S_2|^{|S_1|}$). For each, check the homomorphism conditions. Decidable (finite check).

(b) **Decidability of isomorphism existence**: An isomorphism requires h to be bijective (so $|S_1| = |S_2|$). Check all bijections: $(|S_1|)!$ many. For each, verify the isomorphism conditions. Decidable (finite check for finite S_1).

(c) **Decidability of > 3 isomorphisms**: Similarly, count the number of valid isomorphisms (at most $|S_1|!$ total). If the count exceeds 3, answer yes. Decidable. □

12.31. **Decidability: R_1 describes infinitely many strings (no DFA construction).**

Algorithm (structural induction on R_1): - $R_1 = \emptyset$: finite (empty). No. - $R_1 = \lambda$ or $R_1 = a$: finite. No. - $R_1 = R \cup R'$: infinite iff R or R' is infinite. - $R_1 = R \cdot R'$: infinite iff (R is infinite and $L(R')$ is nonempty) or ($L(R)$ is nonempty and R' is infinite). - $R_1 = R^*$: infinite iff $L(R)$ contains a nonempty string.

Each check is computable by structural recursion on R_1. Terminates since R_1 is finite. □

12.32. **Decidability of Mealy machine equivalent to Moore machine.**

Algorithm:

(a) Minimize the Mealy machine M and the Moore machine A.

(b) Convert one to the other's representation (Mealy to Moore or vice versa) using the standard transformation.

(c) Check equivalence (Exercise 12.19 or 12.20). Decidable. □

12.33. **Decidability: $L(R_2)$ properly contains $L(R_1)$ ($L(R_1) \subset L(R_2)$).**

Algorithm:

(a) Convert R_1, R_2 to DFAs.

(b) Check $L(R_1) \subseteq L(R_2)$ (decidable, Exercise 12.4).

(c) Check $L(R_1) \neq L(R_2)$ (decidable, Exercise 12.14 or 12.15).

(d) $L(R_1) \subset L(R_2)$ iff both conditions hold. □

12.34. **Efficient method for $L(A) = \emptyset$ for NDFA A, without DFA conversion.**

(a) **Method**: BFS/DFS directly on the NDFA A (viewed as a graph). Start from s_0, compute the λ-closure, then follow transitions. $L(A) = \emptyset$ iff no final state is reachable from s_0 (via any sequence of transitions and λ-moves). Reachability on the finite graph of A (without constructing the exponential subset DFA).

(b) **Method for infinite $L(A)$**: BFS on A to find all states reachable from s_0 and from which a final state is reachable. Among these "useful" states, check if there is a cycle (a state that appears twice on some accepting path). BFS/DFS to find a cycle among useful states. If yes, $L(A)$ is infinite; else finite. No DFA conversion needed. □

12.35. **Decidability: $L(M)$ is a regular set for DPDA M.**

Reference result: It is known (Stearns 1967) that it is decidable whether the language accepted by a DPDA is regular. The decision procedure involves analyzing the structure of the DPDA (looking for certain patterns of stack behavior called "bounded stack"), but the proof is complex.

Outline: A DPDA accepts a regular language iff its stack depth is bounded (the stack never grows beyond a fixed bound). This can be checked by analyzing the DPDA's transition structure for cycles that strictly increase stack depth. Such analysis is decidable since the DPDA has finite state and transition structure. □

12.36. **Algorithm for path-finding in graphs (Theorem 12.9).**

(a) **Appropriate algorithm**: BFS (breadth-first search) from the source node to find all reachable nodes and shortest paths. For directed graphs (as in the PDA product machine):

a. Initialize a queue with the start node; mark it visited.

b. At each step, dequeue a node, enqueue all unvisited neighbors, mark them visited.

c. Terminate when the queue is empty or the target is found.

Time complexity: $O(|V| + |E|)$ where V is vertices and E is edges.

(b) **More efficient recursive algorithm**: DFS with memoization. `Reach`(v, t): returns true if t is reachable from v. - If $v = t$: return true. - If v is marked: return false. - Mark v; for each neighbor u of v: if `Reach`(u, t): return true. - Return false.

This avoids reprocessing nodes and runs in $O(|V| + |E|)$ time. □

12.37. **$\mathfrak{H}_\Sigma$ (halting-TM languages) closed under union, intersection, concatenation, reversal.**

(a) **Union**: Given halting TMs M_1 (L_1) and M_2 (L_2), build M for $L_1 \cup L_2$: run M_1 and M_2 in parallel (interleaved simulation). Since both halt on all inputs, M also halts. M accepts iff M_1 or M_2 accepts. □

(b) **Intersection**: Same parallel simulation; accept iff both accept. Both halt, so M halts. □

(c) **Concatenation**: M for $L_1 \cdot L_2$ on input x: enumerate all splits $x = uv$, run M_1 on u and M_2 on v. Since both halt, each check halts. There are $|x| + 1$ splits (finitely many), all decidable. Accept if any split works. □

(d) **Reversal**: M for L_1^r on input x: compute x^r (finitely computable), run M_1 on x^r. M_1 halts on x^r; so M halts. Accept iff M_1 accepts x^r. □

12.38. **Halting TM T' equivalent to $L_2(T)$ for halting TM T.**

$L = L_2(T)$: T halts on all inputs and L consists of strings for which T halts with ***Y*** on the tape.

Construct T':

1. Simulate T on input w.
2. T halts (since it halts on all inputs by assumption).
3. If T halted with ***Y*** somewhere on the tape: erase the tape, write w then ***Y*** after w on an otherwise blank tape; halt in the accepting state.
4. If T halted with no ***Y***: erase the tape, write w then ***N*** after w; halt in a rejecting state.

T' satisfies all three conditions: (1) halts on all input (since T does and the post-processing is finite); (2) writes ***Y*** after accepted words; (3) writes ***N*** after rejected words. And $L(T') = L_2(T) = L$. □

12.39. **Machine $M_{\mathfrak{M}}$ and application to Theorem 12.15.**

(a) **Proof**: Let X be the set of encodings of halting TMs (all in $\mathfrak{M}$). Suppose every encoding in X represents a halting TM. Assume $M_{\mathfrak{M}}$ decides membership in X.

By diagonalization: construct a TM D that on input $\langle T\rangle$ (encoding of TM T): - Simulates $M_{\mathfrak{M}}$ on $\langle T\rangle$. - If $M_{\mathfrak{M}}$ accepts (i.e., $T \in X$): loop forever (don't halt). - If $M_{\mathfrak{M}}$ rejects: accept and halt.

D is a halting TM on inputs not in X, but loops on inputs in X. What about D itself? If $\langle D\rangle \in X$, then D is a halting TM, but by construction D loops on $\langle D\rangle$ (contradiction). If $\langle D\rangle \notin X$, then D halts on $\langle D\rangle$ (accepting), so $\langle D\rangle$ represents a halting TM, contradicting $\langle D\rangle \notin X$.

So $M_{\mathfrak{M}}$ cannot exist: there is a halting TM D whose language is not in $\mathfrak{M}$. □

(b) **Application to Theorem 12.15**: Theorem 12.15 states that the halting problem is undecidable. Applying part (a) with $X = \{\langle T\rangle \mid T \text{ halts on all inputs}\}$ and $\mathfrak{M}$ = all Turing-acceptable languages: if there were an $M_{\mathfrak{M}}$ deciding whether T halts on all inputs, then by part (a), some halting TM D exists whose language is not Turing-acceptable — contradiction. Hence no such $M_{\mathfrak{M}}$ exists. □

12.40. **Encoding context-sensitive grammars.**

(a) **Encoding scheme**: Assume 2 terminal symbols $\{a, b\}$ and arbitrarily many nonterminals $N_1, N_2, N_3, \ldots$

Encoding: - Terminals: $a \mapsto \texttt{00}$, $b \mapsto \texttt{01}$. - Nonterminal $N_i \mapsto \texttt{1}^i\texttt{0}$ (unary encoding with terminator). - Productions: $\alpha \to \beta$ encoded as $\overline{\alpha}\texttt{##}\overline{\beta}$ where $\overline{x}$ is the encoding of x symbol by symbol. - Grammar: list of productions separated by ###, preceded by the start symbol encoding.

This encoding handles an unbounded number of nonterminals via unary encoding and terminates on any finite grammar.

(b) **Algorithm to decide valid CSG encoding**:

i. Parse the encoding to extract the alphabet, nonterminal list, start symbol, and productions.

ii. Check that: (a) terminal symbols are from $\{a, b\}$; (b) nonterminals are distinct and properly encoded; (c) each production $\alpha \to \beta$ has $|\beta| \geq |\alpha|$ (context-sensitive condition), except possibly $Z \to \lambda$.

iii. Verify the syntactic structure of the encoding (valid separators, well-formed strings).

iv. The TM accepts valid encodings and rejects invalid ones. Terminates in polynomial time in the encoding length. □

12.41. **Undecidability of $L(X) = \emptyset$.**

(a) **Arbitrary Turing machines**: Undecidable. If it were decidable, we could solve the halting problem: reduce the question "does T halt on w?" to "does $L(T_w) = \emptyset$?" where T_w runs T on w and accepts iff T halts.

(b) **Halting Turing machines**: Still undecidable. By Rice's theorem applied to halting TMs: any nontrivial property of the language of a halting TM is undecidable.

(c) **Context-sensitive grammars**: Undecidable (same reduction: membership in the halting set reduces to emptiness of the language of a CSG constructed from the TM encoding).

(d) **Linear bounded automata**: Undecidable. The emptiness problem for LBAs is undecidable (since CSG = LBL and CSG emptiness is undecidable). □

12.42. **Undecidability of $L(X) = \Sigma^*$.**

(a) **Arbitrary TMs**: Undecidable (reduce from halting problem).

(b) **Halting TMs**: Undecidable (Rice's theorem for halting TMs).

(c) **Context-sensitive grammars**: Undecidable.

(d) **Linear bounded automata**: Undecidable.

(e) **Context-free grammars**: Undecidable. The universality problem for CFGs ("does G generate Σ^*?") is undecidable. This is a classical result: it can be shown by reducing from the Post Correspondence Problem.

(f) **Pushdown automata**: Undecidable (equivalent to CFG universality). □

12.43. **Set E of encodings of TMs halting on λ.**

(a) $E \in \mathfrak{T}_\Sigma$: Yes. The TM that recognizes E: on input $\langle T \rangle$, simulate T on λ. If T halts, accept (since that means T halts on λ, so $\langle T \rangle \in E$). If T doesn't halt, the TM doesn't halt either. So E is Turing-acceptable ($\mathfrak{T}_\Sigma$). But $E \notin \mathfrak{O}_\Sigma$ (not decidable): if it were, we could decide the halting problem.

(b) $E \in \mathfrak{H}_\Sigma$: No. E is not recursive (decidable). If $E \in \mathfrak{H}_\Sigma$ (halting TM language = recursive language), then E would be decidable. But the halting problem for T on λ is undecidable. So $E \notin \mathfrak{H}_\Sigma$.

(c) $E \in \mathfrak{O}_\Sigma$: No. $\mathfrak{O}_\Sigma = \mathfrak{H}_\Sigma$ (halting = recursive = decidable). As argued above, E is not decidable. $E \notin \mathfrak{O}_\Sigma$.

12.44. **Set N of encodings of TMs that do NOT halt on λ.**

(a) $N \in \mathfrak{T}_\Sigma$: No. $N = \overline{E}$ (complement of E from Exercise 12.41). Since $E \in \mathfrak{T}_\Sigma$ but $E \notin \mathfrak{O}_\Sigma$, the complement $N = \overline{E} \notin \mathfrak{T}_\Sigma$ (if $E \in \mathfrak{T}_\Sigma$ and $\overline{E} \in \mathfrak{T}_\Sigma$, then E would be recursive, contradicting $E \notin \mathfrak{O}_\Sigma$). So $N \notin \mathfrak{T}_\Sigma$.

(b) $N \in \mathfrak{H}_\Sigma$: No. Same argument: N would be recursive (decidable) if $N \in \mathfrak{H}_\Sigma$, contradicting undecidability of the non-halting problem.

(c) $N \in \mathfrak{O}_\Sigma$: No. $\mathfrak{O}_\Sigma = \mathfrak{H}_\Sigma$. □

12.45. **Decidability questions about equivalence classes of $R_{L(A)}$.**

(a) **At least one equivalence class is finite**: For a DFA A, the equivalence classes of $R_{L(A)}$ correspond to states of the minimal DFA A_{min}. An equivalence class $[x]$ is finite iff the state $q = \overline{\delta}_{min}(s_0, x)$ has no cycles reachable from itself that lead back to q. More precisely, $[x] = \{y \mid \overline{\delta}_{min}(s_0, y) = q\}$ is the set of strings leading to state q.
This set is finite iff q is not reachable from itself in A_{min} (i.e., there is no cycle involving q). Check: in the DFA graph, does state q have any path from itself back to itself? BFS/DFS in the state graph. Decidable.

(b) **All equivalence classes are finite**: All classes are finite iff the DFA graph is acyclic (no cycles, since any cycle would create an infinite class). Check if the DFA graph has any cycles: DFS looking for back edges. Decidable. □

12.46. **Decide $L(G_1) \subset L(G_2)$ for right-linear grammars G_1, G_2.**

Algorithm:

(a) Convert G_1, G_2 to DFAs A_1, A_2 (via Lemma 8.1).

(b) Check $L(A_1) \subseteq L(A_2)$ (decidable, Exercise 12.4).

(c) Check $L(A_1) \neq L(A_2)$ (decidable, Exercise 12.14).

(d) $L(G_1) \subset L(G_2)$ (proper subset) iff $L(A_1) \subseteq L(A_2)$ and $L(A_1) \neq L(A_2)$. □